Statistical Machine Learning

Lecture Notes for STA 314 and STA 414

Haocheng Sun

University of Toronto, Fall 2024 – Winter 2025

Based on the STA 314 lectures and tutorials (Fall 2024) and the STA 414/2104 lectures and tutorials (Winter 2025).

Contents

Overview vi

I Foundations 1

1 The Learning Problem 2
- 1.1 Supervised Learning . . . 2
- 1.2 Parametric and Non-Parametric Methods . . . 3
- 1.3 Loss, Risk, and the Best Possible Predictor . . . 4
- 1.4 Training Error, Test Error, and Overfitting . . . 6
- 1.5 The Bias–Variance Trade-off . . . 7

2 Probabilistic Models and Estimation 9
- 2.1 Probabilistic Models . . . 9
- 2.2 Maximum Likelihood Estimation . . . 10
- 2.3 Sufficient Statistics and Exponential Families . . . 11
- 2.4 Bayesian Inference . . . 12
- 2.5 The Multivariate Gaussian . . . 13

3 Optimization 16
- 3.1 Learning as Optimization . . . 16
- 3.2 Gradient Descent . . . 17
- 3.3 Convexity . . . 18
- 3.4 Stochastic Gradient Descent . . . 19
- 3.5 Non-Convex Optimization . . . 20

II Prediction 22

4 Linear Regression and Model Selection 23
- 4.1 The Linear Model and Ordinary Least Squares . . . 23
- 4.2 Sampling Properties and Inference . . . 24
- 4.3 Qualitative Predictors, Interactions and Diagnostics . . . 25
- 4.4 Why Look Beyond OLS . . . 25
- 4.5 Validation Sets and Cross-Validation . . . 26
- 4.6 Adjusting the Training Error: Cp, AIC, BIC and Adjusted R^2 . . . 27
- 4.7 Subset Selection . . . 28

5 Shrinkage and Bayesian Linear Regression **30**
5.1 Ridge Regression . . . 30
5.2 The Lasso . . . 31
5.3 Shrinkage in an Orthogonal Design . . . 32
5.4 Choosing the Penalty and Other Penalties . . . 33
5.5 Penalties as Priors . . . 34
5.6 Bayesian Linear Regression . . . 34

6 Beyond Linearity **37**
6.1 Basis Expansions . . . 37
6.2 Regression Splines . . . 38
6.3 Local Regression . . . 39
6.4 Generalized Additive Models . . . 40

7 Classification: Logistic Regression and Discriminant Analysis **42**
7.1 The Classification Problem . . . 42
7.2 Logistic Regression . . . 43
7.3 Evaluating a Classifier . . . 45
7.4 Multinomial Logistic Regression . . . 45
7.5 Generative Classifiers and Discriminant Analysis . . . 46
7.6 Logistic Regression versus LDA . . . 48

8 Support Vector Machines **50**
8.1 Separating Hyperplanes . . . 50
8.2 The Maximal Margin Classifier . . . 51
8.3 The Soft-Margin SVM . . . 51
8.4 The Hinge-Loss View . . . 52
8.5 Limitations and Comparison . . . 54

9 Kernel Methods and Gaussian Processes **56**
9.1 Feature Maps and Their Cost . . . 56
9.2 Kernel Ridge Regression and the Kernel Trick . . . 57
9.3 Kernels . . . 58
9.4 Gaussian Processes . . . 59

10 Trees and Ensembles **63**
10.1 Regression Trees . . . 63
10.2 Pruning . . . 64
10.3 Classification Trees . . . 65
10.4 Trees Versus Linear Models . . . 66
10.5 The Bootstrap . . . 66
10.6 Bagging . . . 67
10.7 Random Forests . . . 68
10.8 Boosting . . . 69

11 Neural Networks **71**
11.1 The Limits of Linear Classifiers . . . 71
11.2 Multilayer Perceptrons . . . 71
11.3 Expressive Power and Feature Learning . . . 72
11.4 Backpropagation . . . 73
11.5 Training Neural Networks . . . 75

III Probabilistic Models and Inference **77**

12 Graphical Models **78**
12.1 Joint Distributions and Conditional Independence . . . 78
12.2 Directed Graphical Models . . . 78
12.3 Reading Independences from a DAG: d-Separation . . . 79
12.4 Markov Random Fields . . . 81
12.5 The Ising Model and Gaussian Graphical Models . . . 82
12.6 Models for Sequences . . . 83

13 Exact Inference and Message Passing **85**
13.1 Inference Queries . . . 85
13.2 Variable Elimination . . . 86
13.3 Belief Propagation on Trees . . . 88
13.4 Loopy Belief Propagation . . . 89

14 Monte Carlo and Markov Chain Monte Carlo **92**
14.1 Sampling for Approximate Inference . . . 92
14.2 Monte Carlo Estimation and Ancestral Sampling . . . 93
14.3 Importance Sampling . . . 94
14.4 Rejection Sampling . . . 95
14.5 Markov Chains . . . 96
14.6 The Metropolis–Hastings Algorithm . . . 97
14.7 Gibbs Sampling . . . 98
14.8 Hamiltonian Monte Carlo . . . 99
14.9 MCMC in Practice: Diagnostics . . . 100

15 Variational Inference **102**
15.1 Posterior Inference in Latent Variable Models . . . 102
15.2 The Kullback–Leibler Divergence . . . 103
15.3 Mean-Field Approximation . . . 104
15.4 The Evidence Lower Bound . . . 106
15.5 Stochastic Gradients and the Reparameterization Trick . . . 107
15.6 Variational Inference versus MCMC . . . 108

16 Latent Variable Models: Mixtures, EM, PCA and VAEs **110**
16.1 Gaussian Mixture Models . . . 110
16.2 The EM Algorithm . . . 111
16.3 K-Means as a Limit of EM . . . 113
16.4 Principal Component Analysis . . . 114

16.5 Probabilistic PCA . . . 114
16.6 Autoencoders . . . 116
16.7 Variational Autoencoders . . . 117

17 Embeddings and Attention **120**
17.1 Representing Text . . . 120
17.2 Word Embeddings . . . 121
17.3 Contrastive Learning . . . 123
17.4 Attention . . . 125
17.5 The Transformer . . . 126

18 Diffusion Models **130**
18.1 The Forward Process . . . 130
18.2 The Reverse Process . . . 131
18.3 Training by the Variational Bound . . . 131
18.4 The DDPM Algorithms . . . 133
18.5 Diffusion Models and Variational Autoencoders . . . 134
18.6 Conditional Generation and Guidance . . . 134
18.7 Trade-offs in Generative Modeling . . . 136

Overview

These notes merge two courses into one story. STA 314 (Fall 2024) is about *prediction*: given examples (x_i, y_i), build a function that predicts y from x well on new data, and choose among competing methods honestly. STA 414 (Winter 2025) is about *probabilistic modeling*: write down a joint distribution over everything that is observed and unobserved, and compute with it. The two views are not separate subjects. Least squares is maximum likelihood under Gaussian noise, ridge regression is a posterior mode, a Gaussian process is Bayesian kernel regression, and the variational autoencoder trains a neural network with the same evidence lower bound that justifies the EM algorithm. The notes are therefore organized by ideas rather than by the order of the lectures.

The Roadmap

Part I sets up the common language. Chapter 1 defines the prediction problem, the risk of a predictor, the Bayes-optimal rule, and the bias–variance trade-off that governs every method that follows. Chapter 2 introduces likelihood, Bayesian inference, exponential families, and the Gaussian identities used throughout. Chapter 3 covers gradient descent and its stochastic version, the computational engine of almost every fitting procedure in the notes.

Prediction track (Part II). Linear regression is the reference method: it is simple, interpretable, and fitted in closed form, and its test error can be estimated by cross-validation or information criteria (Chapter 4). When there are many predictors, its variance is too high, and shrinkage trades a little bias for a large reduction in variance; the same estimators reappear as posterior modes in Bayesian linear regression (Chapter 5). Relaxing linearity in x while keeping linearity in the parameters gives splines and local regression (Chapter 6). Classification replaces squared error by the 0–1 loss: logistic regression models $\mathbb{P}(Y = k \mid X = x)$ directly, while discriminant analysis models the joint distribution of (X, Y) (Chapter 7). Support vector machines drop probabilities altogether and maximize a margin (Chapter 8). The kernel trick turns every method that depends on the data only through inner products into a non-linear one, and its Bayesian version is the Gaussian process (Chapter 9). Trees learn a partition of the input space adaptively, and averaging or boosting many trees gives some of the most accurate off-the-shelf predictors (Chapter 10). Neural networks, finally, learn the features themselves (Chapter 11).

Probabilistic track (Part III). A joint distribution over many variables is too large to write down, so graphical models describe it by local factors and read conditional independences off a graph (Chapter 12). On trees, marginals can be computed exactly by passing messages (Chapter 13). In general they cannot, and two families of approximations take over: sampling, in particular Markov chain Monte Carlo (Chapter 14), and optimization over a family of simpler distributions, variational inference (Chapter 15). Latent variable models combine both tracks: mixtures and probabilistic PCA are fitted by EM, and variational autoencoders use neural networks for the encoder and the decoder (Chapter 16). The last two chapters describe the components of modern generative models: learned embeddings and attention

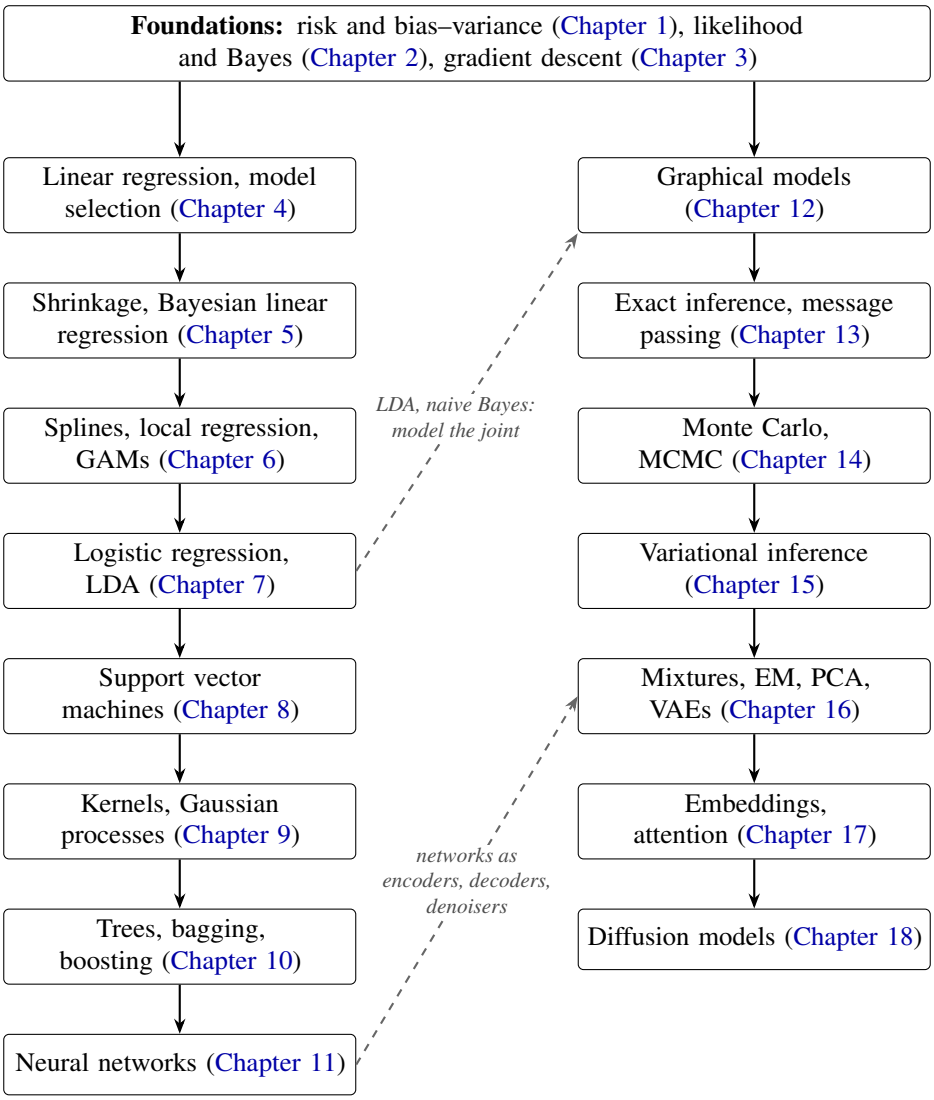


Figure 1. *Two tracks through the notes. Solid arrows give the reading order within each track; dashed arrows mark the main bridges between them.*

(Chapter 17), and diffusion models, which can be read as very deep variational autoencoders with a fixed encoder (Chapter 18).

Method	Problem it solves	Difficulty it leaves
k-nearest neighbors	prediction with no model for f	curse of dimensionality
OLS	interpretable linear fit in closed form	high variance with many predictors
Ridge, lasso	variance reduction, variable selection	tuning λ; still linear in x
Splines, GAMs	smooth non-linear effects	interactions must be added by hand
Logistic regression, LDA	probabilistic classification	linear boundaries
SVM	large-margin classification	no probabilities; linear unless kernelized
Kernels, GPs	non-linear fits, calibrated uncertainty (GP)	$n \times n$ Gram matrix; features fixed in advance
Trees and ensembles	adaptive, non-linear, interactions	ensembles lose interpretability
Neural networks	learned features	non-convex training, many choices
Graphical models	compact joint distributions	inference is hard in general
Variable elimination, BP	exact inference on trees	exponential cost with large cliques
MCMC	asymptotically exact posterior samples	slow mixing, diagnostics
Variational inference	fast approximate posteriors	bias from the variational family
EM, PPCA, VAEs	learning with latent variables	local optima; blurry VAE samples
Attention, transformers	context-dependent representations	quadratic cost in sequence length
Diffusion models	high-quality generation	slow sampling

How to Read These Notes

Each chapter opens with the problem it addresses and closes with a box describing what is still missing, which motivates the next chapter. Definitions and results are stated precisely, but the emphasis is on what the methods do and why, not on proofs; a short derivation is given only when it shows where an algorithm comes from. Every algorithm is written in the same boxed format, with inputs, numbered steps, and outputs, and the main methods end with a box of pros and cons. Remarks titled *Connection* link a topic to related material elsewhere in the notes; following them is the fastest way to see the two courses as one subject.

Where the lecture slides contain typos or imprecise statements, these notes give the corrected version and, where it matters, point out the difference in a remark.

Correspondence with the Lectures

Chapter	STA 314 (Fall 2024)	STA 414 (Winter 2025)
1 The learning problem	Lectures 1, 5 (k-NN), 6	Week 1; Week 4 (decision theory)
2 Probabilistic models	Tutorial 6	Weeks 1, 2 (Gaussians), 10
3 Optimization	Lecture 8	Week 6
4 Linear regression, model selection	Lectures 2, 3; Tutorials 1, 2	–
5 Shrinkage, Bayesian regression	Lecture 4; Tutorial 3	Week 10
6 Beyond linearity	Lecture 5	–
7 Classification	Lectures 6, 7, 9; Tutorials 5, 7	–
8 Support vector machines	Lecture 10; Tutorial 8	–
9 Kernels, Gaussian processes	Tutorial 8	Week 12 (first part)
10 Trees and ensembles	Lectures 11, 12	–
11 Neural networks	–	Week 6
12 Graphical models	–	Week 2; Week 5 (Markov chains); Tutorial 2
13 Exact inference	–	Weeks 3, 4; Tutorial 3
14 Monte Carlo, MCMC	–	Weeks 3, 5; TrueSkill notes
15 Variational inference	–	Weeks 6, 9
16 Latent variable models	–	Weeks 9, 10, 13
17 Embeddings, attention	–	Week 11 (CSC 412 slides)
18 Diffusion models	–	Week 12 (second part), Week 13

Notation

Symbol	Meaning
$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$	training data; $x_i \in \mathbb{R}^p$, responses y_i
$X,\ y$	design matrix (with a leading column of ones when there is an intercept), response vector
$f,\ \hat{f}$	true regression function in $Y = f(X) + \varepsilon$, and its estimate
$\beta,\ \hat{\beta}$	coefficients of a linear model and their estimates
$\phi(x),\ k(x, x'),\ K$	feature map, kernel, Gram matrix
$L(y, \hat{y}),\ \mathrm{Err}(\hat{f})$	loss function, expected test error
$p_k(x) = \mathbb{P}(Y = k \mid X = x)$	posterior class probability; π_k class prior or mixture weight
$\theta,\ \eta$	generic parameters, learning rate
$W^{(\ell)},\ b^{(\ell)}$	weights and biases of layer ℓ of a neural network
$x,\ z$	observed and latent variables in a probabilistic model
q_ϕ	variational distribution or encoder
ELBO	evidence lower bound
$D_{\mathrm{KL}}(q \,\|\, p)$	Kullback–Leibler divergence
$\psi_C,\ Z$	clique potential, partition function
$m_{j \to i}$	message from node j to node i
$p = \tilde{p}/Z,\ q,\ T$	target density, proposal, Markov transition kernel
$f \sim \mathcal{GP}(m, k)$	Gaussian process with mean m and covariance k
$\beta_t,\ \bar{\alpha}_t$	noise schedule of a diffusion model (Chapter 18; unrelated to regression coefficients)
$A \perp\!\!\!\perp B \mid C$	A and B are conditionally independent given C

Part I

Foundations

Chapter 1

The Learning Problem

Every method in these notes answers the same question: given examples of inputs and outputs, how do we predict the output for a new input, and how good can such a prediction be? This chapter sets up that question precisely. Section 1.1 introduces the supervised learning setting and the probabilistic viewpoint that STA 414 takes from the start. Section 1.2 contrasts parametric and non-parametric estimators and presents k-nearest neighbors, the simplest non-parametric method. Section 1.3 asks what the best possible predictor is when the data distribution is known; the answer, the conditional mean for regression and the Bayes classifier for classification, is the target that every later method tries to estimate. Sections 1.4 and 1.5 explain why the error on the training data is a poor guide to performance and decompose the test error into bias, variance, and irreducible noise.

1.1 ■ Supervised Learning

Machine learning programs an algorithm to learn a task from data instead of coding the solution by hand. This is useful when a hand-written solution is hard to find (recognizing objects or speech), when the system must adapt to a changing environment (spam filtering), or when we want it to outperform its programmers. Three types of problems are usually distinguished.

(i) In *supervised learning* we observe pairs (x_i, y_i) of inputs and outputs and learn the mapping from x to y.

(ii) In *unsupervised learning* we observe only inputs x_i and look for structure: groups of similar observations, directions of largest variation, a model of how the data were generated.

(iii) In *reinforcement learning* an agent collects its own data by interacting with an environment and learns to maximize a reward. It is the subject of the FINM 33165 notes and does not appear here.

Statistical learning and machine learning share their models, algorithms, and mathematical tools. The difference is one of emphasis: statistics stresses models, interpretability, and the uncertainty of conclusions, while machine learning stresses algorithms, scalability, and predictive accuracy. STA 314 leans toward the first view and STA 414 toward the second, and these notes use both.

Definition 1.1 (Supervised Learning Problem). *The* response *(output, target)* Y *and the* predictors *(inputs, features, covariates)* $X = (X_1, \dots, X_p)$ *are random with a joint distribution* $p(x, y)$*. The* training data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ *consist of* n *i.i.d. draws from this distribution. When* Y *is quantitative the problem is called* regression*; when* Y *takes values in a finite unordered set* $\mathcal{C}$ *it is called* classification*. We write*

$$Y = f(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0,$$

where $f(x) = \mathbb{E}[Y \mid X = x]$ *is the* regression function *and* ε *collects measurement error and everything that* X *does not explain. A* learning method *maps the training data to an estimate* $\hat{f}$*, used to predict* $\hat{y} = \hat{f}(x_0)$ *at a new input* x_0. □

With the training data in hand, the tasks of supervised learning are *prediction* (accurately predict future responses), *estimation* (understand how each feature affects the response), *model selection* (choose among candidate models or feature sets), and *inference* (quantify the uncertainty of all of the above). In the advertising example of the course, sales are regressed on TV, radio, and newspaper budgets; a good $\hat{f}$ predicts the sales of a new product, while the estimated coefficient of TV tells us how sales change with one more unit of TV budget.

Remark 1.2 (The Probabilistic Viewpoint). STA 414 observes that all these problems are special cases of estimating a distribution from data. Supervised learning estimates $p(y \mid x)$, unsupervised learning estimates $p(x)$, and if we had the joint $p(x, y)$ we could answer both, since

$$p(y \mid x) = \frac{p(x, y)}{\int p(x, y)\, \mathrm{d}y}, \qquad p(c \mid x) = \frac{p(x, c)}{\sum_{c'} p(x, c')}$$

for a continuous response y and a class label c. Methods that model $p(y \mid x)$ directly are called *discriminative*; methods that model the joint $p(x, y)$ are called *generative*. Logistic regression and linear discriminant analysis (Sections 7.2 and 7.5.1) are the standard pair. Chapter 2 develops the tools for fitting such distributions. □

1.2 ▪ Parametric and Non-Parametric Methods

There are two broad strategies for estimating f.

A *parametric* method assumes that f belongs to a family indexed by finitely many parameters. The most important example is the linear model

$$f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p,$$

which reduces the estimation of a function to the estimation of $p + 1$ numbers. Computing the estimates $\hat{\beta}_0, \ldots, \hat{\beta}_p$ from the data is called *fitting* the model, and the same model can be fitted by different procedures (least squares in Section 4.1, penalized least squares in Chapter 5). If a linear model fits poorly, one can enlarge the family: quadratic or polynomial terms, step functions such as $\beta_0 + \beta_1 \mathbf{1}\{X \leq 0.5\} + \beta_2 \mathbf{1}\{X > 0.5\}$ (Chapter 6), or a two-layer neural network $W^{(2)} \sigma(W^{(1)} X + b^{(1)}) + b^{(2)}$ (Chapter 11). More flexible families fit more shapes of f but are harder to interpret: the effect of each feature can be read off a linear model, but not off a neural network. This trade-off between flexibility and interpretability recurs throughout Part II.

A *non-parametric* method makes few assumptions about the form of f and lets the data determine it locally. The prototype predicts at x_0 by averaging the responses of nearby training points.

1.2.1 ▪ k-Nearest Neighbors

Algorithm 1.1 k-Nearest Neighbors

Input: Training data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, number of neighbors $k \in \{1, \dots, n\}$, a distance on $\mathbb{R}^p$ (usually Euclidean), query point x_0.

1: Compute the distances $d_i = \|x_i - x_0\|$ for $i = 1, \dots, n$.
2: Let $N_k(x_0)$ be the indices of the k smallest distances, breaking ties arbitrarily.
3: **if** regression **then**
4: $\quad \hat{f}(x_0) \leftarrow \frac{1}{k} \sum_{i \in N_k(x_0)} y_i$ ▷ local average
5: **else**
6: $\quad \hat{p}_j(x_0) \leftarrow \frac{1}{k} \sum_{i \in N_k(x_0)} \mathbf{1}\{y_i = j\}$ for every class j
7: $\quad \hat{f}(x_0) \leftarrow \arg\max_j \hat{p}_j(x_0)$ ▷ majority vote
8: **end if**

Output: The prediction $\hat{f}(x_0)$; for classification also the class proportions $\hat{p}_j(x_0)$.

There is no training step: the method stores the data and does all its work at prediction time, at a cost of $O(np)$ per query. The number of neighbors k controls the flexibility of the fit. With $k = 1$ the estimate interpolates the training data and is a jagged step function that follows every noisy observation; as k grows the estimate becomes smoother, and with $k = n$ it is the constant $\bar{y}$. Roughly, the fit behaves like a model with n/k parameters. The right k is chosen by estimating the test error, for example by cross-validation (Section 4.5).

Remark 1.3 (Curse of Dimensionality). Local averaging works well when p is small (the lectures suggest at most three or four predictors) and n is large, and it degrades quickly as p grows. In high dimensions the k nearest neighbors are no longer near. For points spread uniformly over the unit cube $[0, 1]^p$, a sub-cube that contains a fraction r of the data has edge length $r^{1/p}$; for $p = 10$ and $r = 0.01$ this is 0.63, so a "neighborhood" with only 1% of the data spans most of the range of every coordinate. Parametric assumptions, additive structure (Section 6.4), and learned features (Chapter 11) are the usual ways around this. □

Pros and Cons: k-NN assumes almost nothing about f, is trivial to implement, and with a suitable k approaches the best possible predictor as $n \to \infty$ in low dimensions. However, it gives no interpretable summary of how X affects Y, prediction is slow for large n, all features enter the distance with equal weight, and performance collapses in high dimensions. Weighted k-NN and local regression (Section 6.3) refine the local average.

1.3 ▪ Loss, Risk, and the Best Possible Predictor

To compare predictors we need a numerical criterion. A *loss function* $L(y, \hat{y}) \geq 0$ measures the cost of predicting $\hat{y}$ when the truth is y. The quality of a predictor g is its *risk*, the expected loss on a new observation,

$$R(g) = \mathbb{E}\big[L(Y, g(X))\big] = \iint L(y, g(x))\, p(x, y)\, \mathrm{d}x\, \mathrm{d}y.$$

In this section the distribution $p(x, y)$ is treated as known, and we ask which g minimizes the risk. This is *decision theory*. The answer is a benchmark: no learning method can do better, and every method can

be viewed as an attempt to approximate it from data.

1.3.1 ▪ Regression

Proposition 1.4 (Best Regression Function). *Under the squared loss $L(y,\hat{y}) = (y-\hat{y})^2$, the risk of any g decomposes as*

$$\mathbb{E}\big[(Y - g(X))^2\big] = \mathbb{E}\big[(g(X) - \mathbb{E}[Y \mid X])^2\big] + \mathbb{E}\big[\operatorname{Var}(Y \mid X)\big].$$

The risk is therefore minimized by the regression function $f(x) = \mathbb{E}[Y \mid X = x]$, and the minimum $\mathbb{E}[\operatorname{Var}(Y \mid X)] = \operatorname{Var}(\varepsilon)$ is the irreducible error.

The decomposition follows by adding and subtracting $\mathbb{E}[Y \mid X]$ inside the square: the cross term vanishes because $\mathbb{E}\big[\mathbb{E}[Y \mid X] - Y \mid X\big] = 0$. The second term does not depend on g and the first is zero exactly when $g = f$. Other losses lead to other summaries of the conditional distribution; for example, the absolute loss $|y - \hat{y}|$ is minimized by the conditional median. The choice of loss should follow the application.

1.3.2 ▪ Classification

For classification with classes $\mathcal{C} = \{1, \dots, K\}$, the natural loss is the 0–1 loss $L(y,\hat{y}) = \mathbf{1}\{y \neq \hat{y}\}$, whose risk is the probability of a mistake, the *expected error rate* $\mathbb{E}[\mathbf{1}\{Y \neq g(X)\}]$. Write $p_k(x) = \mathbb{P}(Y = k \mid X = x)$.

Definition 1.5 (Bayes Classifier). *The* Bayes classifier *assigns each x to its most probable class,*

$$f^*(x) = \arg\max_{k \in \mathcal{C}} p_k(x).$$

It minimizes the expected error rate among all classifiers. Its error rate, the Bayes error rate*, is $\mathbb{E}\big[1 - \max_k p_k(X)\big]$; conditionally on $X = x$ it equals $1 - \max_k p_k(x)$.* □

The proof is one line: for fixed x, predicting class k is wrong with probability $1 - p_k(x)$, which is smallest for the largest $p_k(x)$. The Bayes error is typically positive, because classes overlap: two patients with identical X-ray images can differ in whether they have cancer. In the binary case $\mathcal{C} = \{0, 1\}$ the Bayes classifier predicts 1 if and only if $p_1(x) = \mathbb{P}(Y = 1 \mid X = x) \geq 0.5$, so *learning a classifier amounts to estimating the conditional probability* $p_1(x)$. Since $p_1(x) = \mathbb{E}[Y \mid X = x]$ for a $0/1$ response, this is again a regression problem, but one whose answer must lie in $[0, 1]$; Chapter 7 explains why ordinary least squares is not the right tool for it.

Equivalently, a classifier divides the input space into *decision regions* $\mathcal{R}_1, \dots, \mathcal{R}_K$, with $x \in \mathcal{R}_j$ assigned to class j. The probability of a mistake is $\sum_j \int_{\mathcal{R}_j} \sum_{k \neq j} p(x,k)\,\mathrm{d}x$, and it is minimized pointwise by putting x in the region of the class with the largest joint density $p(x,k)$, which is the same as the largest $p_k(x)$.

1.3.3 ▪ Asymmetric Losses and the Reject Option

Mistakes rarely cost the same. Missing a cancer is worse than a false alarm. Let L_{kj} be the loss incurred when the true class is k and we decide j, collected in a *loss matrix*. The expected loss of deciding j at x is $\sum_k L_{kj}\, p_k(x)$, and the risk is minimized by choosing, at every x, the decision with the smallest expected loss. With $L_{kj} = \mathbf{1}\{k \neq j\}$ this reduces to the Bayes classifier. When the class probabilities are close, it can be better not to decide at all and refer the case to a human; this is the *reject option*.

Algorithm 1.2 Plug-in Bayes Decision Rule

Input: Estimated class probabilities $\hat{p}_1(x), \dots, \hat{p}_K(x)$ at a point x (from any classifier); loss matrix (L_{kj}); optional reject threshold $\tau \in (0, 1)$.

1: $\rho_j(x) \leftarrow \sum_{k=1}^{K} L_{kj}\, \hat{p}_k(x)$ for $j = 1, \dots, K$ ▷ expected loss of deciding j
2: **if** the reject option is used and $\max_k \hat{p}_k(x) < \tau$ **then**
3: **return** "reject"
4: **end if**

Output: The decision $\hat{\jmath}(x) = \arg\min_j \rho_j(x)$.

The algorithm separates two tasks that are often confused. *Inference* estimates the probabilities $p_k(x)$; *decision* turns them into an action using the loss. The same fitted model supports different decisions when the costs change, which is why it pays to estimate probabilities rather than only labels. In the binary case, an asymmetric loss simply moves the threshold on $\hat{p}(x)$ away from 0.5; tracing all thresholds gives the ROC curve of Section 7.3.

Loss	Optimal prediction at x	Minimum risk
Squared $(y-\hat{y})^2$	conditional mean $\mathbb{E}[Y \mid X = x]$	$\mathbb{E}[\mathrm{Var}(Y \mid X)]$, the irreducible error
Absolute $\lvert y-\hat{y}\rvert$	conditional median	expected absolute deviation from the median
0–1 $\mathbf{1}\{y \neq \hat{y}\}$	most probable class $\arg\max_k p_k(x)$	Bayes error rate $\mathbb{E}[1 - \max_k p_k(X)]$
Loss matrix L_{kj}	$\arg\min_j \sum_k L_{kj} p_k(x)$	$\mathbb{E}[\min_j \sum_k L_{kj} p_k(X)]$

1.4 ▪ Training Error, Test Error, and Overfitting

In practice $p(x, y)$ is unknown and $\hat{f}$ is computed from the training data, so the risk must be estimated. For regression the target is the *expected test error*

$$\mathrm{Err}(\hat{f}) = \mathbb{E}\big[(Y_* - \hat{f}(X_*))^2\big],$$

where (X_*, Y_*) is a new pair independent of $\mathcal{D}$, and the expectation is over both the new pair and the training data that produced $\hat{f}$. The obvious estimate is the *training error*

$$\overline{\mathrm{err}} = \mathrm{MSE}_{\text{train}}(\hat{f}) = \frac{1}{n}\sum_{i=1}^{n} \big(y_i - \hat{f}(x_i)\big)^2.$$

It is not a valid measure of quality. Most methods choose $\hat{f}$ precisely by making the training error small over a model class, so a richer class always achieves a smaller training error, and the extreme choice $\hat{f}(x_i) = y_i$ achieves zero. Such a fit follows the noise ε rather than the signal f, which is called *overfitting*. A fit that is too rigid to capture f, such as a plane fitted to a curved income surface, *underfits*.

The honest alternative is the *test error* on data $\{(x_{Ti}, y_{Ti})\}_{i=1}^{m}$ that were not used to fit $\hat{f}$,

$$\mathrm{MSE}_{\text{test}}(\hat{f}) = \frac{1}{m}\sum_{i=1}^{m} \big(y_{Ti} - \hat{f}(x_{Ti})\big)^2,$$

and for classification the analogous training and test error rates $\frac{1}{n}\sum_i \mathbf{1}\{y_i \neq \hat{f}(x_i)\}$ and $\frac{1}{m}\sum_i \mathbf{1}\{y_{Ti} \neq \hat{f}(x_{Ti})\}$. As flexibility increases, the training error decreases monotonically, while the test error first

decreases and then increases: a U-shape. When no test set is available, the test error is estimated by holding out part of the data or by cross-validation (Section 4.5), or by correcting the training error for the complexity of the model (Section 4.6). The best model is the one with the smallest expected test error, and among models with similar test error we prefer the simpler one.

1.5 ▪ The Bias–Variance Trade-off

Why is the test error U-shaped? The answer is a decomposition of the expected test error at a point.

Theorem 1.6 (Bias–Variance Decomposition). *Let $Y_* = f(x_*) + \varepsilon_*$ with $\mathbb{E}[\varepsilon_*] = 0$, where ε_* is independent of the training data, and let $\hat{f}$ be any estimator computed from the training data. Then*

$$\mathbb{E}\big[(Y_* - \hat{f}(x_*))^2\big] = \underbrace{\operatorname{Var}\big(\hat{f}(x_*)\big)}_{\text{variance}} + \underbrace{\big(\mathbb{E}[\hat{f}(x_*)] - f(x_*)\big)^2}_{\text{squared bias}} + \underbrace{\operatorname{Var}(\varepsilon_*)}_{\text{irreducible error}},$$

where the expectations are over the training data and ε_*. In particular, the expected test error is at least $\operatorname{Var}(\varepsilon_*)$.

The identity comes from writing $Y_* - \hat{f}(x_*) = \varepsilon_* + \big(f(x_*) - \mathbb{E}\hat{f}(x_*)\big) + \big(\mathbb{E}\hat{f}(x_*) - \hat{f}(x_*)\big)$. The three terms are uncorrelated, since ε_* is independent of the training data and the last term has mean zero, so the expected square is the sum of the three expected squares. Averaging over x_* gives the same decomposition for $\operatorname{Err}(\hat{f})$.

The *variance* measures how much $\hat{f}$ would change if it were estimated from a different training set. The *bias* is the error introduced by approximating a complicated f with a simpler model; for example, fitting a line when f is a cubic polynomial produces a bias that no amount of data removes. As the flexibility of the method increases, the bias typically decreases and the variance increases. The variance also depends on the sample size, roughly in proportion to (complexity of $\hat{f}$)$/n$: with little data a flexible method is dominated by variance, and with much data its smaller bias wins. Figure 1.1 shows the resulting trade-off.

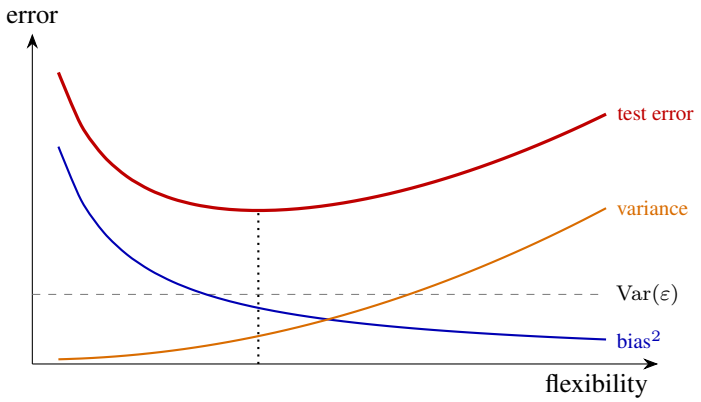


Figure 1.1. *Schematic bias–variance trade-off. The expected test error is the sum of the squared bias, the variance, and the irreducible error; it is smallest at an intermediate flexibility (dotted line). The training error, not shown, keeps decreasing and eventually falls below* $\operatorname{Var}(\varepsilon)$.

Remark 1.7 (Bias and Variance of k-NN). Suppose the inputs $x_1, \dots, x_n$ are fixed and $\mathrm{Var}(\varepsilon) = \sigma^2$. The k-NN estimate at x_0 averages k independent responses, so

$$\mathbb{E}\big[(Y_* - \hat{f}_k(x_0))^2\big] = \sigma^2 + \Big(f(x_0) - \frac{1}{k}\sum_{i \in N_k(x_0)} f(x_i)\Big)^2 + \frac{\sigma^2}{k}.$$

Increasing k shrinks the variance σ^2/k but averages f over a wider neighborhood, which increases the bias wherever f is not flat. The choice of k is exactly a bias–variance trade-off. □

Remark 1.8 (The Trade-off Organizes Part II). Much of Part II consists of ways to move along the bias–variance curve deliberately. Subset selection and shrinkage (Sections 4.7 and 5.1) accept some bias for a large reduction in variance. Splines and local regression (Chapter 6) have a smoothing parameter that plays the role of k. Bagging and random forests (Algorithms 10.4 and 10.5) reduce the variance of flexible trees by averaging, while boosting (Algorithm 10.6) reduces the bias of simple trees by adding them up. Bayesian averaging over parameters (Section 5.6) is yet another way to control variance. □

For classification there is no equally clean additive decomposition of the 0–1 loss, but the qualitative picture is the same: rigid classifiers have decision boundaries that cannot follow the Bayes boundary, flexible ones have boundaries that change with every training set, and the test error rate is U-shaped in flexibility.

Limitations and what comes next: The best predictors of this chapter are functionals of the unknown distribution $p(x, y)$, and the bias–variance trade-off tells us that estimating them well requires choosing a model family of the right complexity. Two ingredients are still missing. We need principles for fitting a model family to data, and for describing the uncertainty of the fit: maximum likelihood and Bayesian inference, developed in Chapter 2. And because fitting means solving an optimization problem that rarely has a closed-form solution, we need general algorithms for minimizing a training objective, developed in Chapter 3.

Chapter 2

Probabilistic Models and Estimation

Chapter 1 showed that the best predictor is a summary of the conditional distribution of the response given the inputs. To learn it we need two things: families of distributions that are flexible enough to describe the data yet simple enough to estimate, and principles for choosing a member of the family given data. This chapter supplies both. Section 2.1 sets up the probabilistic viewpoint of STA 414, in which a model is a joint distribution over observed and unobserved variables. Section 2.2 introduces maximum likelihood, the estimation principle behind least squares, logistic regression, and most of the models in these notes. Section 2.3 describes exponential families, the class of distributions for which maximum likelihood is especially well behaved. Section 2.4 treats parameters as random and introduces priors, posteriors, MAP estimation, and predictive distributions. Section 2.5 collects the facts about the multivariate Gaussian that are used again and again in later chapters.

2.1 ▪ Probabilistic Models

Consider a random vector $X = (X_1, \ldots, X_d)$ whose components may be observed or only partially observed. A *probabilistic (generative) model* relates all the variables through their joint distribution $p(x) = p(x_1, \ldots, x_d)$. Once the joint is available, every question about the variables is answered by marginalizing and conditioning. For a supervised problem with inputs x and a label c, the classifier of Definition 1.5 needs $p(c \mid x) = p(x, c) / \sum_{c'} p(x, c')$, and a regression needs $p(y \mid x) = p(x, y) / \int p(x, y)\, \mathrm{d}y$.

Remark 2.1 (Bayes' Rule and Base Rates). The STA 414 tutorial illustrates conditioning with a diagnostic test. Let $H = 1$ if a person is infected and $Y = 1$ if the test is positive, with prevalence $p(H = 1) = 0.1$, sensitivity $p(Y = 1 \mid H = 1) = 0.875$, and specificity $p(Y = 0 \mid H = 0) = 0.975$. These three numbers determine the joint distribution of (Y, H), and Bayes' rule gives

$$p(H = 1 \mid Y = 1) = \frac{p(Y = 1 \mid H = 1)\, p(H = 1)}{p(Y = 1)} = \frac{0.875 \cdot 0.1}{0.11} \approx 0.795,$$
$$p(H = 1 \mid Y = 0) = \frac{p(Y = 0 \mid H = 1)\, p(H = 1)}{p(Y = 0)} = \frac{0.125 \cdot 0.1}{0.89} \approx 0.014.$$

With a prevalence of 1% instead, a positive test raises the probability of infection only to about 26%: when the condition is rare, a positive result is more often a false positive than a true one. The example also shows that a distribution can be identified with a few parameters, and that marginalizing and conditioning are operations on those parameters. □

The distinction between observed and unobserved variables organizes much of the course. In supervised classification we observe pairs $(x_i, c_i) \overset{\text{i.i.d.}}{\sim} p(x, c)$. If the data are still generated from $p(x, c)$

but we observe only x_i, the label becomes a *latent variable*, usually called a cluster, and the data have the marginal distribution $p(x) = \sum_c p(x, c)$. Mixture models (Section 16.1) are of this form, and fitting models with latent variables is the main subject of Part III.

To make learning feasible we make two modeling assumptions. First, the observations $x_1, \dots, x_n$ are i.i.d. draws from an unknown true distribution p^*. Second, we search for an approximation of p^* in a *parametric family* $\mathcal{F} = \{p(x \mid \theta) : \theta \in \Theta\}$, for example the Gaussians $\{\mathcal{N}(\theta, 1) : \theta \in \mathbb{R}\}$. The parameter space Θ may still be very high-dimensional, as it is for a neural network.

2.2 ▪ Maximum Likelihood Estimation

Definition 2.2 (Likelihood and Maximum Likelihood Estimate). *For i.i.d. data $\mathcal{D} = \{x_1, \dots, x_n\}$ and a parametric family $p(x \mid \theta)$, the* likelihood *and* log-likelihood *are*

$$L(\theta; \mathcal{D}) = p(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} p(x_i \mid \theta), \qquad \ell(\theta; \mathcal{D}) = \log L(\theta; \mathcal{D}) = \sum_{i=1}^{n} \log p(x_i \mid \theta).$$

The maximum likelihood estimate *(MLE) is* $\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta \in \Theta} \ell(\theta; \mathcal{D})$. □

For discrete data, $L(\theta; \mathcal{D})$ is the probability of observing exactly $\mathcal{D}$ if it had been generated with parameter θ, so the MLE picks the parameter under which the data are most probable. Maximizing the log-likelihood gives the same answer and is easier, because the product becomes a sum. For a supervised model $p(y \mid x, \theta)$ the inputs are treated as fixed and the conditional log-likelihood $\sum_i \log p(y_i \mid x_i, \theta)$ is maximized instead.

Algorithm 2.1 Maximum Likelihood Estimation

Input: Data $x_1, \dots, x_n$ (or pairs (x_i, y_i)); parametric family $p(x \mid \theta)$, $\theta \in \Theta$.

1: Form $\ell(\theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta)$ and drop the terms that do not depend on θ.
2: Compute the gradient $\nabla_\theta \ell$, adding a Lagrange multiplier for each equality constraint on θ (for example $\sum_k q_k = 1$).
3: **if** $\nabla_\theta \ell(\theta) = 0$ can be solved explicitly **then**
4: $\quad \hat{\theta} \leftarrow$ the solution, after checking that it is a maximum and comparing with the boundary of Θ.
5: **else**
6: $\quad \hat{\theta} \leftarrow$ the result of gradient descent on $-\ell$ (Algorithm 3.1) or its stochastic version (Algorithm 3.2).
7: **end if**

Output: The estimate $\hat{\theta}_{\mathrm{MLE}}$ and the fitted distribution $p(x \mid \hat{\theta}_{\mathrm{MLE}})$.

The examples worked out in the two courses cover all branches of the algorithm.

Model	MLE	Remark
Bernoulli(θ), $x_i \in \{0,1\}$	$\hat\theta = \frac{1}{n}\sum_i x_i$	the fraction of heads
Categorical$(q_1,\dots,q_K)$	$\hat q_k = n_k/n$, with $n_k = \sum_i \mathbf{1}\{x_i = k\}$	needs a Lagrange multiplier for $\sum_k q_k = 1$
$\mathcal{N}(\mu,\sigma^2)$	$\hat\mu = \bar x$, $\hat\sigma^2 = \frac{1}{n}\sum_i (x_i - \bar x)^2$	the variance estimate divides by n, not $n-1$
Uniform$(0,\theta)$	$\hat\theta = \max_i x_i$	$L(\theta) = \theta^{-n}\mathbf{1}\{\theta \ge \max_i x_i\}$ is not differentiable at the maximum
$y_i = x_i^\top\beta + \varepsilon_i$, $\varepsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,\sigma^2)$	$\hat\beta = (X^\top X)^{-1}X^\top y$	ordinary least squares (Section 4.1)
$\mathbb{P}(y_i = 1 \mid x_i) = \frac{1}{1+e^{-x_i^\top\beta}}$	no closed form	solved iteratively (Algorithm 7.1)

Remark 2.3 (Negative Log-Likelihoods Are Losses). Maximum likelihood connects the losses of Chapter 1 with probability models. Under Gaussian noise,

$$-\sum_{i=1}^{n} \log p(y_i \mid x_i, \beta) = \frac{1}{2\sigma^2}\|y - X\beta\|^2 + \text{const},$$

so least squares is maximum likelihood. Under a Bernoulli model with $p_i = \mathbb{P}(y_i = 1 \mid x_i)$, the negative log-likelihood is the cross-entropy $-\sum_i \big(y_i \log p_i + (1-y_i)\log(1-p_i)\big)$, which becomes the logistic loss of Section 7.2. Choosing a noise model is therefore the same as choosing a loss, and the probabilistic view tells us which loss is natural for which type of response. □

Remark 2.4 (MLE Minimizes a KL Divergence). By the law of large numbers, $\frac{1}{n}\ell(\theta;\mathcal{D}) \to \mathbb{E}_{p^*}[\log p(X \mid \theta)] = -D_{\mathrm{KL}}(p^* \,\|\, p_\theta) - \mathcal{H}(p^*)$, where $\mathcal{H}(p^*)$ does not depend on θ. Maximum likelihood therefore looks for the member of the family closest to the truth in the Kullback–Leibler divergence $D_{\mathrm{KL}}(p^* \,\|\, p_\theta)$. This direction of the KL divergence reappears as the "moment projection" in variational inference (Section 15.2). □

2.3 ▪ Sufficient Statistics and Exponential Families

In the Bernoulli example the likelihood depends on the data only through the count $\sum_i x_i$; any two data sets with the same count lead to the same inference about θ.

Definition 2.5 (Sufficient Statistic). *A statistic $T(x)$ is* sufficient *for θ in the model $p(x \mid \theta)$ if it conveys all the information about θ that the full data contain. By the Fisher–Neyman factorization theorem, T is sufficient if and only if $p(x \mid \theta) = h(x)\, g_\theta\big(T(x)\big)$ for some functions h, which does not depend on θ, and g_θ.* □

Definition 2.6 (Exponential Family). *A family of densities (or probability mass functions) is an* exponential family *if it can be written as*

$$p(x \mid \eta) = h(x)\exp\big\{\eta^\top T(x) - A(\eta)\big\},$$

where $T(x)$ is the vector of sufficient statistics, η the natural parameter*, $h(x)$ the base measure, and $A(\eta) = \log \int h(x)\exp\{\eta^\top T(x)\}\,\mathrm{d}x$ the* log-partition function*, which makes the density integrate to one.* □

The parameter interacts with the data only through $\eta^\top T(x)$, so T is sufficient by the factorization theorem. Many familiar distributions are exponential families: Gaussian, Bernoulli, categorical, Poisson, exponential, gamma, beta, and Dirichlet.

Distribution	$T(x)$	η	$A(\eta)$
Bernoulli(θ)	x	$\log \frac{\theta}{1-\theta}$ (log-odds)	$\log(1+e^{\eta})$
Categorical$(q_1,\dots,q_K)$	$\big(\mathbf{1}\{x=k\}\big)_{k=1}^{K-1}$	$\eta_k = \log \frac{q_k}{q_K}$	$\log\big(1+\sum_{k<K} e^{\eta_k}\big)$
$\mathcal{N}(\mu,\sigma^2)$	$(x,\, x^2)$	$\big(\frac{\mu}{\sigma^2},\, -\frac{1}{2\sigma^2}\big)$	$-\frac{\eta_1^2}{4\eta_2} - \frac{1}{2}\log(-2\eta_2)$

Two entries of the table reappear later under other names. Inverting the Bernoulli natural parameter gives $\theta = 1/(1+e^{-\eta})$, the logistic (sigmoid) function; logistic regression (Section 7.2) is the model in which the log-odds η is linear in x. Inverting the categorical one gives $q_k = e^{\eta_k}/(1+\sum_{j<K} e^{\eta_j})$, the softmax function with $\eta_K = 0$ as reference, which is the link of multinomial logistic regression (Section 7.4). The categorical family must be written with $K-1$ free parameters as in the table. With all K parameters $\eta_k = \log q_k$ the components of η are tied by $\sum_k e^{\eta_k} = 1$, and the moment formulas below do not apply.

Proposition 2.7 (Moments from the Log-Partition Function). *For an exponential family,* $\nabla A(\eta) = \mathbb{E}_\eta[T(X)]$ *and* $\nabla^2 A(\eta) = \mathrm{Cov}_\eta\big(T(X)\big)$. *In particular, A is convex.*

The first identity follows by differentiating $\int h(x)\exp\{\eta^\top T(x) - A(\eta)\}\,\mathrm{d}x = 1$ with respect to η. For the Bernoulli family, $A'(\eta) = e^\eta/(1+e^\eta) = \theta$ is the mean and $A''(\eta) = \theta(1-\theta)$ the variance.

Proposition 2.8 (MLE in an Exponential Family). *For i.i.d. data the log-likelihood is* $\ell(\eta) = \sum_i \log h(x_i) + \eta^\top \sum_i T(x_i) - nA(\eta)$, *a concave function of η. The MLE solves the* moment-matching *equation*

$$\nabla A(\hat\eta_{\mathrm{MLE}}) = \mathbb{E}_{\hat\eta}[T(X)] = \frac{1}{n}\sum_{i=1}^{n} T(x_i),$$

and any solution is a global maximum.

The fitted model reproduces the empirical averages of the sufficient statistics. The equation may have no explicit solution, but because ℓ is concave, gradient ascent cannot get stuck in a spurious local maximum (Section 3.3).

Remark 2.9 (Where Exponential Families Reappear). Exponential families are the common thread of several later topics: generalized linear models such as logistic regression (Section 7.2); Markov random fields and the Ising model, whose clique potentials are exponentials of linear functions of sufficient statistics (Sections 12.4 and 12.5); Gaussian graphical models (Section 2.5); and the closed-form coordinate updates of mean-field variational inference (Section 15.3). □

2.4 ▪ Bayesian Inference

Maximum likelihood returns a single parameter value and ignores how uncertain it is. The Bayesian approach treats θ as a random variable with a *prior* distribution $p(\theta)$, which encodes what is known before seeing the data, and updates it by Bayes' rule to the *posterior*

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D}\mid\theta)\,p(\theta)}{p(\mathcal{D})} \propto p(\theta)\prod_{i=1}^{n} p(x_i\mid\theta), \qquad p(\mathcal{D}) = \int p(\mathcal{D}\mid\theta)\,p(\theta)\,\mathrm{d}\theta.$$

The normalizing constant $p(\mathcal{D})$ is the *marginal likelihood* or *evidence*. The posterior can be used in two ways.

Definition 2.10 (MAP Estimate and Posterior Predictive). *The* maximum a posteriori *(MAP) estimate is the posterior mode,*

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_{\theta} \big[\log p(\mathcal{D} \mid \theta) + \log p(\theta)\big].$$

The posterior predictive distribution *of a new observation averages the model over the posterior,*

$$p(x_{\mathrm{new}} \mid \mathcal{D}) = \int p(x_{\mathrm{new}} \mid \theta)\, p(\theta \mid \mathcal{D})\, \mathrm{d}\theta.$$

□

The MAP estimate is maximum likelihood with an additional term $\log p(\theta)$, which acts as a penalty that pulls the estimate toward values the prior considers plausible. The normalizer $p(\mathcal{D})$ is not needed to find the mode. Full Bayesian inference instead keeps the whole posterior and makes predictions by averaging over all parameter values that explain the data, weighted by how well they do. This smooths out the predictions and quantifies their uncertainty: the model "knows what it doesn't know".

Remark 2.11 (Worked Example: Beta–Bernoulli). For coin flips $x_i \in \{0, 1\}$ with n_1 heads and n_0 tails, take the prior $\theta \sim \mathrm{Beta}(a, b)$, with density proportional to $\theta^{a-1}(1-\theta)^{b-1}$. Multiplying by the likelihood $\theta^{n_1}(1-\theta)^{n_0}$ shows that the posterior is again a beta distribution,

$$\theta \mid \mathcal{D} \sim \mathrm{Beta}(a + n_1,\, b + n_0).$$

A prior with this property is called *conjugate*. The prior acts like a pseudo-heads and b pseudo-tails. The three estimates are

$$\hat{\theta}_{\mathrm{MLE}} = \frac{n_1}{n}, \qquad \hat{\theta}_{\mathrm{MAP}} = \frac{n_1 + a - 1}{n + a + b - 2}, \qquad p(x_{\mathrm{new}} = 1 \mid \mathcal{D}) = \mathbb{E}[\theta \mid \mathcal{D}] = \frac{n_1 + a}{n + a + b}.$$

With little data the prior matters; after three heads in three flips the MLE predicts heads with certainty, while the posterior predictive does not. As $n \to \infty$ all three agree. □

Approach	What is computed	Typical difficulty
MLE	$\arg\max_\theta \log p(\mathcal{D} \mid \theta)$	an optimization problem; overfits with little data
MAP	$\arg\max_\theta [\log p(\mathcal{D} \mid \theta) + \log p(\theta)]$	an optimization problem; a regularized MLE
Full Bayes	$p(\theta \mid \mathcal{D})$ and $\int p(x \mid \theta) p(\theta \mid \mathcal{D})\, \mathrm{d}\theta$	an integration problem; closed form only in conjugate cases

Remark 2.12 (Connections). The MAP view explains regularization: with a Gaussian prior on regression coefficients, MAP estimation is ridge regression, and with a Laplace prior it is the lasso (Section 5.5). Full Bayesian linear regression has a Gaussian posterior and predictive distribution in closed form (Propositions 5.8 and 5.9), and its kernelized version is the Gaussian process (Section 9.4). Outside conjugate models the posterior integral is intractable, and Part III develops the two general ways to approximate it: sampling with Markov chain Monte Carlo (Chapter 14) and optimization with variational inference (Chapter 15). □

2.5 ▪ The Multivariate Gaussian

The Gaussian distribution is the workhorse of continuous probabilistic models, because sums, marginals, and conditionals of Gaussians are again Gaussian and can be computed with linear algebra.

Definition 2.13 (Multivariate Gaussian). *For $\mu \in \mathbb{R}^m$ and a symmetric positive definite $\Sigma \in \mathbb{R}^{m\times m}$ (that is, $u^\top \Sigma u > 0$ for all $u \neq 0$), $X \sim \mathcal{N}(\mu, \Sigma)$ if X has density*

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{m/2}(\det \Sigma)^{1/2}} \exp\Big(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big).$$

Then $\mathbb{E}[X] = \mu$ and $\operatorname{Cov}(X) = \Sigma$. The inverse covariance $\Lambda = \Sigma^{-1}$ is called the precision matrix. □

Proposition 2.14 (Marginals and Conditionals). *Partition $X = (X_A, X_B) \sim \mathcal{N}(\mu, \Sigma)$ with $\mu = (\mu_A, \mu_B)$ and $\Sigma = \begin{pmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{pmatrix}$. Then*

$$X_A \sim \mathcal{N}(\mu_A, \Sigma_{AA}),$$
$$X_A \mid X_B = x_B \sim \mathcal{N}\big(\mu_A + \Sigma_{AB}\Sigma_{BB}^{-1}(x_B - \mu_B),\ \Sigma_{AA} - \Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA}\big).$$

Moreover, for any matrix A with m columns, $AX \sim \mathcal{N}(A\mu, A\Sigma A^\top)$.

Marginalizing means deleting rows and columns. Conditioning shifts the mean linearly in the observed value and shrinks the covariance by a term that does not depend on x_B: observing X_B always reduces uncertainty about X_A by the same amount, whatever value is observed. This formula is the whole of Gaussian process regression (Algorithm 9.2).

Independence can be read off the covariance and conditional independence off the precision: $X_i \perp\!\!\!\perp X_j$ if and only if $\Sigma_{ij} = 0$, and $X_i \perp\!\!\!\perp X_j \mid X_{\text{rest}}$ if and only if $\Lambda_{ij} = 0$. Writing the density as $p(x) \propto \prod_s \exp\{-\frac{1}{2}\Lambda_{ss}(x_s-\mu_s)^2\} \prod_{s<t} \exp\{-\Lambda_{st}(x_s-\mu_s)(x_t-\mu_t)\}$ shows that a Gaussian is a Markov random field whose edges are the nonzero entries of the precision matrix, a *Gaussian graphical model* (Section 12.4).

2.5.1 ▪ Completing the Square

The most useful computational fact about Gaussians is that a density whose logarithm is a quadratic function is Gaussian, with mean and covariance that can be read off the quadratic. Indeed, $\log \mathcal{N}(w \mid \mu, \Sigma) = -\frac{1}{2} w^\top \Sigma^{-1} w + w^\top \Sigma^{-1}\mu + \text{const}$.

Algorithm 2.2 Identifying a Gaussian by Completing the Square

Input: An unnormalized log-density $\log p(w)$ known to be a quadratic function of $w \in \mathbb{R}^m$ (for example, log-prior plus log-likelihood in a linear-Gaussian model).

1: Expand $\log p(w)$ and collect the second-order and first-order terms in w, ignoring everything that does not depend on w:
$\log p(w) = -\frac{1}{2} w^\top P w + w^\top b + \text{const}$, with P symmetric.

2: Check that P is positive definite (otherwise p is not a normalizable Gaussian).

Output: $w \sim \mathcal{N}(P^{-1}b,\ P^{-1})$: precision P, covariance P^{-1}, mean $P^{-1}b$.

Applied to a Gaussian prior and a Gaussian likelihood that is linear in the unknown, the recipe gives the following result, which covers Bayesian linear regression, Gaussian processes, and probabilistic PCA.

Proposition 2.15 (Linear-Gaussian Model). *Let $z \sim \mathcal{N}(m, S)$ and $x \mid z \sim \mathcal{N}(Az + b, \Sigma)$. Then*

$$x \sim \mathcal{N}\big(Am + b,\ \Sigma + ASA^\top\big),$$
$$z \mid x \sim \mathcal{N}\big(m', S'\big), \qquad S' = \big(S^{-1} + A^\top \Sigma^{-1} A\big)^{-1}, \quad m' = S'\big(A^\top \Sigma^{-1}(x-b) + S^{-1}m\big).$$

For the posterior, $\log p(z \mid x) = \log p(z) + \log p(x \mid z) + \text{const}$ has quadratic term $-\frac{1}{2} z^\top (S^{-1} + A^\top \Sigma^{-1} A) z$ and linear term $z^\top (S^{-1} m + A^\top \Sigma^{-1}(x - b))$, and Algorithm 2.2 does the rest. The posterior precision is the prior precision plus the precision contributed by the data, and the posterior mean is a precision-weighted combination of the prior mean and the observation. For the marginal, $x = Az + b + \varepsilon$ is a linear function of the independent Gaussians z and $\epsilon \sim \mathcal{N}(0, \Sigma)$.

Remark 2.16 (Where the Gaussian Toolbox Is Used). Linear and quadratic discriminant analysis model each class by a Gaussian (Sections 7.5.1 and 7.5.2). Bayesian linear regression applies Proposition 2.15 with z the coefficients and x the responses (Proposition 5.8). Gaussian process regression conditions a joint Gaussian with Proposition 2.14 (Section 9.4). Mixtures of Gaussians (Section 16.1) and probabilistic PCA (Section 16.5) are latent variable models built from Gaussian pieces, the VAE uses Gaussian encoders and decoders (Section 16.7), and the forward process of a diffusion model is a chain of linear-Gaussian steps whose marginals follow from Proposition 2.15 (Section 18.1). □

Limitations and what comes next: Maximum likelihood and MAP estimation turn learning into optimization, but closed-form solutions such as the sample mean or the least-squares formula are the exception. Logistic regression, neural networks, and most latent variable models require an iterative algorithm, and even when a closed form exists it may be too expensive to compute for large data. Chapter 3 develops gradient descent and its stochastic version, the algorithms used to fit almost every model in these notes. Full Bayesian inference raises a harder problem, the computation of high-dimensional integrals, which is postponed to Part III.

Chapter 3

Optimization

The first two chapters reduced learning to choosing parameters that make an objective small: the training error of Chapter 1, or the negative log-likelihood and negative log-posterior of Chapter 2. Except in special cases, such as least squares, the minimizer has no closed form, and even when it does, computing it may be too expensive. This chapter presents the algorithms used to fit almost every model in these notes. Section 3.1 states the optimization problem and the closed-form solutions that exist for linear regression. Section 3.2 derives gradient descent and discusses step sizes and stopping rules, and Section 3.3 explains when it is guaranteed to find the global minimum. Section 3.4 introduces stochastic gradient descent, which scales to large data sets, and Section 3.5 describes what changes when the objective is not convex, as it is for neural networks.

3.1 ▪ Learning as Optimization

Almost every training procedure in these notes solves a problem of the form

$$\hat{\theta} = \arg\min_{\theta \in \Theta} J(\theta), \qquad J(\theta) = \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, f_\theta(x_i)\big) + \lambda\, \Omega(\theta), \tag{3.1}$$

where f_θ is a model with parameters θ, L is a loss, Ω is an optional penalty with weight $\lambda \geq 0$, and the parameter space Θ is usually all of $\mathbb{R}^p$. Minimizing the average training loss is called *empirical risk minimization*: the risk $R(f) = \mathbb{E}[L(Y, f(X))]$ of Section 1.3 is replaced by its average over the training data. By Remark 2.3, maximum likelihood is the special case in which L is a negative log-likelihood, and MAP estimation adds the penalty $\Omega(\theta) = -\log p(\theta)$. The factor $\frac{1}{n}$ does not change the minimizer but makes the scale of J, and hence a good step size, independent of n.

If J is differentiable, a minimizer in the interior of Θ must be a *critical point*, a point where the gradient vanishes,

$$\nabla J(\theta) = \Big(\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_p} \Big)^\top = 0.$$

The gradient points in the direction of steepest increase of J, and its length is the slope in that direction. Sometimes the equation $\nabla J = 0$ can be solved directly. For least squares, $J(\beta) = \|y - X\beta\|^2$ has gradient $-2X^\top(y - X\beta)$, and setting it to zero gives the *normal equations* and the familiar solution

$$X^\top X \hat{\beta} = X^\top y, \qquad \hat{\beta} = (X^\top X)^{-1} X^\top y.$$

For ridge regression, $J(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2$ gives $(X^\top X + \lambda I)\hat{\beta}_\lambda = X^\top y$ (Section 5.1). For the logistic regression likelihood, in contrast, the equation $\nabla J = 0$ is nonlinear in β and has no explicit solution, and for a neural network it is hopeless. We then turn to iterative methods, which start from an initial guess and improve it step by step until a stopping criterion is met.

3.2 ▪ Gradient Descent

Suppose we are at θ and want a direction d with $\|d\| = 1$ along which J decreases fastest. For a small step size $\eta > 0$, the first-order Taylor expansion gives

$$J(\theta + \eta d) \approx J(\theta) + \eta \, \nabla J(\theta)^\top d,$$

and by the Cauchy–Schwarz inequality the inner product $\nabla J(\theta)^\top d$ is most negative for $d = -\nabla J(\theta)/\|\nabla J(\theta)\|$. The approximation becomes exact as $\eta \to 0$, so a small enough step in the direction of the negative gradient decreases J unless the gradient is zero. Repeating this step is *gradient descent*.

Algorithm 3.1 Gradient Descent (GD)

Input: Differentiable objective $J : \mathbb{R}^p \to \mathbb{R}$; learning rate $\eta > 0$; tolerance $\epsilon > 0$; maximum number of iterations M.
Initialize: $\theta^{(0)}$, for example $\theta^{(0)} = 0$.
for $k = 0, 1, \dots, M-1$ **do**
  $g^{(k)} \leftarrow \nabla J(\theta^{(k)})$
  $\theta^{(k+1)} \leftarrow \theta^{(k)} - \eta \, g^{(k)}$ ▷ coordinatewise: $\theta_j \leftarrow \theta_j - \eta \, \partial J / \partial \theta_j$
  if $|J(\theta^{(k+1)}) - J(\theta^{(k)})| \le \epsilon$ **or** $\|\theta^{(k+1)} - \theta^{(k)}\| \le \epsilon \|\theta^{(k)}\|$ **then**
    break
  end if
end for
Output: The final iterate $\theta^{(k+1)}$, an approximate minimizer of J.

The three stopping rules are that the objective stops changing (for example, by less than 10^{-6}), that the parameters stop changing in absolute or relative terms, or that the iteration budget M (for example, 1000) is exhausted. A small gradient norm $\|\nabla J(\theta^{(k)})\| \le \epsilon$ is an alternative criterion; an exactly zero gradient is never reached in practice. To maximize a log-likelihood, apply the algorithm to $J = -\ell$, or equivalently use *gradient ascent* $\theta \leftarrow \theta + \eta \nabla \ell(\theta)$.

Remark 3.1 (Two Examples: Error Times Input). For least squares, $J(\beta) = \|y - X\beta\|^2$ and the update is

$$\beta^{(k+1)} = \beta^{(k)} + 2\eta \, X^\top \big(y - X\beta^{(k)}\big) = \beta^{(k)} + 2\eta \sum_{i=1}^{n} \big(y_i - x_i^\top \beta^{(k)}\big) x_i.$$

For logistic regression with $y_i \in \{0, 1\}$, the negative log-likelihood is $J(\beta) = \sum_i \big[-y_i x_i^\top \beta + \log(1 + e^{x_i^\top \beta}) \big]$ and the update is

$$\beta^{(k+1)} = \beta^{(k)} + \eta \sum_{i=1}^{n} \big(y_i - \sigma(x_i^\top \beta^{(k)})\big) x_i, \qquad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

Both updates add to β the inputs weighted by the prediction errors. The same "error times input" structure appears in multinomial logistic regression (Section 7.4) and in the error signals of backpropagation (Section 11.4). The full logistic regression procedure is Algorithm 7.1. □

3.2.1 ▪ The Learning Rate

The learning rate (step size) η is a hyperparameter. If it is too small, progress is slow; if it is too large, the iterates overshoot and oscillate across the valley; if it is much too large, they diverge. Good values are

typically small, such as 0.1, 0.01, or 0.0001, and are found by trying a grid such as $0.1, 0.03, 0.01, \dots$. A simple adaptive strategy starts with a large η and halves it whenever a step fails to decrease J. The sequence of step sizes used over the iterations is called the *learning rate schedule*.

The *training curve*, the objective $J(\theta^{(k)})$ plotted against k, is the basic diagnostic for an optimizer: a curve that decreases slowly suggests a larger η, and one that oscillates or increases suggests a smaller one. The training curve only tells us whether the optimization has converged. It says neither whether a global minimum was found nor how well the fitted model predicts new data, which is a question about test error (Section 1.4).

Remark 3.2 (Why Large Steps Diverge). For least squares the effect of η can be computed exactly. Subtracting the normal equations from the update gives

$$\beta^{(k+1)} - \hat{\beta} = \big(I - 2\eta X^\top X\big)\big(\beta^{(k)} - \hat{\beta}\big).$$

The error is multiplied at every step by a matrix with eigenvalues $1 - 2\eta\lambda_j$, where λ_j are the eigenvalues of $X^\top X$. When X has full column rank, gradient descent converges if and only if all of them lie in $(-1, 1)$, that is, $0 < \eta < 1/\lambda_{\max}(X^\top X)$. With a step close to this limit the direction of largest curvature oscillates; with a smaller step the direction of smallest curvature converges slowly, at a rate governed by the ratio $\lambda_{\max}/\lambda_{\min}$. Elongated contours therefore make gradient descent zigzag (Fig. 3.1). □

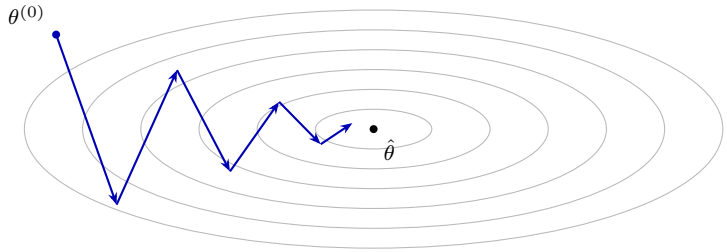


Figure 3.1. *Schematic gradient descent path on the elongated elliptical contours of a quadratic objective. With a step size near the stability limit, the iterates oscillate across the narrow direction while making slow progress along the long one.*

Remark 3.3 (Why Iterate When a Closed Form Exists). Even for least squares, gradient descent can be preferable. Forming $X^\top X$ costs $O(np^2)$ operations and solving the normal equations $O(p^3)$, whereas one gradient step costs $O(np)$, and a stochastic step (Section 3.4) costs $O(p)$. When p is large, a moderate number of cheap steps is much faster than the direct solution. □

3.3 ▪ Convexity

When can we expect gradient descent to find the minimizer and not just some point where it gets stuck? The key property is convexity.

Definition 3.4 (Convex Set and Convex Function). *A set $S \subseteq \mathbb{R}^p$ is* convex *if $(1-t)\theta_0 + t\theta_1 \in S$ for all $\theta_0, \theta_1 \in S$ and $t \in [0,1]$. A function J on a convex set is* convex *if*

$$J\big((1-t)\theta_0 + t\theta_1\big) \le (1-t)J(\theta_0) + tJ(\theta_1) \qquad \text{for all } \theta_0, \theta_1 \text{ and } t \in [0,1],$$

and strictly convex *if the inequality is strict whenever $\theta_0 \neq \theta_1$ and $t \in (0,1)$.* □

Geometrically, a convex function is bowl-shaped: the chord between any two points of its graph lies above the graph, and the region above the graph is a convex set. $\mathbb{R}^p$ itself is convex. In practice convexity is verified with a few rules rather than the definition.

(i) A twice-differentiable function of one variable is convex if $J'' \geq 0$; in several variables, if the Hessian $\nabla^2 J$ is positive semidefinite everywhere. For example, the squared loss $(y-t)^2$ and the logistic loss $-yt + \log(1+e^t)$, whose second derivative is $\sigma(t)(1-\sigma(t)) \geq 0$, are convex in t.

(ii) Composition with an affine map preserves convexity: if J is convex, so is $\beta \mapsto J(x^\top \beta)$. Hence $(y_i - x_i^\top \beta)^2$ and $-y_i x_i^\top \beta + \log(1+e^{x_i^\top \beta})$ are convex in β.

(iii) Nonnegative weighted sums of convex functions are convex. Hence the least-squares objective, the logistic negative log-likelihood, and their versions with a ridge or lasso penalty are convex in β.

Proposition 3.5 (Why Convexity Matters). *Let J be convex and differentiable on $\mathbb{R}^p$. Then every critical point is a global minimizer, and if J is strictly convex there is at most one minimizer. If moreover the curvature of J is bounded (its gradient is Lipschitz continuous), gradient descent with a sufficiently small constant learning rate converges to the minimum value.*

For a convex objective there are no spurious local minima, so a point where gradient descent stops is a solution of the learning problem. This is the case for linear and logistic regression, ridge and lasso (Section 5.2, where the penalty is convex but not differentiable), the support vector machine with the hinge loss (Section 8.4), and the negative log-likelihood of any exponential family in its natural parameter (Proposition 2.8).

Remark 3.6 (Two Corrections to the Slides). The STA 314 slides state that if f is convex and g is linear, then both $f \circ g$ and $g \circ f$ are convex. The first is true (rule (ii)), but the second requires g to be nondecreasing: with $g(t) = -t$, the function $g \circ f = -f$ is concave. The slides also state that the least-squares loss is convex "so there is a unique solution". Convexity guarantees that every critical point is a global minimizer, not uniqueness. The minimizer is unique if and only if X has full column rank; when $p > n$, for example, the normal equations have infinitely many solutions, which is one motivation for ridge regression (Section 5.1). □

3.4 ▪ Stochastic Gradient Descent

In (3.1) the objective is an average over data points, $J(\theta) = \frac{1}{n}\sum_{i=1}^n J_i(\theta)$ with $J_i(\theta) = L(y_i, f_\theta(x_i))$ (plus the penalty), and so is its gradient. Every gradient descent step therefore touches every training example, which is called *batch* training. With millions of examples a single step becomes expensive, and much of the work is redundant, since many examples carry similar information. *Stochastic gradient descent* (SGD) replaces the full gradient by an average over a small random subset of the data, a *minibatch*.

Algorithm 3.2 Minibatch Stochastic Gradient Descent (SGD)

Input: Objective $J(\theta) = \frac{1}{n}\sum_{i=1}^{n} J_i(\theta)$; minibatch size B; learning rate schedule $(\eta_k)_{k\geq 0}$; number of steps K.

Initialize: $\theta^{(0)}$.

for $k = 0, 1, \ldots, K-1$ **do**
  Draw a minibatch $S_k \subset \{1, \ldots, n\}$ of size B uniformly at random.
  $\hat{g}^{(k)} \leftarrow \frac{1}{B}\sum_{i\in S_k} \nabla J_i(\theta^{(k)})$ ▷ noisy estimate of $\nabla J(\theta^{(k)})$
  $\theta^{(k+1)} \leftarrow \theta^{(k)} - \eta_k\, \hat{g}^{(k)}$
end for

Output: The final iterate $\theta^{(K)}$ (or an average of the last iterates).

With $B = 1$ this is plain SGD: pick one example i uniformly at random and step along $-\nabla J_i$. For least squares the step is $\beta \leftarrow \beta + 2\eta\,(y_i - x_i^\top \beta)x_i$, and for logistic regression $\beta \leftarrow \beta + \eta\,(y_i - \sigma(x_i^\top \beta))x_i$. One pass through the data is called an *epoch*; in practice the data are reshuffled once per epoch and split into consecutive minibatches, so that every example is used once per epoch.

The justification is that the minibatch gradient is an unbiased estimate of the full gradient: for a uniformly chosen index, $\mathbb{E}[\nabla J_i(\theta)] = \frac{1}{n}\sum_i \nabla J_i(\theta) = \nabla J(\theta)$. Batch gradient descent moves straight downhill; SGD moves in a noisy direction that is downhill on average. The variance of $\hat{g}^{(k)}$ decreases like $1/B$, so the minibatch size trades the cost of a step against its accuracy, and it is itself a hyperparameter.

Because the gradient estimate is noisy, the learning rate also controls the size of the fluctuations around the minimum. The typical schedule uses a large learning rate early in training, to approach the minimum quickly, and then decays it, to damp the fluctuations. A constant learning rate leaves the iterates bouncing in a neighborhood of the minimizer whose size is proportional to η; convergence requires $\eta_k \to 0$, classically with $\sum_k \eta_k = \infty$ and $\sum_k \eta_k^2 < \infty$, such as $\eta_k \propto 1/k$.

Pros and Cons: The cost of an SGD step is independent of n, so the method makes substantial progress before it has even seen all the data, and it is the only practical choice for very large data sets and models. The noise can even help on non-convex problems (Section 3.5). However, single-example gradients have high variance, the final iterate fluctuates unless the learning rate is decayed, and the method adds two hyperparameters, the minibatch size and the schedule. Practical variants such as momentum and Adam, used for neural networks in Chapter 11, adapt the step to the history of the gradients.

Method	Cost per step	When to use
Closed form	$O(np^2 + p^3)$, once	small p; only for special objectives such as least squares and ridge
Gradient descent	$O(np)$ for a linear model	moderate n; smooth objectives; deterministic, easy to diagnose
Minibatch SGD	$O(Bp)$ for a linear model	large n; neural networks; any objective that is an average

3.5 ▪ Non-Convex Optimization

Many important objectives are not convex: the training loss of a neural network (Chapter 11), the likelihood of a mixture model (Section 16.1), the k-means objective (Algorithm 16.3), and the evidence

lower bound of a variational autoencoder (Section 16.7). For such objectives a critical point may be a local minimum that is worse than the global one, or a *saddle point*, a minimum in some directions and a maximum in others. Gradient descent run to convergence finds a point where the gradient (approximately) vanishes, and which one depends on the initialization. Two practical responses are common: restart from several random initializations and keep the best solution, and use stochastic gradients, whose noise gives the iterates a better chance of escaping poor local minima and saddle points and of settling in broad, favorable regions. For deep networks, local minima found by SGD are often good enough, and the difficulty lies as much in generalization as in optimization.

Remark 3.7 (Gradient-Based Optimization Throughout the Notes). Gradient descent and SGD are the computational backbone of the rest of these notes. They fit logistic and softmax regression (Algorithm 7.1) and neural networks via backpropagation (Algorithm 11.2); they optimize the evidence lower bound in stochastic variational inference and variational autoencoders (Algorithms 15.2 and 16.5); and they train word embeddings and diffusion models (Algorithms 17.1 and 18.1). Boosting with the squared loss fits each new tree to the residuals, which are the negative gradient of the loss with respect to the current predictions, so it can be read as gradient descent in the space of functions (Algorithm 10.6). Hamiltonian Monte Carlo uses the gradient of a log-density to propose distant moves (Algorithm 14.6). The idea behind SGD, replacing an average that is expensive to compute by an unbiased estimate from a random subsample, is the Monte Carlo principle of Section 14.2. □

Limitations and what comes next: Part I has provided the three ingredients of a learning method: a target (the best predictor of Chapter 1), a principle for fitting (likelihood and Bayes, Chapter 2), and algorithms for computing the fit (this chapter). What it has not provided is a model. Part II starts with the simplest and most thoroughly understood one, the linear regression model, where the fit has a closed form, the sampling properties of the estimates are known exactly, and the question of how to choose among competing models (Section 1.5) can be studied in detail.

Part II

Prediction

Chapter 4

Linear Regression and Model Selection

Chapter 1 framed supervised learning as estimating f in $Y = f(X) + \varepsilon$ and showed that the expected test error splits into bias and variance. Linear regression is the simplest parametric answer: it restricts f to linear functions of the features, which makes fitting a least squares problem with a closed-form solution, and makes the variance of the fit explicit. Section 4.1 defines the model and the ordinary least squares estimator, Section 4.2 studies its sampling properties and the resulting inference, and Section 4.3 shows how qualitative predictors and interactions fit into the same framework. The second half of the chapter turns to a question that recurs throughout these notes: how to choose among several candidate models when only training data are available. Section 4.5 estimates the test error directly by cross-validation, Section 4.6 corrects the training error analytically, and Section 4.7 applies both ideas to the selection of predictors.

4.1 ▪ The Linear Model and Ordinary Least Squares

Let $Y \in \mathbb{R}$ be the response and $X = (X_1, \dots, X_p) \in \mathbb{R}^p$ the vector of features.

Definition 4.1 (Linear Regression Model). *The* linear regression model *assumes*

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0,$$

with unknown constants β_0 (the intercept*) and $\beta_1, \dots, \beta_p$ (the* coefficients*). For inference one usually adds that the errors of different observations are uncorrelated with common variance $\mathrm{Var}(\varepsilon) = \sigma^2$ (homoscedasticity), and sometimes that $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.* □

Given estimates $\hat\beta_0, \dots, \hat\beta_p$, the prediction at x is the linear predictor $\hat f(x) = \hat\beta_0 + \hat\beta_1 x_1 + \cdots + \hat\beta_p x_p$. To choose the estimates, recall from Section 1.3 that under squared loss the best predictor minimizes $\mathbb{E}[(Y - g(X))^2]$. Restricting g to linear functions and replacing the expectation by the average over the training data gives ordinary least squares. In matrix form, let $x_i = (1, x_{i1}, \dots, x_{ip})^\top \in \mathbb{R}^{p+1}$, let $X \in \mathbb{R}^{n \times (p+1)}$ have rows $x_i^\top$, and let $y = (y_1, \dots, y_n)^\top$.

Definition 4.2 (Ordinary Least Squares). *The* ordinary least squares *(OLS) estimator is*

$$\hat\beta = \operatorname*{arg\,min}_{\beta \in \mathbb{R}^{p+1}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - x_i^\top \beta\right)^2 = \operatorname*{arg\,min}_{\beta \in \mathbb{R}^{p+1}} \frac{1}{n} \|y - X\beta\|_2^2.$$

The quantity $\mathrm{RSS}(\beta) = \|y - X\beta\|_2^2$ is the residual sum of squares*; $y_i - x_i^\top \hat\beta$ is the i-th* residual. □

OLS is an instance of a general recipe, *empirical risk minimization*: choose

$$\hat{f} = \arg\min_{g \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, g(x_i)\big) \tag{4.1}$$

for a loss L and a class of functions $\mathcal{F}$. OLS takes $L(y, \hat{y}) = (y - \hat{y})^2$ and $\mathcal{F} = \{x \mapsto x^\top \beta\}$. Most methods in Part II are obtained by changing L, changing $\mathcal{F}$, or adding a penalty to (4.1). What makes OLS special is that (4.1) can be solved in closed form. Setting the gradient of $\|y - X\beta\|_2^2$ to zero gives the *normal equations*

$$X^\top X\, \hat{\beta} = X^\top y, \qquad \text{so} \qquad \hat{\beta} = (X^\top X)^{-1} X^\top y \quad \text{if } X \text{ has full column rank.}$$

The fitted values are $\hat{y} = X\hat{\beta} = Hy$ with the *hat matrix* $H = X(X^\top X)^{-1}X^\top$. Geometrically, H is the orthogonal projection onto the column space of X: OLS finds the point of that subspace closest to y, and the residual vector $y - \hat{y}$ is orthogonal to every column of X. The diagonal entries $h_i = H_{ii}$ are the *leverages* of the observations and reappear in Proposition 4.7.

Remark 4.3 (Connection to Maximum Likelihood). If $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, the log-likelihood of β is $-\frac{1}{2\sigma^2}\|y - X\beta\|_2^2$ up to terms that do not depend on β. OLS is therefore the maximum likelihood estimator of Section 2.2 under Gaussian noise. This probabilistic reading is what allows the Bayesian treatment of Section 5.6 and the likelihood-based criteria AIC and BIC in Section 4.6. □

Remark 4.4 (Connection to Gradient Descent). Solving the normal equations costs $O(np^2 + p^3)$ operations. When p is large, or when the loss is changed so that no closed form exists, the same objective is minimized iteratively by gradient descent (Algorithm 3.1); OLS is the first worked example in Chapter 3. □

4.2 ▪ Sampling Properties and Inference

The training data are realizations of random variables, so $\hat{\beta}$ is random: a different training set gives a different fitted line, although the fits scatter around the true regression line. For the analysis we treat the design matrix X as fixed and the errors $\varepsilon_1, \dots, \varepsilon_n$ as i.i.d. with mean zero and variance σ^2.

Proposition 4.5 (Mean and Covariance of OLS). *If X has full column rank, then*

$$\mathbb{E}[\hat{\beta}] = \beta, \qquad \operatorname{Cov}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1}, \qquad \mathbb{E}\big[\|\hat{\beta} - \beta\|_2^2\big] = \sigma^2 \operatorname{tr}\big[(X^\top X)^{-1}\big].$$

In particular, if $X^\top X = n I_{p+1}$, then $\mathbb{E}\big[\|\hat{\beta} - \beta\|_2^2\big] = \sigma^2 (p+1)/n$.

The last identity is the key message for what follows: OLS is unbiased, but its estimation error grows linearly in the number of parameters. Every additional predictor, relevant or not, adds variance.

The noise level is estimated by $\hat{\sigma}^2 = \operatorname{RSS}(\hat{\beta})/(n - p - 1)$, where dividing by $n - p - 1$ rather than n makes the estimate unbiased. The standard error of $\hat{\beta}_j$ is $\operatorname{SE}(\hat{\beta}_j) = \sqrt{\hat{\sigma}^2 [(X^\top X)^{-1}]_{jj}}$, and an approximate 95% confidence interval is $\hat{\beta}_j \pm 1.96\, \operatorname{SE}(\hat{\beta}_j)$. To test $H_0 \colon \beta_j = 0$ (no linear relationship between Y and X_j given the other predictors), one uses

$$t = \frac{\hat{\beta}_j}{\operatorname{SE}(\hat{\beta}_j)},$$

which has a t-distribution with $n-p-1$ degrees of freedom under H_0 and Gaussian errors. A small p-value is evidence that X_j is useful; it says nothing about the size of β_j. Joint hypotheses such as $\beta_{p-q+1}=\cdots=\beta_p=0$ are tested with an F-statistic.

Definition 4.6 (Coefficient of Determination). *With* $\mathrm{TSS}=\sum_{i=1}^{n}(y_i-\bar{y})^2$*, the* coefficient of determination *is* $R^2=1-\mathrm{RSS}/\mathrm{TSS}\in[0,1]$*, the fraction of the sample variability of* Y *explained by the regression.* □

R^2 is a training quantity: it can only increase when a predictor is added, so a high R^2 does not mean that the model predicts well. Correcting this optimism is the subject of Section 4.6.

4.3 ▪ Qualitative Predictors, Interactions and Diagnostics

The word "linear" refers to the parameters, not to the raw inputs. Many useful models fit the framework after the features are transformed.

Qualitative predictors. A categorical predictor with L levels, such as ethnicity in the Credit data, is encoded by $L-1$ dummy (indicator) variables. The level with all dummies equal to zero is the *baseline*, and each dummy coefficient is the average difference in the response between its level and the baseline. Using L dummies together with an intercept would make X rank deficient.

Interactions. In $Y=\beta_0+\beta_1X_1+\beta_2X_2+\varepsilon$ the effect of a unit increase of X_1 is β_1, whatever the value of X_2. Adding the product term,

$$Y=\beta_0+\beta_1X_1+\beta_2X_2+\beta_3X_1X_2+\varepsilon=\beta_0+(\beta_1+\beta_3X_2)X_1+\beta_2X_2+\varepsilon,$$

makes the slope of X_1 depend on X_2. With X_1 a gender dummy and X_2 years of education, β_2 is the education slope for the baseline group and $\beta_2+\beta_3$ the slope for the other group.

Transformed predictors. Terms such as X_1^2 or X_2^3 make the fit nonlinear in X while keeping it linear in β. Chapter 6 develops this idea systematically.

Diagnostics. The assumptions of Definition 4.1 should be checked, mostly with residual plots: nonlinearity of the response–predictor relationship, correlated errors, non-constant error variance, outliers, and collinearity among the predictors (which inflates $(X^\top X)^{-1}$ and hence the variance in Proposition 4.5).

4.4 ▪ Why Look Beyond OLS

Proposition 4.5 identifies two weaknesses of OLS with all p predictors. For *prediction*, the variance is large when p is large relative to n, and when $p>n$ the matrix $X^\top X$ is singular, so the estimator is not even unique. For *interpretation*, a model in which irrelevant coefficients are set exactly to zero is easier to understand. There are three families of alternatives:

(1) *Subset selection*: identify a subset of the predictors and fit OLS on it (Section 4.7).

(2) *Shrinkage*: fit all predictors but shrink the coefficients toward zero, trading a little bias for a large reduction in variance (Chapter 5).

(3) *Dimension reduction*: replace the p predictors by $M<p$ linear combinations, for instance principal components (Section 16.4), and fit OLS on those.

Each family produces not one model but a list of candidates, indexed by the subset, the amount of shrinkage, or M. Choosing among them requires an estimate of the expected test error. With a separate test set one simply compares test errors; without one there are two routes, which the next two sections describe: estimate the test error directly by splitting the data, or adjust the training error for model complexity.

4.5 ▪ Validation Sets and Cross-Validation

The training error is an optimistic estimate of the test error, because the model was fitted to those very points; the gap grows with flexibility (Section 1.5). Data-splitting methods hold out some observations, which then play the role of a test set.

Validation set approach. Randomly split the data into a training part and a validation (hold-out) part, fit on the first, and use the mean squared error on the second as the estimate. This is simple but has two drawbacks. The estimate depends strongly on the random split; on the Auto data, repeating the split ten times gives visibly different validation curves for polynomial fits of mpg on horsepower. And since only part of the data is used for fitting, the validation error tends to overestimate the test error of the model fitted on all data.

Leave-one-out cross-validation. Hold out one observation at a time: fit $\hat{f}^{(-i)}$ on all data except (x_i, y_i), record $\mathrm{MSE}_i = (y_i - \hat{f}^{(-i)}(x_i))^2$, and average,

$$\mathrm{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^{n} \mathrm{MSE}_i \,.$$

Each training set is almost the full data set, and there is no randomness in the splitting. The price is n fits. For least squares this price disappears.

Proposition 4.7 (LOOCV Shortcut for Least Squares). *For OLS (and for least squares on any fixed set of basis functions, $\hat{y} = Hy$, including polynomial regression and regression splines),*

$$\mathrm{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2 ,$$

where $\hat{y}_i$ is the fitted value from the full data and $h_i = H_{ii}$ is the leverage of observation i.

High-leverage points have residuals that the full fit has pulled toward zero; dividing by $1 - h_i$ undoes this.

k-fold cross-validation. The general procedure divides the data into k folds of roughly equal size and uses each fold once as the validation set. LOOCV is the special case $k = n$; in practice $k = 5$ or $k = 10$ is common, which needs only k fits and gives estimates that are close to LOOCV.

Algorithm 4.1 k-Fold Cross-Validation for Model Selection

Input: Data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$; candidate fitting procedures $\mathcal{M}_1, \dots, \mathcal{M}_m$ (models, or one method with different tuning parameters); number of folds k; loss L.

Randomly partition $\{1, \dots, n\}$ into folds $F_1, \dots, F_k$ of roughly equal size.
for $j = 1, \dots, m$ **do**
  for $\ell = 1, \dots, k$ **do**
    Fit $\mathcal{M}_j$ on $\{(x_i, y_i) : i \notin F_\ell\}$, giving $\hat{f}_j^{(-\ell)}$.
    $\mathrm{Err}_{j\ell} \leftarrow \frac{1}{|F_\ell|} \sum_{i \in F_\ell} L\big(y_i, \hat{f}_j^{(-\ell)}(x_i)\big)$ ▷ error on the held-out fold
  end for
  $\mathrm{CV}_j \leftarrow \frac{1}{k} \sum_{\ell=1}^k \mathrm{Err}_{j\ell}$
end for
$j^* \leftarrow \arg\min_j \mathrm{CV}_j$
Refit $\mathcal{M}_{j^*}$ on all of $\mathcal{D}$.

Output: The selected model $\mathcal{M}_{j^*}$, its refitted predictor, and the CV estimates $\mathrm{CV}_1, \dots, \mathrm{CV}_m$.

The same procedure works for classification with the 0-1 loss $L(y, \hat{y}) = \mathbf{1}\{y \neq \hat{y}\}$, in which case CV_j estimates the misclassification rate. Algorithm 4.1 is the workhorse for tuning throughout these notes: the penalty λ in ridge and lasso (Algorithm 5.1), the number of neighbors in k-NN, the number of knots of a spline, and the size of a tree are all chosen this way.

Remark 4.8 (Every Step of the Fit Must Be Cross-Validated). Cross-validation is valid only if the held-out fold plays no role in fitting. Consider 100 samples and 5000 predictors: select the 10 predictors most correlated with y, then fit OLS on them. Cross-validating only the OLS step, after screening on all data, gives a badly optimistic error estimate, because the held-out responses already influenced which predictors were chosen. The screening must be repeated inside every fold, on the training part only. □

4.6 ▪ Adjusting the Training Error: Cp, AIC, BIC and Adjusted R^2

The second route avoids refitting by adding to the training error a penalty that grows with model complexity. For a fitted linear model with d predictors (plus intercept), let $\mathrm{RSS} = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$ and let $\hat{\sigma}^2$ be an estimate of the noise variance, typically from the model with all predictors.

Definition 4.9 (Model Selection Criteria).

(i) Mallows' C_p $= \frac{1}{n}\big(\mathrm{RSS} + 2d\hat{\sigma}^2\big)$.

(ii) $\mathrm{AIC} = -2\log \hat{L} + 2d$ *and* $\mathrm{BIC} = -2\log \hat{L} + (\log n)\, d$, *where $\hat{L}$ is the maximized likelihood of the fitted model.*

(iii) Adjusted $R^2 = 1 - \dfrac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}$.

One selects the model with the smallest C_p, AIC or BIC, or the largest adjusted R^2. □

C_p adds $2d\hat{\sigma}^2/n$ to the training MSE, which, up to a constant that is the same for all models, is the amount by which the training MSE underestimates the test MSE of a least squares fit on average. Maximizing adjusted R^2 is the same as minimizing $\mathrm{RSS}/(n-d-1)$: the RSS always decreases when a predictor is added, but $\mathrm{RSS}/(n-d-1)$ increases if the added predictor removes too little of it. BIC

replaces the factor 2 by $\log n$, which exceeds 2 as soon as $n \geq 8$; it therefore penalizes model size more heavily and selects smaller models than AIC and C_p.

Proposition 4.10 (AIC and Cp Agree for Gaussian Linear Models). *For the linear model with* $\varepsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$ *and* σ^2 *known,* $\text{AIC} = \text{RSS}/\sigma^2 + 2d + \text{const}$*, so* $\frac{\sigma^2}{n}\text{AIC} = C_p + \text{const}$ *and the two criteria select the same model.*

This follows directly from the Gaussian log-likelihood in Remark 4.3: $-2\log \hat{L} = n\log(2\pi\sigma^2) + \text{RSS}/\sigma^2$.

Remark 4.11 (Notation in the Slides). The slides define RSS once with a factor $1/n$ (as the training MSE) and then write $C_p = \frac{1}{n}\text{RSS} + \frac{2p\sigma^2}{n}$, counting the same $1/n$ twice; the worked example uses $C_p = \frac{1}{n}(\text{RSS} + 2d\sigma^2)$ with RSS an unnormalized sum, which is the convention adopted here. The tutorial's Gaussian likelihood has normalizing constant $(2\pi\sigma)^{-n}$; it should be $(2\pi\sigma^2)^{-n/2}$, which does not affect the conclusion of Proposition 4.10. □

Data Splitting versus Adjusting the Training Error: Cross-validation estimates the test error directly and applies to any fitting procedure, including those for which the number of parameters or the noise variance is hard to pin down (trees, k-NN, neural networks). It needs a reasonably large sample and many fits, and it is hard to give guarantees for the model it selects. C_p, AIC, BIC and adjusted R^2 need only one fit per candidate, but they are tied to parametric models with a correctly specified likelihood and a consistent estimate of σ^2. When those conditions hold, the adjusted criteria are a cheap alternative; otherwise cross-validation is the safer choice.

4.7 ▪ Subset Selection

We now apply both routes to choosing which predictors enter a linear model. With p predictors there are 2^p subsets.

Algorithm 4.2 Best Subset Selection

Input: Data with predictors $X_1, \ldots, X_p$; a criterion for comparing models of different sizes (CV error from Algorithm 4.1, C_p, AIC, BIC, or adjusted R^2).

1: $\mathcal{M}_0 \leftarrow$ the null model with no predictors, which predicts $\bar{y}$ for every observation.
2: **for** $k = 1, \ldots, p$ **do**
3: Fit all $\binom{p}{k}$ least squares models that contain exactly k predictors.
4: $\mathcal{M}_k \leftarrow$ the one with the smallest RSS (equivalently, the largest R^2).
5: **end for**
6: Select a single model among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ using the criterion.

Output: The selected model.

The two comparison steps use different tools on purpose. Within a fixed size k all candidates have the same complexity, so the training RSS ranks them correctly. Across sizes it does not: RSS decreases and R^2 increases with k, so choosing among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ by RSS would always pick $\mathcal{M}_p$. Line 6 therefore needs an estimate of the test error. The same algorithm applies to logistic regression (Section 7.2) with the RSS replaced by the deviance, -2 times the maximized log-likelihood.

Best subset selection fits $\sum_k \binom{p}{k} = 2^p$ models, which is infeasible beyond a few dozen predictors, and searching such a large space also increases the chance of finding a model that fits the training data

well by luck. Stepwise methods search a much smaller, nested path of models.

Algorithm 4.3 Forward Stepwise Selection

Input: Data with predictors $X_1, \ldots, X_p$; a criterion for comparing models of different sizes.

1: $\mathcal{M}_0 \leftarrow$ the null model.
2: **for** $k = 0, \ldots, p-1$ **do**
3: Fit all $p-k$ models that add one predictor not yet in $\mathcal{M}_k$ to the predictors of $\mathcal{M}_k$.
4: $\mathcal{M}_{k+1} \leftarrow$ the one with the smallest RSS.
5: **end for**
6: Select a single model among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ using the criterion.

Output: The selected model.

Forward stepwise selection fits $1 + \sum_{k=0}^{p-1}(p-k) = 1 + p(p+1)/2$ models instead of 2^p. *Backward stepwise selection* runs the same idea in reverse: start from the full model $\mathcal{M}_p$ and, for $k = p, \ldots, 1$, remove the predictor whose deletion increases the RSS the least, giving $\mathcal{M}_{k-1}$; it also fits $1 + p(p+1)/2$ models but requires $n > p$ so that the full model can be fitted. Forward selection can be used even when $p > n$, by stopping before the model becomes too large. Both procedures are greedy: once a predictor has been added (or removed), the decision is never revisited, so the best model of a given size may be missed. On the Credit data, for instance, best subset and forward selection agree on the best models with one to three predictors but choose different four-predictor models.

Pros and Cons: Subset selection produces interpretable models that use only some of the predictors, and it combines naturally with any of the criteria of Sections 4.5 and 4.6. Best subset selection is exact but costs 2^p fits; stepwise selection is cheap but greedy. In both cases the fitted model is a discontinuous function of the data, since a predictor is either in or out, so small changes in the data can change the selected model and the fit, which keeps the variance high.

Remark 4.12 (Connection to Resampling). Cross-validation is one of two resampling ideas in these notes. The other, the bootstrap (Section 10.5), resamples the training data with replacement to estimate the variability of an estimator, and it underlies bagging and random forests. □

Limitations and what comes next: OLS on all predictors has high variance when p is large, and subset selection reduces it only through a discrete, expensive or greedy search whose output jumps when the data change slightly. Chapter 5 keeps all predictors and instead shrinks the coefficients continuously toward zero. Ridge regression does so with a quadratic penalty, the lasso with an absolute-value penalty that also sets some coefficients exactly to zero and thus performs variable selection as a by-product. The penalty will turn out to be a prior in disguise, which leads to Bayesian linear regression.

Chapter 5

Shrinkage and Bayesian Linear Regression

Subset selection (Section 4.7) reduces the variance of OLS by removing predictors, but it searches a discrete space: best subset selection is exponentially expensive, stepwise selection is greedy, and either way a predictor is fully in or fully out. Shrinkage methods keep all p predictors and instead pull the estimated coefficients continuously toward zero by adding a penalty to the least squares objective. Section 5.1 introduces ridge regression and Section 5.2 the lasso, whose penalty also sets coefficients exactly to zero. Section 5.3 computes the shrinkage of both in a design where everything is explicit, Section 5.4 chooses the penalty level by cross-validation and lists other penalties, and Section 5.5 reinterprets the penalty as a prior. Taking that interpretation seriously leads to Bayesian linear regression (Section 5.6), which returns a full distribution over predictions instead of a point estimate.

5.1 ▪ Ridge Regression

Definition 5.1 (Ridge Regression). *For a tuning parameter $\lambda \geq 0$, the* ridge regression *estimate $\hat{\beta}_\lambda^R$ minimizes*

$$\sum_{i=1}^{n} \Big(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \Big)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 = \mathrm{RSS}(\beta) + \lambda \|\beta_{1:p}\|_2^2.$$

The term $\lambda\|\beta_{1:p}\|_2^2$ is a shrinkage *or* regularization penalty. *The intercept β_0 is not penalized.* □

For $\lambda = 0$ ridge regression is OLS; as $\lambda \to \infty$ all slopes are driven to zero and the fit tends to the constant $\bar{y}$. In between, ridge accepts a slightly larger RSS in exchange for smaller coefficients. If the predictors are centered and y is centered, so that $\hat{\beta}_0 = \bar{y}$, the minimizer has a closed form,

$$\hat{\beta}_\lambda^R = (X^\top X + \lambda I_p)^{-1} X^\top y,$$

where $X \in \mathbb{R}^{n \times p}$ now has no column of ones. The matrix $X^\top X + \lambda I_p$ is invertible for every $\lambda > 0$, even when $p > n$, so ridge regression is well defined exactly where OLS breaks down. Computing it costs about the same as OLS for a given λ, far less than best subset selection.

Unlike OLS, ridge regression is not *scale equivariant*: multiplying a predictor by c divides its OLS coefficient by c, but changes the ridge fit, since the penalty treats all coefficients alike. Predictors are therefore standardized before fitting,

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2}},$$

so that all have standard deviation one.

As λ grows, the variance of the fit falls and its squared bias rises. On a simulated example with nearly as many predictors as observations, where OLS has very high variance, the test MSE of ridge first decreases sharply with λ and then increases again, with a minimum well below the OLS error; this is the bias–variance trade-off of Theorem 1.6 traced out by a single continuous knob.

Pros and Cons: Ridge regression improves on OLS in prediction whenever the OLS estimates have high variance (many predictors, collinearity, p close to or larger than n), it is computationally as cheap as OLS, and it varies smoothly with the data and with λ. However, it shrinks all coefficients but sets none of them exactly to zero, so the fitted model always contains all p predictors and is hard to interpret.

5.2 ▪ The Lasso

Definition 5.2 (Lasso). *For $\lambda \geq 0$, the* lasso *estimate $\hat{\beta}^L_\lambda$ minimizes*

$$\sum_{i=1}^{n} \Big(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \Big)^2 + \lambda \sum_{j=1}^{p} |\beta_j| = \mathrm{RSS}(\beta) + \lambda \|\beta_{1:p}\|_1.$$

□

The only change is the ℓ_1 penalty in place of the squared ℓ_2 penalty, but the consequence is qualitative: for λ large enough, some lasso coefficients are *exactly zero*. The lasso therefore performs variable selection, and its fitted models are *sparse*. The lasso objective has no closed-form minimizer, but it is convex (Section 3.3), and efficient algorithms compute it quickly for a whole grid of λ values.

Why exact zeros? Both methods can be written as constrained least squares problems: for every λ there is an $s \geq 0$ such that

$$\hat{\beta}^L \in \arg\min_{\beta} \ \mathrm{RSS}(\beta) \ \text{ subject to } \ \sum_{j=1}^{p} |\beta_j| \leq s, \qquad \hat{\beta}^R \in \arg\min_{\beta} \ \mathrm{RSS}(\beta) \ \text{ subject to } \ \sum_{j=1}^{p} \beta_j^2 \leq s.$$

The level sets of the RSS are ellipses centered at the OLS estimate, and the solution is the point where the smallest ellipse touches the constraint region (Fig. 5.1). The ℓ_2 ball is round, so the contact point generically has all coordinates nonzero. The ℓ_1 ball has corners on the coordinate axes, and an ellipse often touches it first at a corner, where some coordinates vanish. In higher dimensions the ℓ_1 ball has many corners and edges, which makes sparse solutions even more typical.

Neither method dominates the other in prediction. When the true coefficients are all nonzero and of similar size, ridge and lasso have similar bias but ridge has smaller variance and wins. When only a few predictors matter, the lasso has both smaller bias and smaller variance. One expects the lasso to do better when the response depends on a relatively small number of predictors; in practice cross-validation decides.

Pros and Cons: The lasso reduces variance like ridge regression and, in addition, selects variables, so its models are sparse and interpretable; the penalized problem is convex and fast to solve. On the other hand, the lasso has no closed form, it tends to pick one predictor arbitrarily from a group of highly correlated ones, and it can be outperformed by ridge regression when the true signal is spread over many predictors.

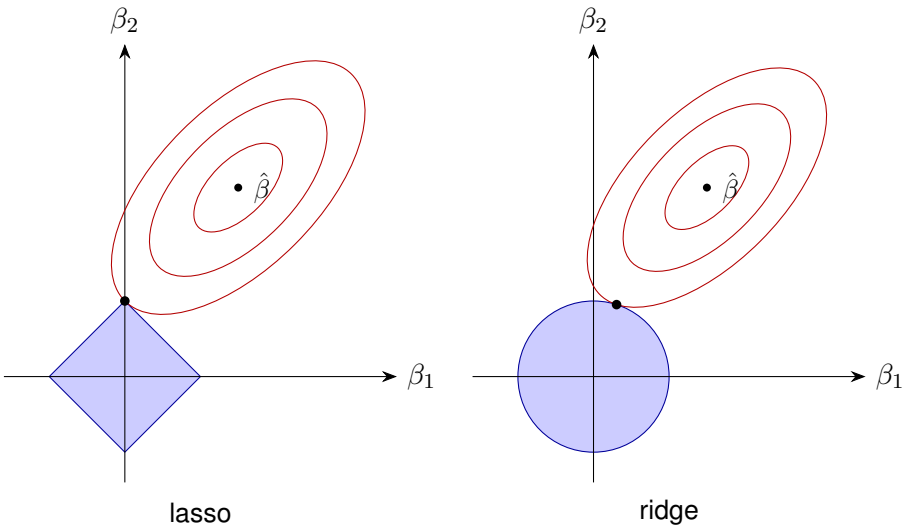


Figure 5.1. *Constraint regions $|\beta_1| + |\beta_2| \le s$ (left) and $\beta_1^2 + \beta_2^2 \le s$ (right) with contours of the RSS around the OLS estimate $\hat{\beta}$. The lasso solution (dot) lies on a corner, so $\hat{\beta}_1^L = 0$; the ridge solution has both coordinates nonzero.*

5.3 ▪ Shrinkage in an Orthogonal Design

The difference between the two penalties is easiest to see when the design is trivial. Take $n = p$, $X = I_p$, no intercept, so that

$$y_j = \beta_j + \varepsilon_j, \qquad \mathbb{E}[\varepsilon_j] = 0, \quad \mathbb{E}[\varepsilon_j^2] = \sigma^2, \qquad j = 1, \dots, p.$$

All three objectives separate across coordinates and can be minimized one β_j at a time.

Proposition 5.3 (OLS, Ridge and Lasso for $X = I$). *Minimizing $\sum_j (y_j - \beta_j)^2$ without penalty, with penalty $\lambda \sum_j \beta_j^2$, or with penalty $\lambda \sum_j |\beta_j|$ gives*

$$\hat{\beta}_j = y_j, \qquad \hat{\beta}_j^R = \frac{y_j}{1+\lambda}, \qquad \hat{\beta}_j^L = \begin{cases} y_j - \lambda/2, & y_j > \lambda/2, \\ y_j + \lambda/2, & y_j < -\lambda/2, \\ 0, & |y_j| \le \lambda/2, \end{cases}$$

that is, $\hat{\beta}_j^L = \operatorname{sign}(y_j)\,(|y_j| - \lambda/2)_+$.

Ridge *scales* every coefficient by the same factor $1/(1+\lambda)$, so no coefficient becomes zero. The lasso *soft-thresholds*: it moves every coefficient toward zero by the same amount $\lambda/2$ and sets to zero those that would cross it. Small, noise-dominated coefficients are removed; large ones are kept, though biased by $\lambda/2$.

In this design the bias and variance are explicit. OLS is unbiased with $\operatorname{Var}(\hat{\beta}_j) = \sigma^2$, so its total mean squared error is $\mathbb{E}\|\hat{\beta} - \beta\|_2^2 = p\sigma^2$. For ridge,

$$\mathbb{E}[\hat{\beta}_j^R] = \frac{\beta_j}{1+\lambda}, \qquad \operatorname{Var}(\hat{\beta}_j^R) = \frac{\sigma^2}{(1+\lambda)^2}, \qquad \mathbb{E}\|\hat{\beta}^R - \beta\|_2^2 = \frac{\lambda^2 \|\beta\|_2^2 + p\sigma^2}{(1+\lambda)^2}.$$

The first term is the squared bias, which grows with λ; the second is the variance, which shrinks.

Remark 5.4 (Some Shrinkage Always Helps Here). Differentiating the ridge MSE gives a derivative proportional to $\lambda\|\beta\|_2^2 - p\sigma^2$, which is negative at $\lambda = 0$. Hence a small amount of shrinkage always beats OLS in this model, and the best value is $\lambda^* = p\sigma^2/\|\beta\|_2^2$, with MSE $p\sigma^2 \cdot \frac{\|\beta\|_2^2}{\|\beta\|_2^2 + p\sigma^2} < p\sigma^2$. The gain is large when the signal $\|\beta\|_2^2$ is small compared with the total noise $p\sigma^2$. Since λ^* depends on the unknown β, it must be estimated from the data, which is the subject of the next section. □

5.4 ▪ Choosing the Penalty and Other Penalties

Each value of λ defines a different model, so choosing λ is a model selection problem, solved with the tools of Chapter 4.

Algorithm 5.1 Choosing the Penalty by Cross-Validation

Input: Data $\mathcal{D}$; penalized method (ridge or lasso); grid $\lambda_1 > \lambda_2 > \cdots > \lambda_m \geq 0$; number of folds k.

1: Standardize the predictors (using the training folds only, inside each fold).
2: For each λ_j, compute the k-fold cross-validation error $\mathrm{CV}(\lambda_j)$ with Algorithm 4.1.
3: $\lambda^* \leftarrow \arg\min_{\lambda_j} \mathrm{CV}(\lambda_j)$
4: Refit the penalized method with λ^* on all of $\mathcal{D}$.

Output: λ^* and the fitted coefficients $\hat{\beta}_{\lambda^*}$.

The grid is usually logarithmic, and with warm starts the fits along the grid are cheap. On the Credit data, the cross-validation curve for ridge regression is fairly flat near its minimum, and the selected λ shrinks the coefficients only moderately compared with OLS.

Other penalties. Ridge and lasso are two points in a large family.

(i) The *elastic net* minimizes $\|y - X\beta\|_2^2 + \lambda\big[(1-\alpha)\|\beta\|_1 + \alpha\|\beta\|_2^2\big]$ with $\alpha \in [0,1]$, so $\alpha = 1$ is ridge and $\alpha = 0$ the lasso (conventions differ: in the R package glmnet, $\alpha = 1$ is the lasso). It keeps the sparsity of the lasso while behaving more stably when predictors are strongly correlated, a situation to which the lasso is very sensitive.

(ii) The *group lasso* penalizes groups of coefficients through their Euclidean norms. In $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2 + \varepsilon$, the penalty $\lambda\big(\sqrt{\beta_1^2 + \beta_2^2} + \sqrt{\beta_3^2 + \beta_4^2}\big)$ sets both coefficients of a variable to zero together, so whole variables are removed rather than individual terms.

More generally, regularization applies to any parametric model:

$$\hat{\beta}_\lambda = \arg\min_{\beta} \Big[\mathcal{L}(\beta; \mathcal{D}) + \lambda\,\mathrm{Pen}(\beta)\Big], \tag{5.1}$$

where $\mathcal{L}$ is a training loss, for instance a negative log-likelihood, and Pen a penalty. OLS, ridge and lasso are the squared loss with $\mathrm{Pen} = 0$, $\|\beta\|_2^2$ and $\|\beta\|_1$. Penalized logistic regression (Section 7.2), the soft-margin SVM (Section 8.3) and weight decay in neural networks (Chapter 11) are further instances.

Remark 5.5 (Connection to Gradient Descent: Weight Decay). With the scaling $\frac{1}{2}\|y - X\beta\|_2^2 + \frac{\lambda}{2}\|\beta\|_2^2$, a gradient descent step (Algorithm 3.1) with learning rate η reads

$$\beta \leftarrow (1 - \eta\lambda)\,\beta - \eta\, X^\top (X\beta - y).$$

Before the usual least squares step, the coefficients are multiplied by $1 - \eta\lambda < 1$; this is why the ℓ_2 penalty is called *weight decay* when it is used to train neural networks. □

Method	How complexity is controlled	Computation	Sparse?
OLS	not at all	closed form	no
Best subset	number of predictors, discrete	2^p fits	yes
Forward stepwise	number of predictors, greedy path	$1 + p(p+1)/2$ fits	yes
Ridge	$\lambda\|\beta\|_2^2$, continuous	closed form per λ	no
Lasso	$\lambda\|\beta\|_1$, continuous	convex optimization	yes

5.5 ▪ Penalties as Priors

So far the penalty was a device to trade bias for variance. A probabilistic model gives it a second meaning. Assume $y_i \mid x_i, \beta \sim \mathcal{N}(x_i^\top \beta, \sigma^2)$, so that the log-likelihood is $-\frac{1}{2\sigma^2}\|y - X\beta\|_2^2 + \text{const}$ (Remark 4.3), and put a prior $p(\beta)$ on the coefficients. The maximum a posteriori (MAP) estimate of Section 2.4 maximizes $\log p(\beta) + \log p(\mathcal{D} \mid \beta)$.

Proposition 5.6 (Ridge and Lasso as MAP Estimates).

(i) With the Gaussian prior $\beta_j \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \tau^2)$, the MAP estimate is the ridge estimate with $\lambda = \sigma^2/\tau^2$.

(ii) With the Laplace prior $p(\beta_j) \propto \exp(-|\beta_j|/b)$, independently over j, the MAP estimate is the lasso estimate with $\lambda = 2\sigma^2/b$.

In both cases the prior is placed on $\beta_1, \dots, \beta_p$ only; the intercept gets a flat prior (or the data are centered), matching the unpenalized intercept of Definition 5.1.

Indeed, in case (i) the negative log-posterior is $\frac{1}{2\sigma^2}\|y - X\beta\|_2^2 + \frac{1}{2\tau^2}\|\beta\|_2^2 + \text{const}$; multiplying by $2\sigma^2$ gives the ridge objective. Case (ii) is the same computation with $\frac{1}{b}\|\beta\|_1$. A strong prior belief that coefficients are small (small τ^2) means heavy shrinkage (large λ), and $\lambda \to 0$ corresponds to a flat prior, under which MAP reduces to maximum likelihood, that is, to OLS. The Laplace density has a sharp peak at zero, which is the probabilistic counterpart of the corners of the ℓ_1 ball.

5.6 ▪ Bayesian Linear Regression

The MAP estimate uses the prior but still returns a single β. Full Bayesian inference keeps the whole posterior distribution and averages predictions over it. The model then "knows what it does not know": predictions come with uncertainty, which can be used to report confidence, to make more robust decisions, and to decide where more data are needed.

We work with a general feature map $\phi\colon \mathcal{X} \to \mathbb{R}^m$, for instance $\phi(x) = (1, x_1, \dots, x_p)$ or the basis functions of Chapter 6. Let $\Phi \in \mathbb{R}^{n \times m}$ have rows $\phi(x_i)^\top$.

Definition 5.7 (Bayesian Linear Regression). *The prior is $\beta \sim \mathcal{N}(0, S)$ and the likelihood $y_i \mid x_i, \beta \sim \mathcal{N}(\beta^\top \phi(x_i), \sigma^2)$, independently over i, with S and σ^2 known.* □

Proposition 5.8 (Posterior). *The posterior is Gaussian, $\beta \mid \mathcal{D} \sim \mathcal{N}(\mu, \Sigma)$, with*

$$\Sigma = \sigma^2\big(\Phi^\top\Phi + \sigma^2 S^{-1}\big)^{-1}, \qquad \mu = \big(\Phi^\top\Phi + \sigma^2 S^{-1}\big)^{-1}\Phi^\top y.$$

The posterior follows by completing the square (Section 2.5): as a function of β,

$$\log p(\beta \mid \mathcal{D}) = -\tfrac{1}{2}\beta^\top S^{-1}\beta - \tfrac{1}{2\sigma^2}\|\Phi\beta - y\|_2^2 + \text{const} = -\tfrac{1}{2}\beta^\top\big(\sigma^{-2}\Phi^\top\Phi + S^{-1}\big)\beta + \sigma^{-2}y^\top\Phi\beta + \text{const},$$

and a log-density of the form $-\frac{1}{2}\beta^\top A\beta + b^\top\beta$ with A positive definite belongs to $\mathcal{N}(A^{-1}b, A^{-1})$. The Gaussian prior is *conjugate*: the posterior is again Gaussian. With $S = (\sigma^2/\lambda)I$ the posterior mean is $\mu = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top y$, the ridge estimate (with the intercept penalized too if Φ contains a constant feature), in agreement with Proposition 5.6; as the prior variance tends to infinity, μ tends to the OLS estimate. Because the posterior after one batch of data is the prior for the next, the posterior can also be updated one observation at a time. For $y = \beta_0 + \beta_1 x$, a single point only restricts β to a line in parameter space, and the posterior concentrates around the true coefficients as points accumulate.

To predict at a new input x, average the likelihood over the posterior.

Proposition 5.9 (Posterior Predictive Distribution).

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, \beta)\, p(\beta \mid \mathcal{D})\, \mathrm{d}\beta = \mathcal{N}\big(y;\ \mu_{\text{pred}}(x),\ \sigma^2_{\text{pred}}(x)\big),$$
$$\mu_{\text{pred}}(x) = \mu^\top\phi(x), \qquad \sigma^2_{\text{pred}}(x) = \phi(x)^\top\Sigma\,\phi(x) + \sigma^2.$$

This is immediate from writing $y = \beta^\top\phi(x) + \varepsilon$ with $\beta \mid \mathcal{D} \sim \mathcal{N}(\mu, \Sigma)$ independent of $\varepsilon \sim \mathcal{N}(0, \sigma^2)$: an affine function of Gaussians is Gaussian (Proposition 2.15). The predictive variance has two parts. The noise σ^2 cannot be reduced by more data. The term $\phi(x)^\top\Sigma\phi(x)$ is the uncertainty about β; it is small near the training inputs and grows away from them, within the region covered by the features. With radial basis features $\phi_j(x) = \exp\big(-(x - c_j)^2/(2s^2)\big)$ centered at c_j, functions sampled from the posterior agree near the data and fan out between the data points, and the predictive bands widen accordingly. Far from all centers, however, $\phi(x) \to 0$ and the predictive variance falls back to σ^2: the model becomes overconfident exactly where it has no data, a defect that Gaussian processes (Section 9.4) avoid.

Algorithm 5.2 Bayesian Linear Regression

Input: Data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$; feature map ϕ; prior covariance S; noise variance σ^2; test inputs $x_1^*, \dots, x_r^*$.

Form $\Phi \in \mathbb{R}^{n\times m}$ with rows $\phi(x_i)^\top$.
$\Sigma \leftarrow \sigma^2\big(\Phi^\top\Phi + \sigma^2 S^{-1}\big)^{-1}$; $\mu \leftarrow \sigma^{-2}\,\Sigma\,\Phi^\top y$ ▷ posterior, Proposition 5.8
for $j = 1, \dots, r$ **do**
  $\mu_j^* \leftarrow \mu^\top\phi(x_j^*)$; $(\sigma_j^*)^2 \leftarrow \phi(x_j^*)^\top\Sigma\,\phi(x_j^*) + \sigma^2$ ▷ predictive, Proposition 5.9
end for

Output: Posterior $\mathcal{N}(\mu, \Sigma)$ over β and predictive distributions $\mathcal{N}\big(\mu_j^*, (\sigma_j^*)^2\big)$; e.g. 95% bands $\mu_j^* \pm 1.96\,\sigma_j^*$.

Pros and Cons: Bayesian linear regression has a closed-form posterior, its predictive mean coincides with ridge regression, and it adds calibrated uncertainty that increases away from the data. However, it assumes that S and σ^2 are known, which is a strong assumption, and the fit is only as flexible as the fixed feature map ϕ; inverting an $m \times m$ matrix also becomes expensive when there are many features.

Remark 5.10 (Connection to Gaussian Processes). The predictive mean and variance depend on the features only through inner products of the form $\phi(x)^\top S\,\phi(x')$. Replacing these inner products by a

kernel $k(x, x')$ allows infinitely many features at finite cost (Section 9.2) and turns Bayesian linear regression into Gaussian process regression (Section 9.4). There the unknown quantities S and σ^2, now kernel hyperparameters, are learned by maximizing the marginal likelihood instead of being fixed in advance. □

Limitations and what comes next: Every model in this and the previous chapter is linear in a fixed set of features, and so far those features were essentially the raw inputs. Regularization controls the variance of such models, and the Bayesian view quantifies their uncertainty, but neither helps when the true regression function is strongly nonlinear in X. Chapter 6 keeps the linear-in-parameters structure, and with it all the fitting and selection tools developed so far, but enriches the features: polynomials, step functions, splines, local fits, and additive combinations of them.

Chapter 6

Beyond Linearity

The linear model of Chapter 4 is linear in two senses: in the parameters β and in the features X. Only the first is needed for the machinery built so far. Least squares, its closed form, cross-validation, subset selection, ridge and lasso, and the Bayesian treatment of Section 5.6 all work for any model of the form $y = \beta_0 + \sum_j \beta_j b_j(x) + \varepsilon$ with fixed functions b_j. Linearity in X, on the other hand, is almost always an approximation, and sometimes a poor one. This chapter relaxes it in two ways. Sections 6.1 and 6.2 replace x by a set of basis functions (polynomials, step functions, splines) and then fit a linear model; Section 6.3 abandons a global formula and fits a simple model near each target point, generalizing k-nearest neighbors; and Section 6.4 combines one-dimensional fits of either kind into additive models for many predictors.

6.1 ▪ Basis Expansions

The model $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \varepsilon$ is quadratic in $X = (X_1, X_2)$ but linear in β: it is an ordinary linear regression on the derived features $X_1, X_2, X_1^2, X_2^2, X_1 X_2$. This observation is the whole idea of the section.

Definition 6.1 (Basis Expansion). *Given fixed functions $b_1, \dots, b_K$ of the input, the* basis function model *is*

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_K b_K(x_i) + \varepsilon_i.$$

We write $\phi(x) = (1, b_1(x), \dots, b_K(x))$ for the resulting feature map. □

Algorithm 6.1 Regression with a Basis Expansion

Input: Data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$; a family of bases indexed by a complexity parameter K (degree, number of bins, number of knots); candidate values of K.

1: **for** each candidate K **do**

2: Form the design matrix B_K with rows $\phi(x_i)^\top = (1, b_1(x_i), \dots, b_M(x_i))$: one column per basis function of the size-K family plus the intercept (e.g. $M = K$ for degree K, $M = K + 3$ for a cubic spline with K knots).

3: Fit $\hat\beta$ by least squares on (B_K, y) (or by ridge or lasso, Chapter 5); record the CV error from Algorithm 4.1.

4: **end for**

5: Choose K^* with the smallest CV error and refit on all data.

Output: $\hat f(x) = \hat\beta^\top \phi(x)$ with basis size K^*.

Because the fit is still least squares, standard errors and pointwise confidence bands for $\hat{f}(x)$ follow from Proposition 4.5 applied to B_K. The same expansion can be used inside logistic regression (Section 7.2), for instance to model $\mathbb{P}(\text{wage} > 250 \mid \text{age})$ with $\operatorname{logit} \mathbb{P}(Y = 1 \mid x) = \beta^\top \phi(x)$, fitted by maximum likelihood. The choice of basis is what distinguishes the methods below.

Polynomial regression. Take $b_k(x) = x^k$, $k = 1, \dots, d$. The individual coefficients are hard to interpret, and one looks at the fitted curve instead. The degree is chosen by cross-validation and is rarely larger than 3 or 4, because high-degree polynomials become wildly flexible. Polynomials impose a *global* structure: changing the data in one region moves the fit everywhere, and their tail behavior makes them notoriously bad at extrapolation.

Step functions. Cut the range of X at points $c_1 < \cdots < c_K$ and use the indicators

$$C_0(x) = \mathbf{1}\{x < c_1\}, \quad C_k(x) = \mathbf{1}\{c_k \le x < c_{k+1}\}\ (1 \le k < K), \quad C_K(x) = \mathbf{1}\{c_K \le x\}.$$

These $K + 1$ dummy variables sum to one, so with an intercept one drops C_0, exactly as with qualitative predictors in Section 4.3. The fit is piecewise constant; β_k is the average change in the response for $c_k \le X < c_{k+1}$ relative to $X < c_1$. Step functions are easy to fit and interpret, and are popular in biostatistics and epidemiology, but they miss trends within bins, and good cut points are hard to specify.

Remark 6.2 (Connection to Trees). A step function with well-chosen cut points can be very effective, but the cut points must be fixed in advance. Regression trees (Chapter 10) choose the cut points from the data, recursively and in several variables at once, which turns this weakness into a method. □

6.2 ▪ Regression Splines

Polynomials are smooth but global; step functions are local but discontinuous. Piecewise polynomials combine the two: fit a separate low-degree polynomial on each interval between *knots* $\xi_1 < \cdots < \xi_K$. With K knots one fits $K + 1$ polynomials, and more knots give a more flexible fit. Unconstrained, however, the pieces jump at the knots, which looks unnatural and wastes parameters. Splines add constraints at the knots.

Definition 6.3 (Spline). *A* spline of degree d *with knots* $\xi_1 < \cdots < \xi_K$ *is a function that is a polynomial of degree* d *on each interval between consecutive knots (and beyond the extreme knots) and has continuous derivatives up to order* $d - 1$ *at every knot.* □

The constraints are linear in the coefficients, so the space of splines with given knots is a linear space, and a spline regression is a basis function model. A convenient basis uses the *truncated power functions* $(x - \xi)_+^d$, where $(u)_+ = \max(u, 0)$.

Proposition 6.4 (Truncated Power Bases).

(i) Linear splines *(continuous, piecewise linear) with knots* $\xi_1, \dots, \xi_K$ *are exactly the functions*

$$f(x) = \beta_0 + \beta_1 x + \sum_{k=1}^{K} \beta_{k+1} (x - \xi_k)_+.$$

(ii) Cubic splines *(piecewise cubic with continuous first and second derivatives) are exactly the functions*

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \beta_{k+3} (x - \xi_k)_+^3,$$

so a cubic spline with K knots has $K + 4$ parameters.

Each truncated power function is zero to the left of its knot, so adding it changes the fit only to the right of the knot, and $(x-\xi)_+^3$ has continuous first and second derivatives but a jump in the third derivative at ξ. In the linear spline, β_1 is the slope for $X < \xi_1$ and β_{k+1} is the change in slope at ξ_k. Polynomial regression (no knots) and step functions (degree 0, where no continuity is required) are special cases of splines.

Remark 6.5 (Notation in the Slides). The slides list the linear spline basis as $b_1(x) = x$ and $b_k(x) = (x-\xi_{k-1})_+$ for $k = 1, \dots, K$; the index should run over $k = 2, \dots, K+1$, as in Proposition 6.4(i). □

Splines can still behave badly near the boundary of the data, where a cubic piece is constrained by few observations and the confidence bands flare out. A *natural spline* adds boundary constraints: the function is required to be linear beyond the outermost knots. For a natural cubic spline this removes two parameters at each end and gives more stable estimates near the boundary.

Choosing the knots. In practice the knots are placed at quantiles of the observed x_i, or evenly over the range of X; their exact placement is usually not crucial. The number of knots K is the complexity parameter of Algorithm 6.1 and is chosen by cross-validation. A related method, the *smoothing spline*, avoids choosing knots altogether: it places a knot at every data point and controls flexibility with a roughness penalty instead, in the same spirit as ridge regression.

Pros and Cons: Regression splines are flexible but stable: the knots give local flexibility where polynomials would need a high global degree, the continuity constraints keep the fit smooth, and natural splines tame the boundary. Since they are linear models in a basis, all inference and selection tools apply. They are, however, designed for a single predictor (for several, they are combined additively as in Section 6.4), and the number and placement of knots must still be chosen.

6.3 ▪ Local Regression

The basis function methods fit one global formula. The k-nearest neighbor method of Algorithm 1.1 instead predicts at a target point x_0 from the training observations near x_0 only,

$$\hat{f}(x_0) = \frac{1}{k} \sum_{i:\, x_i \in N_k(x_0)} y_i,$$

where $N_k(x_0)$ is the set of the k nearest training inputs. Recall that k controls the bias–variance trade-off: small k gives a flexible, high-variance fit, large k a smooth, biased one, and k is chosen by cross-validation. Two refinements improve on the plain local average.

Weighted k-NN. Give closer neighbors more weight,

$$\hat{f}(x_0) = \sum_{i:\, x_i \in N_k(x_0)} w_i(x_0)\, y_i, \qquad w_i(x_0) \geq 0, \qquad \sum_{i:\, x_i \in N_k(x_0)} w_i(x_0) = 1.$$

A popular choice is *inverse distance weighting*, $w_i(x_0) \propto 1/\|x_i - x_0\|_2$; another is a smooth bump such as the tricube function of the scaled distance. The slides write the weights as $K(x_i, x_0)$ and call them kernel weights; we use $w_i(x_0)$ to keep K and k for other purposes.

Local linear regression. The local average fits a constant near x_0. Fitting a line instead removes much of the bias near the boundary and where f has a slope:

$$(\hat\beta_0, \hat\beta_1) = \arg\min_{\beta_0,\beta_1} \sum_{i:\, x_i \in N_k(x_0)} w_i(x_0)\,\big(y_i - \beta_0 - \beta_1^\top x_i\big)^2, \qquad \hat f(x_0) = \hat\beta_0 + \hat\beta_1^\top x_0.$$

This is a weighted least squares problem solved separately for every target point. With a constant instead of a line, the minimizer is the weighted average above, so weighted k-NN is local regression of degree 0.

Algorithm 6.2 Local Regression

Input: Training data $\{(x_i, y_i)\}_{i=1}^n$; span $s \in (0, 1]$ (neighborhood of $k = \lceil sn \rceil$ points) or number of neighbors k; weight function; local degree $q \in \{0, 1\}$; target point x_0.

1: Find the k training inputs closest to x_0, forming $N_k(x_0)$.
2: Assign weights $w_i(x_0)$ to $i \in N_k(x_0)$, decreasing in $\|x_i - x_0\|$ (e.g. inverse distance or tricube); $w_i(x_0) = 0$ otherwise.
3: **if** $q = 0$ **then**
4: $\quad \hat f(x_0) \leftarrow \sum_i w_i(x_0)\, y_i \big/ \sum_i w_i(x_0)$ ▷ weighted k-NN
5: **else**
6: $\quad$ Solve the weighted least squares problem for $(\hat\beta_0, \hat\beta_1)$.
7: $\quad \hat f(x_0) \leftarrow \hat\beta_0 + \hat\beta_1^\top x_0$ ▷ local linear fit
8: **end if**

Output: $\hat f(x_0)$. Repeat for every point at which a prediction is needed; choose s (or k) by cross-validation.

Local regression is a *memory-based* method: like k-NN it keeps the training data and does its fitting at prediction time. With two predictors one uses two-dimensional neighborhoods and fits a plane. Beyond three or four predictors, however, the neighborhoods that contain enough points are no longer local, and local regression suffers from the curse of dimensionality discussed in Section 1.2.1.

Remark 6.6 (Connection to Attention). The prediction $\hat f(x_0) = \sum_i w_i(x_0)\, y_i$ with nonnegative weights summing to one, computed from a similarity between x_0 and each x_i, is a *kernel smoother*. The attention mechanism of transformers (Section 17.4) has exactly this form: a query is compared with every key, the similarities are turned into weights by a softmax, and the output is the weighted average of the values. □

6.4 ▪ Generalized Additive Models

Splines and local regression work well for one or two predictors. For larger p, generalized additive models keep a separate nonlinear function for each predictor and add them up.

Definition 6.7 (Generalized Additive Model). *A* generalized additive model *(GAM) for regression is*

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \varepsilon_i,$$

where each f_j is a function of one predictor: linear, polynomial, a step function, a spline, or a local regression fit. For a binary response, the same additive form models the log-odds, $\operatorname{logit} \mathbb{P}(Y = 1 \mid x) = \beta_0 + \sum_j f_j(x_j)$. □

When each f_j is represented in a basis (polynomials, steps, regression or natural splines), the GAM is one large basis function model: concatenate the basis columns of all predictors into a single design matrix and fit by least squares, or by maximum likelihood in the logistic case. Components fitted by a smoother rather than by least squares on a fixed basis, such as local regression or smoothing splines, are fitted by *backfitting*: cycle through the predictors and refit each f_j to the partial residuals $y_i - \beta_0 - \sum_{l \neq j} f_l(x_{il})$ until the fits stabilize. On the Wage data, wage $= \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \varepsilon$ with natural splines for year and age and a step function for the qualitative variable education gives curves that can be read off one at a time.

Pros and Cons: GAMs fit a nonlinear function of each predictor automatically, which can improve prediction substantially over a linear model. Because the model is additive, the effect of each X_j can still be examined with the other variables held fixed, and additivity sidesteps the curse of dimensionality that defeats local regression. The price is that GAMs miss interactions: an effect of X_1 that depends on X_2 must be added by hand, for example as a product term or a two-dimensional smoother $f_{12}(X_1, X_2)$.

Method	Structure	Complexity parameter	Suited for
Polynomial regression	global, smooth	degree d	$p = 1$, mild curvature
Step functions	local, discontinuous	number of cut points	$p = 1$, interpretable bins
Regression / natural splines	local and smooth	number of knots K	$p = 1$ or 2
k-NN, local regression	local, memory based	k or span s	$p \leq 3$ or 4
GAM	additive in the predictors	one per component	many predictors, no interactions

Remark 6.8 (Connection to Kernels and Neural Networks). All basis function methods choose a finite set of features by hand. Two later ideas remove this limitation from opposite directions. Kernel methods (Section 9.2) work with infinitely many basis functions at once, by noticing that a ridge fit in feature space only needs inner products $\phi(x)^\top \phi(x')$; combined with the Bayesian view of Section 5.6, this gives Gaussian processes. Neural networks (Chapter 11) instead *learn* the basis functions from the data, as the hidden units of a network. □

Limitations and what comes next: Linear regression with basis expansions, local regression and GAMs already covers a wide range of regression models, but they were developed mainly for a quantitative response with squared-error loss. Many problems ask instead for a class label: whether a transaction is fraudulent, which digit is written, whether a patient has a disease. Chapter 7 turns to classification, where the squared loss is replaced by the 0-1 loss and the Bayes classifier of Definition 1.5 becomes the target. The same tension between global parametric models (logistic regression) and flexible local ones (k-NN), and the same basis expansions, will reappear there.

Chapter 7

Classification: Logistic Regression and Discriminant Analysis

When the response is qualitative, the target of learning is no longer a conditional mean but the Bayes classifier of Definition 1.5, which assigns each input to its most probable class. Every classifier in this chapter estimates the class probabilities $p_k(x) = \mathbb{P}(Y = k \mid X = x)$ and then plugs them into the Bayes rule. There are two ways to do this. A *discriminative* method models $p_k(x)$ directly; logistic regression (Sections 7.2 and 7.4) is the basic example, and it is fitted by the maximum likelihood principle of Section 2.2 and the gradient methods of Chapter 3. A *generative* method models how the inputs are distributed within each class and inverts this with Bayes' theorem; linear and quadratic discriminant analysis and naive Bayes (Sections 7.5.1 to 7.5.3) are of this type. Section 7.3 explains how to judge a classifier beyond its error rate, and Section 7.6 compares the two routes.

7.1 ▪ The Classification Problem

The response Y takes values in an unordered set $\mathcal{C}$ of K classes, for instance $\{\text{spam}, \text{non-spam}\}$ ($K = 2$, binary classification) or the ten digits ($K = 10$, multi-class classification). Given training data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^p$ and $y_i \in \mathcal{C}$, we want a classifier $\hat{f} \colon \mathbb{R}^p \to \mathcal{C}$ that predicts well, and we would like to understand the role each feature plays in it. A classifier is judged by its expected error rate under the 0-1 loss,

$$\operatorname{Err}(\hat{f}) = \mathbb{E}\big[\mathbf{1}\{Y \neq \hat{f}(X)\}\big] = \mathbb{P}\big(Y \neq \hat{f}(X)\big),$$

where (X, Y) is a new observation independent of the training data.

By Section 1.3, the minimizer of this risk is the Bayes classifier $f^*(x) = \arg\max_k p_k(x)$, its conditional error at x is $1 - \max_k p_k(x)$, and the Bayes error is the average $\mathbb{E}[1 - \max_k p_k(X)]$, which is typically positive. For example, if $K = 3$ and $p_1(x_0), p_2(x_0), p_3(x_0) = 0.5, 0.3, 0.2$, the Bayes classifier predicts class 1 and errs with probability 0.5 at x_0, while guessing a class uniformly at random errs with probability $2/3$. In the binary case $\mathcal{C} = \{0, 1\}$ the Bayes classifier predicts 1 exactly when

$$p(x) := \mathbb{P}(Y = 1 \mid X = x) = \mathbb{E}[Y \mid X = x] \geq \tfrac{1}{2},$$

so learning the Bayes rule amounts to estimating the function $p(x)$.

Since $p(x)$ is a conditional mean, one could estimate it by least squares (Section 4.1) on the 0/1 responses. This is sometimes done in practice, but the fitted values $\hat{\beta}_0 + \hat{\beta}^\top x$ are not confined to $[0, 1]$ and cannot be read as probabilities; with $K > 2$ classes, coding the labels as numbers would moreover impose an order and spacing that the classes do not have. The methods of this chapter respect the structure of the problem.

Classifiers are compared through the training error rate $\frac{1}{n}\sum_i \mathbf{1}\{y_i \neq \hat{f}(x_i)\}$, which is optimistic, and through estimates of the test error from a validation set or cross-validation (Section 4.5), which apply to the 0-1 loss without change. Besides the parametric methods of this chapter, there are nonparametric classifiers: k-nearest neighbors (Algorithm 1.1), support vector machines (Chapter 8), and classification trees (Chapter 10).

7.2 ▪ Logistic Regression

Definition 7.1 (Logistic Regression). *The* logistic function *is* $\sigma(t) = e^t/(1+e^t) = 1/(1+e^{-t})$. *The* logistic regression model *for a binary response* $Y \in \{0,1\}$ *postulates*

$$p(x;\beta) = \mathbb{P}(Y=1 \mid X=x) = \sigma(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p).$$

Equivalently, the log-odds *or* logit *is linear in* x*:*

$$\log \frac{p(x;\beta)}{1-p(x;\beta)} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

□

The odds $p/(1-p)$ range over $(0,\infty)$ and the log-odds over $\mathbb{R}$, so a linear predictor is natural on the logit scale, while $p(x;\beta)$ itself always lies in $[0,1]$. The model is linear neither in x nor in β on the probability scale, but the coefficients keep the interpretation of linear regression one level up: β_j is the change in the log-odds for a unit increase in x_j, the other features held fixed. Given estimates $\hat{\beta}$, the model predicts at three levels: the logit $\hat{\beta}_0 + \hat{\beta}^\top x$, the probability $\hat{p}(x) = \sigma(\hat{\beta}_0 + \hat{\beta}^\top x)$, and the label $\hat{y} = \mathbf{1}\{\hat{p}(x) \geq \frac{1}{2}\}$. The decision boundary $\{x : \hat{\beta}_0 + \hat{\beta}^\top x = 0\}$ is a hyperplane.

7.2.1 ▪ Maximum Likelihood

From here on we absorb the intercept by prepending a 1 to each x_i, so that $\beta \in \mathbb{R}^{p+1}$ and $p(x_i;\beta) = \sigma(x_i^\top \beta)$. The responses are modeled as independent, $y_i \sim \text{Bernoulli}(p(x_i;\beta))$, and β is estimated by maximizing the conditional likelihood of $y_1,\dots,y_n$ given the inputs (Section 2.2):

$$\ell(\beta) = \sum_{i=1}^{n} \Big[y_i \log p(x_i;\beta) + (1-y_i)\log\big(1-p(x_i;\beta)\big)\Big] = \sum_{i=1}^{n} \Big[y_i\, x_i^\top \beta - \log\big(1+e^{x_i^\top \beta}\big)\Big].$$

The second form follows from $\log \frac{p}{1-p} = x_i^\top \beta$ and $\log(1-p) = -\log(1+e^{x_i^\top \beta})$. The negative log-likelihood $-\ell$ is also called the *cross-entropy* or *logistic loss*.

Proposition 7.2 (Gradient and Concavity). *With* $p_i = \sigma(x_i^\top \beta)$, $p = (p_1,\dots,p_n)^\top$, *and* $W = \operatorname{diag}\big(p_i(1-p_i)\big)$,

$$\nabla \ell(\beta) = \sum_{i=1}^{n} (y_i - p_i)\, x_i = X^\top (y-p), \qquad \nabla^2 \ell(\beta) = -X^\top W X.$$

The Hessian is negative semidefinite, so ℓ *is concave and* $-\ell$ *is convex.*

The gradient has an appealing form: each observation pulls β in the direction of its input, weighted by the residual "observed minus predicted". The same form reappears for the multinomial model below and for the output layer of neural networks. Setting the gradient to zero gives equations that have no closed-form solution, so the maximizer is found iteratively. Because $-\ell$ is convex (Section 3.3), gradient descent with a suitable step size converges to a global minimizer.

Algorithm 7.1 Logistic Regression by Gradient Ascent

Input: Data $\{(x_i, y_i)\}_{i=1}^{n}$ with $y_i \in \{0, 1\}$ and x_i including a leading 1; learning rate η; tolerance ϵ; maximum number of iterations M.

Initialize: $\beta^{(0)} \leftarrow 0$.

1: **for** $k = 0, 1, \ldots, M-1$ **do**
2: $\quad p_i \leftarrow \sigma(x_i^\top \beta^{(k)})$ for $i = 1, \ldots, n$ ▷ current fitted probabilities
3: $\quad g \leftarrow \frac{1}{n} \sum_{i=1}^{n} (y_i - p_i)\, x_i$ ▷ $\frac{1}{n} \nabla \ell(\beta^{(k)})$
4: $\quad \beta^{(k+1)} \leftarrow \beta^{(k)} + \eta\, g$ ▷ ascent on ℓ = descent on $-\ell$
5: $\quad$ **if** $\|g\| \le \epsilon$ **then break**
6: $\quad$ **end if**
7: **end for**

Output: $\hat{\beta}$; probabilities $\hat{p}(x) = \sigma(x^\top \hat{\beta})$; classifier $\hat{y}(x) = \mathbf{1}\{\hat{p}(x) \ge t\}$ with threshold $t = \frac{1}{2}$ by default.

This is Algorithm 3.1 applied to $-\ell/n$; dividing by n only rescales the learning rate, which must be tuned as discussed in Section 3.2. For large n the full sum in line 3 is replaced by a minibatch, as in Algorithm 3.2. Other stopping criteria, such as a small change in ℓ between iterations, work equally well.

Remark 7.3 (Separable Data). If the two classes can be separated perfectly by a hyperplane, scaling a separating β up increases ℓ toward its supremum 0, so no finite maximizer exists and the iterates of Algorithm 7.1 diverge. This is the precise version of the remark in the lectures that the estimates are "surprisingly unstable" when the classes are well separated. A ridge penalty $\frac{\lambda}{2}\|\beta\|^2$ on $-\ell$ (Section 5.1) restores a unique finite solution. □

7.2.2 ▪ Inference

When the model is correct, the MLE is consistent, asymptotically normal, $\sqrt{n}(\hat{\beta} - \beta) \to \mathcal{N}(0, \Sigma)$, and asymptotically efficient: no other asymptotically unbiased estimator has a smaller asymptotic covariance. The *z-statistic* $\hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j)$, with the standard error read off the estimated asymptotic covariance, plays the role of the t-statistic of linear regression and gives a p-value for $H_0 : \beta_j = 0$. The interpretation of a coefficient depends on the other features in the model, exactly as in linear regression: in the Default data, being a student raises the estimated probability of default when student status is the only predictor, but its coefficient becomes negative once the credit card balance is included, because students tend to carry larger balances.

Pros and Cons: Logistic regression produces calibrated probabilities, interpretable coefficients with standard errors, and a convex fitting problem with a unique solution in non-separable cases. It makes no assumption about the distribution of X. On the other hand, its decision boundary is linear in the features, it is unstable when the classes are (nearly) separable, and with small n it can be less efficient than a generative model whose assumptions hold.

7.3 ▪ Evaluating a Classifier

The overall error rate treats all mistakes alike, which is often inappropriate. In binary classification, call class 1 "positive" and class 0 "negative". The *confusion matrix* counts the four combinations of predicted and true labels.

	True $Y = 0$ (negative)	True $Y = 1$ (positive)
Predicted $\hat{Y} = 0$	true negatives (TN)	false negatives (FN)
Predicted $\hat{Y} = 1$	false positives (FP)	true positives (TP)

Definition 7.4 (Error Rates). *The* false positive rate *is* $\mathrm{FPR} = \mathrm{FP}/(\mathrm{FP} + \mathrm{TN})$*, the fraction of negatives classified as positive, and the* false negative rate *is* $\mathrm{FNR} = \mathrm{FN}/(\mathrm{FN} + \mathrm{TP})$*, the fraction of positives classified as negative. The* true positive rate *or* sensitivity *is* $\mathrm{TPR} = 1 - \mathrm{FNR}$*, the* specificity *is* $1 - \mathrm{FPR}$*, and the* precision *is* $\mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$. □

For the Default data, a linear classifier with threshold $\frac{1}{2}$ (the lecture attributes these figures to logistic regression; the ISLR textbook reports them for LDA) has a training error of only 2.75%, yet it misses 75.7% of the customers who actually default, while its false positive rate is 0.2%. For a credit card company the missed defaults are the expensive errors. Lowering the threshold, predicting "default" whenever $\hat{p}(x) \geq t$ for some $t < \frac{1}{2}$, trades a lower FNR for a higher FPR; $t = 0$ classifies everyone as positive and $t = 1$ essentially everyone as negative.

Remark 7.5 (Thresholds from Decision Theory). The threshold $\frac{1}{2}$ is optimal for the 0-1 loss. If a false negative costs c times as much as a false positive, the expected loss of predicting 1 at x is $1 - p(x)$ and that of predicting 0 is $c\,p(x)$, so the Bayes rule of Section 1.3 predicts 1 when $p(x) \geq 1/(1 + c)$. Choosing the threshold is therefore the same as choosing the loss. □

The *ROC curve* plots $\mathrm{TPR}(t)$ against $\mathrm{FPR}(t)$ as the threshold t runs from 1 down to 0; it starts at $(0, 0)$ and ends at $(1, 1)$. The *area under the curve* (AUC) summarizes performance over all thresholds: an ideal classifier has $\mathrm{AUC} = 1$, and a classifier that ignores x and predicts 1 with probability q has $\mathrm{FPR} = \mathrm{TPR} = q$, so its ROC curve is the diagonal and its AUC is $\frac{1}{2}$. The ROC curve depends only on the ranking of the scores $\hat{p}(x)$, so it also applies to classifiers that output scores rather than probabilities, such as the SVM of Chapter 8.

7.4 ▪ Multinomial Logistic Regression

For $K > 2$ classes $\{1, \ldots, K\}$, choose a baseline class, here K, and model the log-odds of every other class against it linearly:

$$\log \frac{p_k(x)}{p_K(x)} = x^\top \beta^{(k)}, \qquad k = 1, \ldots, K - 1,$$

with x again including a leading 1. Solving for the probabilities, and setting $\beta^{(K)} = 0$,

$$p_k(x) = \frac{\exp\left(x^\top \beta^{(k)}\right)}{\sum_{l=1}^{K} \exp\left(x^\top \beta^{(l)}\right)} = \operatorname{softmax}\left(x^\top \beta^{(1)}, \dots, x^\top \beta^{(K)}\right)_k. \tag{7.1}$$

The choice of baseline is arbitrary and does not change the fitted probabilities. For $K = 2$ the model reduces to binary logistic regression.

A naive way to fit the model would run $K - 1$ separate binary logistic regressions, the k-th on the observations in classes k and K only. This ignores the data from the other classes, although the events $\{y_i = k\}$ for different k are dependent. The MLE pools all observations. The log-likelihood is

$$\ell\big(\beta^{(1)}, \dots, \beta^{(K-1)}\big) = \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbf{1}\{y_i = k\} \log p_k(x_i),$$

and its gradient has the same "observed minus predicted" form as in Proposition 7.2:

$$\frac{\partial \ell}{\partial \beta^{(k)}} = \sum_{i=1}^{n} \big(\mathbf{1}\{y_i = k\} - p_k(x_i)\big)\, x_i, \qquad k = 1, \dots, K-1.$$

Algorithm 7.1 carries over verbatim: compute all $p_k(x_i)$ from (7.1), and update every $\beta^{(k)}$ with its gradient. Each update now uses the data from all classes. The negative log-likelihood is again convex, so gradient descent finds the MLE. The fitted classifier predicts $\arg\max_k \hat{p}_k(x)$, whose decision boundaries are pieces of hyperplanes.

Remark 7.6 (A Sign Error in Tutorial 7). For the multinomial logit model of the second midterm, the tutorial writes the update as $\hat{\beta}_1^{(t+1)} = \hat{\beta}_1^{(t)} - \alpha \sum_k \big[n_k - n p_k\big] x_k$. The sum is the gradient of the log-likelihood, which is to be maximized, so the correct update has a plus sign (equivalently, a minus sign in front of the gradient of $-\ell$). □

Remark 7.7 (Connection to Neural Networks and Attention). The softmax in (7.1) turns K real scores into a probability vector. It is the output layer of every neural network classifier (Section 11.2), where the linear scores $x^\top \beta^{(k)}$ are computed from learned features instead of raw inputs, and it produces the attention weights of the transformer (Section 17.4). □

7.5 ▪ Generative Classifiers and Discriminant Analysis

Logistic regression models $p_k(x)$ directly. Discriminant analysis instead models the *class-conditional* distributions. Let $\pi_k = \mathbb{P}(Y = k)$ be the prior probability of class k and $f_k(x)$ the density of X given $Y = k$. By Bayes' theorem,

$$p_k(x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^{K} \pi_l f_l(x)}, \tag{7.2}$$

and since the denominator does not depend on k, the Bayes classifier is

$$f^*(x) = \arg\max_k \, \pi_k f_k(x) = \arg\max_k \, \big(\log \pi_k + \log f_k(x)\big).$$

A parametric model for f_k is fitted to the inputs of each class, π_k is estimated by the class frequency, and the estimates are plugged in. The methods below differ only in the model for f_k.

7.5.1 ▪ Linear Discriminant Analysis

Definition 7.8 (LDA). Linear discriminant analysis *assumes Gaussian classes with a common covariance matrix:* $X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma)$ *for* $k = 1, \dots, K$. □

With the Gaussian density of Section 2.5,

$$\begin{aligned}\log \pi_k f_k(x) &= \log \pi_k - \tfrac{1}{2}(x-\mu_k)^\top \Sigma^{-1}(x-\mu_k) + \text{const} \\ &= \log \pi_k + x^\top \Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k - \tfrac{1}{2}x^\top \Sigma^{-1} x + \text{const}.\end{aligned}$$

The term $-\frac{1}{2}x^\top \Sigma^{-1} x$ and the constant are the same for all classes and can be dropped. What remains defines the *discriminant functions*

$$\delta_k(x) = x^\top \Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \log \pi_k, \tag{7.3}$$

and the Bayes classifier is $\arg\max_k \delta_k(x)$. Each δ_k is linear in x, hence the name, and the boundaries $\{\delta_k(x) = \delta_l(x)\}$ are hyperplanes. For $p = 1$ the discriminant is $\delta_k(x) = x\mu_k/\sigma^2 - \mu_k^2/(2\sigma^2) + \log \pi_k$; with two classes, equal priors, and $\mu_0 < \mu_1$, the Bayes boundary is the midpoint $x = (\mu_0 + \mu_1)/2$.

Algorithm 7.2 Linear Discriminant Analysis

Input: Data $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{1, \dots, K\}$.

1: **for** $k = 1, \dots, K$ **do**
2: $\quad n_k \leftarrow \sum_i \mathbf{1}\{y_i = k\}; \quad \hat\pi_k \leftarrow n_k/n$
3: $\quad \hat\mu_k \leftarrow \frac{1}{n_k}\sum_{i:y_i=k} x_i$ ▷ class mean
4: **end for**
5: $\hat\Sigma \leftarrow \frac{1}{n-K}\sum_{k=1}^K \sum_{i:y_i=k}(x_i - \hat\mu_k)(x_i - \hat\mu_k)^\top$ ▷ pooled within-class covariance
6: $\hat\delta_k(x) \leftarrow x^\top \hat\Sigma^{-1}\hat\mu_k - \frac{1}{2}\hat\mu_k^\top \hat\Sigma^{-1}\hat\mu_k + \log \hat\pi_k$ for $k = 1, \dots, K$

Output: Classifier $\hat f(x) = \arg\max_k \hat\delta_k(x)$; probabilities $\hat p_k(x) = \operatorname{softmax}\big(\hat\delta_1(x), \dots, \hat\delta_K(x)\big)_k$.

Fitting is a single pass over the data with no iterative optimization. The probabilities in the output line follow from (7.2), because $\pi_k f_k(x) \propto \exp \delta_k(x)$ with a proportionality factor common to all classes.

Remark 7.9 (MLE versus Unbiased Estimates). The lectures state that the estimates in Algorithm 7.2 are the MLEs. This holds for $\hat\pi_k$ and $\hat\mu_k$, but maximizing the joint likelihood of (x_i, y_i) gives $\hat\Sigma$ with denominator n, as derived in Tutorial 7; the denominator $n - K$ makes the estimate unbiased. The difference is negligible unless n is close to K, and it does not affect the decision boundaries much. □

7.5.2 ▪ Quadratic Discriminant Analysis

Quadratic discriminant analysis (QDA) allows a separate covariance matrix per class, $X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma_k)$. The quadratic term no longer cancels, and

$$\delta_k(x) = -\tfrac{1}{2}x^\top \Sigma_k^{-1} x + x^\top \Sigma_k^{-1}\mu_k - \tfrac{1}{2}\mu_k^\top \Sigma_k^{-1}\mu_k - \tfrac{1}{2}\log|\Sigma_k| + \log \pi_k$$

is quadratic in x, as are the decision boundaries. Fitting follows Algorithm 7.2 with line 5 replaced by the per-class covariances $\hat\Sigma_k = \frac{1}{n_k - 1}\sum_{i:y_i=k}(x_i - \hat\mu_k)(x_i - \hat\mu_k)^\top$.

The price of flexibility is the number of parameters. Besides the $K - 1$ free priors and the pK means, LDA estimates one covariance matrix with $p(p+1)/2$ entries, while QDA estimates K of them,

$Kp(p+1)/2$ in total. When p is large relative to n the QDA covariance estimates are very noisy, and even LDA breaks down once $\hat{\Sigma}$ is singular. This is the bias–variance trade-off of Section 1.5: QDA has lower bias when the class covariances truly differ, LDA has lower variance.

7.5.3 ▪ Naive Bayes

Definition 7.10 (Naive Bayes). *The* naive Bayes *classifier assumes that the features are conditionally independent given the class: $f_k(x) = \prod_{j=1}^{p} f_{kj}(x_j)$, where f_{kj} is the density of X_j given $Y = k$.* □

Each one-dimensional density f_{kj} is estimated separately, from any suitable family: a Gaussian for a quantitative feature, a table of frequencies for a categorical one. Mixed feature types therefore pose no difficulty. With Gaussian $f_{kj} = \mathcal{N}(\mu_{kj}, \sigma_{kj}^2)$, naive Bayes is QDA with diagonal covariances $\Sigma_k = \operatorname{diag}(\sigma_{k1}^2, \dots, \sigma_{kp}^2)$, and

$$\delta_k(x) = -\frac{1}{2}\sum_{j=1}^{p}\frac{(x_j - \mu_{kj})^2}{\sigma_{kj}^2} - \frac{1}{2}\sum_{j=1}^{p}\log \sigma_{kj}^2 + \log \pi_k.$$

Only $2pK$ parameters are needed besides the priors. The independence assumption is rarely true, yet the classifier often works well: it needs only the ranking of $\pi_k f_k(x)$ to be right, not the probabilities themselves, and the variance saved by the assumption usually outweighs its bias when p is large.

Remark 7.11 (Connection to Graphical Models and Mixtures). Naive Bayes is the simplest directed graphical model: the class Y is the single parent of every feature, and the factorization $p(x, y) = \pi_y \prod_j f_{yj}(x_j)$ is read off the graph (Section 12.2). The Gaussian generative classifiers also return in Part III. A Gaussian mixture model (Section 16.1) is exactly the QDA model in which the labels y_i are *not observed*; the EM algorithm (Algorithm 16.1) then replaces the class counts and class means of Algorithm 7.2 by their expectations under the current posterior probabilities (7.2). □

Pros and Cons: Generative classifiers are fitted in closed form, handle many classes naturally, and give the most efficient estimates when their distributional assumptions hold. They also remain stable when the classes are well separated. However, the Gaussian assumption may be badly wrong, for example for skewed or categorical features; QDA needs many parameters; and the probabilities are only as good as the model for f_k.

7.6 ▪ Logistic Regression versus LDA

The two methods lead to the same functional form. For LDA with two classes, subtracting the discriminants (7.3) gives

$$\log\frac{p_1(x)}{p_0(x)} = \delta_1(x) - \delta_0(x) = c_0 + c^\top x,$$

where $c = \Sigma^{-1}(\mu_1 - \mu_0)$ and $c_0 = \log(\pi_1/\pi_0) - \frac{1}{2}(\mu_1 + \mu_0)^\top \Sigma^{-1}(\mu_1 - \mu_0)$, which is exactly the logistic regression model; in general the LDA posterior (7.2) has the softmax form (7.1). The difference lies entirely in how the coefficients are estimated.

Remark 7.12 (Discriminative versus Generative Learning). Logistic regression maximizes the *conditional* likelihood $\prod_i p(y_i \mid x_i)$ and makes no assumption about the distribution of X; this is *discriminative* learning. LDA maximizes the *joint* likelihood $\prod_i p(x_i, y_i) = \prod_i \pi_{y_i} f_{y_i}(x_i)$, which also

uses the information in the marginal distribution of the inputs, at the cost of assuming it is Gaussian; this is *generative* learning. If the Gaussian assumption holds, LDA uses the data more efficiently; if it fails, logistic regression is more robust. The same distinction runs through Part III, where generative models of the data are the main object of study. □

Method	Models	Fitting	Works well when
Logistic regression	$p_k(x)$, softmax-linear	iterative MLE of $p(y \mid x)$	classes overlap; X non-Gaussian
LDA	$X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma)$	closed form, joint MLE	Gaussian classes; small n; many classes
QDA	$X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma_k)$	closed form	covariances differ; n large relative to p^2
Naive Bayes	independent features within class	one density per feature	p large; mixed feature types
k-NN (Algorithm 1.1)	nothing (local vote)	none	nonlinear boundary; p small, n large

Limitations and what comes next: Every method of this chapter goes through an estimate of the class probabilities, and every parametric one produces linear or quadratic decision boundaries. When the goal is only to separate the classes, one can ask for the separating hyperplane directly and choose the one that is geometrically most robust, the one with the widest margin. This leads to the support vector machine of Chapter 8, which replaces the logistic loss by the hinge loss and, through its dual formulation, opens the door to nonlinear boundaries via kernels.

Chapter 8

Support Vector Machines

The classifiers of Chapter 7 draw linear decision boundaries, but they arrive at them indirectly, through a probability model fitted by likelihood. The support vector machine (SVM) asks for the separating hyperplane directly and selects it by a geometric criterion: among all hyperplanes that separate the classes, take the one that stays farthest from the data. Section 8.1 sets up the geometry, Section 8.2 solves the separable case, and Section 8.3 relaxes it to overlapping classes. Section 8.4 shows that the result is a regularized loss minimization problem of the same type as ridge regression and logistic regression, differing only in the loss, and Section 8.5 compares the three linear classifiers. Throughout this chapter the labels are coded as $y \in \{-1, +1\}$, which makes the formulas symmetric in the two classes.

8.1 ▪ Separating Hyperplanes

Logistic regression predicts class 1 when $\beta_0 + \beta^\top x \geq 0$, and binary LDA (Section 7.6) also compares an affine function of x with zero. Both are *linear classifiers* of the form

$$\hat{y}(x) = \operatorname{sign}\big(f(x)\big), \qquad f(x) = \beta_0 + \beta^\top x,$$

with decision boundary the *hyperplane* $\{x \in \mathbb{R}^p : \beta_0 + \beta^\top x = 0\}$, a $(p-1)$-dimensional affine subspace. The vector β is orthogonal to the hyperplane: if x_1, x_2 both lie on it, then $\beta^\top(x_1 - x_2) = 0$. A point is classified correctly exactly when $y\, f(x) > 0$, so the quantity $y_i f(x_i)$ measures how well the i-th observation is classified; it is called the *functional margin*.

A first attempt is to look for (β_0, β) with $\operatorname{sign}(\beta_0 + \beta^\top x_i) = y_i$ for all i. This has two defects. If the data are linearly separable, there are infinitely many such hyperplanes, and some pass very close to the observations, which suggests poor generalization. If the data are not separable, there is no solution at all. The maximal margin principle addresses the first problem, slack variables the second.

Proposition 8.1 (Distance to a Hyperplane). *The Euclidean distance from a point x' to the hyperplane $\{x : \beta_0 + \beta^\top x = 0\}$ is*

$$\frac{|\beta_0 + \beta^\top x'|}{\|\beta\|}.$$

The closest point x_{proj} on the hyperplane differs from x' by a multiple of the unit normal $\beta/\|\beta\|$, and $\beta_0 + \beta^\top x_{\mathrm{proj}} = 0$, so the distance is $|(x' - x_{\mathrm{proj}})^\top \beta|/\|\beta\| = |\beta^\top x' + \beta_0|/\|\beta\|$. For a correctly classified point this equals $y\, f(x')/\|\beta\|$, the functional margin divided by $\|\beta\|$, which is called the *geometric margin*.

8.2 ▪ The Maximal Margin Classifier

Suppose the data are separable. We require every point to be classified correctly with some room to spare,

$$y_i(\beta_0 + \beta^\top x_i) \geq M, \qquad i = 1, \dots, n,$$

for some $M > 0$; these are the *margin constraints*. By Proposition 8.1, every point is then at distance at least $M/\|\beta\|$ from the hyperplane, and we want this distance, the *margin*, to be as large as possible. The pair (β_0, β) is only determined up to a positive factor, since multiplying it by $c > 0$ changes neither the hyperplane nor the classifier but multiplies the functional margins by c. We may therefore fix the scale by setting $M = 1$. The two parallel hyperplanes $\beta_0 + \beta^\top x = \pm 1$ then bound a slab of width $2/\|\beta\|$ that contains no data points, and maximizing this width is the same as minimizing $\|\beta\|^2$.

Definition 8.2 (Hard-Margin SVM). *The* maximal margin classifier *or* hard-margin SVM *is* $\hat{y}(x) = \operatorname{sign}(\hat{\beta}_0 + \hat{\beta}^\top x)$*, where* $(\hat{\beta}_0, \hat{\beta})$ *solves*

$$\min_{\beta_0, \beta} \ \tfrac{1}{2}\|\beta\|^2 \qquad \textit{subject to} \qquad y_i(\beta_0 + \beta^\top x_i) \geq 1, \quad i = 1, \dots, n.$$

The observations with $y_i(\hat{\beta}_0 + \hat{\beta}^\top x_i) = 1$*, which lie on the boundary of the slab, are the* support vectors. □

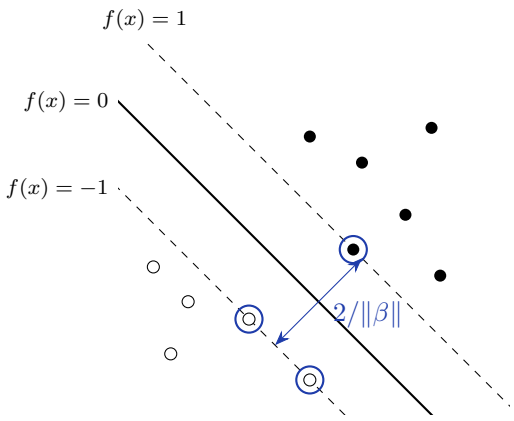


Figure 8.1. *The maximal margin hyperplane* $f(x) = \beta_0 + \beta^\top x = 0$ *(solid) and the margin boundaries* $f(x) = \pm 1$ *(dashed). Filled points have* $y = +1$*, open points* $y = -1$*; the circled points are support vectors. Moving any other point slightly does not change the solution.*

The problem is a convex quadratic program: a convex quadratic objective with linear constraints, so it has a unique solution that standard solvers find efficiently. Its solution depends only on the support vectors. If the constraint of observation i is not tight, removing x_i from the training set leaves the optimal hyperplane unchanged (this can be made rigorous with the Karush–Kuhn–Tucker conditions). The classifier is thus determined by the few points that are hardest to classify, which explains the name.

8.3 ▪ The Soft-Margin SVM

When the classes overlap, no hyperplane satisfies all margin constraints. We allow violations through *slack variables* $\zeta_i \geq 0$ and relax the constraints to $y_i(\beta_0 + \beta^\top x_i) \geq 1 - \zeta_i$. The value of ζ_i records

where observation i lies: $\zeta_i = 0$ if it is on the correct side of the slab, $0 < \zeta_i \le 1$ if it is inside the slab but still correctly classified, and $\zeta_i > 1$ if it is misclassified.

Definition 8.3 (Soft-Margin SVM). *For a budget $B \ge 0$, the* soft-margin SVM *solves*

$$\min_{\beta_0,\beta,\zeta} \tfrac{1}{2}\|\beta\|^2 \quad \textit{subject to} \quad y_i(\beta_0 + \beta^\top x_i) \ge 1 - \zeta_i, \quad \zeta_i \ge 0, \quad \sum_{i=1}^{n} \zeta_i \le B.$$

Equivalently, for a cost parameter $C > 0$ that corresponds to B,

$$\min_{\beta_0,\beta,\zeta} \tfrac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{n} \zeta_i \quad \textit{subject to} \quad y_i(\beta_0 + \beta^\top x_i) \ge 1 - \zeta_i, \quad \zeta_i \ge 0. \tag{8.1}$$

□

Since every misclassified point has $\zeta_i > 1$, the budget bounds the number of training errors by B; $B = 0$ recovers the hard margin. (The lectures call the budget K; we write B to avoid a clash with the number of classes.) The tuning parameter controls a bias–variance trade-off (Section 1.5). A large C (small B) tolerates few violations, gives a narrow margin determined by few support vectors, and has low bias but high variance; a small C gives a wide margin with many support vectors, which is more stable. In practice C is chosen by cross-validation (Section 4.5). The support vectors are now all observations with $\zeta_i > 0$ or on the margin boundary.

Proposition 8.4 (Dual Problem). *The solution of* (8.1) *has the form $\hat\beta = \sum_{i=1}^{n} \hat\alpha_i y_i x_i$, where $\hat\alpha$ solves the dual problem*

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j\, x_i^\top x_j \quad \textit{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0, \quad 0 \le \alpha_i \le C.$$

Moreover $\hat\alpha_i > 0$ only for support vectors, and the fitted decision function is

$$\hat f(x) = \hat\beta_0 + \sum_{i\in\mathcal{S}} \hat\alpha_i y_i\, x_i^\top x, \qquad \mathcal{S} = \{i : \hat\alpha_i > 0\}.$$

The dual is obtained from the Lagrangian of (8.1), with one multiplier α_i per margin constraint, by minimizing over (β_0, β, ζ); the hard margin corresponds to $C = \infty$. Two features of the dual matter for what follows. First, the data enter only through the inner products $x_i^\top x_j$ and $x_i^\top x$. Second, the decision function is a weighted sum over the support vectors only, which is usually a small subset of the data.

Remark 8.5 (Constants in Tutorial 8). Tutorial 8 writes the dual objective with a factor $\frac{1}{4}$ instead of $\frac{1}{2}$. That factor is what one obtains from the primal objective $\|\beta\|^2$ without the $\frac{1}{2}$, as in the lecture slides; with the $\frac{1}{2}\|\beta\|^2$ written in the tutorial's own Lagrangian the factor is $\frac{1}{2}$. Rescaling the objective changes the multipliers by a constant but not the classifier. □

8.4 ▪ The Hinge-Loss View

The slack variables can be eliminated. For fixed (β_0, β), the smallest admissible ζ_i is $\max\{0, 1 - y_i f(x_i)\}$. Substituting this into (8.1) and dividing by nC shows that the soft-margin SVM solves

$$\min_{\beta_0,\beta} \frac{1}{n}\sum_{i=1}^{n} \max\left\{0,\, 1 - y_i(\beta_0 + \beta^\top x_i)\right\} + \lambda\|\beta\|^2, \qquad \lambda = \frac{1}{2nC}. \tag{8.2}$$

(The lectures put $\|\beta\|^2$ instead of $\frac{1}{2}\|\beta\|^2$ in the primal, which gives $\lambda = 1/(nC)$.) The first term is the average *hinge loss* $L_{\text{hinge}}(y, f) = \max\{0, 1 - yf\}$, the second a ridge penalty. So the SVM is a linear classifier fitted by penalized empirical risk minimization, with the same penalty as ridge regression (Section 5.1) and only a different loss. The hinge loss is zero when $yf(x) \geq 1$ (correct and outside the margin), positive but below one when $0 < yf(x) < 1$ (correct but inside the margin), and larger than one for a misclassified point.

Remark 8.6 (Comparing Classification Losses). Each linear classifier we have met minimizes, possibly with a penalty, an average of a loss that depends on the data only through the margin $m = yf(x)$, with $y \in \{-1, +1\}$.

- The *0-1 loss* $\mathbf{1}\{m < 0\}$ is the error rate we care about, but it is discontinuous and non-convex, and minimizing it over hyperplanes is computationally hard.
- The *hinge loss* $\max\{0, 1 - m\}$ of the SVM is a convex upper bound on the 0-1 loss; it is exactly zero for confidently correct points, which is why only the support vectors matter.
- The *logistic loss* $\log(1 + e^{-m})$ is the negative log-likelihood of logistic regression (Section 7.2) rewritten for $y \in \{-1, +1\}$, since $\mathbb{P}(Y = y \mid x) = \sigma(yf(x))$. It is smooth and never exactly zero, so every point influences the fit, and its minimizer estimates the log-odds.
- The *squared loss* $(1 - m)^2 = (y - f(x))^2$ corresponds to least squares on the ± 1 labels; it also penalizes points that are classified correctly with a large margin.

□

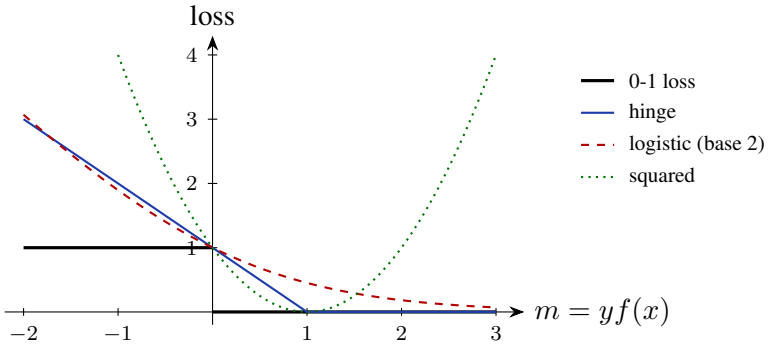


Figure 8.2. *Classification losses as functions of the margin $m = yf(x)$. The logistic loss is drawn in base 2 so that it passes through $(0, 1)$; this rescaling does not change its minimizer.*

The hinge-loss form (8.2) is an unconstrained convex problem, so it can be solved by gradient methods, as the lectures point out. The hinge loss is not differentiable at $m = 1$, but it has a *subgradient*: $-yx$ (and $-y$ for β_0) when $m < 1$, and 0 when $m > 1$; at the kink either choice works. Subgradient descent replaces the gradient of Algorithm 3.1 by a subgradient.

Algorithm 8.1 Linear SVM by Subgradient Descent

Input: Data $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, +1\}$; penalty $\lambda > 0$; step sizes η_t; number of iterations T.
Initialize: $\beta \leftarrow 0$, $\beta_0 \leftarrow 0$.

1: **for** $t = 1, \dots, T$ **do**
2: $\quad \mathcal{V} \leftarrow \{i : y_i(\beta_0 + \beta^\top x_i) < 1\}$ ▷ points violating the margin
3: $\quad g \leftarrow 2\lambda\beta - \frac{1}{n}\sum_{i\in\mathcal{V}} y_i x_i; \quad g_0 \leftarrow -\frac{1}{n}\sum_{i\in\mathcal{V}} y_i$ ▷ subgradient of (8.2)
4: $\quad \beta \leftarrow \beta - \eta_t\, g; \quad \beta_0 \leftarrow \beta_0 - \eta_t\, g_0$
5: **end for**

Output: Classifier $\hat{y}(x) = \operatorname{sign}(\beta_0 + \beta^\top x)$.

Only the observations currently inside the margin or misclassified contribute to the update, mirroring the role of the support vectors; the penalty term shrinks β toward zero in every step, which widens the margin. Because a subgradient does not vanish near the optimum, the step sizes must decrease, typically like $\eta_t \propto 1/t$. Restricting lines 2–3 to a single random observation or a minibatch instead of all n gives the stochastic version (Algorithm 3.2). For moderate n, or when kernels are used (Chapter 9), the dual quadratic program of Proposition 8.4 is usually solved instead.

8.5 ▪ Limitations and Comparison

The SVM outputs only a label $\operatorname{sign}(\hat{f}(x))$. The score $\hat{f}(x)$ can be used for ranking and ROC curves (Section 7.3), but it is not a probability, so the SVM does not estimate $\mathbb{P}(Y = 1 \mid X = x)$ and cannot be used directly with cost-sensitive thresholds as in Remark 7.5. The notion of a margin also does not extend naturally to $K > 2$ classes. The usual workarounds combine binary SVMs, either one-versus-one (fit an SVM for every pair of classes and take a majority vote) or one-versus-all (fit K SVMs, each separating one class from the rest, and pick the class with the largest score).

	Logistic regression	LDA	SVM
Loss / criterion	logistic loss (conditional likelihood)	joint Gaussian likelihood	hinge loss + ridge penalty
Probabilities	yes	yes	no
Assumptions on X	none	Gaussian, common Σ	none
Influential points	all, smoothly weighted	all, through means and Σ	support vectors only
(Nearly) separable classes	unstable	good	good
Overlapping classes	similar to SVM (with ridge penalty)	good if Gaussian	similar to penalized LR
Multi-class	softmax extension	natural	ad hoc combinations

In essence the SVM is closer to logistic regression than to LDA: both are discriminative and differ only in the loss, and the two losses behave similarly for misclassified points (Fig. 8.2). When the classes are nearly separable, the SVM and LDA tend to do better than unpenalized logistic regression; when Gaussianity is justified, LDA is best.

Pros and Cons: The SVM has a clear geometric motivation, a convex training problem with a unique solution, and a sparse solution that depends only on the support vectors, which makes it robust to points far from the boundary. Its dual depends on the data only through inner products, which is the gateway to kernels. On the other hand, it gives no class probabilities, it needs ad hoc extensions for more than two classes, its cost parameter C must be tuned, and the dual quadratic program becomes expensive for very large n.

Limitations and what comes next: A linear SVM still draws a hyperplane, and many problems need curved boundaries. Tutorial 8 gives a simple example: two classes arranged as an inner disk and a surrounding ring cannot be separated by a line in $\mathbb{R}^2$, but after the feature map $\phi(a, b) = (a, b, a^2 + b^2)$ they are separated by a plane in $\mathbb{R}^3$. Running the SVM on $\phi(x_i)$ only requires the inner products $\phi(x_i)^\top \phi(x_j)$ in the dual of Proposition 8.4. Chapter 9 shows that these can be computed without ever forming ϕ, even when the feature space is infinite-dimensional, and that the same trick applies to ridge regression and leads to Gaussian processes.

Chapter 9

Kernel Methods and Gaussian Processes

Two earlier observations point to the same idea. Chapter 6 made linear methods flexible by expanding the input into basis functions, and the more flexibility we want, the more features we need. Chapter 8 showed that the SVM touches the data only through inner products $x_i^\top x_j$. This chapter combines the two. If an algorithm can be written in terms of inner products, then replacing them by inner products of features, $\phi(x_i)^\top \phi(x_j)$, runs the algorithm in feature space, and for many useful feature maps this inner product can be computed directly, without forming ϕ at all, even when the feature space is infinite-dimensional. Section 9.1 explains why explicit features become expensive, Section 9.2 derives kernel ridge regression and the kernel SVM, and Section 9.3 describes which functions qualify as kernels. Section 9.4 takes the Bayesian view of the same model: Bayesian linear regression in feature space turns into a prior over functions, the Gaussian process, whose posterior mean coincides with kernel ridge regression and which in addition quantifies uncertainty.

9.1 ▪ Feature Maps and Their Cost

A *feature map* $\phi\colon \mathbb{R}^p \to \mathbb{R}^M$ turns the linear model into $y = \beta^\top \phi(x) + \varepsilon$, which is linear in the parameters but not in x (Section 6.1). Collect the features of the training inputs in the $n \times M$ matrix Φ whose i-th row is $\phi(x_i)^\top$. Least squares and its relatives then apply with Φ in place of X. The same idea works for classification: two classes arranged as a disk inside a ring in $\mathbb{R}^2$ are not linearly separable, but with $\phi(a, b) = (a, b, a^2 + b^2)$ the third coordinate separates them by a plane.

Explicit feature maps have two drawbacks. First, flexibility may require a number of features M that is large, may need to grow with n, and may exceed n, in which case the least squares solution is not unique and regularization is needed. Second, computing the features is expensive. For $x \in \mathbb{R}^m$ the map that produces all monomials of degree at most two,

$$\phi(x) = \big(x_1^2, \dots, x_p^2,\ \sqrt{2}x_1x_2, \dots, \sqrt{2}x_{p-1}x_p,\ \sqrt{2c}\,x_1, \dots, \sqrt{2c}\,x_p,\ c\big),$$

has $2p + \binom{p}{2} + 1$ coordinates, so evaluating $\phi(x)^\top \phi(x')$ costs $O(p^2)$ operations. Yet a direct calculation shows that

$$\phi(x)^\top \phi(x') = (x^\top x' + c)^2,$$

which costs $O(p)$. For degree d the gap grows to $O(p^d)$ versus $O(p)$. The question is which algorithms need only these inner products.

9.2 ▪ Kernel Ridge Regression and the Kernel Trick

Ridge regression (Section 5.1) in feature space minimizes

$$E(\beta) = \tfrac{1}{2}\|y - \Phi\beta\|^2 + \tfrac{\lambda}{2}\|\beta\|^2, \qquad \beta \in \mathbb{R}^M.$$

Setting the gradient $\Phi^\top\Phi\beta - \Phi^\top y + \lambda\beta$ to zero and rearranging gives

$$\beta = \frac{1}{\lambda}\Phi^\top(y - \Phi\beta) = \Phi^\top a, \qquad a := \frac{1}{\lambda}(y - \Phi\beta) \in \mathbb{R}^n.$$

So the solution is a linear combination of the training features, $\beta = \sum_i a_i\phi(x_i)$. Substituting $\beta = \Phi^\top a$ into the definition of a yields $\lambda a = y - \Phi\Phi^\top a$, that is,

$$a = (K + \lambda I_n)^{-1}y, \qquad K := \Phi\Phi^\top, \quad K_{ij} = \phi(x_i)^\top\phi(x_j).$$

The prediction at a new input is $\hat{y}(x) = \phi(x)^\top\beta = \phi(x)^\top\Phi^\top a$, and $\Phi\phi(x)$ is the vector of inner products $\phi(x_i)^\top\phi(x)$. Every quantity is an inner product of features.

Definition 9.1 (Kernel and Gram Matrix). *The function $k(x, x') = \phi(x)^\top\phi(x')$ is the* kernel *associated with ϕ. For inputs $x_1, \dots, x_n$, the $n \times n$ matrix K with entries $K_{ij} = k(x_i, x_j)$ is the* Gram *or* kernel matrix*, and $k(x) \in \mathbb{R}^n$ denotes the vector with entries $k(x_i, x)$.* □

Proposition 9.2 (Kernel Ridge Regression). *The ridge regression prediction in the feature space of ϕ is*

$$\hat{y}(x) = k(x)^\top(K + \lambda I_n)^{-1}y = \sum_{i=1}^{n} a_i\, k(x_i, x), \qquad a = (K + \lambda I_n)^{-1}y.$$

This is the *dual* or *kernel* form of ridge regression. The primal solution $\beta = (\Phi^\top\Phi + \lambda I_M)^{-1}\Phi^\top y$ requires solving an $M \times M$ system, the dual an $n \times n$ system; the two agree because of the identity $(\Phi^\top\Phi + \lambda I_M)^{-1}\Phi^\top = \Phi^\top(\Phi\Phi^\top + \lambda I_n)^{-1}$. The decisive advantage of the dual form is that it never mentions ϕ: we may choose k directly, as long as it is the inner product of *some* feature map, and work in feature spaces of any dimension. This substitution of a kernel for an inner product is called the *kernel trick*.

Algorithm 9.1 Kernel Ridge Regression

Input: Data $\{(x_i, y_i)\}_{i=1}^n$; kernel k; penalty $\lambda > 0$.

1: $K_{ij} \leftarrow k(x_i, x_j)$ for $i, j = 1, \dots, n$ ▷ $O(n^2)$ kernel evaluations

2: Solve $(K + \lambda I_n)\, a = y$ for a ▷ e.g. by Cholesky; $O(n^3)$

Output: Predictor $\hat{y}(x) = \sum_{i=1}^n a_i\, k(x_i, x)$; each prediction costs n kernel evaluations.

The penalty λ and any parameters of the kernel are tuned by cross-validation (Section 4.5). The cost is governed by n, not by the dimension of the feature space, which is the reverse of ordinary ridge regression: kernels pay off when M is large or infinite and n is moderate.

Remark 9.3 (Kernel Regression as a Weighted Average). The prediction $\hat{y}(x) = \sum_i a_i k(x_i, x)$ combines the training points, weighted by how close x is to each x_i as measured by the kernel. This looks very much like k-nearest neighbors (Algorithm 1.1) and local regression (Section 6.3), with a smooth similarity in place of a hard neighborhood. The weights $a = (K + \lambda I)^{-1}y$, however, are obtained by solving a linear system rather than by normalizing kernel values, so they account for the correlations among the training points. □

Remark 9.4 (Kernelized Least Squares in Tutorial 8). Tutorial 8 writes ordinary least squares in dual form using $(X^\top X)^{-1}X^\top = X^\top(XX^\top)^{-1}$, which requires both matrices to be invertible and hence a square X of full rank. With a ridge penalty the corresponding identity holds for every $\lambda > 0$ and every Φ, which is why the kernel trick for regression is always stated for ridge regression. □

9.2.1 ▪ The Kernel SVM

The SVM dual of Proposition 8.4 also depends on the data only through inner products. Replacing $x_i^\top x_j$ by $k(x_i, x_j)$ gives the *kernel SVM*:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i\alpha_j y_i y_j\, k(x_i, x_j) \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0, \quad 0 \le \alpha_i \le C,$$

with decision function

$$\hat{f}(x) = \hat{\beta}_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i y_i\, k(x_i, x).$$

The decision boundary $\{\hat{f}(x) = 0\}$ is a hyperplane in feature space and a curved surface in input space. For the disk-and-ring example, the kernel of $\phi(a, b) = (a, b, a^2 + b^2)$ is $k(x, x') = x^\top x' + \|x\|^2\|x'\|^2$, computed without forming ϕ.

Remark 9.5 (One Trick, Two Losses). Kernel ridge regression and the kernel SVM both minimize a loss plus a ridge penalty $\|\beta\|^2$ in feature space; the first uses the squared loss, the second the hinge loss (Section 8.4). In both cases the solution is a combination $\sum_i c_i\, k(x_i, \cdot)$ of kernel functions centered at the training points. This is a general fact, known as the representer theorem: for any loss plus a ridge penalty, the minimizer lies in the span of the training features. The hinge loss makes most coefficients zero (only support vectors), the squared loss makes all of them nonzero. □

9.3 ▪ Kernels

Since we want to choose k directly, we need to know which functions are inner products of feature maps.

Definition 9.6 (Positive Semidefinite Kernel). *A symmetric function $k(x, x') = k(x', x)$ is a* kernel *if for every n and all points $x_1, \dots, x_n$ the Gram matrix $K_{ij} = k(x_i, x_j)$ is positive semidefinite: $u^\top K u = \sum_{i,j} u_i u_j k(x_i, x_j) \ge 0$ for all $u \in \mathbb{R}^n$.* □

Every feature map gives a kernel, since $u^\top K u = u^\top \Phi\Phi^\top u = \|\Phi^\top u\|^2 \ge 0$. Conversely, one can show that every kernel in this sense is the inner product of some feature map, possibly into an infinite-dimensional space. So we can forget the feature map, choose a kernel as a measure of similarity between inputs, and let the dimension of the feature space take care of itself. (The tutorial writes the condition as $\sum_{i,j} \alpha_i\alpha_j k(x, x') \ge 0$; the points must vary with the indices, $k(x_i, x_j)$.)

Kernel	$k(x, x')$	Feature space
Linear	$x^\top x'$	$\mathbb{R}^p$; recovers the linear method
Polynomial	$(x^\top x' + c)^d$, $c > 0$	all monomials of degree $\le d$
Gaussian (RBF)	$\exp\big(-\|x - x'\|^2/2\ell^2\big)$	infinite-dimensional
Exponential	$\exp\big(-\|x - x'\|/\ell\big)$	infinite-dimensional; rough functions
Cosine	$x^\top x'/(\|x\|\|x'\|)$	$\phi(x) = x/\|x\|$; no savings

For the Gaussian kernel, write $\exp(-\|x-x'\|^2/2\ell^2) = e^{-\|x\|^2/2\ell^2} e^{-\|x'\|^2/2\ell^2} \exp(x^\top x'/\ell^2)$ and expand the last factor in its Taylor series: it is an infinite sum of polynomial kernels of every degree, so the feature space contains monomials of all degrees. The cosine kernel is a counterexample to the claim that kernels always save computation; its feature map is as cheap as the kernel.

New kernels can be built from old ones. If k_1, k_2 are kernels, then so are ck_1 for $c > 0$, $f(x)k_1(x,x')f(x')$ for any function f, $k_1 + k_2$, $k_1 k_2$, $\exp(k_1)$, $q(k_1)$ for a polynomial q with nonnegative coefficients, and $x^\top A x'$ for a positive semidefinite matrix A. The Gaussian kernel, for example, arises from the linear kernel through the scaling, exponential, and $f(x)k_1(x,x')f(x')$ rules, as its factorization above shows.

Kernels also differ in how far their influence reaches. A *local* kernel such as the Gaussian decays with distance, so a prediction depends mostly on nearby training points, and the bandwidth ℓ plays the role of k in k-NN. A *global* kernel such as the polynomial kernel lets distant points interact. Closely related are *radial basis function* expansions, $f(x) = \sum_{i=1}^{n} w_i\, h(\|x - x_i\|)$ with one basis function centered at every data point, for instance a Gaussian bump h, fitted by least squares; with h a Gaussian this has the form of the kernel ridge predictor with the Gaussian kernel, and it coincides with it when the weights are penalized by $\lambda\, w^\top K w$ (a plain ridge penalty $\lambda\|w\|^2$ gives a different fit). Fitting noisy samples of a sine curve this way produces a smooth interpolant whose wiggliness is controlled by ℓ and λ.

Remark 9.7 (Fixed versus Learned Features). A kernel fixes the features in advance through the choice of k; only the weights are learned. A neural network (Chapter 11) learns the features: its hidden layers compute $h = \phi_\theta(x)$, and its last layer is a linear model $w^\top h + b$ for regression or a logistic model $\sigma(w^\top h + b)$ for binary classification, exactly the models of Chapters 4 and 7 applied to learned features. □

9.4 ▪ Gaussian Processes

Kernel ridge regression gives a point prediction. The Bayesian treatment of the same model also gives its uncertainty, and leads to a way of specifying a prior directly over functions.

9.4.1 ▪ From Bayesian Linear Regression to a Prior over Functions

Consider Bayesian linear regression (Section 5.6) in feature space: $y = f(x) + \varepsilon$ with $f(x) = \beta^\top \phi(x)$, noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, and prior $\beta \sim \mathcal{N}(0, \tau^2 I_M)$. The prior on β induces a distribution on the vector of function values at the training inputs, $f = (f(x_1), \dots, f(x_n))^\top = \Phi\beta$. As a linear transformation of a Gaussian vector it is Gaussian (Section 2.5):

$$f \sim \mathcal{N}\big(0, \tau^2 \Phi\Phi^\top\big), \qquad \operatorname{Cov}\big(f(x_i), f(x_j)\big) = \tau^2 \phi(x_i)^\top \phi(x_j) = \tau^2 k(x_i, x_j).$$

The parameters have disappeared: the prior on β is equivalent to a joint Gaussian distribution on the function values at any finite set of inputs, with covariance given by a kernel. We can therefore skip the feature map and the parameters and place the prior on f directly. From now on we absorb τ^2 into the kernel.

Definition 9.8 (Gaussian Process). *A* Gaussian process *is a distribution over functions f such that for every $n \geq 1$ and every set of inputs $x_1, \dots, x_n$ the vector $(f(x_1), \dots, f(x_n))$ is jointly Gaussian. It is specified by a mean function $m(x) = \mathbb{E}[f(x)]$ and a covariance function $k(x, x') = \operatorname{Cov}(f(x), f(x'))$, which must be a kernel; we write $f \sim \mathcal{GP}(m, k)$. In most applications $m = 0$.* □

The kernel encodes our prior beliefs about f. Samples from a GP with the Gaussian kernel are smooth, those with the exponential kernel are continuous but rough, and the bandwidth sets the length scale on which the function varies.

9.4.2 ▪ Gaussian Process Regression

Let $f \sim \mathcal{GP}(0, k)$ and $y_i = f(x_i) + \varepsilon_i$ with independent $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The responses $y = (y_1, \dots, y_n)^\top$ combine two independent Gaussian sources of randomness, so their marginal distribution is

$$y \sim \mathcal{N}(0, C), \qquad C = K + \sigma^2 I_n,$$

where K is the Gram matrix of the training inputs. To predict the response y_* at a new input x_*, append it to the vector:

$$\begin{pmatrix} y \\ y_* \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} C & k_* \\ k_*^\top & c_* \end{pmatrix}\right), \qquad k_* = \big(k(x_i, x_*)\big)_{i=1}^n, \quad c_* = k(x_*, x_*) + \sigma^2.$$

The inputs are treated as fixed throughout.

Proposition 9.9 (GP Predictive Distribution). *Given the training responses y, the response at x_* is Gaussian,*

$$y_* \,|\, y \sim \mathcal{N}\big(\mu_*,\ s_*^2\big), \qquad \mu_* = k_*^\top C^{-1} y, \qquad s_*^2 = c_* - k_*^\top C^{-1} k_*.$$

For the noise-free function value $f(x_*)$, the mean is the same and the variance is $s_*^2 - \sigma^2$.

This is the conditioning formula for jointly Gaussian vectors, Proposition 2.14, applied to the block covariance above. The mean is a linear combination of the observed responses, and both the mean and the variance depend on x_* only through kernel evaluations. The variance is the prior variance minus a nonnegative term that measures how much the training data inform x_*: near the data the term is large and the variance small; far away $k_* \approx 0$ for a local kernel, and the variance returns to its prior level.

Algorithm 9.2 Gaussian Process Regression

Input: Data $\{(x_i, y_i)\}_{i=1}^n$; kernel k; noise variance σ^2; test inputs $x_{*,1}, \dots, x_{*,r}$.

$C \leftarrow K + \sigma^2 I_n$ with $K_{ij} = k(x_i, x_j)$

Compute the Cholesky factor L with $C = LL^\top$ ▷ $O(n^3)$, once

$a \leftarrow C^{-1} y$ by two triangular solves with L

for $j = 1, \dots, r$ **do**

  $k_* \leftarrow \big(k(x_i, x_{*,j})\big)_{i=1}^n$

  $\mu_{*,j} \leftarrow k_*^\top a$ ▷ predictive mean

  $v \leftarrow L^{-1} k_*$; $s_{*,j}^2 \leftarrow k(x_{*,j}, x_{*,j}) + \sigma^2 - v^\top v$ ▷ predictive variance

end for

$\log p(y) \leftarrow -\frac{1}{2} y^\top a - \sum_i \log L_{ii} - \frac{n}{2}\log(2\pi)$ ▷ log marginal likelihood, (9.1)

Output: Predictive means $\mu_{*,j}$ and variances $s_{*,j}^2$ (subtract σ^2 for the noise-free f); $\log p(y)$.

A pointwise interval $\mu_* \pm 2 s_*$ gives an approximate 95% predictive band, which widens between and beyond the observations. As with kernel ridge regression, training costs $O(n^3)$ and each prediction $O(n)$ for the mean and $O(n^2)$ for the variance.

Remark 9.10 (GP Regression Is Bayesian Kernel Ridge Regression). The predictive mean $\mu_* = k_*^\top (K + \sigma^2 I)^{-1} y$ is exactly the kernel ridge prediction of Proposition 9.2 with penalty $\lambda = \sigma^2$. This parallels Section 5.5, where ridge regression appeared as the posterior mode under a Gaussian prior; for Gaussian posteriors the mode and the mean coincide. The Gaussian process adds what ridge regression lacks, the predictive variance, just as Bayesian linear regression adds a predictive variance to ridge regression (Proposition 5.9). With the linear kernel, GP regression *is* Bayesian linear regression. □

9.4.3 ▪ Learning the Hyperparameters

So far nothing has been learned except through the choice of the kernel. In practice the kernel belongs to a parametric family, such as the Gaussian kernel with an amplitude and a bandwidth, and together with σ^2 its parameters form a vector θ of hyperparameters. Since $y \sim \mathcal{N}(0, C_\theta)$, they can be estimated by maximizing the *log marginal likelihood*

$$\log p(y \mid \theta) = -\frac{1}{2} y^\top C_\theta^{-1} y - \frac{1}{2} \log |C_\theta| - \frac{n}{2} \log(2\pi), \tag{9.1}$$

by gradient-based optimization (Chapter 3) or grid search. The first term rewards fitting the data, the second penalizes covariances that are too flexible, so the criterion balances fit against complexity without a validation set. This is the GP counterpart of choosing λ for kernel ridge regression by cross-validation, and it is an instance of maximizing the evidence, which reappears in Chapter 16.

9.4.4 ▪ Gaussian Processes for Classification

For a binary response $y \in \{0, 1\}$, place a GP prior on a latent function $a(x)$ and squash it, $\mathbb{P}(y = 1 \mid a) = \sigma(a(x))$, as in logistic regression (Section 7.2). The process $\sigma(a(x))$ takes values in $(0, 1)$ but is no longer Gaussian. The predictive probability

$$p(y_* \mid y) = \int p(y_* \mid a_*)\, p(a_* \mid y)\, \mathrm{d}a_*$$

requires the posterior of the latent values given binary observations, which has no closed form because the Bernoulli likelihood is not Gaussian. It must be approximated, by Markov chain Monte Carlo (Chapter 14), by numerical integration, or by fitting a Gaussian approximation to the posterior in the spirit of Chapter 15. In practice a small jitter $\nu\, \delta_{ij}$ is added to the covariance of a for numerical stability.

	Bayesian linear regression	Kernel ridge regression	GP regression
Model	$\beta^\top \phi(x)$, Gaussian prior on β	$\beta^\top \phi(x)$, ridge penalty	$f \sim \mathcal{GP}(0, k)$
Output	posterior mean and variance	point prediction	posterior mean and variance
Linear algebra	$M \times M$	$n \times n$	$n \times n$
Tuning	prior and noise variances	λ and kernel by CV	θ by marginal likelihood

Pros and Cons: Kernel methods turn linear algorithms into flexible nonlinear ones at the cost of choosing a kernel, and Gaussian processes add calibrated uncertainty and a principled way to tune hyperparameters, all with closed-form formulas for regression. They work best with moderate n and when a good notion of similarity between inputs is available. However, training scales as $O(n^3)$ in time and $O(n^2)$ in memory, predictions require all training points, the results depend heavily on the kernel, and classification with a GP needs approximate inference.

Limitations and what comes next: Kernel methods and Gaussian processes are smoothers: the prediction at x is driven by training points that are similar to x under a similarity chosen in advance, and every prediction involves the whole training set. In many applications the relevant notion of similarity is not known beforehand, varies across the input space, or depends on only a few of many features. Decision trees (Chapter 10) take a different route: they learn an adaptive partition of the input space from the data and predict by a constant on each cell, and their ensembles are among the most accurate off-the-shelf predictors. Neural networks (Chapter 11) go further and learn the features themselves.

Chapter 10

Trees and Ensembles

The methods of Chapters 4 to 9 all describe the regression function or the decision boundary by a single global formula: a linear combination of the predictors, of basis functions, or of kernel evaluations. Tree-based methods take a different route. They partition the feature space into rectangles and predict a constant in each one, and the partition itself is learned from the data. A single tree is easy to explain but unstable: a small change in the training data can produce a very different tree. The second half of the chapter shows how combining many trees fixes this, at the price of interpretability. Sections 10.1 to 10.3 build, prune, and adapt single trees; Section 10.5 introduces the bootstrap, a general resampling tool; Sections 10.6 and 10.7 use it to average many trees; and Section 10.8 grows trees sequentially, each one correcting the errors of the current ensemble.

10.1 ▪ Regression Trees

A decision tree makes a prediction by asking a sequence of questions about the features. Each internal node tests one feature, $x_j < s$ for a continuous feature or membership in a subset of levels for a qualitative one; the branches partition the range of that feature; and each leaf (terminal node) stores a prediction. Every root-to-leaf path defines a region of the feature space, and the regions of all leaves form a partition.

Definition 10.1 (Tree Predictor). *A tree with J leaves partitions the feature space into disjoint rectangles $R_1, \dots, R_J$ and predicts*

$$\hat{f}(x) = \sum_{m=1}^{J} c_m \, \mathbf{1}\{x \in R_m\},$$

where, for the training observations falling in R_m, $c_m = \bar{y}_{R_m}$ is their mean response (regression tree) or their most common class (classification tree). □

The Hitters example of the lectures predicts a baseball player's salary from *Years* and *Hits*. The fitted tree splits first on Years < 4.5 and then, for experienced players only, on Hits < 117.5: experience matters most, and the number of hits matters only once a player is established. A linear model cannot express such an interaction as directly, and the tree can be read without any statistics.

Ideally we would choose the rectangles to minimize the residual sum of squares $\sum_{m=1}^{J} \sum_{i: x_i \in R_m} (y_i - \bar{y}_{R_m})^2$, but enumerating all partitions is infeasible. Trees are therefore grown by a *top-down, greedy* procedure: start with the whole space and repeatedly split one region into two, choosing at each step the split that helps most *now*, without looking ahead. For a region R with n_R observations write $Q(R)$ for its *impurity*; for regression $Q(R) = \frac{1}{n_R} \sum_{i: x_i \in R} (y_i - \bar{y}_R)^2$, so that $n_R Q(R)$ is the RSS of the region.

The impurities used for classification appear in Section 10.3. A split of R on feature j at cutpoint s produces

$$R_1(j,s) = \{x \in R : x_j < s\}, \qquad R_2(j,s) = \{x \in R : x_j \geq s\},$$

and we choose (j,s) to minimize $n_{R_1}Q(R_1) + n_{R_2}Q(R_2)$. For each feature only the (at most $n_R - 1$) midpoints between consecutive observed values need to be tried, so the search is cheap.

Algorithm 10.1 Growing a Tree by Recursive Binary Splitting

Input: Training data $\{(x_i, y_i)\}_{i=1}^n$, impurity Q, minimum leaf size $n_{\min}$.
Initialize: One leaf $R = \mathbb{R}^p$ containing all observations; list of leaves $\mathcal{L} \leftarrow \{R\}$.
while some leaf in $\mathcal{L}$ has more than $n_{\min}$ observations and can be split **do**
  Pick such a leaf R.
  for $j = 1, \ldots, p$ and each midpoint s between consecutive values of x_{ij}, $x_i \in R$ **do**
    Form $R_1(j,s)$, $R_2(j,s)$ and compute the cost $n_{R_1}Q(R_1) + n_{R_2}Q(R_2)$.
  end for
  $(j^*, s^*) \leftarrow$ the pair with the smallest cost.
  Replace R in $\mathcal{L}$ by $R_1(j^*, s^*)$ and $R_2(j^*, s^*)$. ▷ one new branch pair
end while
Output: The tree T_0 with leaves $R_1, \ldots, R_J$; $\hat{f}(x) = \bar{y}_{R_m}$ (or the majority class) for $x \in R_m$.

Recursive binary splitting can only produce partitions that are obtained by cutting a box into two boxes along one axis, again and again. Many rectangular partitions, such as a "pinwheel" arrangement of five boxes, cannot arise this way; this restriction is what makes the result a tree.

Remark 10.2 (Connection to Step Functions and k-NN). A tree is a step-function regression (Section 6.1) whose steps are chosen by the data rather than fixed in advance, and in several dimensions at once. It is also a local averaging method like k-nearest neighbors (Section 1.2.1): the prediction at x is the average response over a neighborhood of x. The difference is that the neighborhoods of a tree adapt to the response, becoming small where f changes quickly and large where it is flat, and they ignore features that never appear in a split. □

10.2 ▪ Pruning

Grown until every leaf holds a single observation, a tree interpolates the training data: its training error is zero and its variance is enormous. A smaller tree has lower variance and is easier to interpret, at the cost of some bias (Section 1.5). Stopping the growth early is short-sighted, because a split that looks useless may enable a very good split below it. The standard strategy is to grow a large tree T_0 and then *prune* it back.

Definition 10.3 (Cost Complexity Criterion). *For a subtree $T \subseteq T_0$ with $|T|$ leaves $R_1, \ldots, R_{|T|}$ and a tuning parameter $\alpha \geq 0$,*

$$C_\alpha(T) = \sum_{m=1}^{|T|} n_{R_m} Q(R_m) + \alpha|T|.$$

T_α denotes the subtree of T_0 that minimizes C_α. □

The penalty $\alpha|T|$ plays the role of the penalties in C_p and AIC (Section 4.6) and in the lasso (Section 5.2): $\alpha = 0$ returns T_0, and larger α favors smaller trees. As α increases, branches are removed

in a nested and predictable order, so the whole family $\{T_\alpha\}$ is cheap to compute. This is *weakest-link pruning*: repeatedly collapse the internal node whose removal increases the training cost least per leaf removed. The value of α is chosen by cross-validation (Section 4.5).

Algorithm 10.2 Building a Regression Tree with Cost Complexity Pruning

Input: Training data, number of folds K, grid of values of α.

1: Grow a large tree T_0 on all training data with Algorithm 10.1.
2: Apply weakest-link pruning to T_0 to obtain the best subtree T_α for every α in the grid.
3: Split the training data into K folds.
4: **for** $k = 1, \dots, K$ **do**
5: Repeat lines 1–2 on all folds except fold k.
6: Compute the mean squared prediction error $E_k(\alpha)$ on fold k for every α.
7: **end for**
8: $\hat{\alpha} \leftarrow \arg\min_\alpha \frac{1}{K} \sum_{k=1}^{K} E_k(\alpha)$.

Output: The subtree $T_{\hat{\alpha}}$ from line 2.

Cross-validation is applied to the one-dimensional parameter α, not to every possible subtree, of which there are far too many. In the Hitters example the cross-validated error is minimized by a tree with only three leaves, the same three regions described above.

10.3 ▪ Classification Trees

A classification tree predicts the most common class among the training observations in a leaf, and it is grown by the same recursive splitting. Only the impurity changes. Let $\hat{p}_{mk} = \frac{1}{n_{R_m}} \sum_{i : x_i \in R_m} \mathbf{1}\{y_i = k\}$ be the proportion of class k in region R_m.

Definition 10.4 (Node Impurities). *For a region R_m with class proportions $\hat{p}_{m1}, \dots, \hat{p}_{mK}$:*

(i) the misclassification rate $E_m = 1 - \max_k \hat{p}_{mk}$*;*

(ii) the Gini index $G_m = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$*;*

(iii) the deviance *or* entropy $D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$.

All three vanish when a node is pure (one $\hat{p}_{mk}$ equals 1) and are largest when the classes are equally mixed. □

Entropy measures the uncertainty of a random variable: a fair coin has entropy 1 bit, a coin with $\mathbb{P}(\text{heads}) = 8/9$ about 0.5 bit, and a deterministic coin 0. A split is good if the children are, on average, more certain than the parent. The lectures illustrate this with seven fruits, five oranges and two lemons, and two candidate splits. Split A sends two oranges to the left and $(3, 2)$ to the right; split B sends $(3, 1)$ to the left and $(2, 1)$ to the right. Both misclassify two fruits. Weighting each child by its share of the observations gives:

Impurity	Parent $(5, 2)$	Split A: $(2, 0) \mid (3, 2)$	Split B: $(3, 1) \mid (2, 1)$
Misclassification rate	0.286	0.286	0.286
Gini index	0.408	0.343	0.405
Entropy (bits)	0.863	0.694	0.857

The misclassification rate cannot tell the splits apart, whereas the Gini index and the entropy both prefer split A, which creates a pure node. The decrease in entropy, here $0.863 - 0.694 \approx 0.17$ bits for split A against 0.006 for split B, is called the *information gain*.

Remark 10.5 (Weighting the Children). The slides compare the splits by the *unweighted sum* of the two children's entropies (1.73 for B, 0.97 for A). The criterion used in Algorithm 10.1 weights each child by its number of observations, $n_{R_1}Q(R_1) + n_{R_2}Q(R_2)$; otherwise a split that isolates a single observation in a pure leaf would look artificially good. In this example both versions prefer split A. □

The Gini index and the deviance are numerically similar and more sensitive to node purity than the misclassification rate, so either is used to grow the tree. Any of the three can be used for pruning; if prediction accuracy is the final goal, the misclassification rate is the natural choice. In the Heart data (303 patients, 13 features, predicting heart disease), cross-validation selects a pruned tree with six leaves.

10.4 ▪ Trees Versus Linear Models

A linear model assumes $f(x) = \beta_0 + \sum_j \beta_j x_j$, while a tree assumes $f(x) = \sum_m c_m \mathbf{1}\{x \in R_m\}$. If the true relationship is close to linear, the tree wastes many splits approximating a plane by a staircase; if it is highly nonlinear with interactions, the tree can win. Which assumption fits better is an empirical question, answered by estimating the test error with cross-validation or a validation set.

Pros and Cons: Trees are easy to explain, arguably closer to human decision-making than regression, can be displayed graphically, and handle qualitative predictors without dummy variables. They are invariant to monotone transformations of the features. However, a single tree is usually less accurate than the methods of the previous chapters, and it is not robust: a small change in the data can change the first split and hence the entire tree. The rest of the chapter exploits exactly this high variance: averaging many different trees reduces it.

10.5 ▪ The Bootstrap

Before combining trees we need a way to create many training sets from one. The bootstrap does this, and it is useful far beyond trees: it quantifies the uncertainty of almost any statistical procedure.

Consider the lectures' portfolio example. We invest a fraction α in an asset with return X and $1 - \alpha$ in an asset with return Y. The variance $\mathrm{Var}(\alpha X + (1-\alpha)Y)$ is minimized by

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}},$$

and from 100 past observations we estimate $\hat{\alpha}$ by plugging in sample variances and the sample covariance. How variable is $\hat{\alpha}$? If we could draw new data sets from the population, we would compute $\hat{\alpha}$ on each of, say, 1000 simulated data sets and take the sample variance. This *oracle* approach is impossible in practice, because the population distribution is unknown. The bootstrap replaces the unknown population by the data themselves.

Algorithm 10.3 Bootstrap Estimate of Uncertainty

Input: Data $Z = (z_1, \dots, z_n)$, an estimator $\hat{\theta} = s(Z)$, number of bootstrap replicates B.

1: **for** $b = 1, \dots, B$ **do**
2: Draw n indices uniformly from $\{1, \dots, n\}$ *with replacement*; let Z^{*b} be the corresponding observations.
3: $\hat{\theta}^{*b} \leftarrow s(Z^{*b})$
4: **end for**
5: $\bar{\theta}^{*} \leftarrow \frac{1}{B} \sum_{b=1}^{B} \hat{\theta}^{*b}$

Output: $\widehat{\mathrm{SE}}(\hat{\theta}) = \left(\frac{1}{B-1} \sum_{b=1}^{B} (\hat{\theta}^{*b} - \bar{\theta}^{*})^2 \right)^{1/2}$; more generally, the empirical distribution of $\hat{\theta}^{*1}, \dots, \hat{\theta}^{*B}$ (its quantiles, histogram) as an estimate of the sampling distribution of $\hat{\theta}$.

A bootstrap sample has the same size as the original data, so some observations appear several times and others not at all. In the portfolio example the histogram of 1000 bootstrap estimates is very close to the histogram from 1000 truly new data sets. The same recipe applies to OLS (Section 4.1): resample the pairs (x_i, y_i), recompute $\hat{\beta}^{*b} = (X^{*b\top} X^{*b})^{-1} X^{*b\top} y^{*b}$, and read off the standard error, quantiles, or whole distribution of each $\hat{\beta}_j$. The textbook formula $\mathrm{Cov}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1}$ relies on the linear model being correct; the bootstrap does not.

Remark 10.6 (Connection to Monte Carlo). The oracle approach is plain Monte Carlo estimation (Section 14.2) under the true distribution F of the data. The bootstrap is Monte Carlo under the *empirical distribution* $\hat{F}_n$, which puts mass $1/n$ on each observation; sampling n points from $\hat{F}_n$ is exactly sampling with replacement. Two approximations are involved: $\hat{F}_n \approx F$, a statistical error that does not shrink as B grows, and the finite number of replicates, a Monte Carlo error of order $B^{-1/2}$ that can be made as small as computing time allows. □

A fact used below: a given observation is left out of a bootstrap sample with probability $(1-1/n)^n \to e^{-1} \approx 0.37$, so each bootstrap sample contains about two thirds of the distinct observations.

10.6 ▪ Bagging

Averaging reduces variance without adding bias. If $\hat{\beta}^{(1)}, \dots, \hat{\beta}^{(N)}$ are *independent* estimates of β, each with $|\mathbb{E}[\hat{\beta}^{(i)}] - \beta| \le b$ and variance σ^2, then their average $\bar{\beta}$ still has bias at most b but variance σ^2/N. With B independent training sets we could grow B trees and average them. We have only one training set, so *bootstrap aggregation*, or *bagging*, uses bootstrap samples instead.

Algorithm 10.4 Bagging Trees

Input: Training data $\{(x_i, y_i)\}_{i=1}^{n}$, number of trees B.

for $b = 1, \dots, B$ **do**
  Draw a bootstrap sample $\mathcal{D}^{*b}$ of size n (Algorithm 10.3); record the out-of-bag set $O_b = \{i : (x_i, y_i) \notin \mathcal{D}^{*b}\}$.
  Grow a deep tree $\hat{f}^{*b}$ on $\mathcal{D}^{*b}$ with Algorithm 10.1, without pruning.
end for
for $i = 1, \dots, n$ **do** ▷ out-of-bag error
  $\hat{y}_i^{\text{oob}} \leftarrow$ average (regression) or majority vote (classification) of $\hat{f}^{*b}(x_i)$ over $\{b : i \in O_b\}$.
end for

Output: $\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$, or the majority vote of the B trees for classification; OOB error: the MSE or misclassification rate of $\hat{y}^{\text{oob}}$ against y.

The trees are grown deep and not pruned: each has low bias and high variance, and averaging attacks the variance. Each observation is out of bag for about a third of the trees, so its OOB prediction comes from trees that never saw it. The OOB error is therefore a valid estimate of the test error, obtained for free without cross-validation; with large B it is essentially leave-one-out cross-validation for the bagged model.

Bagging gives up the interpretability of a single tree. A summary is the *variable importance* of each predictor: the total decrease in RSS (regression) or in the Gini index (classification) due to splits on that predictor, averaged over the B trees and often expressed relative to the largest value.

Remark 10.7 (Connection to the Bias–Variance Trade-off). By Theorem 1.6, the expected test error is the irreducible noise plus squared bias plus variance. A single tree chooses its size to balance the last two. Bagging changes the trade-off: because averaging removes much of the variance, each tree can be grown to have very low bias. Bagging helps unstable, high-variance procedures such as deep trees and does little for stable ones such as OLS. □

10.7 ▪ Random Forests

The bootstrap samples overlap, so bagged trees are not independent. If B identically distributed predictions each have variance σ^2 and pairwise correlation ρ, their average has variance

$$\rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

The second term vanishes as $B \to \infty$, but the first does not: correlation sets a floor on what averaging can achieve. Bagged trees are often strongly correlated. If one predictor is very strong, almost every tree uses it for the first split, and the trees look alike. Random forests decorrelate the trees by restricting the features each split may use.

Algorithm 10.5 Random Forest

Input: Training data, number of trees B, number of split candidates $m \le p$ (commonly $m \approx \sqrt{p}$).

1: **for** $b = 1, \dots, B$ **do**
2: Draw a bootstrap sample $\mathcal{D}^{*b}$.
3: Grow a deep tree on $\mathcal{D}^{*b}$ with Algorithm 10.1, except that *at each split* a fresh random subset of m of the p predictors is drawn and only these are considered.
4: **end for**

Output: The average (regression) or majority vote (classification) of the B trees, with OOB error as in Algorithm 10.4.

With $m = p$ a random forest is bagging. With $m < p$, many splits do not even consider the strong predictor, other predictors get a chance, and the trees become less similar; this lowers ρ at the price of slightly more bias per tree. In the gene expression example of the lectures (349 patients, 15 classes, the 500 genes of highest variance), random forests with $m < p$ improve slightly on bagging, and both are far better than a single tree, whose error rate is 45.7%. Neither bagging nor random forests overfit as B grows: B only needs to be large enough for the error to settle.

10.8 ▪ Boosting

Boosting also combines many trees, but it builds them *sequentially*, without bootstrap sampling: each tree is fit to a modified version of the data that reflects what the current ensemble gets wrong. Instead of fitting one large tree, which could overfit, boosting learns slowly by adding many small trees, each shrunk toward zero.

Algorithm 10.6 Boosting for Regression Trees

Input: Training data $\{(x_i, y_i)\}_{i=1}^n$, number of trees B, shrinkage $\lambda > 0$, number of splits per tree d.
Initialize: $\hat{f}(x) \leftarrow 0$ and residuals $r_i \leftarrow y_i$ for all i.

1: **for** $b = 1, \dots, B$ **do**
2: Fit a tree $\hat{f}^b$ with d splits ($d + 1$ leaves) to the data $\{(x_i, r_i)\}_{i=1}^n$.
3: $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$ ▷ add a shrunken copy
4: $r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$ for all i. ▷ update the residuals
5: **end for**

Output: The boosted model $\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x)$.

Each tree is small and fit to the residuals, so it improves $\hat{f}$ a little in the regions where the current fit is poor. The method has three tuning parameters.

(i) The number of trees B. Unlike bagging and random forests, boosting can overfit if B is too large, although slowly; B is chosen by cross-validation.

(ii) The shrinkage λ, a small positive number such as 0.01 or 0.001. It controls the rate of learning; a smaller λ requires a larger B.

(iii) The number of splits d, which controls the complexity of each tree and the order of interactions the ensemble can represent. Often $d = 1$ works well: each tree is a *stump* with a single split, and the boosted model is a sum of functions of one variable each, that is, an additive model as in Section 6.4.

Remark 10.8 (Connection to Gradient Descent). For the squared loss $\frac{1}{2}\sum_i (y_i - F_i)^2$, viewed as a function of the vector of fitted values $F_i = \hat{f}(x_i)$, the negative gradient with respect to F_i is exactly the residual $r_i = y_i - F_i$. Lines 2–3 of Algorithm 10.6 therefore fit a tree to the negative gradient and take a step of size λ in that direction: boosting is gradient descent (Algorithm 3.1) in the space of functions, with the tree serving as a regularized, generalizable approximation of the gradient and λ as the learning rate. Replacing the squared loss by another differentiable loss, such as the logistic loss, gives *gradient boosting*, the idea behind libraries such as XGBoost. □

In the gene expression data, boosting with stumps ($\lambda = 0.01$) slightly outperforms depth-two trees and random forests, although the differences are within their standard errors.

Method	How the trees are built	Main effect	Tuning
Single tree	one tree, grown and pruned	interpretable; high variance	α by CV
Bagging	deep trees on bootstrap samples, in parallel	reduces variance	B (large enough)
Random forest	as bagging, m random candidates per split	reduces variance further by decorrelating	B, m
Boosting	small trees fit to residuals, in sequence	reduces bias step by step	B by CV, λ, d

Pros and Cons: Random forests and boosting are among the most accurate off-the-shelf methods for tabular data. They need little preprocessing, handle mixed feature types and interactions automatically, and come with built-in error estimates (OOB) and importance measures. However, a model made of hundreds of trees is hard to interpret, and boosting must be tuned with care to avoid overfitting.

Limitations and what comes next: Every split of a tree is a threshold on one raw feature. This suits tabular data with meaningful columns but not images, audio, or text, where a single pixel or character means little and useful features are complex combinations of many inputs. Kernel methods (Chapter 9) handle this by fixing a rich feature map in advance. Chapter 11 takes the next step: a neural network *learns* its features, and it is trained by the same principle that boosting uses, gradient descent on a loss, now in the space of network weights.

Chapter 11

Neural Networks

Every flexible method so far rests on features chosen in advance: polynomial and spline bases (Chapter 6), kernels (Chapter 9), or thresholds on the raw coordinates (Chapter 10). Once the features are fixed, the model on top of them is linear or piecewise constant, and its quality is limited by how well the features were designed. Neural networks remove this limitation by learning the features together with the predictor, and they are trained with the gradient methods of Chapter 3. Section 11.1 shows why fixed linear models are not enough and why feature maps help; Section 11.2 defines multilayer perceptrons; Section 11.3 discusses what they can represent and how they learn features; Section 11.4 derives backpropagation, the algorithm that computes their gradients; and Section 11.5 assembles the training procedure. The lectures write t for the target and y for the output of the network; to stay consistent with the rest of the notes, here the target (the response) is y and the network output is $\hat{y}$.

11.1 ▪ The Limits of Linear Classifiers

A data set with binary targets is *linearly separable* if some hyperplane $w^\top x + b = 0$ puts all examples with $y = 1$ on one side and all examples with $y = 0$ on the other. The logical function NOT, for example, is separable. The function XOR is not:

x_1	x_2	y	$\psi_1(x) = x_1$	$\psi_2(x) = x_2$	$\psi_3(x) = x_1 x_2$	y
0	0	0	0	0	0	0
0	1	1	0	1	0	1
1	0	1	1	0	0	1
1	1	0	1	1	1	0

The reason is convexity: the set where a linear classifier predicts 0 is a half-space, hence convex, so if it contains $(0, 0)$ and $(1, 1)$ it contains their midpoint $(\frac{1}{2}, \frac{1}{2})$; the same argument for the other class puts $(\frac{1}{2}, \frac{1}{2})$, the midpoint of $(0, 1)$ and $(1, 0)$, in the half-space for class 1 as well, which is impossible.

A *feature map* can fix this. With $\psi(x) = (x_1, x_2, x_1 x_2)$ the data become separable; for instance, the rule $\psi_1 + \psi_2 - 2\psi_3 - \frac{1}{2} > 0$ reproduces the table. This is the same trick as polynomial regression (Section 6.1) and the kernel trick (Section 9.2): a linear method in a richer feature space. The difficulty is that good features are hard to design by hand, especially for raw inputs such as the 784 pixels of an image. Neural networks learn them.

11.2 ▪ Multilayer Perceptrons

A neural network is built from simple units. A *unit* (artificial neuron) computes a weighted sum of its inputs plus a bias and applies a nonlinearity, $\hat{y} = \varphi(w^\top x + b)$. With the logistic function $\varphi = \sigma$,

$\sigma(z) = 1/(1+e^{-z})$, a single unit is exactly a logistic regression model (Section 7.2). A *feed-forward network* connects many units in a directed acyclic graph, grouped into layers; in a *fully connected* layer every input unit feeds every output unit.

Definition 11.1 (Multilayer Perceptron). *A* multilayer perceptron *(MLP) with L layers maps an input x to an output $\hat{y}$ through hidden representations $h^{(1)}, \dots, h^{(L-1)}$:*

$$h^{(0)} = x, \qquad h^{(\ell)} = \varphi\big(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\big), \quad \ell = 1, \dots, L-1, \qquad \hat{y} = f^{(L)}\big(h^{(L-1)}\big),$$

where $W^{(\ell)}$ are weight matrices, $b^{(\ell)}$ bias vectors (the superscript (L) marks the last layer; an unadorned L later denotes the loss), and the activation function φ is applied componentwise. The output layer depends on the task: $\hat{y} = w^{(L)\top} h^{(L-1)} + b^{(L)}$ for regression, $\hat{y} = \sigma(w^{(L)\top} h^{(L-1)} + b^{(L)})$ for binary classification, and $\hat{y} = \mathrm{softmax}(W^{(L)} h^{(L-1)} + b^{(L)})$ for K classes. □

The network computes a composition $\hat{y} = f^{(L)} \circ \cdots \circ f^{(1)}(x)$. This modularity matters in practice: each layer can be implemented as a black box with a forward computation and, as we shall see, a backward one.

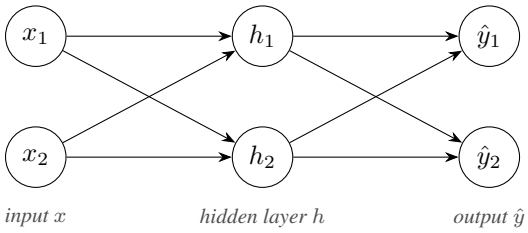


Figure 11.1. *The two-layer network of the lectures, $h = \sigma(W^{(1)}x + b^{(1)})$ and $\hat{y} = W^{(2)}h + b^{(2)}$. Every arrow carries a weight; biases are not drawn.*

Activation	$\varphi(z)$	Remarks
Identity	z	a network of linear layers is linear
Hard threshold	$\mathbf{1}\{z > 0\}$	the original perceptron; derivative is zero almost everywhere
Logistic	$1/(1+e^{-z})$	smooth version of the threshold
Hyperbolic tangent	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	a rescaled logistic, with values in $(-1,1)$
ReLU	$\max(0, z)$	the default in modern networks
Soft ReLU	$\log(1+e^{z})$	smooth version of ReLU

Remark 11.2 (Connection to Softmax Regression). Everything except the last layer computes a feature vector $h^{(L-1)} = \phi_\theta(x)$, and the last layer is a linear, logistic, or softmax regression (Section 7.4) on these features. A neural network is therefore a generalized linear model whose features are learned jointly with the coefficients. □

11.3 ▪ Expressive Power and Feature Learning

The nonlinearity is essential. If φ is the identity, each layer is an affine map, and a composition of affine maps is affine: $W^{(3)}W^{(2)}W^{(1)}x = W'x$. A deep linear network is no more expressive than linear regression. With a nonlinear activation the picture changes completely.

Theorem 11.3 (Universal Approximation, Informal). *A feed-forward network with one hidden layer and a non-polynomial activation such as the threshold, the logistic function, or ReLU can approximate any continuous function on a compact set arbitrarily well, provided the hidden layer has enough units.*

XOR shows the mechanism on a small scale. With hard thresholds, take $h_1 = \mathbf{1}\{x_1 + x_2 - 0.5 > 0\}$, which computes $x_1 \vee x_2$, and $h_2 = \mathbf{1}\{x_1 + x_2 - 1.5 > 0\}$, which computes $x_1 \wedge x_2$. Then $\hat{y} = \mathbf{1}\{h_1 - h_2 - 0.5 > 0\}$ is x_1 XOR x_2: the hidden layer computes features in which the problem is linearly separable. A logistic unit $\sigma(c(w^\top x + b))$ approaches the threshold as the scale c grows, so logistic networks can do the same, and, unlike threshold networks, they are differentiable and can be trained by gradient descent.

Each hidden unit acts as a *feature detector*. In a network trained to classify handwritten digits, the weight vector of a first-layer unit, reshaped into a 28×28 image, shows the pattern it responds to, such as a diagonal stroke. Unlike the hard-coded features of polynomial regression, these features adapt to the data.

Remark 11.4 (Connection to Kernels and Basis Functions). Basis expansions (Section 6.1) and kernel methods (Section 9.2) fix a feature map ϕ, possibly infinite-dimensional, and learn only a linear function of $\phi(x)$; the optimization is then convex. A neural network learns ϕ_θ as well. This makes it far more adaptive, but the training problem becomes non-convex (Section 3.3), and the features have to be found by gradient descent from a random start. □

Expressivity has a price. A network may need very many units to represent a given function, the weights that represent it may be hard to find, and a model that can fit anything can also overfit. For polynomial features this concern led to the L^2 penalty of ridge regression; for neural networks the remedies are discussed in Section 11.5.

11.4 ▪ Backpropagation

Training by gradient descent requires the gradient of the loss with respect to every weight in every layer. Backpropagation computes all of them at a cost comparable to evaluating the network once. It is the chain rule, organized so that no intermediate derivative is computed twice.

11.4.1 ▪ Computation Graphs and Error Signals

Consider the simplest case, a single logistic unit with squared loss ("logistic least squares"):

$$z = wx + b, \qquad \hat{y} = \sigma(z), \qquad L = \tfrac{1}{2}(\hat{y} - y)^2.$$

The *computation graph* has a node for each input and each computed quantity, and an edge from every quantity to those computed directly from it: $x, w, b \to z \to \hat{y} \to L$, with $y \to L$. By the univariate chain rule,

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w} = (\hat{y} - y)\, \sigma'(z)\, x, \qquad \frac{\partial L}{\partial b} = (\hat{y} - y)\, \sigma'(z) \cdot 1.$$

The two gradients share the factor $(\hat{y} - y)\sigma'(z) = \partial L / \partial z$, and computing it once is the whole idea. Write $\bar{v} = \partial L / \partial v$ for the *error signal* of a node v; it is a number the program computes, not a new mathematical operation. Going backward through the graph,

$$\bar{\hat{y}} = \hat{y} - y, \qquad \bar{z} = \bar{\hat{y}}\, \sigma'(z), \qquad \bar{w} = \bar{z}\, x, \qquad \bar{b} = \bar{z}.$$

When a node feeds several others (*fan-out* greater than one), its error signal collects contributions from all of them, by the multivariate chain rule $\frac{\mathrm{d}}{\mathrm{d}t} f(u(t), v(t)) = \frac{\partial f}{\partial u}\frac{\mathrm{d}u}{\mathrm{d}t} + \frac{\partial f}{\partial v}\frac{\mathrm{d}v}{\mathrm{d}t}$. In L^2-regularized logistic least squares, $L_{\mathrm{reg}} = L + \lambda R$ with $R = \frac{1}{2}w^2$, the weight w feeds both z and R, so

$$\bar{L} = 1, \quad \bar{R} = \lambda, \quad \bar{\hat{y}} = \bar{L}\,(\hat{y} - y), \quad \bar{z} = \bar{\hat{y}}\,\sigma'(z), \quad \bar{w} = \bar{z}\,x + \bar{R}\,w, \quad \bar{b} = \bar{z}.$$

Softmax regression is another example: each logit z_m affects every output $\hat{y}_k = e^{z_k}/\sum_j e^{z_j}$, so $\bar{z}_m$ is a sum over k.

11.4.2 ▪ The Algorithm

Algorithm 11.1 Backpropagation (Reverse-Mode Differentiation)

Input: A computation graph with nodes $v_1, \dots, v_N$ ordered so that parents come before children, inputs and parameters among the first nodes, and the scalar loss $v_N = L$.

Forward pass:

for $i = 1, \dots, N$ **do**
  Compute v_i as a function of its parents; store it.
end for

Backward pass:

$\bar{v}_N \leftarrow 1$
for $i = N-1, \dots, 1$ **do**
  $\bar{v}_i \leftarrow \sum_{j \in \mathrm{Children}(v_i)} \bar{v}_j\, \partial v_j / \partial v_i$ ▷ local derivatives use stored values
end for

Output: $\bar{v}_i = \partial L / \partial v_i$ for every node, in particular for every parameter.

The order of the backward loop guarantees that when $\bar{v}_i$ is computed, the error signals of all its children are already known. Each local derivative $\partial v_j / \partial v_i$ involves only the values stored in the forward pass. For vector-valued nodes the product becomes a vector–Jacobian product, $\bar{v}_i = \sum_j (\partial v_j / \partial v_i)^\top \bar{v}_j$.

For the two-layer network of Fig. 11.1 with squared loss, the passes in vectorized form are

Forward pass	Backward pass
$z = W^{(1)}x + b^{(1)}$	$\bar{\hat{y}} = \hat{y} - y$
$h = \sigma(z)$	$\bar{W}^{(2)} = \bar{\hat{y}}\,h^\top, \quad \bar{b}^{(2)} = \bar{\hat{y}}$
$\hat{y} = W^{(2)}h + b^{(2)}$	$\bar{h} = W^{(2)\top}\bar{\hat{y}}$
$L = \frac{1}{2}\lVert y - \hat{y}\rVert^2$	$\bar{z} = \bar{h} \circ \sigma'(z)$
	$\bar{W}^{(1)} = \bar{z}\,x^\top, \quad \bar{b}^{(1)} = \bar{z}$

where $\circ$ is the componentwise product. The pattern is general: the gradient of a weight matrix is the outer product of the error signal at its output and the activation at its input, and error signals travel backward through the transposed weights.

Remark 11.5 (Cost). The forward pass costs about one multiply–add per weight ($z_i = \sum_j W_{ij}x_j + b_i$). The backward pass costs about two per weight: one for $\bar{W} = \bar{\hat{y}}\,h^\top$ and one for $\bar{h} = W^\top\bar{\hat{y}}$. So one backward pass is about as expensive as two forward passes, regardless of the number of parameters. Approximating the gradient by finite differences would instead require one forward pass per parameter, which is hopeless for millions of weights. □

Remark 11.6 (Connection to Variable Elimination). Computation graphs are directed acyclic graphs, like the Bayesian networks of Chapter 12. Backpropagation is dynamic programming on such a graph: the derivative of L with respect to v_i is a sum over all paths from v_i to L of products of local derivatives, and the backward pass computes this sum without enumerating the paths, by reusing the error signals of the children. Variable elimination (Section 13.2) evaluates sums of products over a graph by the same principle, caching intermediate factors instead of error signals. □

Modern libraries such as PyTorch, TensorFlow, and JAX perform this procedure automatically (*automatic differentiation*): the program records the computation graph during the forward pass and runs Algorithm 11.1 on request. Even optimizers more sophisticated than gradient descent rely on backpropagation for their gradients.

11.5 ▪ Training Neural Networks

Training minimizes the average loss over the training data, possibly plus a penalty,

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, f_\theta(x_i)\big) + \lambda \Omega(\theta),$$

where θ collects all weights and biases. For regression L is usually the squared error; for classification it is the cross-entropy $-\sum_k y_k \log \hat{y}_k$ with one-hot targets, which is the negative log-likelihood of the categorical model, so minimizing J with $\lambda = 0$ is maximum likelihood (Section 2.2). With a sigmoid or softmax output and the cross-entropy, the error signal at the logits takes the simple form $\bar{z} = \hat{y} - y$, exactly as in the gradient of logistic regression.

Unlike the losses of linear and logistic regression, J is not convex in θ: it has many local minima and saddle points, and permuting the hidden units of a layer gives an equivalent network. Gradient descent can therefore only be expected to find a good local minimum. Stochastic gradient descent (Section 3.4) is the method of choice, both because each step is cheap and because its noise helps the iterates escape poor minima and settle in favorable regions.

Overfitting in neural networks is not fully understood, but two remedies are standard. An L^2 penalty $\Omega(\theta) = \frac{1}{2}\|\theta\|^2$ (*weight decay*) shrinks the weights as in ridge regression (Section 5.1). *Early stopping* monitors the error on a validation set (Section 4.5) and stops when it starts to increase, since overfitting typically grows as training proceeds.

Algorithm 11.2 Training a Neural Network with Minibatch SGD and Early Stopping

Input: Training and validation data, network f_θ, loss L, penalty weight λ, learning rate η, minibatch size B, maximum number of epochs E, patience P.
Initialize: Small random weights θ; $\theta_{\text{best}} \leftarrow \theta$; best validation error $e_{\text{best}} \leftarrow \infty$.

for epoch $= 1, \dots, E$ **do**
  Shuffle the training data and split it into minibatches of size B.
  for each minibatch S **do**
    Forward pass: $J_S(\theta) = \frac{1}{|S|} \sum_{i \in S} L\big(y_i, f_\theta(x_i)\big) + \lambda \Omega(\theta)$.
    Backward pass: $g \leftarrow \nabla_\theta J_S(\theta)$ by Algorithm 11.1.
    $\theta \leftarrow \theta - \eta\, g$ ▷ in practice often Adam or momentum
  end for
  Compute the validation error e of f_θ.
  if $e < e_{\text{best}}$ **then**
    $e_{\text{best}} \leftarrow e$; $\theta_{\text{best}} \leftarrow \theta$
  end if
  if e_{best} has not improved for P epochs **then**
    stop ▷ early stopping
  end if
end for

Output: The network $f_{\theta_{\text{best}}}$.

This is Algorithm 3.2 with the gradient supplied by backpropagation. The weights are initialized randomly rather than at zero: if all hidden units started equal, they would receive identical gradients and remain identical forever.

Pros and Cons: Neural networks learn features directly from raw data, scale to very large data sets through minibatch SGD, and, thanks to automatic differentiation, can be assembled from modular layers for almost any input and output. However, training is non-convex and sensitive to initialization, learning rate, and architecture; the models need a lot of data; and the learned function is hard to interpret.

Limitations and what comes next: Like every method in Part II, a neural network as used here returns a point prediction $f_\theta(x)$ for a single response. It does not describe uncertainty, it does not model the relations among many variables, and it cannot generate new data. Part III changes the question from predicting y given x to modeling a joint distribution, beginning with graphical models in Chapter 12. Neural networks return later as components of probabilistic models: as the encoder and decoder of variational autoencoders (Section 16.7), as transformers (Section 17.5), and as the denoisers of diffusion models (Chapter 18).

Part III

Probabilistic Models and Inference

Chapter 12

Graphical Models

Part II was about prediction: given an input x, estimate a response y, through $f(x)$ or the conditional distribution $p(y \mid x)$. Part III asks a broader question. We model the *joint distribution* $p(x_1, \dots, x_N)$ of many variables, some observed and some hidden, and then answer any query about it: the distribution of one variable given others, the most likely configuration of the hidden variables, or new samples that look like the data. Prediction becomes a special case, conditioning on x to obtain $p(y \mid x)$. The difficulty is size: a general joint distribution of N binary variables has $2^N - 1$ free parameters. Graphical models make the problem manageable by encoding *conditional independences* in a graph, which makes the representation compact (this chapter), the computation of queries feasible (Chapters 13 to 15), and learning from data possible (Chapter 16). Section 12.1 reviews conditional independence; Sections 12.2 and 12.3 treat directed models and how to read independences from them; Sections 12.4 and 12.5 treat undirected models; and Section 12.6 applies both ideas to sequences, ending with hidden Markov models.

12.1 ▪ Joint Distributions and Conditional Independence

By the chain rule of probability, every joint distribution factorizes as

$$p(x_1, \dots, x_N) = \prod_{j=1}^{N} p(x_j \mid x_1, \dots, x_{j-1}),$$

for any ordering of the variables. This factorization is general and therefore saves nothing: for binary variables the j-th factor needs 2^{j-1} parameters, and the total is $2^N - 1$. Savings come from dropping variables from the conditioning sets, which is justified by conditional independence. For $A \subseteq \{1, \dots, N\}$ write $x_A = \{x_i : i \in A\}$.

Definition 12.1 (Conditional Independence). *For disjoint index sets A, B, C, the variables x_A and x_B are* conditionally independent given x_C*, written $x_A \perp\!\!\!\perp x_B \mid x_C$, if*

$$p(x_A, x_B \mid x_C) = p(x_A \mid x_C)\, p(x_B \mid x_C),$$

or equivalently $p(x_A \mid x_B, x_C) = p(x_A \mid x_C)$, or $p(x_B \mid x_A, x_C) = p(x_B \mid x_C)$: once x_C is known, learning x_B tells us nothing more about x_A. □

12.2 ▪ Directed Graphical Models

Definition 12.2 (Directed Graphical Model). *Let G be a directed acyclic graph (DAG) whose nodes are the variables $x_1, \dots, x_N$, and let $\mathrm{pa}(i)$ be the set of nodes with an edge into x_i. A distribution*

factorizes according to G *if*

$$p(x_1, \dots, x_N) = \prod_{i=1}^{N} p\big(x_i \,|\, x_{\mathrm{pa}(i)}\big).$$

The pair of G and the conditional distributions is a directed graphical model*, or* Bayesian network. □

Comparing with the chain rule, ordered so that parents precede children, a DAG asserts that each variable depends on its predecessors only through its parents. For the graph in Fig. 12.1,

$$p(x_1, \dots, x_6) = p(x_1)\, p(x_2 \,|\, x_1)\, p(x_3 \,|\, x_1)\, p(x_4 \,|\, x_2)\, p(x_5 \,|\, x_3)\, p(x_6 \,|\, x_2, x_5).$$

For discrete variables each factor is a *conditional probability table* (CPT). A binary node with K_i parents needs 2^{K_i} parameters, one probability for each configuration of its parents, so a DAG needs about $N\, 2^{\max_i K_i}$ parameters instead of $2^N - 1$. The saving is exponential when every node has few parents.

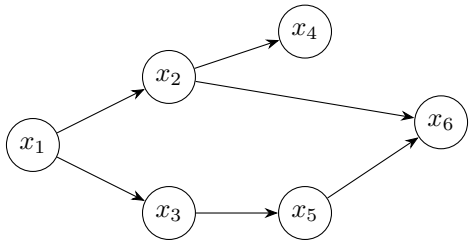


Figure 12.1. *The running example of a DAG from the lectures.*

12.2.1 ▪ Example: Naive Bayes

The tutorial classifies texts as spam ($C = 1$) or not ($C = 0$) from binary indicators $X_1, \dots, X_d$ of which dictionary words appear. Applying the chain rule in the order $C, X_1, \dots, X_d$ gives a fully connected DAG with $1 + \sum_{i=1}^{d} 2^i = 2^{d+1} - 1$ parameters, as many as the unrestricted joint distribution. The *naive Bayes* model deletes every edge between words and keeps only the edges $C \to X_i$ (Fig. 12.2, left):

$$p(C, X_1, \dots, X_d) = p(C) \prod_{i=1}^{d} p(X_i \,|\, C),$$

which says that the words are conditionally independent given the class. With $p(C = 1) = \pi$ and $p(X_j = 1 \,|\, C = c) = \theta_{j,c}$ the model has $1 + 2d$ parameters, linear rather than exponential in d, and maximum likelihood (Section 2.2) gives frequencies:

$$\hat{\pi} = \frac{\#\{\text{spam texts}\}}{n}, \qquad \hat{\theta}_{j,c} = \frac{\#\{\text{texts of class } c \text{ containing word } j\}}{\#\{\text{texts of class } c\}}.$$

The posterior $p(C \,|\, X) \propto p(C) \prod_i p(X_i \,|\, C)$ then classifies new texts. This is the naive Bayes classifier of Section 7.5.3, now read off a graph.

12.3 ▪ Reading Independences from a DAG: d-Separation

A DAG implies many conditional independences beyond the ones used to define it. All of them can be read off the graph by examining three elementary configurations of three nodes (Fig. 12.3).

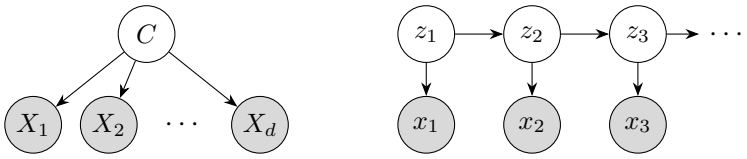
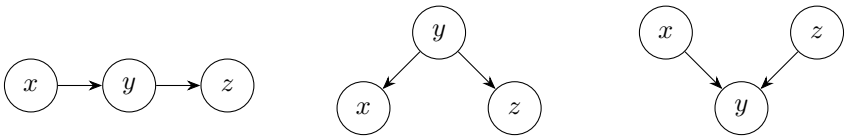


Figure 12.2. *Left: naive Bayes. Right: a hidden Markov model (Section 12.6.2). Shaded nodes are observed.*



(a) causal chain *(b) common cause* *(c) common effect*

Figure 12.3. *The three canonical triples. In (a) and (b), observing y blocks the flow of information between x and z; in (c), observing y (or one of its descendants) opens it.*

(a) *Causal chain* $x \to y \to z$: $p(x,y,z) = p(x)p(y\,|\,x)p(z\,|\,y)$, so $p(z\,|\,x,y) = p(z\,|\,y)$ and $x \perp\!\!\!\perp z\,|\,y$. Without conditioning, x and z are in general dependent.

(b) *Common cause* $x \leftarrow y \to z$: $p(x,z\,|\,y) = p(y)p(x\,|\,y)p(z\,|\,y)/p(y) = p(x\,|\,y)p(z\,|\,y)$, so again $x \perp\!\!\!\perp z\,|\,y$. Two effects of a common cause are correlated until the cause is known.

(c) *Common effect* $x \to y \leftarrow z$: $p(x,y,z) = p(x)p(z)p(y\,|\,x,z)$, so $x \perp\!\!\!\perp z$ marginally, but in general $x \not\perp\!\!\!\perp z\,|\,y$. This is *explaining away*: if an alarm y can be set off by a burglary x or an earthquake z, then given that the alarm rang, learning that there was an earthquake makes a burglary less likely.

Call a triple u–v–w on a path *active* given the observed set C if information can flow through v: for a chain or a common cause, if $v \notin C$; for a common effect $u \to v \leftarrow w$, if v or one of its descendants is in C. Otherwise the triple is *blocked*.

Definition 12.3 (d-Separation). *A path between two nodes is* active *given C if every consecutive triple along it is active. Sets A and B are* d-separated *by C if no path from a node of A to a node of B is active given C.* □

Theorem 12.4 (d-Separation and Conditional Independence). *If A and B are d-separated by C in G, then $x_A \perp\!\!\!\perp x_B\,|\,x_C$ for every distribution that factorizes according to G. Conversely, if they are not d-separated, then there is some distribution factorizing according to G in which x_A and x_B are dependent given x_C.*

Remark 12.5 (What "Vice Versa" Means). The slides state that d-separation implies conditional independence "and vice versa". The converse holds only in the sense of the theorem: a particular distribution may have extra independences, for instance because its CPT entries happen to cancel, that the graph does not show. d-Separation lists exactly the independences that hold for *all* distributions compatible with the graph. □

The lectures check d-separation with the *Bayes ball* procedure: shade the observed nodes, then try to travel from A to B, passing through a node only when the triple formed there is active. If x_C is clamped and we "wiggle" x_A, the question is whether any node in B can move.

Algorithm 12.1 Checking $x_A \perp\!\!\!\perp x_B \mid x_C$ by d-Separation (Bayes Ball)

Input: A DAG G, disjoint node sets A, B, C.
Initialize: Shade the nodes in C; mark every node that is in C or has a descendant in C.
for each $a \in A$ **do**
  Explore, by depth-first search, the paths that start at a and do not revisit a node; the first step may go to any neighbor of a along an edge in either direction.
  Extend a path $\dots, u, v$ to a neighbor w of v only if the triple u–v–w is active:
    chain or common cause at v: v is not shaded;
    common effect $u \to v \leftarrow w$: v is marked.
  if some path reaches a node of B **then**
    return "$x_A \not\perp\!\!\!\perp x_B \mid x_C$ in general"
  end if
end for
Output: "$x_A \perp\!\!\!\perp x_B \mid x_C$" (no active path exists).

In Fig. 12.1, is $x_1 \perp\!\!\!\perp x_6 \mid \{x_2, x_3\}$? The paths from x_1 to x_6 are $x_1 \to x_2 \to x_6$, blocked by the shaded chain node x_2, and $x_1 \to x_3 \to x_5 \to x_6$, blocked at x_3. So the answer is yes. Is $x_2 \perp\!\!\!\perp x_3 \mid \{x_1, x_6\}$? The path $x_2 \to x_6 \leftarrow x_5 \leftarrow x_3$ is active: x_6 is an observed common effect and x_5 an unobserved chain node. So the answer is no, even though the other path $x_2 \leftarrow x_1 \to x_3$ is blocked.

Two consequences of d-separation are used repeatedly. Every node is conditionally independent of its non-descendants given its parents. And the *Markov blanket* of x_i, the smallest set that makes x_i independent of all other nodes, consists of its parents, its children, and the other parents of its children. The co-parents enter because of explaining away, and in large DAGs they can make the blanket look unintuitive: nodes far from x_i in the picture belong to it while closer ones do not. For some problems, moreover, there is no natural direction for the edges. Both observations motivate undirected models.

12.4 ▪ Markov Random Fields

In an undirected graphical model, or *Markov random field* (MRF), edges represent symmetric interactions between neighboring variables rather than conditional distributions. A *clique* is a set of nodes that are all pairwise connected; a *maximal clique* cannot be enlarged by adding another node.

Definition 12.6 (Markov Random Field). *Let G be an undirected graph on $x = (x_1, \dots, x_N)$ with set of maximal cliques $\mathcal{C}$. A distribution* factorizes according to *G if*

$$p(x) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(x_C), \qquad Z = \sum_{x} \prod_{C \in \mathcal{C}} \psi_C(x_C),$$

for some nonnegative potential functions *ψ_C. The normalizing constant Z is the* partition function *(an integral for continuous variables).* □

Potentials are not probabilities: they are arbitrary nonnegative scores, large for compatible configurations of the clique, and only the product is normalized. For discrete variables a potential is a table of nonnegative numbers. The price of this freedom is Z, a sum over all configurations, whose computation is in general as hard as inference itself.

Theorem 12.7 (Hammersley–Clifford). *If $p(x) > 0$ for all x, the following are equivalent:*

(i) p *factorizes according to* G*;*

(ii) p *satisfies the* global Markov property*:* $x_A \perp\!\!\!\perp x_B \mid x_S$ *whenever* S *separates* A *from* B *in* G*, that is, every path from* A *to* B *passes through* S*.*

In particular, if x_i *and* x_j *are not adjacent then* $x_i \perp\!\!\!\perp x_j \mid x_{rest}$*, and the Markov blanket of* x_i *is the set of its neighbors.*

Reading independences off an MRF is simpler than in a DAG: it is plain graph separation, with no special case for common effects. For the graph in Fig. 12.4, the maximal cliques give

$$p(x) \propto \psi_{123}(x_1,x_2,x_3)\,\psi_{235}(x_2,x_3,x_5)\,\psi_{245}(x_2,x_4,x_5)\,\psi_{356}(x_3,x_5,x_6)\,\psi_{4567}(x_4,x_5,x_6,x_7),$$

and separation yields, for example, $(x_1,x_2) \perp\!\!\!\perp (x_6,x_7) \mid (x_3,x_4,x_5)$ and $x_1 \perp\!\!\!\perp x_5 \mid (x_2,x_3)$.

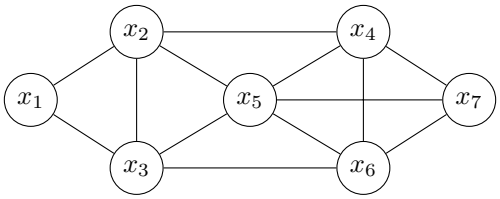


Figure 12.4. *The MRF example of the lectures, with maximal cliques* $\{1,2,3\}$*,* $\{2,3,5\}$*,* $\{2,4,5\}$*,* $\{3,5,6\}$*,* $\{4,5,6,7\}$*.*

Directed and undirected models express different families of independences, and neither contains the other. The four-cycle A–B–C–D–A has the two independences $A \perp\!\!\!\perp C \mid \{B,D\}$ and $B \perp\!\!\!\perp D \mid \{A,C\}$, and no DAG on four nodes has exactly these. Conversely, the common effect $x \to y \leftarrow z$ has $x \perp\!\!\!\perp z$ but $x \not\perp\!\!\!\perp z \mid y$; an undirected graph that connects x and z through y cannot make them marginally independent while keeping them dependent given y.

Remark 12.8 (Connection to Exponential Families). Write each potential in log-linear form, $\log \psi_C(x_C \mid \theta_C) = \theta_C^\top \phi_C(x_C)$. Then

$$p(x \mid \theta) = \exp\Big\{ \sum_{C \in \mathcal{C}} \theta_C^\top \phi_C(x_C) - A(\theta) \Big\}, \qquad A(\theta) = \log Z(\theta),$$

an exponential family (Section 2.3) with sufficient statistics ϕ_C. For discrete variables this is no restriction: let $\phi_C(x_C)$ be the indicator vector of the configuration x_C and θ_C the table of log-potentials. The usual exponential-family identity gives $\partial A / \partial \theta_C = \mathbb{E}[\phi_C(X_C)]$: derivatives of the log-partition function are marginal probabilities of the cliques. This is one reason why computing marginals (Chapter 13) is the central computational task for MRFs. □

12.5 ▪ The Ising Model and Gaussian Graphical Models

The *Ising model*, originally a model of magnetism, is the prototypical MRF. The variables are spins $x_s \in \{-1,+1\}$ on the nodes of a graph, often a grid. Each edge $s \sim t$ carries a pairwise potential $\psi_{st}(x_s,x_t) = e^{J_{st} x_s x_t}$, equal to $e^{J_{st}}$ when the spins agree and $e^{-J_{st}}$ when they disagree, and each node may carry a field $\psi_s(x_s) = e^{b_s x_s}$. Together,

$$p(x) \propto \exp\Big\{ \sum_{s \sim t} J_{st}\, x_s x_t + \sum_s b_s\, x_s \Big\}.$$

A coupling $J_{st} > 0$ favors equal neighboring spins, and by Theorem 12.7 $J_{st} = 0$ means $x_s \perp\!\!\!\perp x_t \mid x_{\text{rest}}$. In an image, the spins are black or white pixels and positive couplings express that neighboring pixels tend to agree. This model reappears throughout Part III: for image denoising with loopy belief propagation (Section 13.4), with Gibbs sampling (Algorithm 14.5), and with mean-field variational inference (Section 15.3).

The continuous analogue is the multivariate Gaussian (Section 2.5). With precision matrix $K = \Sigma^{-1}$,

$$p(x) \propto \prod_s \exp\Big\{ -\tfrac{1}{2} K_{ss}(x_s - \mu_s)^2 \Big\} \prod_{s<t} \exp\big\{ -K_{st}(x_s - \mu_s)(x_t - \mu_t) \big\},$$

a pairwise MRF in which $-K_{st}$ plays the role of the coupling J_{st}. Hence $K_{ij} = 0$ if and only if $x_i \perp\!\!\!\perp x_j \mid x_{\text{rest}}$: the graph of a *Gaussian graphical model* is the sparsity pattern of the precision matrix, not of the covariance matrix. A zero covariance $\Sigma_{ij} = 0$ instead means marginal independence.

12.6 ▪ Models for Sequences

Much data comes in sequences: time series such as prices or speech, and ordered data such as text or genes. The i.i.d. assumption of Part II no longer applies, and the chain rule $p(x_{1:T}) = \prod_t p(x_t \mid x_{1:t-1})$ needs factors with ever longer histories. The simplest remedy is a Markov assumption.

12.6.1 ▪ Markov Chains

A *first-order Markov chain* assumes $p(x_t \mid x_{1:t-1}) = p(x_t \mid x_{t-1})$, so

$$p(x_{1:T}) = p(x_1) \prod_{t=2}^{T} p(x_t \mid x_{t-1}),$$

the DAG $x_1 \to x_2 \to \cdots \to x_T$. The chain is *time-homogeneous* (the slides call it stationary) if $p(x_{t+1} = j \mid x_t = i)$ does not depend on t. An m-th order chain conditions on $x_{t-m:t-1}$; this is useful for language, where dependencies reach further back. For $x_t \in \{1, \dots, K\}$ the transitions form a $K \times K$ *transition matrix* A with $A_{ij} = p(x_t = j \mid x_{t-1} = i)$ and rows summing to one, drawn as a state transition diagram whose arrows are the nonzero entries. The marginal distribution evolves as $p(x_t = j) = \sum_i A_{ij}\, p(x_{t-1} = i)$, and the n-step transition probabilities are the entries of A^n (the Chapman–Kolmogorov equations, $A(m+n) = A(m)A(n)$). The long-run behavior of A^n, the stationary distribution, is the basis of Markov chain Monte Carlo and is studied in Section 14.5.

A first-order chain over words is a simple *language model*. Its parameters, the initial distribution $\pi_j = p(x_1 = j)$ and the matrix A, are estimated by maximum likelihood from sentences treated as independent, and the log-likelihood $\sum_j N^1_j \log \pi_j + \sum_{j,k} N_{jk} \log A_{jk}$ depends on the data only through counts.

Algorithm 12.2 Maximum Likelihood for a Markov Chain (Bigram Language Model)

Input: Sequences $x^{(1)}, \dots, x^{(N)}$ over states $\{1, \dots, K\}$, of lengths $T_1, \dots, T_N$.

1: $N^1_j \leftarrow \sum_{i=1}^{N} \mathbf{1}\{x_1^{(i)} = j\}$ for each j ▷ how often state j starts a sequence

2: $N_{jk} \leftarrow \sum_{i=1}^{N} \sum_{t=1}^{T_i - 1} \mathbf{1}\{x_t^{(i)} = j,\ x_{t+1}^{(i)} = k\}$ for each j, k ▷ transition counts

Output: $\hat{\pi}_j = N^1_j / \sum_{j'} N^1_{j'}$ and $\hat{A}_{jk} = N_{jk} / \sum_{k'} N_{jk'}$.

The estimates are the empirical frequencies of starting states and of transitions out of each state, just as the naive Bayes estimates above are empirical frequencies: whenever every factor of a directed model is a free categorical distribution and all variables are observed, maximum likelihood decouples into one counting problem per CPT.

12.6.2 ▪ Hidden Markov Models

In many applications the quantity that evolves in a Markovian way is not observed directly. A *hidden Markov model* (HMM) has hidden states $z_t \in \{1, \dots, K\}$ that form a Markov chain and observations x_t, in any space, that depend only on the current state (Fig. 12.2, right):

$$p(x_{1:T}, z_{1:T}) = p(z_1) \prod_{t=2}^{T} p(z_t \mid z_{t-1}) \prod_{t=1}^{T} p(x_t \mid z_t).$$

By d-separation, z_t separates the past from the future, but the observations alone are not Markov: every path from x_{t-2} to x_t runs through unobserved states, so all past observations remain informative. An HMM is thus a latent variable model, like the mixtures of Chapter 16, with the mixture components linked over time. Computing $p(z_t \mid x_{1:T})$ is an inference problem on a chain, solved by the message passing of Chapter 13.

Graphs can also be designed from domain knowledge rather than generic assumptions. The QMR network for medical diagnosis is bipartite, with about 570 hidden diseases pointing to about 4000 observed symptoms through "noisy-OR" CPTs; the ALARM network for monitoring intensive-care patients has 37 discrete variables but, being sparse, only 504 parameters instead of at least $2^{37} - 1$ (its variables have up to four states).

Pros and Cons: Graphical models give a compact and interpretable representation of a high-dimensional distribution: the graph states the modeling assumptions, the number of parameters scales with the size of the largest family or clique rather than with 2^N, and conditional independences can be read off by d-separation or graph separation. DAGs are natural when there is a causal or generative story and are easy to sample from; MRFs are natural for symmetric interactions such as neighboring pixels. However, the graph only describes structure. Computing with the model, through marginals, conditionals, or the partition function, can still be very expensive, and an MRF's potentials are hard to interpret and to learn because of Z.

Limitations and what comes next: A graphical model defines $p(x)$, but the questions we care about, such as $p(x_A \mid x_B)$, require summing the joint distribution over all configurations of the remaining variables, which is exponential in their number if done naively. Chapter 13 shows how to exploit the factorization to compute such sums exactly by variable elimination and message passing, and when this remains too expensive, Chapters 14 and 15 approximate them by sampling and by optimization.

Chapter 13

Exact Inference and Message Passing

A graphical model (Chapter 12) specifies a joint distribution over many variables through local factors. To use it we must answer questions about a few of the variables after observing some others: the probability of getting a job given a GPA of 3.1, the skill of a player given the outcomes of all games, the true value of a pixel given a noisy image. These questions are called *inference* queries. Section 13.1 states them precisely and explains why the naive computation is exponential in the number of variables. Section 13.2 presents variable elimination, which exploits the factorization to push sums inside products. On trees the same computation can be organized as messages passed along the edges, which gives all marginals at once (Section 13.3). On graphs with cycles the message updates can still be iterated, giving loopy belief propagation (Section 13.4), an approximate method illustrated by image denoising with an Ising model.

13.1 ▪ Inference Queries

Let $x = (x_1, \dots, x_N)$ be the variables of a graphical model, each taking k values. Split them into three groups:

x_E	the observed evidence,
x_F	the unobserved variables we want to infer,
$x_R = x \setminus \{x_F, x_E\}$	the remaining variables, extraneous to the query.

The basic inference task is to compute the conditional distribution

$$p(x_F \mid x_E) = \frac{p(x_F, x_E)}{p(x_E)} = \frac{p(x_F, x_E)}{\sum_{x_F} p(x_F, x_E)}, \qquad p(x_F, x_E) = \sum_{x_R} p(x_F, x_E, x_R). \tag{13.1}$$

The extraneous variables x_R must be marginalized out, and the denominator is obtained by one further sum over x_F. Two related queries are the *marginal* $p(x_F)$ (no evidence) and the *MAP assignment* $\arg\max_{x_F, x_R} p(x_F, x_R \mid x_E)$ of all unobserved variables, in which every sum is replaced by a maximum.

Written out naively, the sum over x_R has $k^{|R|}$ terms, so the cost is exponential in the number of variables. The factorization of the joint distribution is what makes inference feasible. Both kinds of graphical model can be written in the same form,

$$p(x) = \frac{1}{Z} \prod_{C} \psi_C(x_C), \tag{13.2}$$

with nonnegative *factors* ψ_C. For a DAG (Section 12.2) the factors are the conditional probability tables $p(x_i \mid x_{\mathrm{pa}(i)})$, one per scope $\{i\} \cup \mathrm{pa}(i)$, and $Z = 1$. For an MRF (Section 12.4) they are the clique

potentials, and Z is the partition function. The algorithms of this chapter work on the factor form and therefore apply to both. Observing $x_E = \bar{x}_E$ simply fixes the corresponding arguments of the factors at their observed values.

13.2 ▪ Variable Elimination

13.2.1 ▪ A Chain

Consider the chain $A \to B \to C \to D$, so that $p(A,B,C,D) = p(A)p(B \mid A)p(C \mid B)p(D \mid C)$, and suppose we want the marginal $p(D)$. Summing the joint over all (A,B,C) costs $O(k^4)$, and $O(k^n)$ for a chain of length n. Instead, use the distributive law to move each sum as far to the right as possible:

$$p(D) = \sum_C p(D \mid C) \sum_B p(C \mid B) \underbrace{\sum_A p(A)p(B \mid A)}_{=\tau(B)=p(B)} = \sum_C p(D \mid C) \underbrace{\sum_B p(C \mid B)\tau(B)}_{=\tau(C)=p(C)} .$$

Each step sums one variable out of a product of two tables of size $k \times k$, which costs $O(k^2)$, and the whole computation costs $O(nk^2)$. This is dynamic programming: each intermediate result is computed once and reused.

13.2.2 ▪ The Algorithm

In general, eliminating a variable multiplies all factors that contain it, sums it out, and leaves a new *intermediate factor* τ on the variables that appeared together with it. For example, in $p(A,B,C) = \sum_X p(X)p(A \mid X)p(B \mid A)p(C \mid B,X)$,

$$\sum_X \psi_1(X)\psi_2(A,X)\psi_3(A,B)\psi_4(X,B,C) = \psi_3(A,B) \underbrace{\sum_X \psi_1(X)\psi_2(A,X)\psi_4(X,B,C)}_{=\tau(A,B,C)} .$$

Repeating this for every variable in x_R in some order gives the following algorithm. Let Ψ denote the current set of factors.

Algorithm 13.1 Variable Elimination (Sum-Product)

Input: Factors $\Psi = \{\psi_C\}$ of $p(x) \propto \prod_C \psi_C(x_C)$; evidence $x_E = \bar{x}_E$; query variables x_F; an elimination ordering $x_{j_1} \prec x_{j_2} \prec \cdots \prec x_{j_r}$ of the remaining variables x_R.

Restrict: in every factor, fix the arguments in x_E at their observed values $\bar{x}_E$.
for $\ell = 1, \dots, r$ **do**
  $\Psi_\ell \leftarrow \{\psi \in \Psi : \psi \text{ depends on } x_{j_\ell}\}$
  $\tau_\ell \leftarrow \sum_{x_{j_\ell}} \prod_{\psi \in \Psi_\ell} \psi$ ▷ a factor on the variables that share a factor with x_{j_ℓ}
  $\Psi \leftarrow (\Psi \setminus \Psi_\ell) \cup \{\tau_\ell\}$
end for
$\tau(x_F) \leftarrow \prod_{\psi \in \Psi} \psi$ ▷ only x_F is left

Output: $p(x_F \mid \bar{x}_E) = \tau(x_F) \big/ \sum_{x_F} \tau(x_F)$.

The final normalization handles both the evidence and the partition function: $\sum_{x_F} \tau(x_F)$ equals $p(\bar{x}_E)$ for a DAG and $Z\,p(\bar{x}_E)$ for an MRF. For a DAG with no evidence, $\tau(x_F)$ is already the marginal

$p(x_F)$ and no normalization is needed. Replacing every sum by a maximum gives the max-product version, which computes $\max_{x_R} p(x_F, \bar{x}_E, x_R)$ and, with back-tracking of the maximizers, a MAP assignment.

Remark 13.1 (Barren Variables). In a DAG, a variable that is neither queried nor observed and has no descendants in $x_F \cup x_E$ can be deleted before elimination: summing out a leaf gives $\sum_{x_i} p(x_i \mid x_{\mathrm{pa}(i)}) = 1$. In the tutorial example $p(x_1 \mid \bar{x}_6)$ for the DAG with factorization $p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1)p(x_4 \mid x_2)p(x_5 \mid x_3)p(x_6 \mid x_2, x_5)$, the variable x_4 drops out in this way, and eliminating x_5, x_3, x_2 in turn leaves $p(x_1)p(\bar{x}_6 \mid x_1)$, which is normalized over x_1 to give the answer. □

13.2.3 ▪ Elimination Orderings and Cost

The ordering matters. The cost of eliminating x_{j_ℓ} is dominated by the size of the table $\prod_{\psi \in \Psi_\ell} \psi$, which is k^{N_ℓ} if N_ℓ variables appear in that sum (the eliminated variable and the variables it currently shares factors with).

Proposition 13.2 (Cost of Variable Elimination). *With m initial factors and k states per variable, variable elimination runs in time $O\big(m\, k^{N_{\max}}\big)$, where $N_{\max} = \max_\ell N_\ell$ is the number of variables in the largest sum. Finding the ordering that minimizes $N_{\max}$ is NP-hard.*

Example: the Student Network

The slides use a DAG with eight variables (Coherence, Difficulty, Intelligence, Grade, SAT, Letter, Job, Happy) and factorization

$$p(C, D, I, G, S, L, J, H) = p(C)p(D \mid C)p(I)p(G \mid D, I)p(L \mid G)p(S \mid I)p(J \mid S, L)p(H \mid J, G),$$

so $m = 8$. To compute $p(J)$ with the ordering C, D, I, H, G, S, L, the intermediate factors are

$$\tau(D), \quad \tau(G, I), \quad \tau(S, G), \quad \tau(G, J), \quad \tau(J, L, S), \quad \tau(J, L), \quad \tau(J) = p(J).$$

The largest sum is the one over G, which involves $\psi(L, G)\tau(S, G)\tau(G, J)$ and hence $N_G = 4$ variables (G, J, L, S); the cost is $O(8k^4)$ instead of $O(k^8)$. Since H is a barren leaf, $\tau(G, J) = \sum_H p(H \mid G, J) = 1$, so in this example H could have been dropped at the start. In practice orderings are chosen by greedy heuristics, for example eliminating next the variable that creates the smallest new factor.

Remark 13.3 (Connection to Hidden Markov Models). On the chain of an HMM (Section 12.6.2), eliminating the hidden states from the first to the last time step is the forward algorithm, and the max-product version is the Viterbi algorithm. Variable elimination is the general form of these dynamic programs. □

Pros and Cons: Variable elimination is exact, simple, and applies to any DAG or MRF. However, its cost is exponential in the size of the largest intermediate factor, which depends on the graph and the ordering, and a good ordering is hard to find. It also answers one query at a time: computing the marginal of every variable by running it N times repeats most of the work.

13.3 ▪ Belief Propagation on Trees

On a tree, the repetition can be avoided. We consider a pairwise MRF on a tree $G = (V, E)$,

$$p(x) = \frac{1}{Z} \prod_{i \in V} \psi_i(x_i) \prod_{(i,j) \in E} \psi_{ij}(x_i, x_j),$$

with neighbor sets $N(i) = \{j : (i, j) \in E\}$. A directed tree can be written in this form, with $\psi_{ij}(x_i, x_j) = p(x_j \mid x_i)$ for each edge and the root's marginal as its node potential. For trees, any elimination ordering that proceeds from the leaves toward a chosen root is optimal: eliminating a leaf x_j attached to x_i produces a factor of the single variable x_i. That factor is called a *message* from j to i.

Definition 13.4 (Messages and Beliefs). *The* message *from node j to a neighbor i is*

$$m_{j \to i}(x_i) = \sum_{x_j} \psi_j(x_j)\, \psi_{ij}(x_i, x_j) \prod_{u \in N(j) \setminus \{i\}} m_{u \to j}(x_j). \tag{13.3}$$

If x_j is observed as $\bar{x}_j$, the sum is replaced by evaluation at $\bar{x}_j$:

$$m_{j \to i}(x_i) = \psi_j(\bar{x}_j) \psi_{ij}(x_i, \bar{x}_j) \prod_{u \in N(j) \setminus \{i\}} m_{u \to j}(\bar{x}_j).$$

The belief *at node i is*

$$b(x_i) \propto \psi_i(x_i) \prod_{j \in N(i)} m_{j \to i}(x_i).$$

□

A message summarizes everything on j's side of the edge (i, j): it combines j's own potential, the edge potential, and the messages j has received from its other neighbors, and then sums x_j out. Observing x_j is the same as multiplying ψ_j by the indicator $\mathbf{1}[x_j = \bar{x}_j]$, so evidence can always be absorbed into the node potentials.

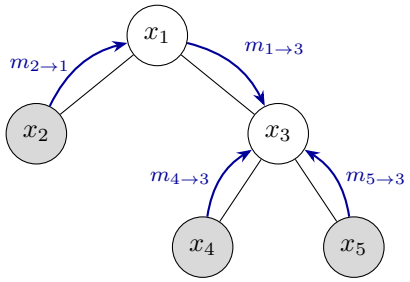


Figure 13.1. *The tree from the lectures, with x_2, x_4, x_5 observed (shaded). With x_3 as root, the four messages shown give $p(x_3 \mid \bar{x}_2, \bar{x}_4, \bar{x}_5) \propto \psi_3(x_3) m_{1\to3}(x_3) m_{4\to3}(x_3) m_{5\to3}(x_3)$.*

In the example of Fig. 13.1, the messages from the observed leaves are $m_{2\to1}(x_1) = \psi_2(\bar{x}_2)\psi_{12}(x_1, \bar{x}_2)$, $m_{4\to3}(x_3) = \psi_4(\bar{x}_4)\psi_{34}(x_3, \bar{x}_4)$, $m_{5\to3}(x_3) = \psi_5(\bar{x}_5)\psi_{35}(x_3, \bar{x}_5)$, followed by $m_{1\to3}(x_3) = \sum_{x_1} \psi_1(x_1)\psi_{13}(x_1, x_3) m_{2\to1}(x_1)$. The normalized belief $b(x_3)$ is exactly the result of variable elimination with x_3 as the query. A single pass toward a root gives the correct marginal only at that root. For $p(x_1 \mid \bar{x}_2, \bar{x}_4, \bar{x}_5)$ we also need $m_{3\to1}(x_1) = \sum_{x_3} \psi_3(x_3)\psi_{13}(x_1, x_3) m_{4\to3}(x_3) m_{5\to3}(x_3)$; the messages $m_{4\to3}$ and $m_{5\to3}$ are reused.

Algorithm 13.2 Belief Propagation on a Tree (Sum-Product)

Input: A tree-structured pairwise MRF with potentials ψ_i, ψ_{ij}; evidence absorbed into the node potentials.

Choose an arbitrary root r.
Collect: from the leaves toward r, each node $j \neq r$ sends $m_{j\to i}$ to its parent i by (13.3) once it has received the messages from all its children.
Distribute: from r toward the leaves, each node sends $m_{i\to j}$ to each of its children j by (13.3).
for each unobserved node i **do**
    $b(x_i) \leftarrow \psi_i(x_i) \prod_{j\in N(i)} m_{j\to i}(x_i)$, then normalize so that $\sum_{x_i} b(x_i) = 1$.
end for

Output: Beliefs $b(x_i) = p(x_i \mid \bar{x}_E)$ for every unobserved node i.

After the two passes every directed edge carries exactly one message, so the whole computation costs $O(|E|k^2)$, about twice the cost of a single query, instead of N separate runs of variable elimination.

Remark 13.5 (Max-Product and MAP). Replacing the sum in (13.3) by a maximum,

$$m_{j\to i}(x_i) = \max_{x_j} \psi_j(x_j)\psi_{ij}(x_i, x_j) \prod_{u\in N(j)\setminus\{i\}} m_{u\to j}(x_j),$$

gives *max-product* belief propagation. The resulting beliefs are *max-marginals*,

$$b(x_i) \propto \max_{x_{-i}} p(x_i, x_{-i} \mid \bar{x}_E),$$

where x_{-i} denotes all variables other than x_i, and on a tree $\hat{x}_i = \arg\max_{x_i} b(x_i)$ is a MAP assignment when the maximizer is unique. In practice messages are computed in the log domain (max-sum) to avoid underflow. On an HMM chain, sum-product BP is the forward–backward algorithm and max-product BP is Viterbi. □

Remark 13.6 (Which Decision?). Sum-product beliefs support the decision $\hat{x}_i = \arg\max_{x_i} p(x_i \mid \bar{x}_E)$ for each node separately, which minimizes the expected number of wrongly predicted variables (a Hamming loss) in the sense of Section 1.3. Max-product gives the single most probable joint configuration, which minimizes the probability of getting anything wrong. The two can differ. □

Pros and Cons: On trees, belief propagation is exact and computes all marginals in two passes, at a cost linear in the number of edges. The message updates are local, so they are easy to implement and to vectorize. However, the exactness depends on the absence of cycles. For general graphs one can group variables into a tree of clusters, but the clusters can be large, which brings back the exponential cost of variable elimination.

13.4 ▪ Loopy Belief Propagation

If the graph has cycles, the collect and distribute passes are no longer well defined, because messages around a cycle depend on themselves. *Loopy belief propagation* ignores this and simply iterates the message updates (13.3) from some initialization until they stop changing much. The beliefs are then used as approximations of the marginals. There is no general guarantee that the iteration converges, or that its limit is close to the true marginals, but on many problems the result is a very good approximation.

Algorithm 13.3 Loopy Belief Propagation with Damping

Input: A pairwise MRF on a graph $G = (V, E)$ (cycles allowed) with potentials ψ_i, ψ_{ij}; damping parameter $\eta \in (0, 1]$; number of iterations T.

Initialize: $m_{j \to i}(x_i) \leftarrow 1/k$ for every edge in both directions.

for $t = 1, \dots, T$ **do**
  for every directed edge $j \to i$ **do**
    $m^{\text{new}}_{j \to i}(x_i) \leftarrow \sum_{x_j} \psi_{ij}(x_i, x_j)\, \psi_j(x_j) \prod_{u \in N(j) \setminus \{i\}} m_{u \to j}(x_j)$
    $m^{\text{new}}_{j \to i}(x_i) \leftarrow m^{\text{new}}_{j \to i}(x_i) \big/ \sum_{x_i'} m^{\text{new}}_{j \to i}(x_i')$ ▷ normalization
    $m_{j \to i} \leftarrow \eta\, m^{\text{new}}_{j \to i} + (1 - \eta)\, m_{j \to i}$ ▷ damping; $\eta = 1$ is the plain update
  end for
end for

$b(x_i) \leftarrow \psi_i(x_i) \prod_{j \in N(i)} m_{j \to i}(x_i)$, normalized, for every $i \in V$.

Output: Approximate marginals $b(x_i) \approx p(x_i \mid \bar{x}_E)$ and decisions $\hat{x}_i = \arg\max_{x_i} b(x_i)$.

Normalizing the messages does not change the beliefs, which are normalized at the end anyway, but it keeps the numbers in a reasonable range. Mixing the new message with the old one, which the lectures call momentum, damps the oscillations that are common on graphs with many short cycles. The messages in the inner loop may be updated all at once from the previous iteration, or sequentially, using the newest messages as soon as they are available; the lectures sweep from an arbitrary root to the leaves and back, as on a tree. Replacing the sum by a maximum gives loopy max-product.

13.4.1 ▪ Example: Image Denoising

A binary image with n pixels is a vector $x \in \{-1, +1\}^n$. The prior is the Ising model of Section 12.5 on the pixel grid,

$$p(x) \propto \prod_{(i,j) \in E} e^{J x_i x_j}, \qquad J > 0,$$

which favors neighboring pixels with equal values. The image is sent through a noisy channel that flips each pixel independently with probability ϵ, and we observe $y \in \{-1, +1\}^n$:

$$p(y_i \mid x_i) = (1 - \epsilon)^{\frac{1 + x_i y_i}{2}} \epsilon^{\frac{1 - x_i y_i}{2}} \;\propto\; e^{\beta y_i x_i}, \qquad \beta = \tfrac{1}{2} \log \frac{1 - \epsilon}{\epsilon}.$$

The posterior is again an Ising model, now with an external field given by the observations:

$$p(x \mid y) \propto \exp\Big(J \sum_{(i,j) \in E} x_i x_j + \beta \sum_{i \in V} y_i x_i \Big), \qquad \psi_{ij}(x_i, x_j) = e^{J x_i x_j}, \quad \psi_i(x_i) = e^{\beta y_i x_i}.$$

The two hyperparameters have clear roles: J controls how strongly a pixel agrees with its neighbors, and β how strongly it agrees with its noisy observation; a small flip probability ϵ means a large β.

For binary variables every function of one pixel is a vector in $\mathbb{R}^2$ with entries for $+1$ and -1, and the updates become linear algebra:

$$\psi_i = \begin{pmatrix} e^{\beta y_i} \\ e^{-\beta y_i} \end{pmatrix}, \qquad \Psi_{ij} = \begin{pmatrix} e^{J} & e^{-J} \\ e^{-J} & e^{J} \end{pmatrix}, \qquad m_{j \to i} = \Psi_{ij} \Big(\psi_j \odot \bigodot_{u \in N(j) \setminus \{i\}} m_{u \to j} \Big),$$

where $\odot$ is the elementwise product: the product over neighbors is elementwise, and the sum over x_j is a matrix–vector product. Starting from $m_{j \to i} = (1/2, 1/2)^\top$, ten to twenty iterations of Algorithm 13.3

suffice, and the decision $\hat{x}_i = \arg\max_{x_i} b(x_i)$ recovers a clean image (the Mona Lisa example of the lectures). Note that nothing is learned from data here: J and β are fixed, and the computation is pure inference for a single image.

Remark 13.7 (Connection to Mean-Field and Gibbs Updates). Loopy BP is one of several schemes that approximate marginals by repeated local updates. Naive mean-field variational inference (Section 15.3) for the same Ising posterior updates $\mu_i = \mathbb{E}_q[x_i]$ by $\mu_i \leftarrow \tanh\left(J \sum_{j \in N(i)} \mu_j + \beta y_i\right)$, and the Gibbs sampler of Algorithm 14.5 draws x_i given the current values of its neighbors. Loopy BP keeps a separate message for each direction of each edge, which avoids feeding a node's own influence straight back to it. □

Pros and Cons: Loopy BP is cheap per iteration, local, and often accurate in practice. However, it may oscillate or fail to converge, it has essentially no guarantees beyond tree-like graphs, and its beliefs tend to be *overconfident*, because evidence that travels around a cycle is counted more than once. Its accuracy deteriorates quickly when the model is wrong, for example when the noise is not independent across pixels. Normalization and damping help with stability; extensions such as generalized belief propagation address some of the other issues.

Method	Graphs	Output	Cost and accuracy
Variable elimination	any DAG or MRF	one query $p(x_F \mid \bar{x}_E)$	exact; $O(mk^{N_{\max}})$, depends on the ordering
Belief propagation	trees	all marginals (or max-marginals)	exact; $O(\lvert E\rvert k^2)$ for two passes
Loopy BP	any pairwise MRF	approximate marginals	$O(\lvert E\rvert k^2)$ per iteration; no guarantees

Limitations and what comes next: Exact inference is exponential in the size of the largest intermediate factor, and loopy BP replaces exactness by a heuristic without guarantees. A second difficulty is continuous variables. Sums become integrals, which have closed forms only in special cases, such as jointly Gaussian models (Section 2.5). In the TrueSkill model, for instance, each skill has a Gaussian prior, but the probability that one player beats another is a logistic function of the difference of their skills, and the posterior over all skills has no closed form. Chapter 14 therefore gives up exact computation and approximates expectations under the posterior by averages over random samples; Chapter 15 instead approximates the posterior by a simpler distribution found by optimization.

Chapter 14

Monte Carlo and Markov Chain Monte Carlo

Exact inference (Chapter 13) is exponential in the size of the largest factor and breaks down for continuous variables without conjugate structure. This chapter takes a different route: instead of computing a posterior exactly, we draw random samples from it and approximate every quantity of interest by an average over the samples. Section 14.1 formulates the two problems that sampling solves and introduces the TrueSkill model as a running example. Section 14.2 covers simple Monte Carlo estimation and ancestral sampling in directed models. Sections 14.3 and 14.4 present importance and rejection sampling, which work with unnormalized densities but need a good proposal distribution, a requirement that fails in high dimensions. Markov chain Monte Carlo removes this requirement by building a Markov chain whose stationary distribution is the target: after the background on Markov chains in Section 14.5, Sections 14.6 to 14.8 present Metropolis–Hastings, Gibbs sampling, and Hamiltonian Monte Carlo, and Section 14.9 discusses how to tell whether a chain has run long enough.

14.1 ▪ Sampling for Approximate Inference

A *sample* from a distribution $p(x)$ is a realization of a random variable with distribution p; x may be a scalar or high-dimensional. Monte Carlo methods address two problems:

(1) *Sampling:* generate samples $x^{(1)}, \dots, x^{(S)}$ from a given distribution p.

(2) *Estimation:* compute expectations $\Phi = \mathbb{E}_{x\sim p}[\varphi(x)] = \int \varphi(x)p(x)\,\mathrm{d}x$ of a *test function* φ.

Solving the first solves the second. Many quantities are expectations: the mean of $f(x)$ uses $\varphi = f$, the variance uses $\varphi_1 = f$ and $\varphi_2 = f^2$ through $\mathrm{Var}(f(x)) = \mathbb{E}[\varphi_2] - \mathbb{E}[\varphi_1]^2$, and a probability $\mathbb{P}(x \in A)$ uses $\varphi = \mathbf{1}_A$.

In inference the target is a posterior, which is known only up to a constant. We therefore assume that we can evaluate an *unnormalized density* $\tilde{p}$ with

$$p(x) = \frac{\tilde{p}(x)}{Z}, \qquad Z = \int \tilde{p}(x)\,\mathrm{d}x.$$

For a posterior $p(z \mid x) \propto p(z)p(x \mid z)$, the unnormalized density is the joint $\tilde{p}(z) = p(z, x)$ and $Z = p(x)$ is the evidence. There are two difficulties. The constant Z is a high-dimensional integral that is usually intractable; and even if Z were known, drawing samples from p in high dimensions would remain hard. Discretizing space does not help: 50 grid points per coordinate in $D = 1000$ dimensions give 50^{1000} points. MacKay's analogy makes the difficulty concrete. Think of $\tilde{p}(x)$ as the depth of a lake

at location x and $\varphi(x)$ as the plankton concentration; then Z is the volume of the lake and $\Phi = \frac{1}{Z}\int \varphi\tilde{p}$ is the average concentration. We can measure depth and concentration at any point we visit, but to estimate Φ correctly a method must discover the deep canyons and their volume relative to the rest of the lake.

Running Example: TrueSkill

TrueSkill is a rating system for competitive games. Each of N players has an unknown skill $z_i \in \mathbb{R}$ with independent prior $z_i \sim \mathcal{N}(0,1)$. In a game between players i and j,

$$p(i \text{ beats } j \mid z_i, z_j) = \sigma(z_i - z_j), \qquad \sigma(y) = \frac{1}{1+e^{-y}},$$

the same logistic link as in logistic regression (Section 7.2), and games are independent given the skills. If game g was won by w_g and lost by ℓ_g, the posterior over skills is

$$p(z \mid \text{games}) \propto \tilde{p}(z) = \prod_{i=1}^{N} \mathcal{N}(z_i; 0, 1) \prod_{g} \sigma\big(z_{w_g} - z_{\ell_g}\big),$$
$$\log \tilde{p}(z) = -\tfrac{1}{2}\|z\|^2 + \sum_{g} \log \sigma\big(z_{w_g} - z_{\ell_g}\big) + \text{const.}$$

The unnormalized log posterior and its gradient are cheap to evaluate, but the evidence is an N-dimensional integral, and so is every marginal such as $p(z_1 \mid \text{games})$. After observing games, skills are no longer independent, even for players who never met. With two players, the posterior after A beats B ten times is confident that $z_A > z_B$; after each beats the other ten times it is confident that the skills are similar but uncertain about their common level, so its mass is stretched along the diagonal $z_A = z_B$. The posterior is not Gaussian, and conjugate Gaussian computations (Section 2.5) do not apply.

14.2 ▪ Monte Carlo Estimation and Ancestral Sampling

Given independent samples $x^{(1)}, \dots, x^{(S)} \sim p$, the *simple Monte Carlo estimator* of $\Phi = \mathbb{E}_p[\varphi(x)]$ is the sample average

$$\hat{\Phi} = \frac{1}{S}\sum_{s=1}^{S} \varphi\big(x^{(s)}\big).$$

Proposition 14.1 (Simple Monte Carlo). *If $x^{(1)}, \dots, x^{(S)}$ are i.i.d. from p and $\mathrm{Var}_p(\varphi(x)) < \infty$, then*

$$\mathbb{E}[\hat{\Phi}] = \Phi, \qquad \mathrm{Var}(\hat{\Phi}) = \frac{1}{S}\,\mathrm{Var}_p\big(\varphi(x)\big),$$

and $\hat{\Phi} \to \Phi$ almost surely as $S \to \infty$ by the law of large numbers.

The error is of order $\sqrt{\mathrm{Var}_p(\varphi)/S}$ whatever the dimension of x: the accuracy depends on the variance of φ, not on the number of coordinates. This is what makes Monte Carlo attractive in high dimensions, in contrast with the grid. The catch is that it presupposes independent samples from p, which is the hard part.

Remark 14.2 (Monte Carlo Elsewhere in These Notes). The same estimator appears throughout the course. A minibatch gradient in SGD (Section 3.4) is a Monte Carlo estimate of the full gradient. The bootstrap (Algorithm 10.3) is Monte Carlo in which the unknown data distribution is replaced by the empirical distribution of the sample. Stochastic variational inference (Section 15.5) estimates the gradient of its objective with Monte Carlo samples from the variational distribution. □

When the joint distribution is a directed graphical model and each conditional can be sampled, exact samples are easy to produce: sample the variables in a topological order, so that the parents of each node have already been drawn when the node is visited. This is *ancestral sampling*; it is simply the factorization $p(x) = \prod_i p(x_i \mid x_{\mathrm{pa}(i)})$ of Section 12.2 read as a recipe.

Algorithm 14.1 Ancestral Sampling

Input: A DAG with nodes ordered topologically, $x_1, \dots, x_N$ (parents before children); samplers for each $p(x_i \mid x_{\mathrm{pa}(i)})$; number of samples S.

for $s = 1, \dots, S$ **do**
  for $i = 1, \dots, N$ **do**
    Draw $x_i^{(s)} \sim p\big(x_i \mid x_{\mathrm{pa}(i)}^{(s)}\big)$ ▷ the parents are already sampled
  end for
end for

Output: Independent samples $x^{(1)}, \dots, x^{(S)}$ from the joint $p(x)$.

For the DAG of the lectures with $p(x_{1:5}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1)p(x_4 \mid x_2, x_3)p(x_5 \mid x_3)$, one draws x_1, then x_2 and x_3, then x_4, and finally x_5. Generating data from a trained VAE (Section 16.7) or a diffusion model (Algorithm 18.2) is ancestral sampling in the same sense: first the latent variables, then the observations given them.

Ancestral sampling produces samples from the joint, or from the prior, not from a posterior. If some variables are observed, one can sample from the prior and discard the samples that disagree with the evidence; this is correct but wasteful, since the fraction kept is $p(\bar{x}_E)$, which is tiny for improbable evidence. It is a special case of rejection sampling (Section 14.4). Keeping every sample but weighting it by the probability of the evidence given its parents, $\prod_{e \in E} p(\bar{x}_e \mid x_{\mathrm{pa}(e)})$, is a special case of importance sampling, to which we turn now.

14.3 ▪ Importance Sampling

Suppose we cannot sample from $p = \tilde{p}/Z_p$ but can sample from a simpler *proposal* $q = \tilde{q}/Z_q$, and can evaluate both $\tilde{p}$ and $\tilde{q}$. Samples from q over-represent regions where $q > p$ and under-represent regions where $q < p$; importance sampling corrects this by weights. If both densities were normalized,

$$\Phi = \int \varphi(x) \frac{p(x)}{q(x)} q(x)\, \mathrm{d}x \approx \frac{1}{S} \sum_{s=1}^{S} \varphi\big(x^{(s)}\big) \frac{p(x^{(s)})}{q(x^{(s)})}, \qquad x^{(s)} \overset{\text{i.i.d.}}{\sim} q.$$

With unnormalized densities, define $\tilde{w}_s = \tilde{p}(x^{(s)})/\tilde{q}(x^{(s)})$. Since $\mathbb{E}_q[\tilde{p}/\tilde{q}] = Z_p/Z_q$, the average of the $\tilde{w}_s$ estimates the unknown ratio of normalizing constants, and dividing by it gives an estimator that uses only $\tilde{p}$ and $\tilde{q}$.

Algorithm 14.2 Importance Sampling (Self-Normalized)

Input: Unnormalized target $\tilde{p}$; proposal q that can be sampled, with unnormalized density $\tilde{q}$; test function φ; number of samples S.

for $s = 1, \dots, S$ **do**
  Draw $x^{(s)} \sim q$.
  $\tilde{w}_s \leftarrow \tilde{p}(x^{(s)}) \big/ \tilde{q}(x^{(s)})$ ▷ unnormalized importance weight
end for
$w_s \leftarrow \tilde{w}_s \big/ \sum_{r=1}^{S} \tilde{w}_r$ for $s = 1, \dots, S$ ▷ self-normalization

Output: $\hat{\Phi}_{\mathrm{iw}} = \sum_{s=1}^{S} w_s \, \varphi(x^{(s)}) \approx \mathbb{E}_p[\varphi(x)]$, and $\frac{1}{S}\sum_s \tilde{w}_s \approx Z_p/Z_q$.

The proposal must be positive wherever $p\varphi$ is nonzero (for the self-normalized estimator, wherever p is nonzero). The self-normalized estimator is consistent but slightly biased for finite S, because it is a ratio of two averages. When q is normalized ($Z_q = 1$), the average weight estimates Z_p itself, which for a posterior is the evidence $p(x)$. The variance is small only when q is close to p: if q has lighter tails than p, a few samples receive enormous weights. In high dimensions this is the typical situation, and almost all of the normalized weight concentrates on one or two samples.

14.4 ▪ Rejection Sampling

Rejection sampling produces exact samples from p, again from a proposal q, under the additional assumption that we know a constant c with

$$c\,\tilde{q}(x) \geq \tilde{p}(x) \qquad \text{for all } x.$$

The idea is to draw points uniformly under the graph of $c\tilde{q}$ and keep those that also lie under the graph of $\tilde{p}$.

Algorithm 14.3 Rejection Sampling

Input: Unnormalized target $\tilde{p}$; proposal q with unnormalized density $\tilde{q}$; constant c with $c\tilde{q} \geq \tilde{p}$; number of samples S.

Initialize: An empty list of accepted samples.

while fewer than S samples have been accepted **do**
  Draw $x \sim q$ and then $u \sim \mathrm{Unif}\big[0,\, c\tilde{q}(x)\big]$.
  if $u \leq \tilde{p}(x)$ **then**
    Accept: append x to the list. ▷ otherwise discard x and u
  end if
end while

Output: Independent samples $x^{(1)}, \dots, x^{(S)}$ from p.

Proposition 14.3 (Correctness and Acceptance Rate). *The accepted samples are independent draws from p. Each proposal is accepted with probability*

$$\mathbb{P}(\textit{accept}) = \mathbb{E}_{x\sim q}\Big[\frac{\tilde{p}(x)}{c\tilde{q}(x)}\Big] = \frac{Z_p}{cZ_q} = \frac{\textit{area under } \tilde{p}}{\textit{area under } c\tilde{q}}.$$

Given x, acceptance has probability $\tilde{p}(x)/(c\tilde{q}(x))$, so the density of accepted points is proportional to $q(x)\tilde{p}(x)/(c\tilde{q}(x)) \propto \tilde{p}(x)$; this is the content of the proposition. Its practical message is the acceptance

rate. In high dimensions the envelope condition forces c to be enormous: even for a Gaussian target and a Gaussian proposal whose standard deviation is only 1% larger, $c = 1.01^D \approx 2 \times 10^4$ for $D = 1000$. In general c, and hence the expected number of proposals per accepted sample, grows exponentially with the dimension. Finding a valid c is itself hard when the location and height of the modes of $\tilde{p}$ are unknown.

Pros and Cons: Importance and rejection sampling need only an unnormalized target, are simple, and produce independent samples (weighted samples in the case of importance sampling); importance sampling also estimates normalizing constants. Both, however, depend on a proposal that resembles the target globally. In high dimensions no fixed proposal does, the importance weights degenerate, and the rejection rate approaches one.

14.5 ▪ Markov Chains

The way out is to give up independence. Markov chain Monte Carlo generates a dependent sequence in which each sample is proposed near the previous one, so that the proposal only needs to be good locally. This section collects the facts about Markov chains that MCMC relies on. Markov chains as models of sequential data, together with higher-order chains and Markov language models, are discussed in Chapter 12.

A sequence $x_1, x_2, \dots$ is a (first-order) *Markov chain* if $p(x_t \mid x_{1:t-1}) = p(x_t \mid x_{t-1})$, so that $p(x_{1:T}) = p(x_1) \prod_{t=2}^{T} p(x_t \mid x_{t-1})$. It is *time-homogeneous* (the slides call it stationary) if the transition probabilities do not depend on t; we only consider such chains. For a finite state space $\{1, \dots, K\}$ the transitions form the *transition matrix*

$$A_{ij} = p(x_t = j \mid x_{t-1} = i), \qquad \sum_{j} A_{ij} = 1.$$

The n-step transition probabilities are A^n (the Chapman–Kolmogorov equations $A(m+n) = A(m)A(n)$). Writing $\pi_t(j) = p(x_t = j)$ as a column vector, the distribution evolves as $\pi_t = A^\top \pi_{t-1} = (A^\top)^t \pi_0$.

Definition 14.4 (Stationary Distribution, Detailed Balance). *A distribution π is* stationary *(invariant) for A if $\pi = A^\top \pi$, that is, $\sum_i \pi_i A_{ij} = \pi_j$ for all j. The chain satisfies* detailed balance *with respect to π if*

$$\pi_i A_{ij} = \pi_j A_{ji} \qquad \text{for all } i, j.$$

For a general state space with transition kernel $T(x' \mid x)$, the conditions read $\int p(x) T(x' \mid x)\, \mathrm{d}x = p(x')$ and $p(x)T(x' \mid x) = p(x')T(x \mid x')$. □

A stationary distribution is an eigenvector of $A^\top$ with eigenvalue 1, normalized to sum to one. Such an eigenvector exists ($A\mathbf{1} = \mathbf{1}$ shows that 1 is an eigenvalue, and the Perron–Frobenius theorem gives a nonnegative eigenvector), but it need not be unique. Detailed balance says that in equilibrium the probability flow from i to j equals the flow back; summing it over i gives $\sum_i \pi_i A_{ij} = \pi_j \sum_i A_{ji} = \pi_j$.

Proposition 14.5 (Detailed Balance Implies Stationarity). *If a chain satisfies detailed balance with respect to π, then π is a stationary distribution of the chain.*

Detailed balance is a local condition on pairs of states and is much easier to verify than stationarity; every MCMC algorithm below is built to satisfy it. For example, the two-state chain with $A = \left(\begin{smallmatrix} 1-a & a \\ b & 1-b \end{smallmatrix}\right)$

satisfies $\pi_1 a = \pi_2 b$ for $\pi = (b, a)/(a + b)$, which is therefore its stationary distribution. A chain is *irreducible* if every state can be reached from every other state. For a finite irreducible and aperiodic chain, the stationary distribution is unique, $\pi_t \to \pi$ from any initial distribution, and time averages along a single trajectory converge to expectations under π. This last property is what MCMC uses.

14.6 ▪ The Metropolis–Hastings Algorithm

Markov chain Monte Carlo (MCMC) constructs a Markov chain whose stationary distribution is the target p, runs it, and uses the trajectory as samples. Since each state depends on the previous one, the samples are correlated, and the first ones, before the chain has reached the high-probability region, are discarded as *burn-in*. Expectations are estimated by the ergodic average $\mathbb{E}_p[\varphi(x)] \approx \frac{1}{S - S_0} \sum_{t=S_0+1}^{S} \varphi(x^{(t)})$.

The Metropolis–Hastings (MH) algorithm uses a proposal $q(x' \mid x)$ that depends on the current state, for example a Gaussian centered at x. Unlike in importance and rejection sampling, q does not need to resemble p: it only has to make local moves, which are then accepted or rejected so that the chain has the right stationary distribution.

Algorithm 14.4 Metropolis–Hastings

Input: Unnormalized target $\tilde{p}$; proposal kernel $q(x' \mid x)$; initial state $x^{(0)}$; number of iterations S; burn-in S_0.

for $t = 0, \dots, S - 1$ **do**
  Draw a candidate $x' \sim q(\cdot \mid x^{(t)})$.
  $\alpha \leftarrow \min\left\{1,\ \dfrac{\tilde{p}(x')\, q(x^{(t)} \mid x')}{\tilde{p}(x^{(t)})\, q(x' \mid x^{(t)})}\right\}$ ▷ Z cancels
  Draw $u \sim \mathrm{Unif}[0, 1]$.
  if $u \le \alpha$ **then**
    $x^{(t+1)} \leftarrow x'$ ▷ accept
  else
    $x^{(t+1)} \leftarrow x^{(t)}$ ▷ reject: the current state is repeated
  end if
end for

Output: Correlated samples $x^{(S_0+1)}, \dots, x^{(S)}$, approximately distributed according to p.

Theorem 14.6 (Correctness of Metropolis–Hastings). *The chain generated by Algorithm 14.4 satisfies detailed balance with respect to p; hence p is a stationary distribution.*

For $x' \neq x$ the transition density is $q(x' \mid x)\alpha(x' \mid x)$, and

$$p(x)\, q(x' \mid x)\, \alpha(x' \mid x) = \min\big\{p(x)q(x' \mid x),\ p(x')q(x \mid x')\big\},$$

which is symmetric in x and x'; this is detailed balance, and Proposition 14.5 finishes the argument. Two features of the algorithm are essential. Only the ratio $\tilde{p}(x')/\tilde{p}(x)$ enters, so the normalizing constant is never needed. And a rejected candidate does not disappear as in rejection sampling: the current state is counted again, which is how the chain spends more time in regions of high probability.

When the proposal is symmetric, $q(x' \mid x) = q(x \mid x')$, the ratio reduces to $\tilde{p}(x')/\tilde{p}(x)$: moves uphill are always accepted, and moves downhill are accepted with probability equal to the density ratio. This is the original *Metropolis* algorithm. Its most common instance is *random-walk Metropolis*, $x' = x + \sigma\xi$

with $\xi \sim \mathcal{N}(0, I)$. The step size σ governs a trade-off: small steps are almost always accepted but explore slowly, while large steps are mostly rejected and the chain gets stuck.

Pros and Cons: MCMC needs only the unnormalized density, works in high dimensions where independent proposals fail, and is asymptotically exact: with enough iterations, averages converge to the true expectations. However, the samples are correlated, so many iterations may carry little information; there is no reliable way to know when the chain has converged; random-walk proposals explore slowly in high dimensions and between separated modes; and the chain is inherently sequential.

14.7 ▪ Gibbs Sampling

Gibbs sampling updates one coordinate at a time, drawing it exactly from its *full conditional* distribution given all other coordinates. Write $x = (x_1, \dots, x_d)$ and x_{-j} for all components except x_j.

Algorithm 14.5 Gibbs Sampling

Input: Samplers for the full conditionals $p(x_j \mid x_{-j})$, $j = 1, \dots, d$; initial state $x^{(0)}$; number of sweeps S; burn-in S_0.

for $t = 1, \dots, S$ **do**
  for $j = 1, \dots, d$ **do**
    Draw $x_j^{(t)} \sim p\big(x_j \mid x_1^{(t)}, \dots, x_{j-1}^{(t)}, x_{j+1}^{(t-1)}, \dots, x_d^{(t-1)}\big)$ ▷ newest values
  end for
end for

Output: Samples $x^{(S_0+1)}, \dots, x^{(S)}$, approximately distributed according to p.

There is no accept/reject step: every draw is kept. In fact a Gibbs update is a Metropolis–Hastings step whose proposal is the full conditional itself, and for this proposal the MH acceptance probability equals one. The full conditionals are often easy even when the joint is not, because $p(x_j \mid x_{-j}) \propto p(x)$ involves only the factors that contain x_j, that is, the Markov blanket of x_j in the graphical model.

Bivariate Gaussian. For (X_1, X_2) with means μ_1, μ_2, unit variances, and correlation ρ, Proposition 2.14 gives $X_1 \mid X_2 = x_2 \sim \mathcal{N}\big(\mu_1 + \rho(x_2 - \mu_2),\ 1 - \rho^2\big)$ and symmetrically for X_2. The sampler alternates these two draws. When $|\rho|$ is close to one, the conditional variance $1 - \rho^2$ is tiny, the chain moves in small axis-parallel steps along a narrow diagonal ridge, and mixing is slow. The TrueSkill posterior after many games between two evenly matched players has exactly this shape.

Ising model. For the denoising posterior of Section 13.4, $p(x_i \mid x_{-i}) \propto \exp(x_i h_i)$ with local field $h_i = J \sum_{j \in N(i)} x_j + \beta y_i$, so $p(x_i = +1 \mid x_{-i}) = 1/(1 + e^{-2h_i})$. The real power of Gibbs sampling is in models like this, where the joint is intractable but every full conditional is a coin flip.

Restricted Boltzmann machines. In an Ising model on a bipartite graph between visible units x and hidden units h, the hidden units are conditionally independent given x and vice versa (by Hammersley–Clifford, Section 12.4). Gibbs sampling can then update whole blocks at once, alternating $h \sim p(h \mid x)$ and $x \sim p(x \mid h)$, each a product of independent coin flips.

For TrueSkill with the logistic likelihood, by contrast, the full conditional of a skill is not a standard distribution, so Gibbs sampling would need an inner sampler; gradient-based methods are more convenient.

Remark 14.7 (Connection to Mean-Field Updates). Gibbs sampling and naive mean-field variational inference (Algorithm 15.1) have the same coordinate-wise structure. Gibbs draws x_j from its conditional given the *sampled* values of its neighbors; mean field sets $q_j(x_j) \propto \exp\big(\mathbb{E}_{q_{-j}}[\log p(x)]\big)$, which replaces the neighbors by their *expectations*. For the Ising posterior, Gibbs uses $h_i = J\sum_j x_j + \beta y_i$ with sampled x_j, while mean field iterates $\mu_i \leftarrow \tanh\big(J\sum_j \mu_j + \beta y_i\big)$ with means $\mu_j = \mathbb{E}_q[x_j]$. □

14.8 ▪ Hamiltonian Monte Carlo

Random-walk proposals ignore the shape of the target. Hamiltonian Monte Carlo (HMC) is a Metropolis–Hastings algorithm with a proposal that follows the gradient of the log density and can therefore make long moves that are still accepted with high probability. It uses a physical analogy. Define the *potential energy* $E(x) = -\log \tilde{p}(x)$, so that $p(x) \propto e^{-E(x)}$ and low energy means high probability. Introduce an auxiliary *momentum* v of the same dimension with *kinetic energy* $K(v) = \frac{1}{2}\|v\|^2$, and the *Hamiltonian* (total energy) $H(x, v) = E(x) + K(v)$. The joint distribution

$$p(x, v) \propto e^{-H(x,v)} = e^{-E(x)}\, e^{-\frac{1}{2}\|v\|^2}$$

has $p(x)$ as its x-marginal and makes $v \sim \mathcal{N}(0, I)$ independent of x, so it suffices to sample (x, v) and discard v.

Picture a frictionless ball rolling on the surface E. Its motion follows Hamilton's equations

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{\partial H}{\partial v} = v, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = -\frac{\partial H}{\partial x} = -\nabla E(x),$$

which conserve the energy H (so $p(x, v)$ is constant along the trajectory), are reversible (flipping the momentum retraces the path), and preserve volume in (x, v)-space. A proposal obtained by following the dynamics and flipping the momentum would therefore always be accepted. In practice the dynamics are simulated with the *leapfrog integrator*, a few steps of size ε of

$$v \leftarrow v - \tfrac{\varepsilon}{2}\nabla E(x), \qquad x \leftarrow x + \varepsilon v, \qquad v \leftarrow v - \tfrac{\varepsilon}{2}\nabla E(x),$$

which is more accurate than Euler's method and is still deterministic, reversible, and volume preserving. Discretization changes H slightly, and a Metropolis–Hastings test corrects for the error.

Algorithm 14.6 Hamiltonian Monte Carlo (HMC)

Input: Energy $E(x) = -\log \tilde{p}(x)$ and its gradient; step size ε; number of leapfrog steps L; initial state $x^{(0)}$; number of iterations S.

for $t = 1, \dots, S$ **do**
  $x \leftarrow x^{(t-1)}$; draw a fresh momentum $v \sim \mathcal{N}(0, I)$.
  $(x', v') \leftarrow (x, v)$
  for $\ell = 1, \dots, L$ **do** ▷ leapfrog integration
    $v' \leftarrow v' - \frac{\varepsilon}{2}\nabla E(x'); \quad x' \leftarrow x' + \varepsilon v'; \quad v' \leftarrow v' - \frac{\varepsilon}{2}\nabla E(x')$
  end for
  $v' \leftarrow -v'$ ▷ makes the proposal its own inverse
  With probability $\min\big\{1, \exp\big(H(x, v) - H(x', v')\big)\big\}$ set $x^{(t)} \leftarrow x'$; otherwise $x^{(t)} \leftarrow x$.
end for

Output: Samples $x^{(1)}, \dots, x^{(S)}$, approximately distributed according to p (momenta are discarded).

Resampling the momentum at every iteration lets the chain move between energy levels; the leapfrog trajectory then carries it far along the current level set. If the integration were exact, $H(x', v') = H(x, v)$ and every proposal would be accepted; with a small step size the acceptance rate stays high even for long trajectories. The step size ε and the trajectory length L must be tuned: too large a step makes the energy error and the rejection rate grow, too short a trajectory turns HMC back into a random walk.

Remark 14.8 (The Acceptance Probability). The slides write the acceptance probability as $\min\{1, \exp(H(x^{(t-1)}, v^{(t-1)}))/\exp(H(x', v'))\}$. The energy in the numerator must be evaluated at the starting position and the *freshly drawn* momentum v of the current iteration, as in Algorithm 14.6; the momentum from the previous iteration plays no role. □

Remark 14.9 (Connection to Gradient Descent and Langevin Monte Carlo). Each leapfrog half-step $v \leftarrow v + \frac{\varepsilon}{2}\nabla \log \tilde{p}(x)$ is a gradient ascent step on the log density, as in Algorithm 3.1, applied to the momentum. With a single leapfrog step, $L = 1$, the proposal becomes

$$x' = x + \eta\, \nabla \log \tilde{p}(x) + \sqrt{2\eta}\,\xi, \qquad \xi \sim \mathcal{N}(0, I), \quad \eta = \varepsilon^2/2,$$

gradient ascent plus Gaussian noise. This is the Langevin proposal; combined with a Metropolis–Hastings test, using the proposal density $q(x' \mid x) = \mathcal{N}\big(x';\, x + \eta\nabla \log \tilde{p}(x),\, 2\eta I\big)$, it is the Metropolis-adjusted Langevin algorithm (MALA). Noisy gradient-like steps of a closely related form reappear in the sampling procedure of diffusion models (Chapter 18). □

Pros and Cons: HMC uses gradient information to make distant proposals with high acceptance probability, and it scales much better with dimension than random-walk Metropolis; with automatic differentiation the gradient of a log posterior such as TrueSkill's is cheap to compute. However, it requires a differentiable log density on a continuous space, each iteration costs L gradient evaluations, its performance is sensitive to ε and L, and it still struggles to move between well-separated modes.

14.9 ▪ MCMC in Practice: Diagnostics

MCMC output is used like any sample from the posterior: posterior means, medians, and quantiles are computed from the draws, and the posterior predictive distribution (Section 2.4) is sampled by simulating new outcomes given each draw. Before doing so, one must ask whether the chain has run long enough, whether it started too far from the bulk of the distribution, and how much information correlated samples actually carry. Some practices follow directly: run several chains in parallel from dispersed starting points, discard a burn-in phase, and examine how well the chains have mixed. Two standard numerical summaries make this precise. (The lectures note that the formulas need not be memorized.)

Definition 14.10 (Potential Scale Reduction $\hat{R}$). *Let x_{ij}, $i = 1, \dots, n$, $j = 1, \dots, m$, be m chains of length n after burn-in, with chain means $\bar{x}_{\cdot j}$ and overall mean $\bar{x}_{\cdot\cdot}$. Define the between-chain and within-chain variances*

$$B = \frac{n}{m-1}\sum_{j=1}^{m}\big(\bar{x}_{\cdot j} - \bar{x}_{\cdot\cdot}\big)^2, \qquad W = \frac{1}{m}\sum_{j=1}^{m} s_j^2, \quad s_j^2 = \frac{1}{n-1}\sum_{i=1}^{n}\big(x_{ij} - \bar{x}_{\cdot j}\big)^2,$$

the pooled estimate $\widehat{\mathrm{var}}^{+} = \frac{n-1}{n}W + \frac{1}{n}B$, and $\hat{R} = \sqrt{\widehat{\mathrm{var}}^{+}/W}$. Split-$\hat{R}$ *applies the same formula after cutting each chain into its first and second halves.* □

If the chains have not mixed, they sit in different regions, the pooled variance exceeds the within-chain variance, and $\hat{R} > 1$. Splitting the chains also detects non-stationarity within a single chain.

Definition 14.11 (Effective Sample Size). *For mn draws with autocorrelations $\rho_t = \operatorname{corr}(x_0, x_t)$ at lag t,*

$$n_{\text{eff}} = \frac{mn}{1 + 2\sum_{t=1}^{\infty} \rho_t},$$

where in practice the ρ_t are estimated from the chains. □

For i.i.d. draws $\operatorname{Var}(\bar{x}) = \sigma^2/n$; for correlated draws the variance is larger by the factor $1 + 2\sum_t \rho_t$, and n_{eff} is the number of independent draws that would give the same precision. The rule of thumb from the lectures is to collect the draws once $\hat{R}$ is near 1 and n_{eff} exceeds about 10 per chain for every scalar quantity of interest. Passing these checks only shows that the chains *appear* stationary and well mixed; a chain can pass every test and still be far from convergence, for example if all chains miss the same mode.

Remark 14.12. The samplers of this chapter, together with Langevin algorithms, tempering, and particle filters, are developed in detail in the CAAM 31511 notes (*A First Course in Monte Carlo Methods*), which use the same layout as these notes. □

Method	Requires	Produces	Main weakness
Ancestral sampling	DAG with samplable conditionals	exact i.i.d. joint samples	no conditioning on evidence
Importance sampling	$\tilde{p}$; proposal q (sample, evaluate)	weighted samples; Z_p/Z_q	weight degeneracy in high dimension
Rejection sampling	$\tilde{p}$; q; envelope constant c	exact i.i.d. samples	acceptance rate $Z_p/(cZ_q)$, exponentially small
Metropolis–Hastings	$\tilde{p}$; local proposal	correlated samples	slow random-walk exploration
Gibbs sampling	samplable full conditionals	correlated samples	slow when variables are strongly dependent
HMC	$\nabla \log \tilde{p}$; continuous x	correlated samples, long moves	tuning of ε, L; cost per iteration

Limitations and what comes next: Sampling methods are asymptotically exact, but they are slow: MCMC needs many sequential iterations, each of which evaluates the unnormalized density, and for a posterior that means a pass over the whole data set; convergence is hard to diagnose; and the evidence $p(x)$, needed to compare models or fit hyperparameters, is not a by-product. Chapter 15 trades exactness for speed. It approximates the posterior by the closest member of a tractable family, which turns inference into an optimization problem that can be solved with stochastic gradients, scales to large data, and yields a lower bound on the evidence.

Chapter 15

Variational Inference

The central computation in a probabilistic model is the posterior distribution of the unobserved variables given the observed ones. Exact inference (Chapter 13) is feasible only when the graph is a tree or has small cliques after elimination, and Markov chain Monte Carlo (Chapter 14) is asymptotically exact but slow and hard to diagnose. Variational inference takes a third route: it gives up exactness from the start and turns inference into *optimization*. We choose a family of tractable distributions and search it for the member closest to the posterior. Section 15.1 states the problem, Section 15.2 introduces the Kullback–Leibler divergence and the two ways of using it, and Section 15.3 derives the classical mean-field algorithm for Markov random fields. Section 15.4 rewrites the objective as the evidence lower bound, which only requires the unnormalized joint density, and Section 15.5 shows how to maximize it by stochastic gradients with the reparameterization trick. Section 15.6 compares the result with MCMC.

15.1 ▪ Posterior Inference in Latent Variable Models

A *latent variable model* specifies a joint distribution

$$p(x, z) = p(z)\, p(x \mid z),$$

where x are the observations, z are the unobserved (latent) variables, $p(z)$ is the *prior*, and $p(x \mid z)$ is the *likelihood*, the part of the model that gives the latent variables their meaning. Everything we want to know about z after seeing x is contained in the posterior

$$p(z \mid x) = \frac{p(x \mid z)\, p(z)}{p(x)}, \qquad p(x) = \int p(x, z)\, \mathrm{d}z.$$

The normalizing constant $p(x)$ is called the *evidence* or *marginal likelihood*.

Remark 15.1 (TrueSkill). The running example of the course is TrueSkill, a rating system for competitive games (Chapter 14). Each player i has an unobserved skill z_i with an independent Gaussian prior, and the probability that i beats j is $\sigma(z_i - z_j)$ with the logistic function $\sigma(t) = 1/(1 + e^{-t})$. With two players and a broad prior, the posterior after a few games is no longer Gaussian. After A beats B ten times it is confident that $z_A > z_B$; after each beats the other ten times it is confident that $z_A \approx z_B$ but remains uncertain about their common level. With N players the posterior is a non-Gaussian density on $\mathbb{R}^N$, and the skills become dependent even for players who never met. □

When z is high-dimensional the integral defining $p(x)$ is intractable, and with it most posterior computations:

(i) the posterior density $p(z \mid x) = p(z)p(x \mid z)/p(x)$ itself;

(ii) the evidence $p(x)$, used to compare models and to fit model parameters;

(iii) marginals such as $p(z_1 \mid x) = \int p(z_1, \dots, z_D \mid x)\, \mathrm{d}z_2 \cdots \mathrm{d}z_D$, for example the skill of one player given all games;

(iv) exact samples $z \sim p(z \mid x)$, used to summarize plausible hypotheses, to predict, and to decide.

What remains easy is the evaluation of the joint density $p(x, z)$ at a given pair (x, z): the posterior is known up to its normalizing constant. Sampling methods exploit exactly this fact, and so does variational inference.

Definition 15.2 (Variational Inference). *Given a target distribution $p(z \mid x)$ and a family $\mathcal{Q}$ of tractable distributions on the latent space,* variational inference *computes*

$$q^* = \arg\min_{q \in \mathcal{Q}} \; D_{\mathrm{KL}}\big(q \,\|\, p(\cdot \mid x)\big)$$

by an iterative optimization method, and uses q^* in place of the posterior. Typically $\mathcal{Q} = \{q_\phi : \phi \in \Phi\}$ is parametric, for example $q_\phi = \mathcal{N}(\mu, \Sigma)$ with $\phi = (\mu, \Sigma)$, or factorized, $q(z) = \prod_j q_j(z_j)$. □

Three choices define a variational method: the family $\mathcal{Q}$, the measure of discrepancy, and the optimizer. Usually no member of $\mathcal{Q}$ equals the posterior; the approximation error is the price paid for tractability.

15.2 ▪ The Kullback–Leibler Divergence

Definition 15.3 (KL Divergence). *For densities (or mass functions) q and p on the same space,*

$$D_{\mathrm{KL}}(q \,\|\, p) = \int q(z) \log \frac{q(z)}{p(z)}\, \mathrm{d}z = \mathbb{E}_{z \sim q}\Big[\log \frac{q(z)}{p(z)}\Big],$$

with the integral replaced by a sum in the discrete case. □

Proposition 15.4 (Properties of the KL Divergence). *$D_{\mathrm{KL}}(q \,\|\, p) \geq 0$, with equality if and only if $q = p$. In general $D_{\mathrm{KL}}(q \,\|\, p) \neq D_{\mathrm{KL}}(p \,\|\, q)$, so the KL divergence is not a metric.*

Nonnegativity follows from Jensen's inequality applied to the concave logarithm. Asymmetry means there are two different ways to fit q to p, and they behave very differently.

Definition 15.5 (I-Projection and M-Projection). *Let $\mathcal{Q}$ be a family of distributions and p a target. The* information projection *(I-projection) and the* moment projection *(M-projection) of p onto $\mathcal{Q}$ are*

$$q_I = \arg\min_{q \in \mathcal{Q}} D_{\mathrm{KL}}(q \,\|\, p) = \arg\min_{q \in \mathcal{Q}} \mathbb{E}_{x \sim q}\Big[\log \frac{q(x)}{p(x)}\Big],$$

$$q_M = \arg\min_{q \in \mathcal{Q}} D_{\mathrm{KL}}(p \,\|\, q) = \arg\min_{q \in \mathcal{Q}} \mathbb{E}_{x \sim p}\Big[\log \frac{p(x)}{q(x)}\Big].$$

□

The I-projection averages $\log(q/p)$ under q. It is large where q puts mass and p has almost none, so q_I avoids such regions: it is *mode seeking*, tends to lock onto one mode of a multimodal target, and underestimates the spread of p. The M-projection averages $\log(p/q)$ under p. It is large where p has mass and q has almost none, so q_M must cover all of p: it is *mass covering* and may put mass between the modes. Figure 15.1 shows both for a bimodal target and the family of Gaussians.

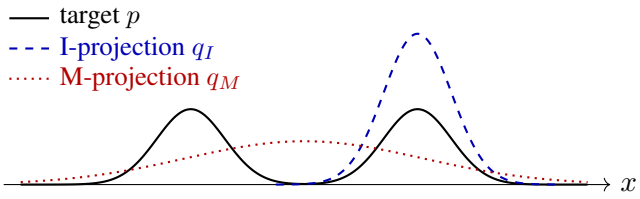


Figure 15.1. *I- and M-projection of a bimodal density onto the Gaussian family. The I-projection fits one mode (the other mode gives an equally good, symmetric solution); the M-projection matches the mean and variance of p and puts mass where p has little.*

Proposition 15.6 (M-Projection onto an Exponential Family). *Let $\mathcal{Q} = \{q_\eta(x) = h(x)\exp(\eta^\top T(x) - A(\eta))\}$ be an exponential family (Section 2.3). If the M-projection $q_M = q_{\eta^*}$ exists, it matches the expected sufficient statistics of the target:*

$$\mathbb{E}_{x\sim q_{\eta^*}}[T(x)] = \mathbb{E}_{x\sim p}[T(x)].$$

In particular, the M-projection onto the Gaussians has the mean and covariance of p.

Setting the gradient of $D_{\mathrm{KL}}(p \,\|\, q_\eta) = \text{const} - \eta^\top \mathbb{E}_p[T(x)] + A(\eta)$ to zero gives $\nabla A(\eta) = \mathbb{E}_p[T(x)]$, and $\nabla A(\eta) = \mathbb{E}_{q_\eta}[T(x)]$ is the mean of the sufficient statistics.

Remark 15.7 (Connection to Maximum Likelihood). If p is the empirical distribution of a sample $x_1, \dots, x_n$, then $D_{\mathrm{KL}}(p \,\|\, q_\theta) = \text{const} - \frac{1}{n}\sum_i \log q_\theta(x_i)$. Maximum likelihood estimation (Section 2.2) is therefore the M-projection of the data onto the model, and Proposition 15.6 is the moment-matching property of exponential-family MLEs from Section 2.3. □

Remark 15.8 (Maximum Entropy). The entropy $\mathcal{H}(p) = -\mathbb{E}_{x\sim p}[\log p(x)]$ measures the uncertainty of p. Among all distributions satisfying the moment constraints $\mathbb{E}_p[f_i(x)] = t_i$, $i = 1, \dots, k$, the entropy is maximized by an exponential family with sufficient statistics $(f_1, \dots, f_k)$. Exponential families are therefore the least committal distributions consistent with given moments, which is another reason they are the natural variational families. □

Which direction should we use for inference? The M-projection is attractive, since it gets the moments right, but it requires expectations under $p(z \mid x)$, which is exactly what we cannot compute. The I-projection only requires expectations under q, which we choose to be tractable. Almost all variational inference algorithms therefore use the I-projection, $D_{\mathrm{KL}}(q \,\|\, p)$, and inherit its mode-seeking behavior.

15.3 ▪ Mean-Field Approximation

We first apply the I-projection to a Markov random field (Section 12.4). In this section $x = (x_1, \dots, x_N)$ denotes all variables of the field (any observed variables have already been fixed, as in the denoising example of Section 13.4), and

$$p(x) = \frac{1}{Z}\prod_{C\in\mathcal{C}} \psi_C(x_C) = \exp\Big(\sum_{C\in\mathcal{C}} \log\psi_C(x_C) - \log Z\Big).$$

For any q,

$$D_{\mathrm{KL}}(q \,\|\, p) = \mathbb{E}_q[\log q(x)] - \sum_{C\in\mathcal{C}} \mathbb{E}_q[\log\psi_C(x_C)] + \log Z,$$

and since $\log Z$ does not depend on q,

$$q^* = \arg\min_{q \in \mathcal{Q}} D_{\mathrm{KL}}(q \,\|\, p) = \arg\max_{q \in \mathcal{Q}} \Big\{ \sum_{C \in \mathcal{C}} \mathbb{E}_q[\log \psi_C(x_C)] + \mathcal{H}(q) \Big\}. \tag{15.1}$$

The intractable partition function has dropped out. The objective trades off fitting the potentials, which favors concentrating q on high-potential configurations, against the entropy, which keeps q spread out. If $p \in \mathcal{Q}$ the solution is $q^* = p$, but this almost never happens for a tractable $\mathcal{Q}$.

Definition 15.9 (Naive Mean-Field Family). *The* (naive) mean-field family *consists of the fully factorized distributions* $q(x) = \prod_{i=1}^{N} q_i(x_i)$. □

Under this family both terms of (15.1) simplify. The marginal of a clique is a product, $q(x_C) = \prod_{i \in C} q_i(x_i)$, and the entropy is additive, $\mathcal{H}(q) = \sum_i \mathcal{H}(q_i)$, because $\log q(x) = \sum_i \log q_i(x_i)$ and each term only involves one coordinate. For a pairwise MRF with edge set E and potentials ψ_{ij} the problem becomes

$$\max_{q_1, \dots, q_N} \sum_{(i,j) \in E} \sum_{x_i, x_j} q_i(x_i)\, q_j(x_j) \log \psi_{ij}(x_i, x_j) - \sum_i \sum_{x_i} q_i(x_i) \log q_i(x_i) \tag{15.2}$$

subject to $q_i(x_i) \geq 0$ and $\sum_{x_i} q_i(x_i) = 1$. This objective is not concave jointly and has many local maxima, but it is easy to maximize over one factor q_i with the others fixed. Writing the Lagrangian for the normalization constraint, differentiating with respect to $q_i(x_i)$, and setting the derivative to zero gives the closed-form update

$$q_i(x_i) \propto \exp\Big(\sum_{j \in N(i)} \sum_{x_j} q_j(x_j) \log \psi_{ij}(x_i, x_j) \Big), \tag{15.3}$$

where $N(i)$ is the set of neighbors of i. If the field also has node potentials $\psi_i(x_i)$, the term $\log \psi_i(x_i)$ is added inside the exponential.

Algorithm 15.1 Naive Mean-Field (Coordinate Ascent)

Input: Pairwise MRF with potentials ψ_{ij} on the edges E (and optionally node potentials ψ_i); tolerance ϵ.

Initialize: q_i uniform on the states of x_i, for every node i.

repeat
  for $i = 1, \dots, N$ **do** ▷ any fixed or random order
    $\ell_i(x_i) \leftarrow \log \psi_i(x_i) + \sum_{j \in N(i)} \sum_{x_j} q_j(x_j) \log \psi_{ij}(x_i, x_j)$ for every state x_i
    $q_i(x_i) \leftarrow \exp(\ell_i(x_i)) \big/ \sum_{x'} \exp(\ell_i(x'))$
  end for
until the objective (15.2), plus the node terms $\sum_i \sum_{x_i} q_i(x_i) \log \psi_i(x_i)$, increases by less than ϵ

Output: Factors $q_1, \dots, q_N$; q_i approximates the marginal $p(x_i)$.

Each update maximizes the objective exactly in one block of variables, so the objective never decreases and the algorithm converges, but only to a local maximum that depends on the initialization and the update order. The output approximates each marginal by q_i and ignores all correlations between variables.

Remark 15.10 (Mean Field for the Ising Model). For the Ising model (Section 12.5) with spins $x_i \in \{-1, +1\}$ and $p(x) \propto \exp\big(\sum_i b_i x_i + \sum_{(i,j)\in E} J_{ij} x_i x_j\big)$, the update (15.3) gives $q_i(x_i) \propto \exp\big(x_i(b_i + \sum_{j\in N(i)} J_{ij}\mu_j)\big)$, where $\mu_j = \mathbb{E}_{q_j}[x_j]$. In terms of the means,

$$\mu_i \leftarrow \tanh\Big(b_i + \sum_{j\in N(i)} J_{ij}\mu_j\Big).$$

Each spin feels the *average* field produced by its neighbors, which is the origin of the name "mean field". Applied to the noisy-image MRF of Section 13.4, the rounded means $\operatorname{sign}(\mu_i)$ give a denoised image. □

Remark 15.11 (Connection to Gibbs Sampling and Belief Propagation). The Gibbs sampler (Algorithm 14.5) visits the nodes in turn and draws x_i from $p(x_i \mid x_{N(i)}) \propto \exp\big(\sum_{j\in N(i)} \log \psi_{ij}(x_i, x_j)\big)$, using the current *values* of the neighbors. Mean field performs the same sweep but replaces the neighbors' values by *averages* under q_j; it is a deterministic, averaged version of Gibbs sampling. Loopy belief propagation (Algorithm 13.3) is another local, iterative scheme on the same graph; it sends edge-specific messages instead of one shared belief per node, and its fixed points correspond to a different variational objective. □

15.4 ▪ The Evidence Lower Bound

We return to a general latent variable model $p(x, z)$ and a parametric family $q_\phi(z)$. The objective $D_{\mathrm{KL}}(q_\phi \,\|\, p(\cdot \mid x))$ cannot be evaluated, because it involves the normalized posterior. Expanding $p(z \mid x) = p(x, z)/p(x)$ separates the unknown constant:

$$D_{\mathrm{KL}}\big(q_\phi(z) \,\|\, p(z \mid x)\big) = \mathbb{E}_{z\sim q_\phi}\Big[\log \frac{q_\phi(z)\, p(x)}{p(x, z)}\Big] = -\mathbb{E}_{z\sim q_\phi}\big[\log p(x, z) - \log q_\phi(z)\big] + \log p(x).$$

Definition 15.12 (Evidence Lower Bound). *The* evidence lower bound *(ELBO) of q_ϕ is*

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{z\sim q_\phi}\big[\log p(x, z) - \log q_\phi(z)\big] = \mathbb{E}_{z\sim q_\phi}[\log p(x, z)] + \mathcal{H}(q_\phi).$$

□

Theorem 15.13 (ELBO Identity). *For every q_ϕ,*

$$\log p(x) = \mathrm{ELBO}(\phi) + D_{\mathrm{KL}}\big(q_\phi(z) \,\|\, p(z \mid x)\big).$$

Consequently $\mathrm{ELBO}(\phi) \le \log p(x)$, with equality if and only if $q_\phi = p(\cdot \mid x)$, and maximizing the ELBO over ϕ is equivalent to minimizing the KL divergence to the posterior.

The theorem is the rearranged display above together with $D_{\mathrm{KL}} \ge 0$. The left side does not depend on ϕ, so every increase of the ELBO is a decrease of the KL divergence by the same amount. The ELBO only involves the joint density $p(x, z)$, which we can evaluate, and expectations under q_ϕ, which we can sample. It has two readings:

- *expected log-joint plus entropy*: $\mathbb{E}_{q_\phi}[\log p(x, z)]$ pulls q_ϕ toward configurations that explain the data and agree with the prior, while $\mathcal{H}(q_\phi)$ prevents it from collapsing onto a single point;

- *expected log-likelihood minus KL to the prior*: since $\log p(x,z) = \log p(x \mid z) + \log p(z)$,

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{z\sim q_\phi}[\log p(x \mid z)] - D_{\mathrm{KL}}\big(q_\phi(z) \,\|\, p(z)\big), \tag{15.4}$$

  which rewards explaining the data and penalizes moving away from the prior. This is the form used by variational autoencoders (Section 16.7).

Remark 15.14 (Mean Field Revisited). For an MRF the role of the evidence is played by the partition function: with the unnormalized density $\tilde p(x) = \prod_C \psi_C(x_C)$, the objective in (15.1) is exactly $\mathbb{E}_q[\log \tilde p(x)] + \mathcal{H}(q)$, and the same argument gives $\log Z = \text{objective} + D_{\mathrm{KL}}(q \,\|\, p)$. Mean field (Algorithm 15.1) is coordinate ascent on an ELBO over a factorized family. The general version of the update, $q_j(z_j) \propto \exp\big(\mathbb{E}_{q_{-j}}[\log p(x,z)]\big)$, where the expectation is over all other factors, is known as coordinate ascent variational inference (CAVI). □

Remark 15.15 (Connection to EM). The ELBO is also a function of model parameters θ when the joint density is $p_\theta(x,z)$, and $\log p_\theta(x) \geq \mathrm{ELBO}(\theta,\phi)$ for every ϕ. Alternating between maximizing over q and over θ is the EM algorithm of Section 16.2; maximizing over both by stochastic gradients is the variational autoencoder of Section 16.7. □

15.5 ▪ Stochastic Gradients and the Reparameterization Trick

Coordinate updates in closed form exist only for special models. For a general differentiable model we maximize the ELBO by gradient ascent, $\phi \leftarrow \phi + \eta\, \nabla_\phi \mathrm{ELBO}(\phi)$, as in Section 3.2. The gradient is an expectation that we cannot compute exactly, so we use an unbiased Monte Carlo estimate (Section 14.2) and stochastic gradient ascent (Section 3.4).

The difficulty is that the distribution over which we average depends on ϕ. If Y had a distribution not depending on ϕ, we could exchange gradient and expectation, $\nabla_\phi \mathbb{E}[f(Y,\phi)] = \mathbb{E}[\nabla_\phi f(Y,\phi)] \approx \frac{1}{m}\sum_{k=1}^m \nabla_\phi f(y_k,\phi)$. For the ELBO, however,

$$\nabla_\phi \mathbb{E}_{z\sim q_\phi}\big[\log p(x,z) - \log q_\phi(z)\big] \neq \mathbb{E}_{z\sim q_\phi}\big[\nabla_\phi\big(\log p(x,z) - \log q_\phi(z)\big)\big],$$

because moving ϕ also moves the samples.

Definition 15.16 (Reparameterization). *A family q_ϕ is* reparameterizable *if there are a fixed base distribution p_0, not depending on ϕ, and a map T, differentiable in ϕ, such that*

$$\varepsilon \sim p_0 \quad\Longrightarrow\quad z = T(\varepsilon,\phi) \sim q_\phi.$$

□

The standard example is the Gaussian family: with $\phi = (\mu,\sigma)$, $\varepsilon \sim \mathcal{N}(0,I)$, and $z = \mu + \sigma \odot \varepsilon$, we have $z \sim \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$. The randomness now enters through ε, whose distribution is fixed, and ϕ enters only through a deterministic, differentiable map. Hence

$$\begin{aligned}\nabla_\phi \mathrm{ELBO}(\phi) &= \mathbb{E}_{\varepsilon\sim p_0}\Big[\nabla_\phi\big(\log p\big(x, T(\varepsilon,\phi)\big) - \log q_\phi\big(T(\varepsilon,\phi)\big)\big)\Big] \\ &\approx \frac{1}{m}\sum_{k=1}^m \nabla_\phi\big(\log p(x,z_k) - \log q_\phi(z_k)\big)\Big|_{z_k = T(\varepsilon_k,\phi)},\end{aligned} \tag{15.5}$$

where the gradient flows through $z_k = T(\varepsilon_k,\phi)$ by the chain rule. In code, one writes the Monte Carlo ELBO as a function of ϕ and lets automatic differentiation (Section 11.4) do the rest.

Algorithm 15.2 Stochastic Variational Inference with Reparameterization

Input: Data x; joint density $p(x, z)$, differentiable in z; reparameterizable family q_ϕ with base distribution p_0 and map T; Monte Carlo sample size m; learning rates η_t; number of iterations K.
Initialize: Variational parameters ϕ, e.g. $\mu = 0$ and $\log \sigma = 0$ for a Gaussian family.

1: **for** $t = 1, \dots, K$ **do**
2: Draw $\varepsilon_1, \dots, \varepsilon_m \overset{\text{i.i.d.}}{\sim} p_0$ and set $z_k \leftarrow T(\varepsilon_k, \phi)$.
3: $\widehat{\mathrm{ELBO}}(\phi) \leftarrow \frac{1}{m} \sum_{k=1}^{m} \big(\log p(x, z_k) - \log q_\phi(z_k) \big)$
4: $\hat{g} \leftarrow \nabla_\phi \widehat{\mathrm{ELBO}}(\phi)$ ▷ differentiate through $z_k = T(\varepsilon_k, \phi)$
5: $\phi \leftarrow \phi + \eta_t \, \hat{g}$ ▷ gradient *ascent*; in practice Adam
6: **end for**
Output: Approximate posterior $q_\phi \approx p(z \mid x)$.

The estimate $\hat{g}$ is unbiased for every $m \geq 1$; larger m reduces its variance at proportional cost, and $m = 1$ is common. The method only needs to evaluate $\log p(x, z)$ and its gradient in z, so it applies to any model with continuous latent variables, such as TrueSkill with a Gaussian approximation to the skills. When the data consist of many conditionally independent points, $\log p(x, z)$ is a sum over data points and can be replaced by a rescaled minibatch sum, which is a second source of stochasticity in the same spirit as Algorithm 3.2.

Remark 15.17 (Score-Function Estimator). An alternative that does not require reparameterization uses the log-derivative trick $\nabla_\phi q_\phi = q_\phi \nabla_\phi \log q_\phi$:

$$\nabla_\phi \mathbb{E}_{z \sim q_\phi}[f(z)] = \mathbb{E}_{z \sim q_\phi}\big[f(z) \, \nabla_\phi \log q_\phi(z)\big].$$

It works for discrete latent variables and non-differentiable f, and it is the same identity that underlies the policy gradient theorem in reinforcement learning. Its variance is usually much larger than that of (15.5), because it only uses the value of f, not its gradient. □

15.6 ▪ Variational Inference versus MCMC

MCMC (Chapter 14) and variational inference answer the same question with opposite trade-offs.

	MCMC	Variational inference
Output	samples whose distribution converges to $p(z \mid x)$	a distribution $q^* \in \mathcal{Q}$
Accuracy	asymptotically exact	biased by the choice of $\mathcal{Q}$, even with infinite compute
Multimodality	can visit several modes (slowly)	I-projection is mode seeking
Cost	many iterations; samples are correlated	optimization; fast and scalable with SGD
Tuning	proposal scales, step sizes, burn-in; convergence is hard to diagnose (Section 14.9)	learning rate, family, initialization; convergence is monitored through the ELBO
Typical use	accurate posterior summaries for moderate models	large models and data sets; inner loop of learning (VAEs)

Pros and Cons: Variational inference replaces sampling by optimization, so it converges quickly, scales to large data sets through stochastic gradients, and is easy to implement with automatic differentiation; the ELBO provides a usable objective and a lower bound on the evidence. However, the result is only as good as the family $\mathcal{Q}$: a factorized or Gaussian q ignores correlations and non-Gaussian shape, the I-projection underestimates posterior uncertainty and picks a single mode, and the answer is sensitive to initialization and learning rates.

Limitations and what comes next: So far the model $p(x, z)$ was fixed and we approximated one posterior. In unsupervised learning the model has parameters θ of its own, to be fitted by maximizing the evidence $p_\theta(x)$, and every data point x_i has its own latent variable z_i and therefore its own posterior. Chapter 16 handles both. When the posterior is tractable, maximizing the ELBO alternately over q and θ gives the EM algorithm; when it is not, a neural network that maps each x_i to the parameters of $q(z_i \mid x_i)$ amortizes the cost of inference, and training it jointly with θ by Algorithm 15.2 gives the variational autoencoder.

Chapter 16

Latent Variable Models: Mixtures, EM, PCA and VAEs

Unsupervised learning looks for structure in unlabeled data $x_1, \dots, x_n \in \mathbb{R}^p$: groups of similar points, a low-dimensional description, or a model that can generate new data. Latent variable models address all three in one framework. Each data point x_i is explained by an unobserved z_i (a cluster label, a low-dimensional code), the model specifies $p_\theta(x, z) = p_\theta(z)\, p_\theta(x \mid z)$, and the parameters are fitted by maximizing the marginal likelihood $\prod_i p_\theta(x_i) = \prod_i \int p_\theta(x_i, z_i)\, \mathrm{d}z_i$. The integral is the same obstacle as in Chapter 15, and the evidence lower bound is again the tool. Section 16.1 introduces Gaussian mixtures, Section 16.2 derives the EM algorithm as coordinate ascent on the ELBO, and Section 16.3 recovers K-means as a limiting case. Sections 16.4 and 16.5 turn to continuous latent variables: principal component analysis and its probabilistic version. Sections 16.6 and 16.7 replace the linear maps by neural networks, which leads to autoencoders and variational autoencoders.

16.1 ▪ Gaussian Mixture Models

A single Gaussian cannot describe data with several clusters or a skewed shape. Combining several Gaussians gives a much richer family. (The model goes back to Karl Pearson, who in 1894 asked whether a skewed distribution of crab measurements from the Bay of Naples was a mixture of two sub-species.)

Definition 16.1 (Gaussian Mixture Model). *A* Gaussian mixture model *(GMM) with K components has density*

$$p(x) = \sum_{k=1}^{K} \pi_k\, \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1,$$

with mixing coefficients π_k*, component means $\mu_k \in \mathbb{R}^p$, and covariances Σ_k. Equivalently, it is the latent variable model*

$$z \sim \mathrm{Categorical}(\pi_1, \dots, \pi_K), \qquad x \mid z = k \sim \mathcal{N}(\mu_k, \Sigma_k),$$

whose marginal is $p(x) = \sum_k p(z = k)\, p(x \mid z = k)$. □

The latent formulation explains how the data are generated: first pick a component, then draw from it. Inference about z is a single application of Bayes' rule. The *responsibility* of component k for a point x is the posterior

$$\gamma_k(x) = p(z = k \mid x) = \frac{\pi_k\, \mathcal{N}(x \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j\, \mathcal{N}(x \mid \mu_j, \Sigma_j)}. \tag{16.1}$$

The prior probability π_k is updated to $\gamma_k(x)$ after seeing x; a soft clustering assigns each point to all components with these weights.

Remark 16.2 (Connection to Discriminant Analysis). If the labels z_i were observed, the GMM would be exactly the generative classification model of quadratic discriminant analysis (Section 7.5.2), or of LDA (Section 7.5.1) if all Σ_k are equal. Its maximum likelihood estimates would be the class proportions, class means, and class covariances, and (16.1) would be the posterior class probability used by the Bayes classifier. A GMM is discriminant analysis with missing labels. □

For data $x_1, \dots, x_n$ and parameters $\theta = (\pi, \mu, \Sigma)$, the log-likelihood is

$$\log p(x_1, \dots, x_n \mid \theta) = \sum_{i=1}^{n} \log \Big(\sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k) \Big). \tag{16.2}$$

The logarithm of a sum does not separate, and there is no closed-form maximizer. Setting the derivatives to zero nevertheless gives informative equations. With $\gamma_{ik} = \gamma_k(x_i)$ and $N_k = \sum_i \gamma_{ik}$, a stationary point satisfies

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \, x_i, \qquad \Sigma_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^\top, \qquad \pi_k = \frac{N_k}{n}. \tag{16.3}$$

These are the usual Gaussian estimates with each point weighted by its responsibility; N_k is the effective number of points in component k. They are not a solution, because the responsibilities depend on the parameters. But they suggest an iteration: fix the responsibilities, update the parameters, recompute the responsibilities, and repeat.

Algorithm 16.1 EM for Gaussian Mixtures

Input: Data $x_1, \dots, x_n \in \mathbb{R}^p$; number of components K; tolerance ϵ.
Initialize: $\pi_k = 1/K$; means μ_k at K randomly chosen data points (or the output of Algorithm 16.3); Σ_k the sample covariance.

1: **repeat**
2: **E-step:** $\gamma_{ik} \leftarrow \dfrac{\pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$ for all i, k
3: **M-step:** $N_k \leftarrow \sum_i \gamma_{ik}, \quad \mu_k \leftarrow \frac{1}{N_k} \sum_i \gamma_{ik} x_i$
4: $\Sigma_k \leftarrow \frac{1}{N_k} \sum_i \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top, \quad \pi_k \leftarrow N_k / n$ ▷ uses the new μ_k
5: Evaluate the log-likelihood (16.2).
6: **until** the log-likelihood increases by less than ϵ

Output: Parameters $(\pi_k, \mu_k, \Sigma_k)_{k=1}^{K}$ and responsibilities γ_{ik} (soft cluster assignments).

Each iteration increases the log-likelihood (Proposition 16.4 below), and the algorithm converges to a stationary point of the likelihood, typically a local maximum. Different initializations can give different solutions, so in practice one runs several and keeps the one with the highest likelihood.

16.2 ▪ The EM Algorithm

The algorithm above is an instance of a general method for maximum likelihood with latent variables. Write $X = (x_1, \dots, x_n)$ for the observed data and $Z = (z_1, \dots, z_n)$ for the latent variables. If Z were

observed, we would maximize the *complete-data log-likelihood* $\log p(X, Z \mid \theta)$, which is usually easy: for the GMM,

$$\log p(X, Z \mid \theta) = \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbf{1}[z_i = k] \, \log \big(\pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\big),$$

and its maximizer is a set of per-component Gaussian fits. Since Z is unknown, EM replaces the complete-data log-likelihood by its expectation under the current posterior.

Definition 16.3 (Expected Complete-Data Log-Likelihood). *For current parameters θ^{old},*

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z \sim p(Z \mid X, \theta^{\text{old}})} \big[\log p(X, Z \mid \theta) \big] = \sum_{Z} p(Z \mid X, \theta^{\text{old}}) \log p(X, Z \mid \theta),$$

with the sum replaced by an integral for continuous Z. □

Algorithm 16.2 Expectation Maximization (EM)

Input: Data X; joint model $p(X, Z \mid \theta)$; number of iterations L (or a tolerance).
Initialize: Parameters $\theta^{(0)}$.
for $\ell = 0, 1, \ldots, L-1$ **do**
  E-step: compute the posterior $p(Z \mid X, \theta^{(\ell)})$ and hence $Q(\theta, \theta^{(\ell)})$.
  M-step: $\theta^{(\ell+1)} \leftarrow \arg\max_{\theta} Q(\theta, \theta^{(\ell)})$.
end for
Output: Parameters $\theta^{(L)}$.

For the GMM the posterior factorizes over data points, $p(z_i = k \mid X, \theta) = \gamma_k(x_i)$, and $\mathbb{E}[\mathbf{1}[z_i = k] \mid X, \theta^{\text{old}}] = \gamma_{ik}$, so

$$Q(\theta, \theta^{\text{old}}) = \sum_{i=1}^{n} \sum_{k=1}^{K} \gamma_{ik} \log \big(\pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\big).$$

Maximizing this weighted sum of Gaussian log-densities gives exactly the M-step of Algorithm 16.1. EM is useful whenever the E-step posterior is tractable and the M-step is a weighted version of a problem with a closed-form solution.

EM as Coordinate Ascent on the ELBO

The ELBO of Section 15.4 explains why EM works. For any distribution q over Z and any θ, Theorem 15.13 gives

$$\log p(X \mid \theta) = \mathrm{ELBO}(q, \theta) + D_{\mathrm{KL}}\big(q \,\|\, p(\cdot \mid X, \theta)\big), \qquad \mathrm{ELBO}(q, \theta) = \mathbb{E}_{Z \sim q}[\log p(X, Z \mid \theta)] + \mathcal{H}(q). \tag{16.4}$$

- *E-step = maximize over q.* With $\theta = \theta^{\text{old}}$ fixed, the ELBO is maximized by $q = p(Z \mid X, \theta^{\text{old}})$, which makes the KL term zero: the bound touches the log-likelihood at θ^{old}.
- *M-step = maximize over θ.* With q fixed, the entropy does not depend on θ, so maximizing the ELBO over θ is maximizing $\mathbb{E}_q[\log p(X, Z \mid \theta)] = Q(\theta, \theta^{\text{old}})$.

Proposition 16.4 (Monotonicity of EM). *The iterates of Algorithm 16.2 satisfy* $\log p(X \mid \theta^{(\ell+1)}) \geq \log p(X \mid \theta^{(\ell)})$ *for every* ℓ.

With $q_\ell = p(Z \mid X, \theta^{(\ell)})$, the two steps and $D_{\mathrm{KL}} \geq 0$ give the chain

$$\log p(X \mid \theta^{(\ell+1)}) \;\geq\; \mathrm{ELBO}(q_\ell, \theta^{(\ell+1)}) \;\geq\; \mathrm{ELBO}(q_\ell, \theta^{(\ell)}) \;=\; \log p(X \mid \theta^{(\ell)}).$$

The likelihood can only go up, but EM may stop at a local maximum or a saddle point, and its convergence can be slow.

Remark 16.5 (Variational Inference versus EM). Variational inference (Chapter 15) and EM maximize the same objective (16.4). EM assumes the exact posterior is available and maximizes exactly in each block. When the posterior is intractable, one restricts q to a tractable family; the E-step then becomes a variational approximation (variational EM). When even that has no closed form, one takes stochastic gradient steps on q and θ jointly, which is the variational autoencoder of Section 16.7. □

16.3 ▪ K-Means as a Limit of EM

Consider a GMM whose covariances are fixed at $\Sigma_k = \epsilon I$, so that only π and μ are learned. The responsibilities become

$$\gamma_{ik} = \frac{\pi_k \exp\big(-\|x_i - \mu_k\|^2/(2\epsilon)\big)}{\sum_j \pi_j \exp\big(-\|x_i - \mu_j\|^2/(2\epsilon)\big)}.$$

As $\epsilon \to 0$, the term with the smallest distance dominates the denominator, and (for $\pi_k > 0$) $\gamma_{ik} \to r_{ik} = \mathbf{1}\big[k = \arg\min_j \|x_i - \mu_j\|\big]$: soft assignments become hard. In the same limit $Q(\theta, \theta^{\mathrm{old}}) = -\frac{1}{2\epsilon}\sum_{i,k} r_{ik}\|x_i - \mu_k\|^2 + \text{const}$, so the M-step minimizes the *distortion*

$$J(r, \mu) = \sum_{i=1}^{n}\sum_{k=1}^{K} r_{ik}\|x_i - \mu_k\|^2.$$

The slides state the limit with $\Sigma_k = I$; the variance must shrink, $\Sigma_k = \epsilon I$ with $\epsilon \to 0$, for the assignments to become hard.

Algorithm 16.3 K-Means

Input: Data $x_1, \ldots, x_n \in \mathbb{R}^p$; number of clusters K.
Initialize: Centers $\mu_1, \ldots, \mu_K$, e.g. K distinct data points chosen at random.
1: **repeat**
2: **Assignment:** $r_{ik} \leftarrow 1$ if $k = \arg\min_j \|x_i - \mu_j\|^2$ and $r_{ik} \leftarrow 0$ otherwise ▷ hard E-step
3: **Update:** $\mu_k \leftarrow \sum_i r_{ik} x_i \big/ \sum_i r_{ik}$ for every nonempty cluster k ▷ M-step
4: **until** the assignments do not change
Output: Centers μ_k and cluster labels $\arg\max_k r_{ik}$.

No step increases J, and there are finitely many assignments, so K-means stops after finitely many iterations, at a local minimum of J.

	K-means	EM for a GMM
Assignments	hard, nearest center	soft, responsibilities γ_{ik}
Cluster shape	spherical, equal size (Euclidean distance)	ellipsoidal, with covariance Σ_k and weight π_k
Objective	distortion J	log-likelihood (16.2)
Cost per iteration	low; converges in few iterations	higher; converges more slowly
Output	a partition	a density model with uncertainty

Both methods find only local optima and need K to be chosen. K-means is often used to initialize EM.

16.4 ▪ Principal Component Analysis

High-dimensional data often lie close to a low-dimensional subspace. A famous example: the first two principal components of genetic data from European individuals reproduce the map of Europe. A low-dimensional representation saves computation, allows visualization, and can improve generalization.

PCA approximates each point by its projection onto a K-dimensional affine subspace, $K < p$:

$$x_i \approx \tilde{x}_i = \bar{x} + U z_i, \qquad z_i = U^\top (x_i - \bar{x}),$$

where $\bar{x} = \frac{1}{n}\sum_i x_i$ is the sample mean, $U \in \mathbb{R}^{p \times K}$ has orthonormal columns spanning the *principal subspace*, and $z_i \in \mathbb{R}^K$ is the *code* of x_i. The basis is chosen to minimize the reconstruction error

$$\min_{U : U^\top U = I_K} \sum_{i=1}^{n} \left\| x_i - \bar{x} - U U^\top (x_i - \bar{x}) \right\|^2. \tag{16.5}$$

Proposition 16.6 (PCA Solution). *Let $\hat{\Sigma} = \frac{1}{n}\sum_i (x_i - \bar{x})(x_i - \bar{x})^\top$ have eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p$ with orthonormal eigenvectors $u_1, \ldots, u_p$. Then $U = (u_1, \ldots, u_K)$ minimizes (16.5), with minimal value $n \sum_{j>K} \lambda_j$. The same U maximizes the variance $\sum_{k=1}^{K} u_k^\top \hat{\Sigma} u_k$ of the projected data, and the variance of the k-th code coordinate is λ_k.*

Minimum reconstruction error and maximum variance are the same criterion, because for an orthogonal projection the squared length of each centered point splits into the part captured by the subspace and the residual (Pythagoras). Any orthonormal basis of the same subspace gives the same reconstructions; the eigenvectors are the canonical choice.

Algorithm 16.4 Principal Component Analysis (PCA)

Input: Data $x_1, \ldots, x_n \in \mathbb{R}^p$; target dimension $K < p$.

1: $\bar{x} \leftarrow \frac{1}{n}\sum_i x_i$; $\quad \hat{\Sigma} \leftarrow \frac{1}{n}\sum_i (x_i - \bar{x})(x_i - \bar{x})^\top$

2: Compute the K leading eigenvectors $u_1, \ldots, u_K$ of $\hat{\Sigma}$ (or the right singular vectors of the centered data matrix) and set $U \leftarrow (u_1, \ldots, u_K)$.

3: $z_i \leftarrow U^\top (x_i - \bar{x})$ and $\tilde{x}_i \leftarrow \bar{x} + U z_i$ for $i = 1, \ldots, n$.

Output: Principal directions U, eigenvalues $\lambda_1, \ldots, \lambda_K$, codes z_i, reconstructions $\tilde{x}_i$.

16.5 ▪ Probabilistic PCA

PCA was motivated geometrically. It can also be obtained as maximum likelihood in a latent variable model, in which the latent variable is continuous instead of a cluster label.

Definition 16.7 (Probabilistic PCA). Probabilistic PCA *(PPCA) with latent dimension $K < p$ is the model*

$$z \sim \mathcal{N}(0, I_K), \qquad x \mid z \sim \mathcal{N}(Wz + \mu,\ \sigma^2 I_p),$$

with parameters $W \in \mathbb{R}^{p \times K}$, $\mu \in \mathbb{R}^p$, and $\sigma^2 > 0$. □

To generate a point, take a random linear combination Wz of the columns of W, shift it by μ, and add a small amount of isotropic noise. The data therefore concentrate near the affine subspace $\mu + \operatorname{span}(W)$, which plays the role of the principal subspace. Given z, the coordinates of x are independent, the same structure as in naive Bayes (Section 7.5.3); all dependence between coordinates comes from the shared latent z.

Unlike the GMM, PPCA has a Gaussian marginal. Since $x = Wz + \mu + \varepsilon$ with independent $z \sim \mathcal{N}(0, I_K)$ and $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_p)$ is an affine function of Gaussians (Proposition 2.15),

$$x \sim \mathcal{N}(\mu, C), \qquad C = \operatorname{Cov}[x] = WW^\top + \sigma^2 I_p.$$

The log-likelihood of the data is that of a Gaussian with this structured covariance,

$$\log p(x_1, \dots, x_n \mid W, \mu, \sigma^2) = -\frac{np}{2}\log(2\pi) - \frac{n}{2}\log|C| - \frac{1}{2}\sum_{i=1}^{n}(x_i - \mu)^\top C^{-1}(x_i - \mu).$$

The model is not identifiable: for any orthogonal $R \in \mathbb{R}^{K \times K}$, $(WR)(WR)^\top = WW^\top$, so W and WR give the same distribution, just as PCA determines the subspace but not a particular basis.

Theorem 16.8 (Maximum Likelihood for PPCA; Tipping and Bishop, 1999). *Let $\hat\Sigma$, $\lambda_1 \geq \cdots \geq \lambda_p$, and $u_1, \dots, u_p$ be as in Proposition 16.6, and let $\hat U_K = (u_1, \dots, u_K)$ and $\hat L_K = \operatorname{diag}(\lambda_1, \dots, \lambda_K)$. The maximum likelihood estimates are*

$$\hat\mu = \bar x, \qquad \hat\sigma^2 = \frac{1}{p-K}\sum_{j=K+1}^{p}\lambda_j, \qquad \hat W = \hat U_K\big(\hat L_K - \hat\sigma^2 I_K\big)^{1/2} R,$$

for an arbitrary orthogonal matrix R.

The fitted covariance is $\hat C = \hat U_K(\hat L_K - \hat\sigma^2 I)\hat U_K^\top + \hat\sigma^2 I$, which is easiest to read along the eigenvectors. For $j \leq K$, $u_j^\top \hat C u_j = (\lambda_j - \hat\sigma^2) + \hat\sigma^2 = \lambda_j$; for $j > K$, $u_j^\top \hat C u_j = \hat\sigma^2$. The model reproduces the sample variance exactly along the K principal directions and replaces it by a single average value $\hat\sigma^2$ in all remaining directions: $\hat\sigma^2$ is the variance that the principal subspace fails to explain.

Proposition 16.9 (Posterior over the Code). *In PPCA, with $M = W^\top W + \sigma^2 I_K$,*

$$z \mid x \sim \mathcal{N}\big(M^{-1}W^\top(x - \mu),\ \sigma^2 M^{-1}\big).$$

As $\sigma^2 \to 0$, the posterior mean tends to $(W^\top W)^{-1}W^\top(x - \mu)$, the coordinates of the orthogonal projection of $x - \mu$ onto $\operatorname{span}(W)$. With the maximum likelihood estimates this limit is $R^\top \hat L_K^{-1/2}\hat U_K^\top(x - \bar x)$, the PCA code of Algorithm 16.4 up to a rescaling of each coordinate and a rotation.

This is another instance of Proposition 2.15. The slides write the posterior covariance with an extra factor $W^\top(x - \mu)$; the covariance does not depend on x, and the correct expression is $\sigma^2 M^{-1}$.

Why bother with the probabilistic version? A full-covariance Gaussian in $\mathbb{R}^p$ needs $p(p+1)/2 + p$ parameters, whereas PPCA models only the K strongest correlations with $O(Kp)$ parameters. Having a

likelihood allows comparison with other density models and a principled choice of K, for example by placing a prior on W (Bayesian PCA). Finally, the parameters can also be fitted by EM (Algorithm 16.2), with the Gaussian posterior of Proposition 16.9 as the E-step, which avoids forming and diagonalizing $\hat{\Sigma}$ and scales to very large, high-dimensional data sets.

Model	Latent variable	Posterior	Use
Gaussian mixture (Section 16.1)	discrete $z \in \{1, \dots, K\}$	exact: responsibilities	clustering, density estimation
Probabilistic PCA (Section 16.5)	continuous $z \in \mathbb{R}^K$, Gaussian	exact Gaussian	dimension reduction
Bayesian linear regression (Section 5.6)	regression weights, Gaussian prior	exact Gaussian	regression with uncertainty
Variational autoencoder (Section 16.7)	continuous $z \in \mathbb{R}^K$, Gaussian	intractable; amortized Gaussian q_ϕ	generation, representation learning

16.6 ▪ Autoencoders

PCA is linear. Neural networks (Chapter 11) provide nonlinear maps, and an *autoencoder* uses them to compress data through a bottleneck.

Definition 16.10 (Autoencoder). *An* autoencoder *consists of an* encoder $g\colon \mathbb{R}^p \to \mathbb{R}^K$ *and a* decoder $f\colon \mathbb{R}^K \to \mathbb{R}^p$ *with $K \ll p$, both neural networks. The code of x is $z = g(x)$, its reconstruction is $\tilde{x} = f(g(x))$, and the networks are trained to minimize the reconstruction error $\frac{1}{2}\sum_{i=1}^{n} \|f(g(x_i)) - x_i\|^2$ by stochastic gradient descent (Algorithm 11.2).* □

Remark 16.11 (Linear Autoencoders Are PCA). If the encoder and decoder are linear, $g(x) = Ax$ and $f(z) = Bz$ (with centered data), the reconstruction BAx is a rank-K linear map, and minimizing the squared error over such maps is the PCA problem (16.5). The K hidden units then span the same subspace as the first K principal components, although they need not be orthonormal or ordered. Adding nonlinear hidden layers before and after the bottleneck gives a nonlinear generalization of PCA: the encoder maps the data nonlinearly to codes in $\mathbb{R}^K$, and the decoder maps the codes onto a curved K-dimensional surface in $\mathbb{R}^p$ instead of a subspace. Deep autoencoders reconstruct images such as MNIST digits far better than PCA with the same code size, but they are harder to train. □

A *denoising autoencoder* receives a corrupted input and is trained to reconstruct the clean original. It learns to remove noise of the type seen in training, which requires many clean–noisy pairs and knowledge of the noise. Loopy belief propagation on an Ising model (Section 13.4) denoises a single image without any training data, using only a prior that neighboring pixels agree.

Deterministic autoencoders have two weaknesses as generative models. First, nothing forces nearby inputs to have nearby codes, or the codes to fill a region of $\mathbb{R}^K$; the code space can have gaps, and decoding a point in a gap gives an unrealistic output, so we cannot generate new data by picking a code. Second, the squared error is a poor measure of perceptual quality: a reconstruction shifted by one pixel looks correct but is heavily penalized. A probabilistic formulation addresses the first issue directly.

16.7 ▪ Variational Autoencoders

A *variational autoencoder* (VAE) is a latent variable model with a neural network decoder, trained by maximizing the ELBO with an encoder that outputs a distribution rather than a point.

Definition 16.12 (Variational Autoencoder). *The* generative model *is*

$$z \sim p(z) = \mathcal{N}(0, I_K), \qquad x \mid z \sim p_\theta(x \mid z),$$

where the parameters of $p_\theta(x \mid z)$ are computed from z by a decoder network f_θ; for binary images such as MNIST, $p_\theta(x \mid z) = \prod_{d=1}^{p} \mathrm{Bernoulli}\big(x_d; f_\theta(z)_d\big)$. The inference model *(encoder, recognition model) is a network g_ϕ that maps x to the mean and log standard deviation of a Gaussian,*

$$q_\phi(z \mid x) = \mathcal{N}\big(z \mid \mu_\phi(x), \mathrm{diag}(\sigma_\phi(x)^2)\big), \qquad g_\phi(x) = \big(\mu_\phi(x), \log \sigma_\phi(x)\big).$$

□

The posterior $p_\theta(z \mid x)$ is intractable because the decoder is nonlinear, so we use variational inference. For one data point, the ELBO in the form (15.4) is

$$\mathrm{ELBO}(\theta, \phi; x) = \mathbb{E}_{z \sim q_\phi(z \mid x)}\big[\log p_\theta(x \mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big) \le \log p_\theta(x), \tag{16.6}$$

and training maximizes $\sum_i \mathrm{ELBO}(\theta, \phi; x_i)$ over both θ and ϕ. Its negative is the VAE loss. The first term is a *reconstruction* term (for the Bernoulli decoder, minus the binary cross-entropy between x and the decoded probabilities); the second is a *regularizer* that keeps each $q_\phi(z \mid x_i)$ close to the prior. For the Gaussian encoder it has a closed form,

$$D_{\mathrm{KL}}\big(\mathcal{N}(\mu, \mathrm{diag}(\sigma^2)) \,\|\, \mathcal{N}(0, I_K)\big) = \frac{1}{2} \sum_{j=1}^{K} \big(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\big).$$

Remark 16.13 (Amortized Inference). A *standard* (non-amortized) VAE gives each data point its own variational parameters $\phi_i = (\mu_i, \log \sigma_i)$ and optimizes them separately, as in Algorithm 15.2. With n data points this means n separate optimization problems, and a new point requires a new optimization. An *amortized* VAE learns one function $x \mapsto (\mu_\phi(x), \sigma_\phi(x))$ shared by all points: the cost of inference is paid once, during training, and inference for a new point is a single forward pass. The number of encoder parameters no longer grows with n. The price is the *amortization gap*: the per-point parameters are more expressive, because nothing is shared across data points. □

The expectation in (16.6) depends on ϕ, so we use the reparameterization trick of Section 15.5: $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I_K)$. The sample is then a differentiable function of ϕ, and gradients flow from the decoder through z into the encoder.

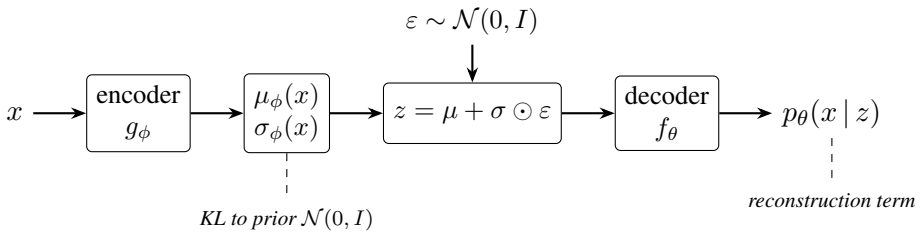


Figure 16.1. *The VAE computation for one data point. The noise ε enters from outside, so the loss is differentiable in both ϕ and θ.*

Algorithm 16.5 Training a Variational Autoencoder

Input: Data $x_1, \dots, x_n$; encoder g_ϕ and decoder f_θ (neural networks); latent dimension K; minibatch size B; learning rate η; number of iterations T.

Initialize: Random θ and ϕ.

1: **for** $t = 1, \dots, T$ **do**
2: Sample a minibatch $\{x_i\}_{i \in \mathcal{B}}$ of size B.
3: **for** $i \in \mathcal{B}$ **do**
4: $(\mu_i, \log \sigma_i) \leftarrow g_\phi(x_i)$ ▷ encode
5: Draw $\varepsilon_i \sim \mathcal{N}(0, I_K)$; $z_i \leftarrow \mu_i + \sigma_i \odot \varepsilon_i$ ▷ reparameterize
6: $\ell_i \leftarrow -\log p_\theta(x_i \mid z_i) + \frac{1}{2} \sum_{j=1}^{K} \left(\mu_{ij}^2 + \sigma_{ij}^2 - 1 - \log \sigma_{ij}^2\right)$ ▷ decode; $-\widehat{\mathrm{ELBO}}$
7: **end for**
8: $(\theta, \phi) \leftarrow (\theta, \phi) - \eta \nabla_{\theta,\phi} \frac{1}{B} \sum_{i \in \mathcal{B}} \ell_i$ ▷ backpropagation; in practice Adam
9: **end for**

Output: Decoder $p_\theta(x \mid z)$ and encoder $q_\phi(z \mid x)$. To generate: draw $z \sim \mathcal{N}(0, I_K)$, then $x \sim p_\theta(x \mid z)$.

The gradient in line 8 is computed by backpropagation (Algorithm 11.1) through decoder, sampling step, and encoder, as in Fig. 16.1. After training, new data are generated by decoding samples from the prior, and because the KL term packs the codes of the training data into the region where the prior has mass, the decoder has learned to produce realistic outputs throughout that region. Codes vary smoothly, so one can interpolate between two inputs by decoding points on the segment between their codes. The codes of a VAE trained on MNIST fill the latent space more evenly than those of a deterministic autoencoder, whose clusters are separated by gaps.

The objective has two equivalent readings, both from Section 15.4:

$$\mathrm{ELBO}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(x, z)] + \mathcal{H}(q_\phi)}_{\text{expected complete-data log-likelihood + entropy}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}(q_\phi \,\|\, p)}_{\text{reconstruction minus distance to the prior}}.$$

The first maximizes the complete-data log-likelihood, as in EM, while penalizing encoders with low entropy; the second maximizes the reconstruction likelihood while penalizing encoders that move far from the prior.

Remark 16.14 (Connections). The VAE combines three earlier ideas. It is stochastic variational inference (Algorithm 15.2) with amortized variational parameters; it is EM (Remark 16.5) with both steps replaced by joint stochastic gradient steps on the same ELBO; and it is a nonlinear PPCA. If the decoder is linear with Gaussian noise, $p_\theta(x \mid z) = \mathcal{N}(Wz + \mu, \sigma^2 I)$, the generative model is exactly

PPCA (Definition 16.7), whose posterior is Gaussian (Proposition 16.9), so a Gaussian encoder can make the bound tight. □

Pros and Cons: VAEs learn a smooth, low-dimensional latent space and a generative model at the same time, they scale to large data sets through minibatch SGD, and the amortized encoder gives approximate posteriors for new data in a single pass. However, they maximize a lower bound rather than the likelihood itself; the Gaussian encoder limits how tight the bound can be, and the KL term can push the model to ignore some latent dimensions altogether. Samples from image VAEs with simple Gaussian or Bernoulli decoders tend to be blurry.

Limitations and what comes next: The models of this chapter learn representations of images and other vectors, and the VAE is the first deep generative model of the course. Two directions follow. Chapter 17 turns to learned representations of text, where the inputs are sequences of tokens, and introduces embeddings and the attention mechanism. Chapter 18 returns to generation: diffusion models keep the ELBO of the VAE but fix the encoder to a gradual noising process with latent variables of the same dimension as the data, and learn a decoder that removes the noise in many small steps, which produces far sharper samples.

Chapter 17

Embeddings and Attention

Every method in this book so far assumed that an input is a vector $x \in \mathbb{R}^p$. Text is not: a document is a sequence of symbols of variable length, and two documents that mean the same thing may share no words at all. Before any model of Part II can be applied, text must be turned into numbers, and the quality of that *representation* often matters more than the choice of model. This chapter follows the development of text representations from hand-crafted to learned. Section 17.1 builds sparse count vectors (bag of words, n-grams, TF-IDF). Section 17.2 learns dense word embeddings with word2vec, and Section 17.3 learns embeddings of whole texts and images by contrastive learning, which also powers retrieval-augmented generation. Section 17.4 introduces attention, which makes the representation of a token depend on its context, and Section 17.5 assembles attention into the transformer, the architecture behind modern language models.

17.1 ▪ Representing Text

We fix some vocabulary. A *token* is a single atom of text, usually a word; a *document* is one complete data point; a *span* is a contiguous group of tokens within a document; and the *vocabulary* $\mathcal{V}$ is the list of the M tokens the model knows. Language tasks come in the familiar flavors: classification of documents, spans or tokens (spam filtering, hate speech detection, named entities), regression (how many retweets will a post get), information extraction, retrieval (search engines, literature review), and generation (question answering, summarization, translation, chatbots).

Consider spam filtering with data $\{(x_i, y_i)\}_{i=1}^n$, where x_i is the text of an email and $y_i = 1$ if it is spam. We want $f_\theta(x) \approx p(y = 1 \mid x)$, which is a logistic regression problem (Section 7.2) once x is a vector.

Definition 17.1 (Bag of Words and n-Grams). *Let the vocabulary consist of the M most frequent tokens in the training corpus. The* binary bag of words *of a document x is the vector $h \in \{0,1\}^M$ with $h_j = 1$ if and only if token $\mathcal{V}_j$ occurs in x; the* count bag of words *replaces the indicator by the number of occurrences. An* n-gram *is a contiguous sequence of n tokens; using n-grams (bigrams, trigrams, ...) instead of single tokens (unigrams) as vocabulary entries lets the representation capture short phrases.* □

For example, with unigram vocabulary $(\text{This}, \text{is}, \text{a}, \text{sentence})$ the document "This is a sentence" is $(1,1,1,1)$ and "A sentence" is $(0,0,1,1)$; with bigram vocabulary $(\text{This is}, \text{is a}, \text{a sentence})$ they become $(1,1,1)$ and $(0,0,1)$. Word order is lost beyond the window of an n-gram, hence the name "bag". Feeding the binary bag of words into naive Bayes (Section 7.5.3) gives the classical spam filter.

Very common words such as "the" or "a" appear in almost every document and carry little information about which document is which. They are either removed as *stopwords* or down-weighted.

Definition 17.2 (TF-IDF). *The* term frequency $\mathrm{TF}(w, x)$ *measures how often token* w *occurs in document* x*: the raw count, its binarized version, the count divided by the length of* x*, or* $\log(1 + \text{count})$*. With* N *documents of which* N_w *contain* w*, the* inverse document frequency *is a decreasing function of* N_w*, most commonly*

$$\mathrm{IDF}(w) = \log \frac{N}{N_w} \qquad \textit{or the smoothed} \qquad \log \frac{N}{N_w + 1}.$$

The TF-IDF *representation of* x *has entries* $\mathrm{TF}(w, x) \cdot \mathrm{IDF}(w)$. □

A token that occurs in every document has $\mathrm{IDF} = \log 1 = 0$ and disappears from the representation; a token concentrated in few documents is up-weighted. In the lecture example with the two documents "This is a sentence. This is another sentence." and "This is not a sentence. Yet another not a sentence", only "not" and "Yet" survive in the second document, with weights $2 \log_{10} 2 \approx 0.602$ and $\log_{10} 2 \approx 0.301$.

Remark 17.3 (BM25). TF-IDF ideas still underlie keyword search. The BM25 score of a document d for a query q is

$$\mathrm{score}(d, q) = \sum_{w \in q} \mathrm{IDF}(w) \, \frac{\mathrm{TF}(w, d)\,(k_1 + 1)}{\mathrm{TF}(w, d) + k_1 \big(1 - b + b\,|d|/\mathrm{avgdl}\big)},$$

where avgdl is the average document length. The parameter $k_1 \in [1.2, 2]$ makes the contribution of a term saturate as its count grows, and $b \in [0, 1]$ (typically 0.75) controls how strongly long documents are penalized; $b = 0$ switches length normalization off. □

To compare documents we need a similarity. The standard choice in language processing is the *cosine similarity*

$$\cos(x, y) = \frac{x^\top y}{\|x\| \, \|y\|},$$

which ignores the overall scale of the vectors, so that a long and a short document on the same topic can still be similar.

Pros and Cons: Count representations are simple, interpretable, fast, and surprisingly strong baselines for classification and search. However, they are sparse and high-dimensional (M is in the tens of thousands), they ignore word order beyond n tokens, and distinct words are orthogonal: "car" and "automobile" are as dissimilar as "car" and "banana". The representation knows nothing about meaning.

17.2 ▪ Word Embeddings

All the vectors of Section 17.1 are fixed functions of the text. The step taken in this section is to *learn* the representation.

Definition 17.4 (Embedding). *An* embedding function *is a map* $g \colon x \mapsto h \in \mathbb{R}^e$ *from raw inputs (tokens, documents, images, or image–text pairs) to vectors in a space of moderate dimension* e*;* h *is the* embedding *of* x*. A predictor then factorizes as* $f_\theta(x) = c_{\theta_1}\big(g_{\theta_2}(x)\big)$*, with* c *a classifier or regressor acting on the embedding.* □

A good embedding places semantically similar inputs close together, so that similarity search, clustering, and semi-supervised learning can be done directly in embedding space. It also enables *transfer learning*: an embedding trained on a large corpus is reused for a new task by training only a small *linear head* (or *linear probe*) on top of it, that is, a logistic or softmax regression (Section 7.4) on h.

Remark 17.5 (Connection to Autoencoders and PCA). The code of an autoencoder (Section 16.6) and the principal component scores of PCA (Section 16.4) are embeddings learned by reconstruction. Word2vec learns embeddings by prediction instead: the representation of a word is whatever helps predict its neighbors. □

17.2.1 ▪ Word2vec and the Skip-Gram Model

Word2vec (Mikolov et al., 2013) rests on the *distributional hypothesis*: words with similar meanings occur in similar contexts. For a corpus $w_1, \dots, w_T$ and a window size k, the context of position t is $\mathrm{context}(t) = \{t-k, \dots, t-1, t+1, \dots, t+k\}$. The *skip-gram* task is to predict each context word from the center word, by maximizing the average log-likelihood

$$\frac{1}{T} \sum_{t=1}^{T} \sum_{j \in \mathrm{context}(t)} \log p(w_j \mid w_t). \tag{17.1}$$

With vocabulary size M and embedding dimension e, the model has two matrices $U, V \in \mathbb{R}^{M \times e}$. Writing u_w for the row of U that belongs to word w (the product of the one-hot vector of w with U) and v_o for the row of V that belongs to a context word o,

$$p(o \mid w) = \frac{\exp(u_w^\top v_o)}{\sum_{r=1}^{M} \exp(u_w^\top v_r)}. \tag{17.2}$$

Algorithm 17.1 Skip-Gram Word2vec

Input: Corpus $w_1, \dots, w_T$ over a vocabulary of size M, window size k, embedding dimension e, learning rate η, number of steps K.

Initialize: Word embeddings $U \in \mathbb{R}^{M \times e}$ and context embeddings $V \in \mathbb{R}^{M \times e}$ with small random entries.

1: **for** $\kappa = 1, \dots, K$ **do**
2: Sample a position t and a context position $j \in \mathrm{context}(t)$; set $w \leftarrow w_t$, $o \leftarrow w_j$.
3: $\ell \leftarrow -\log p(o \mid w)$ with p as in (17.2).
4: $(U, V) \leftarrow (U, V) - \eta \nabla_{U,V} \ell$ ▷ only u_w and the rows of V change
5: **end for**

Output: The matrix U; the embedding of the i-th vocabulary word is the i-th row of U. The matrix V is discarded.

In practice the pairs are processed in minibatches with SGD (Algorithm 3.2). The normalizing sum in (17.2) runs over the entire vocabulary at every step, which is too expensive for M in the hundreds of thousands, and the original paper introduces several tricks to avoid it. The best known, *negative sampling*, replaces the M-way softmax by a binary logistic regression that separates the observed pair (w, o) from a few randomly drawn "negative" words. The reverse task, predicting the center word from its context, is called *continuous bag of words* (CBOW).

Remark 17.6 (Connection to Softmax Regression). For a fixed center word w, (17.2) is multinomial logistic regression (Section 7.4) with M classes, feature vector u_w and coefficient vector v_o for class o. What is new is that the features are learned along with the coefficients: skip-gram is a softmax regression whose inputs are themselves parameters, or equivalently a two-layer linear network (Section 11.2) from a one-hot input to a softmax output. Fitting it is maximum likelihood (Section 2.2) on the pairs (center word, context word). □

The learned vectors have a striking linear structure. With $h(\cdot)$ denoting the word2vec embedding, $h(\text{Athens}) - h(\text{Greece}) + h(\text{Germany})$ is close to $h(\text{Berlin})$, and $h(\text{mice}) - h(\text{mouse}) + h(\text{dollar})$ is close to $h(\text{dollars})$. On a benchmark of such analogy questions in 14 categories, the original paper reports an accuracy of about 55–60%.

Word embeddings are building blocks. A token can be classified by feeding its embedding into logistic regression or naive Bayes; a document can be represented by the average of its word embeddings, possibly concatenated with its TF-IDF vector. If the context is restricted to the k words *preceding* a position, the CBOW softmax becomes a model of the next word given the last k words, and sampling from it word by word generates text: this is a k-th order Markov language model (Chapter 12) with a learned conditional distribution.

17.2.2 ▪ Tokenizers

Word-level vocabularies cannot handle rare or unseen words. Modern language models use *tokenizers* trained on a large corpus that split text into subword units; byte-pair encoding, which repeatedly merges the most frequent adjacent pair of symbols, is the classical construction. Vocabulary sizes are large (about 128 thousand tokens for LLaMA 3 and DeepSeek-V3, about 200 thousand for GPT-4o), and their design trades compression (few tokens per text, also in languages other than English) against representational quality. Images can be tokenized as well, by cutting them into patches, which is how vision–language models consume them.

17.3 ▪ Contrastive Learning

A word2vec vector is the same for every occurrence of a word, whatever the context. Modern embedding models map entire sentences, documents, or images to vectors, and they are trained directly for the property we want: similar inputs should have similar embeddings. Given an anchor x, a *positive* example v^+ that should be close to it, and a *negative* example v^- that should not, two standard losses for an embedding network f_θ are the margin (triplet) loss and the softmax loss with temperature $\tau > 0$,

$$L(x, v^+, v^-) = \max\big(f(x)^\top f(v^-) - f(x)^\top f(v^+) + m,\ 0\big),$$

$$L(x, v^+, v^-) = -\log \frac{\exp\big(f(x)^\top f(v^+)/\tau\big)}{\exp\big(f(x)^\top f(v^+)/\tau\big) + \exp\big(f(x)^\top f(v^-)/\tau\big)}.$$

The margin loss is zero once the positive beats the negative by at least m. The slides write it with a leading minus sign; as a loss to be minimized, the sign must be positive. With many negatives, the softmax loss becomes a classification problem: which of the candidates is the positive? It is a softmax regression (Section 7.4) whose logits are similarities. This loss is often called InfoNCE.

Positives and negatives can come from class labels, from naturally paired data (a question and its answer, an image and its caption), or, without any labels, from data augmentation: two random augmentations of the same image (crops, color distortion, blur) form a positive pair, and augmentations of different images are negatives. This is SimCLR (Chen et al., 2020).

Algorithm 17.2 SimCLR

Input: Unlabeled data, batch size N, temperature τ, encoder f, projection head g, distribution $\mathcal{T}$ of augmentations, learning rate η.

repeat
  Sample a minibatch $x_1, \dots, x_N$.
  for $i = 1, \dots, N$ **do**
    Draw $t, t' \sim \mathcal{T}$; $\quad z_{2i-1} \leftarrow g\big(f(t(x_i))\big), \quad z_{2i} \leftarrow g\big(f(t'(x_i))\big)$.
  end for
  $s_{ij} \leftarrow z_i^\top z_j / (\|z_i\| \, \|z_j\|)$ for all $i, j \in \{1, \dots, 2N\}$ ▷ cosine similarity
  $\ell(i, j) \leftarrow -\log \dfrac{\exp(s_{ij}/\tau)}{\sum_{k \neq i} \exp(s_{ik}/\tau)}$
  $\mathcal{L} \leftarrow \frac{1}{2N} \sum_{i=1}^{N} \big(\ell(2i-1, 2i) + \ell(2i, 2i-1)\big)$
  Update the parameters of f and g by a gradient step on $\mathcal{L}$ with learning rate η.
until converged

Output: The encoder f; the projection head g is thrown away.

Each augmented view has one positive (the other view of the same image) and $2N - 2$ negatives (the other images in the batch), so large batches give many negatives for free. The representation $h = f(\cdot)$ before the projection head is the one kept for downstream tasks.

Text embedding models such as E5 (Wang et al., 2024) are trained in three phases: start from a pretrained language model (for example BERT, Section 17.5), pretrain contrastively on a large set of weakly paired texts, then fine-tune contrastively on smaller labeled data. The resulting embeddings serve classification, clustering, retrieval, and reranking.

17.3.1 ▪ Retrieval-Augmented Generation

Contrastive embeddings make semantic search cheap, which is the basis of *retrieval-augmented generation* (RAG): a language model answers a question with the help of relevant passages retrieved from a document collection.

Algorithm 17.3 Retrieval-Augmented Generation

Input: Document collection, embedding model g, language model, number of retrieved chunks k, user prompt q.

Preprocessing (once):

Split each document into chunks $c_1, \dots, c_J$ and store their embeddings $g(c_1), \dots, g(c_J)$.

Query time:

Compute $g(q)$ and the similarities $\cos\big(g(q), g(c_j)\big)$ for all j.
Retrieve the k chunks with the largest similarity.
Feed the prompt together with the retrieved chunks to the language model.

Output: The generated answer.

Retrieval is k-nearest neighbors (Algorithm 1.1) in embedding space with cosine similarity. With good retrieval the language model's task shifts from recalling facts to summarizing the provided context. Common extensions combine embedding search with keyword search such as BM25, or with graph structures over the documents.

17.4 ▪ Attention

Embeddings so far are static. To represent a token *in context* we need a network that processes sequences.

17.4.1 ▪ Residual and Recurrent Layers

Two building blocks precede attention. When many layers are stacked, the signal (and, by backpropagation, the gradient; Section 11.4) can shrink to zero or blow up. A *residual connection* adds the input of a layer to its output,

$$y = \phi(Wx + b) + x,$$

so that every layer only has to learn a correction to the identity and information can flow unchanged through the stack. For sequences $x_1, x_2, \dots$ a *recurrent layer* carries a state s_t and computes $(y_t, s_{t+1}) = f(x_t, s_t)$; a network built from such layers is a *recurrent neural network* (RNN). During training each item of the sequence is fed as input, whereas at generation time each input is the model's own previous prediction. The weakness of an RNN is that everything it knows about the past must pass through the single state s_t, and the computation is inherently sequential.

17.4.2 ▪ Scaled Dot-Product Attention

Attention lets the model look at all positions at once. At a high level it has three steps: compute a score for every position, normalize the scores with a softmax, and return the score-weighted average of the representations of all positions. To compute scores, each input vector x_i is mapped by three learned matrices to a *query*, a *key*, and a *value*,

$$q_i = (W^Q)^\top x_i, \qquad k_i = (W^K)^\top x_i, \qquad v_i = (W^V)^\top x_i,$$

with $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ and $W^V \in \mathbb{R}^{d \times d_v}$; stacking the $x_i^\top$ as rows of X gives $Q = XW^Q$, as in Algorithm 17.4. The terminology is borrowed from information retrieval: the query is compared with all keys, and the values of the best-matching keys are returned.

Definition 17.7 (Scaled Dot-Product Attention). *For a query $q \in \mathbb{R}^{d_k}$, keys $k_1, \dots, k_m \in \mathbb{R}^{d_k}$, and values $v_1, \dots, v_m \in \mathbb{R}^{d_v}$,*

$$\mathrm{Attn}(q, K, V) = \sum_{i=1}^{m} \alpha_i(q)\, v_i, \qquad \alpha_i(q) = \frac{\exp\left(q^\top k_i / \sqrt{d_k}\right)}{\sum_{j=1}^{m} \exp\left(q^\top k_j / \sqrt{d_k}\right)}.$$

Stacking queries, keys, and values as the rows of matrices $Q \in \mathbb{R}^{m' \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$,

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where the softmax is applied to each row. In self-attention*, queries, keys, and values are all computed from the same sequence.* □

The factor $1/\sqrt{d_k}$ keeps the scores on a fixed scale: if the entries of q and k are independent with mean zero and variance one, then $q^\top k$ has variance d_k, and without the rescaling the softmax would saturate for large d_k, with vanishing gradients. In practice several attention *heads*, each with its own W^Q, W^K, W^V, run in parallel, so that different heads can track different relations (syntax, coreference, position). Their outputs are concatenated and mixed by one more learned matrix W^O.

Algorithm 17.4 Multi-Head Self-Attention (Forward Pass)

Input: Input sequence as rows of $X \in \mathbb{R}^{m\times d}$; number of heads H; for each head h, matrices $W_h^Q, W_h^K \in \mathbb{R}^{d\times d_k}$ and $W_h^V \in \mathbb{R}^{d\times d_v}$; output matrix $W^O \in \mathbb{R}^{Hd_v\times d}$; optional causal mask.

for $h = 1, \dots, H$ **do**
  $Q_h \leftarrow XW_h^Q, \quad K_h \leftarrow XW_h^K, \quad V_h \leftarrow XW_h^V$
  $S_h \leftarrow Q_h K_h^\top / \sqrt{d_k} \in \mathbb{R}^{m\times m}$ ▷ all pairwise scores
  if causal **then** set $(S_h)_{ij} \leftarrow -\infty$ for $j > i$ ▷ no peeking at the future
  $A_h \leftarrow \mathrm{softmax}(S_h)$ row by row; $\quad O_h \leftarrow A_h V_h \in \mathbb{R}^{m\times d_v}$
end for

Output: $\mathrm{MHA}(X) = \big[O_1, \dots, O_H\big]\, W^O \in \mathbb{R}^{m\times d}$.

Row i of the output is a new representation of token i that mixes in information from every other token, weighted by relevance. The causal mask is used when the model must predict the next token and therefore may only attend to earlier positions. In *encoder–decoder* (cross) attention, the queries come from one sequence and the keys and values from another.

Remark 17.8 (Connection to Local Regression and Softmax Regression). The output at a query q is a weighted average $\sum_i \alpha_i(q) v_i$ with nonnegative weights summing to one, and the weight of position i is a kernel $\exp(q^\top k_i/\sqrt{d_k})$ evaluated between the query and the key. This is exactly the form of the weighted k-nearest-neighbor and local regression estimates of Section 6.3, where a prediction at x is a kernel-weighted average of training responses: attention is a kernel smoother whose "training set" is the sequence itself and whose kernel, inputs, and outputs are all learned. Each row of attention weights is a softmax over positions, the same function that turns scores into class probabilities in Section 7.4. □

Self-attention compares every pair of positions, so its cost is $O(m^2 d)$, quadratic in the sequence length m. Much current work reduces this cost or its memory traffic: sliding-window and linear attention, grouped-query attention (several query heads share one key and value head), multi-head latent attention, and FlashAttention, a GPU implementation that never stores the full $m \times m$ matrix. Attention-free sequence models such as RWKV and state space models revive the recurrent idea with better scaling.

Pros and Cons: Attention connects any two positions in one step, regardless of their distance, and all positions are processed in parallel, which suits GPUs far better than recurrence. The price is quadratic cost in the sequence length. Moreover, attention by itself is blind to order: permuting the input rows permutes the output rows in the same way, so position information has to be added explicitly.

17.5 ▪ The Transformer

The transformer (Vaswani et al., 2017, "Attention is all you need") is a stack of identical blocks built from attention, a feed-forward network, residual connections, and normalization.

Definition 17.9 (Transformer Block). *For an input $X \in \mathbb{R}^{m\times d}$ (one row per token), an* encoder block *computes*

$$X' = \mathrm{Norm}\big(X + \mathrm{MHA}(X)\big), \qquad Y = \mathrm{Norm}\big(X' + \mathrm{FFN}(X')\big),$$

where MHA *is Algorithm 17.4,* FFN *is a two-layer network (Section 11.2) applied to each row separately, and* Norm *is layer normalization (each row is standardized and then rescaled by learned parameters).*

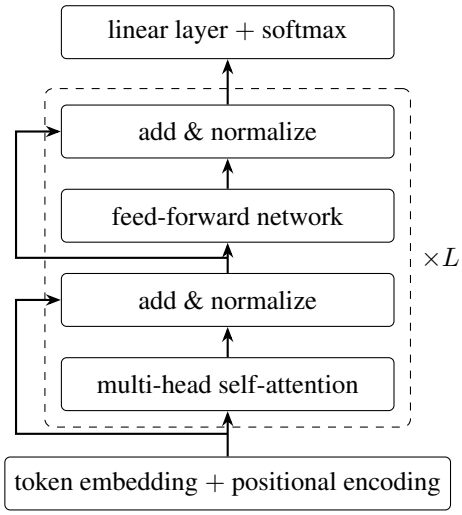


Figure 17.1. *A transformer encoder block, repeated L times. The side arrows are residual connections. A decoder block adds a causal mask to the self-attention and, in the encoder–decoder model, an extra cross-attention sub-block.*

A decoder block *uses masked self-attention and, when it reads an encoder, adds a third sub-block of cross-attention whose queries come from the decoder and whose keys and values come from the encoder output.* □

The original model, built for translation, has an encoder and a decoder of six blocks each. Many later variants place the normalization before each sub-block instead of after it, which stabilizes the training of deep stacks.

The input to the first block is a learned *token embedding*, a lookup table like the matrix U of word2vec, plus a *positional encoding*. Because attention is permutation-equivariant, without positional information a transformer would treat its input as a bag of words (Definition 17.1). The positional encoding is either a learned vector for each position or a fixed one, such as the sinusoids of the original paper,

$$\mathrm{PE}(t, 2i) = \sin\big(t/10000^{2i/d}\big), \qquad \mathrm{PE}(t, 2i+1) = \cos\big(t/10000^{2i/d}\big),$$

which assign to each position t a pattern of waves of geometrically increasing wavelengths across the embedding coordinates.

17.5.1 ▪ Training Language Models

The original transformer was trained on English–German translation: the final decoder state passes through a linear layer and a softmax that outputs probabilities over the next token. Pretraining on unlabeled text uses self-supervised tasks in the spirit of word2vec. *Masked language modeling* (BERT, Devlin et al., 2018) replaces a random 15% of the tokens by a special `[MASK]` token and trains an encoder to recover them. *Next-token prediction* (GPT, Radford et al., 2018) trains a decoder with causal attention to maximize, over a corpus $u_1, \dots, u_n$ and a context window of length k,

$$L_1 = \sum_i \log P\big(u_i \,|\, u_{i-k}, \dots, u_{i-1}; \Theta\big),$$

with

$$h_0 = UW_e + W_p, \qquad h_\ell = \mathrm{block}(h_{\ell-1}) \ \ (\ell = 1, \dots, L), \qquad P(u) = \mathrm{softmax}\left(h_L W_e^\top\right),$$

where U stacks the one-hot vectors of the context tokens, W_e is the token embedding matrix (reused at the output), and W_p is the position embedding matrix.

Algorithm 17.5 Next-Token Pretraining of a Transformer Language Model

Input: Tokenized corpus, context length k, decoder-only transformer with parameters Θ, batch size B, optimizer (SGD or Adam) with learning rate η.

repeat
  Sample B windows $(u_1, \dots, u_{k+1})$ of consecutive tokens from the corpus.
  Run the transformer with causal masking on $(u_1, \dots, u_k)$, obtaining at every position i a distribution $P(\cdot \mid u_{\le i})$.
  $\mathcal{L} \leftarrow -\frac{1}{Bk} \sum_{\text{windows}} \sum_{i=1}^{k} \log P(u_{i+1} \mid u_{\le i})$ ▷ cross-entropy at all positions at once
  $\Theta \leftarrow \Theta - \eta \, \nabla_\Theta \mathcal{L}$ ▷ gradients by backpropagation
until the training budget is spent

Output: Language model $P(u_{i+1} \mid u_{\le i})$; text is generated by sampling one token at a time and appending it to the context.

Thanks to the causal mask, one forward pass yields k training signals, one per position, which is a major reason why transformers train efficiently compared with RNNs.

Remark 17.10 (Connection to Markov Models and Maximum Likelihood). By the chain rule, $p(u_1, \dots, u_n) = \prod_i p(u_i \mid u_{<i})$, so L_1 is the log-likelihood (Section 2.2) of an autoregressive model that conditions on the last k tokens. With a probability table in place of the network this is the k-th order Markov language model of Chapter 12, whose table has $R^k(R-1)$ free entries; the transformer replaces the table by a shared network that generalizes across contexts, as word embeddings generalize across words. □

17.5.2 ▪ Beyond Text

A *vision transformer* (ViT) cuts an image into fixed-size patches, flattens and linearly projects each patch into a token embedding, adds position embeddings, and runs a standard transformer encoder; a special classification token at the start of the sequence is read out by an MLP head. CLIP (Radford et al., 2021) trains a text encoder and an image encoder jointly on N image–caption pairs per batch: it forms the $N \times N$ matrix of similarities between all image and text embeddings and applies the softmax contrastive loss of Section 17.3 along both rows and columns, treating the diagonal as positives. The result is a shared embedding space for images and text, which allows zero-shot classification by comparing an image embedding with embeddings of captions such as “a photo of a dog”.

Representation	How it is obtained	Captures similarity of meaning	Depends on context
Bag of words, n-grams	counting	no; distinct words are orthogonal	only within n tokens
TF-IDF	counting and reweighting	no	no
Word2vec	predicting neighboring words	yes, for words	no; one vector per word
Contrastive embeddings	pulling positives together, pushing negatives apart	yes, for whole texts or images	yes, through the encoder
Transformer hidden states	self-supervised pretraining with attention	yes	yes, every token sees every other

Limitations and what comes next: A transformer language model is a generative model of discrete sequences, trained by maximum likelihood one token at a time. For continuous, high-dimensional data such as images this route is awkward: there is no natural ordering of pixels, and generating hundreds of thousands of values one by one is slow. The VAE of Section 16.7 generates an image in one shot but tends to produce blurry samples. Chapter 18 presents diffusion models, which generate by gradually removing noise from a Gaussian sample; they are trained with the same evidence lower bound as the VAE, and their denoising networks contain the self-attention layers of this chapter.

Chapter 18

Diffusion Models

A generative model learns a distribution $p(x)$ from samples $x^{(1)}, \dots, x^{(n)}$ and produces new samples from it. The book has met several: Gaussian mixtures and probabilistic PCA (Chapter 16), variational autoencoders (Section 16.7), and autoregressive transformers (Section 17.5). Other well-known families are generative adversarial networks (GANs) and normalizing flows. This last chapter treats *diffusion models*, which underlie text-to-image systems such as DALL·E 3 and Stable Diffusion 3. They use two processes: a fixed *forward process* that turns an image into pure noise by adding a little Gaussian noise at a time (Section 18.1), and a learned *reverse process* that starts from noise and removes it step by step (Section 18.2). The reverse process is trained by maximizing an evidence lower bound, exactly as in variational inference (Section 18.3), which reduces to a simple denoising regression (Section 18.4). Section 18.5 compares diffusion models with VAEs, Section 18.6 shows how to condition on a label or a caption, and Section 18.7 places diffusion among the other generative models. Throughout, β_t denotes a noise variance; it is unrelated to the regression coefficients β of Part II.

18.1 ▪ The Forward Process

Definition 18.1 (Forward Process). *Let $x_0 \sim q(x_0)$ be a data point (an image, flattened into a vector in $\mathbb{R}^d$) and let $0 < \beta_1 < \cdots < \beta_T < 1$ be a* noise schedule. *The* forward process *is the Markov chain*

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \qquad q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\big),$$

that is, $x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1}$ with $\varepsilon_{t-1} \sim \mathcal{N}(0, I)$ independent of the past. □

Each step shrinks the signal slightly and adds a little noise. The shrinkage factor $\sqrt{1-\beta_t}$ is chosen so that the variance is preserved: if x_{t-1} has identity covariance, so does x_t. The forward process has no parameters; it is a linear-Gaussian chain $x_0 \to x_1 \to \cdots \to x_T$, a DAG in the sense of Section 12.2.

Proposition 18.2 (Closed-Form Marginals). *Let $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$. Then*

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\big), \qquad \textit{equivalently} \qquad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Two steps show the pattern: $x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \varepsilon_{t-2} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1}$, and the two independent noise terms combine into one Gaussian with variance $\alpha_t(1-\alpha_{t-1}) + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}$. The proposition is the practical heart of the method: a noisy version of x_0 at any level t can be drawn in one shot, without simulating the chain.

The schedule is chosen so that $\bar{\alpha}_T \approx 0$. Then $q(x_T \mid x_0) \approx \mathcal{N}(0, I)$ whatever x_0 was: after T steps all information about the data has been destroyed and x_T is white noise. The standard choice (Ho et al., 2020) is $T = 1000$ with β_t increasing linearly from 10^{-4} to 0.02.

18.2 ▪ The Reverse Process

If we could sample from the reverse transitions, generation would be easy: draw $x_T \sim \mathcal{N}(0, I) \approx q(x_T)$, then $x_{t-1} \sim q(x_{t-1} \mid x_t)$ for $t = T, \dots, 1$, and return x_0. But

$$q(x_{t-1} \mid x_t) = \int q(x_{t-1} \mid x_t, x_0)\, q(x_0 \mid x_t)\, \mathrm{d}x_0$$

involves the unknown data distribution through $q(x_0 \mid x_t)$ and is intractable. When β_t is small, however, a single forward step barely moves x, and the reverse step is approximately Gaussian as well. This motivates a Gaussian model for it.

Definition 18.3 (Reverse Process). *The* reverse process *is the Markov chain*

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \qquad p(x_T) = \mathcal{N}(0, I), \qquad p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\ \sigma_t^2 I\big),$$

where the mean μ_θ is a neural network with parameters θ and the variances σ_t^2 are fixed. □

The marginal $p_\theta(x_0) = \int p_\theta(x_{0:T})\, \mathrm{d}x_{1:T}$ is the generative model. The intermediate states $x_1, \dots, x_T$ play the role of latent variables, which suggests training by variational inference. A key ingredient is that, although $q(x_{t-1} \mid x_t)$ is intractable, the reverse step becomes tractable once we also condition on the clean data point.

Proposition 18.4 (Forward Posterior). *For $t \geq 2$, $q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t I\big)$ with*

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1 - \bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\, x_t, \qquad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\, \beta_t.$$

By Bayes' rule and the Markov property, $q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)$, a product of two Gaussian densities in x_{t-1}; completing the square gives the result. It is an instance of the linear-Gaussian posterior of Proposition 2.15: the prior is $q(x_{t-1} \mid x_0)$ from Proposition 18.2, and the observation is $x_t = \sqrt{\alpha_t}\, x_{t-1} +$ noise. The posterior mean is a weighted average of the clean image and the current noisy image.

18.3 ▪ Training by the Variational Bound

Treat $x_{1:T}$ as latent variables and the forward process $q(x_{1:T} \mid x_0)$ as the variational distribution. The ELBO identity of Section 15.4 reads

$$\log p_\theta(x_0) = \mathbb{E}_{x_{1:T} \sim q}\Big[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\Big] + D_{\mathrm{KL}}\big(q(x_{1:T} \mid x_0) \,\big\|\, p_\theta(x_{1:T} \mid x_0)\big).$$

Since the KL term is nonnegative, minimizing

$$\mathcal{L} := \mathbb{E}_{x_{0:T} \sim q}\Big[\log \frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{0:T})}\Big] \ \geq\ \mathbb{E}_{x_0 \sim q}\big[-\log p_\theta(x_0)\big]$$

minimizes an upper bound on the negative log-likelihood. Unlike in a VAE, the variational distribution has no parameters: only the decoder p_θ is trained.

Proposition 18.5 (Decomposition of the Bound).

$$\mathcal{L} = \mathbb{E}_q\Big[\underbrace{D_{\mathrm{KL}}\big(q(x_T \mid x_0) \,\|\, p(x_T)\big)}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}\big(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\big)}_{L_{t-1}} \underbrace{-\log p_\theta(x_0 \mid x_1)}_{L_0}\Big].$$

The derivation rewrites each forward factor with $t \geq 2$ by Bayes' rule,

$$q(x_t \mid x_{t-1}) = \frac{q(x_{t-1} \mid x_t, x_0)\, q(x_t \mid x_0)}{q(x_{t-1} \mid x_0)};$$

in the product over t the ratios $q(x_t \mid x_0)/q(x_{t-1} \mid x_0)$ telescope, leaving $q(x_T \mid x_0)/q(x_1 \mid x_0)$, and the remaining factor $q(x_1 \mid x_0)$ cancels against the $t = 1$ term. The decomposition turns one intractable objective into a sum of local comparisons, each between two Gaussians. The term L_T contains no parameters and is ignored. Each L_{t-1} asks the learned reverse step to match the tractable forward posterior of Proposition 18.4; since both are Gaussian with fixed isotropic covariances,

$$L_{t-1} = \mathbb{E}_q\Big[\frac{1}{2\sigma_t^2}\,\big\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\big\|^2\Big] + \text{const.}$$

18.3.1 ▪ Predicting the Noise

Substituting $x_0 = \big(x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon\big)/\sqrt{\bar{\alpha}_t}$ from Proposition 18.2 into $\tilde{\mu}_t$ gives

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon\Big).$$

The target mean is the current point minus a multiple of the noise that was added. Since x_t is known at sampling time, the network only has to predict the noise. We therefore parametrize

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon_\theta(x_t, t)\Big) \tag{18.1}$$

with a *noise predictor* ε_θ, and the loss becomes

$$L_{t-1} = \lambda_t\, \mathbb{E}_{x_0 \sim q,\, \varepsilon \sim \mathcal{N}(0,I)}\Big[\big\|\varepsilon_\theta\big(\sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon,\ t\big) - \varepsilon\big\|^2\Big], \qquad \lambda_t = \frac{\beta_t^2}{2\sigma_t^2\,\alpha_t\,(1-\bar{\alpha}_t)}.$$

Remark 18.6. The slides write the weight as $\beta_t^2/\big(2\sigma_t^2\alpha_t(1-\alpha_t)\big)$. Squaring the coefficient of ε in $\tilde{\mu}_t$ shows that the last factor is $1-\bar{\alpha}_t$, not $1-\alpha_t$. □

Ho et al. (2020) observed that samples improve if the weights are simply dropped, $\lambda_t = 1$, giving the *simplified loss*

$$L_{\mathrm{simple}}(\theta) = \mathbb{E}_{t \sim \mathrm{Unif}\{1,\dots,T\},\ x_0 \sim q,\ \varepsilon \sim \mathcal{N}(0,I)}\Big[\big\|\varepsilon_\theta\big(\sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon,\ t\big) - \varepsilon\big\|^2\Big].$$

Training is a least-squares regression: corrupt a data point with a random amount of noise and ask the network to recover the noise. Dropping λ_t reweights the noise levels and puts more emphasis on the harder, noisier ones. The resulting method is the *denoising diffusion probabilistic model* (DDPM).

Remark 18.7 (Connection to the Reparameterization Trick). Writing $x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$ is the reparameterization trick of Section 15.5: the expectation over $x_t \sim q(x_t \mid x_0)$ becomes an expectation over parameter-free noise ε. Each training step draws one t and one ε per data point, so the gradient is an unbiased Monte Carlo estimate of $\nabla_\theta L_{\mathrm{simple}}$, as in stochastic variational inference (Algorithm 15.2). □

18.4 ▪ The DDPM Algorithms

Algorithm 18.1 DDPM Training

Input: Training data, noise schedule $\beta_1, \dots, \beta_T$ with $\bar\alpha_t = \prod_{s \le t}(1-\beta_s)$, noise-prediction network $\varepsilon_\theta(x, t)$, optimizer with learning rate η.

repeat
  Sample a data point x_0 (in practice a minibatch).
  $t \sim \mathrm{Unif}\{1, \dots, T\}$; $\varepsilon \sim \mathcal{N}(0, I)$
  $x_t \leftarrow \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\, \varepsilon$ ▷ Proposition 18.2
  $\theta \leftarrow \theta - \eta \nabla_\theta \big\| \varepsilon_\theta(x_t, t) - \varepsilon \big\|^2$
until converged

Output: Noise predictor ε_θ.

Algorithm 18.2 DDPM Sampling

Input: Trained noise predictor ε_θ, schedule β_t, $\alpha_t = 1 - \beta_t$, $\bar\alpha_t$, reverse variances σ_t^2.

$x_T \sim \mathcal{N}(0, I)$
for $t = T, \dots, 1$ **do**
  $z \sim \mathcal{N}(0, I)$ if $t > 1$, else $z \leftarrow 0$
  $x_{t-1} \leftarrow \dfrac{1}{\sqrt{\alpha_t}} \Big(x_t - \dfrac{1-\alpha_t}{\sqrt{1-\bar\alpha_t}}\, \varepsilon_\theta(x_t, t) \Big) + \sigma_t z$ ▷ sample from $p_\theta(x_{t-1} \mid x_t)$, (18.1)
end for

Output: A sample x_0.

Training touches a single random noise level per example and never runs the chain; sampling runs the whole reverse chain, one network evaluation per step. No noise is added at the last step, so that the output is the predicted mean.

The main design choices are as follows. The noise predictor $\varepsilon_\theta(x_t, t)$ is a U-Net, a convolutional encoder–decoder with residual blocks (Section 17.4) and self-attention layers; its weights are shared across all time steps, and t enters through an embedding similar to the positional encoding of Section 17.5. The variances are fixed, commonly $\sigma_t^2 = \beta_t$ (or $\sigma_t^2 = \tilde\beta_t$); a learned covariance $\Sigma_\theta(x_t, t)$ is also possible. Better schedules than the linear one, such as the cosine schedule of Nichol and Dhariwal (2021), further reduce the loss.

Remark 18.8 (Connection to Langevin Dynamics and MCMC). The score of the forward marginal is $\nabla_{x_t} \log q(x_t \mid x_0) = -(x_t - \sqrt{\bar\alpha_t}\, x_0)/(1-\bar\alpha_t) = -\varepsilon/\sqrt{1-\bar\alpha_t}$, and the minimizer of the noise-prediction loss is $\mathbb{E}[\varepsilon \mid x_t] = -\sqrt{1-\bar\alpha_t}\, \nabla_{x_t} \log q(x_t)$. So the trained network estimates the score $s_\theta(x_t, t) = -\varepsilon_\theta(x_t, t)/\sqrt{1-\bar\alpha_t}$, and the update of Algorithm 18.2 becomes

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \big(x_t + \beta_t\, s_\theta(x_t, t) \big) + \sigma_t z.$$

This is a small step up the gradient of the log-density plus Gaussian noise, the structure of a Langevin step $x \leftarrow x + \frac{h}{2} \nabla \log p(x) + \sqrt{h}\, z$. Like HMC (Algorithm 14.6), it uses gradients of the log target to move toward high-density regions. Unlike MCMC, the target changes from step to step, from pure noise toward the data distribution, and the chain is run for a fixed number of steps rather than to

stationarity. Learning the score directly from noised data is *denoising score matching* (Vincent, 2011), and *score-based generative modeling* (Song et al., 2021) gives the continuous-time view of the same model. □

Pros and Cons: Diffusion models produce samples of very high quality and diversity, and their training is stable: it is plain least-squares regression on a fixed target, with no adversarial game and no learned encoder that can collapse. The main drawback is sampling cost: generating one sample requires T sequential network evaluations ($T = 1000$ in the original model), compared with a single pass for a VAE or a GAN. The likelihood is only bounded, not computed exactly.

18.5 ▪ Diffusion Models and Variational Autoencoders

Both models are trained with an ELBO, and both generate by decoding an isotropic Gaussian sample. A diffusion model is a VAE with T layers of latent variables in which the encoder is fixed and the decoder is shared across layers.

	VAE (Section 16.7)	Diffusion model
Latent variables	one code z, lower-dimensional than x	$x_1, \dots, x_T$, each of the same dimension as x_0
Encoder	learned, $q_\phi(z \mid x)$	fixed forward process $q(x_{1:T} \mid x_0)$
Decoder	learned, $p_\theta(x \mid z)$	learned reverse steps $p_\theta(x_{t-1} \mid x_t)$, one network for all t
Prior	$\mathcal{N}(0, I)$	$\mathcal{N}(0, I)$ for x_T
Objective	ELBO: reconstruction + KL	ELBO, simplified to noise-prediction least squares
Sampling	one decoder pass	T sequential network passes
Typical samples	fast but often blurry	slow but sharp and diverse

Fixing the encoder removes the hardest part of VAE training, the balance between encoder and decoder, and the small Gaussian steps make every reverse transition easy to model, at the price of many steps.

18.6 ▪ Conditional Generation and Guidance

The model so far generates unconditional samples. Text-to-image generation needs samples conditioned on a caption, a class label, or another image. With training pairs (x_0, y), the conditional reverse process is

$$p_\theta(x_{0:T} \mid y) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t, y), \qquad p_\theta(x_{t-1} \mid x_t, y) = \mathcal{N}\big(\mu_\theta(x_t, t, y),\ \Sigma_\theta(x_t, t, y)\big),$$

and the bound of Proposition 18.5 is unchanged except that y is fed to every reverse step. How y enters the U-Net depends on its type: a class label is typically added as an embedding, and a caption is typically injected through cross-attention to the embeddings of a text encoder (Section 17.4).

Plain conditioning often follows y too weakly. *Guidance* strengthens it. The idea comes from Bayes' rule for scores,

$$\nabla_{x_t} \log p(x_t \mid y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y \mid x_t),$$

in which the second term is scaled up by a factor $s > 1$, as if sampling from a distribution proportional to $p(x_t)\,p(y \mid x_t)^s$ that concentrates on inputs the classifier finds unambiguous.

18.6.1 ▪ Classifier Guidance

Dhariwal and Nichol (2021) train a separate classifier $p_\phi(y \mid x_t)$ on noisy images at all noise levels and add its log-gradient to the mean of each reverse step.

Algorithm 18.3 Classifier-Guided Diffusion Sampling

Input: Diffusion model with reverse mean and covariance $\mu_\theta(x_t, t)$, $\Sigma_\theta(x_t, t)$; classifier $p_\phi(y \mid x_t)$; class label y; guidance scale s.

1: $x_T \sim \mathcal{N}(0, I)$
2: **for** $t = T, \dots, 1$ **do**
3: $\quad \mu \leftarrow \mu_\theta(x_t, t), \quad \Sigma \leftarrow \Sigma_\theta(x_t, t)$
4: $\quad x_{t-1} \sim \mathcal{N}\big(\mu + s\,\Sigma\,\nabla_{x_t} \log p_\phi(y \mid x_t),\ \Sigma\big)$
5: **end for**

Output: A sample x_0 conditioned on y.

18.6.2 ▪ Classifier-Free Guidance

Training a classifier on noisy images is a burden. *Classifier-free guidance* (Ho and Salimans, 2022) instead trains a conditional and an unconditional model simultaneously; in practice one network $\varepsilon_\theta(x_t, t, y)$ does both, by replacing y with a null token $\varnothing$ for a random fraction of the training examples. Bayes' rule provides an implicit classifier, $p(y \mid x_t) \propto p(x_t \mid y)/p(x_t)$, so

$$\nabla_{x_t} \log p(y \mid x_t) = \nabla_{x_t} \log p(x_t \mid y) - \nabla_{x_t} \log p(x_t).$$

By Remark 18.8, scores are proportional to noise predictions, with factor $-1/\sqrt{1-\bar\alpha_t}$. The guided score $\nabla \log p(x_t) + s\,\nabla \log p(y \mid x_t)$ therefore corresponds to the guided noise prediction in line 3 below.

Algorithm 18.4 Classifier-Free Guided Sampling

Input: Noise predictor $\varepsilon_\theta(x_t, t, y)$ trained with random label dropout, condition y, guidance scale $s \geq 0$, schedule as in Algorithm 18.2.

1: $x_T \sim \mathcal{N}(0, I)$
2: **for** $t = T, \dots, 1$ **do**
3: $\quad \tilde\varepsilon \leftarrow \varepsilon_\theta(x_t, t, \varnothing) + s\,\big(\varepsilon_\theta(x_t, t, y) - \varepsilon_\theta(x_t, t, \varnothing)\big)$
4: $\quad z \sim \mathcal{N}(0, I)$ if $t > 1$, else $z \leftarrow 0$
5: $\quad x_{t-1} \leftarrow \dfrac{1}{\sqrt{\alpha_t}}\Big(x_t - \dfrac{1-\alpha_t}{\sqrt{1-\bar\alpha_t}}\,\tilde\varepsilon\Big) + \sigma_t z$
6: **end for**

Output: A sample x_0 conditioned on y.

With $s = 0$ the sampler ignores y, with $s = 1$ it is the plain conditional model, and with $s > 1$ the condition is enforced more strongly. The guidance scale controls a trade-off between sample quality and diversity: larger s gives sharper, more prototypical images that match the caption closely, and less variety among them. Conditioning need not be on text; image-to-image diffusion models (for example Palette, Saharia et al., 2022) condition on another image for colorization, inpainting, uncropping, and restoration.

18.7 ▪ Trade-offs in Generative Modeling

Three properties are desirable in a generative model: high-quality samples, fast sampling, and coverage of all modes of the data (diversity). The main families each achieve two of them (Xiao et al., 2021). GANs give high-quality samples quickly but tend to miss modes. VAEs and normalizing flows sample quickly and cover the modes, but their samples are of lower quality. Diffusion models achieve quality and coverage, but sample slowly. Much current research accelerates diffusion sampling to escape this trilemma. The probability flow ODE (Song et al., 2021), a deterministic counterpart of the reverse process, is one route to fewer steps.

Looking back: A small number of ideas carry the whole book. *Likelihood and Bayes' rule* define estimators from least squares and logistic regression (Chapters 4 and 7) to mixtures and language models (Chapters 16 and 17), and posterior distributions from Bayesian linear regression and Gaussian processes (Chapters 5 and 9) to latent variable models. *Gaussian algebra* (Section 2.5) gives closed forms for LDA, Bayesian regression, GPs, probabilistic PCA, and both processes of a diffusion model. *Controlling complexity*, through the bias–variance trade-off (Section 1.5), shrinkage, pruning, averaging, and cross-validation, separates learning from memorizing. When exact inference is impossible (Chapter 13), we either *sample*, by Monte Carlo and MCMC (Chapter 14), or *optimize* a KL divergence through the ELBO (Chapter 15); the ELBO then reappears in EM, VAEs, and diffusion models. Underneath all of it runs *gradient-based optimization* (Chapter 3), made practical for networks by backpropagation (Chapter 11). Diffusion models use nearly every one of these ideas at once: a Gaussian Markov chain, an ELBO, a reparameterized Monte Carlo gradient, least-squares regression, and a deep network trained by SGD.